



БИБЛИОТЕКА РАЗРАБОТЧИКА

Е. Ю. Хрусталева

ЗНАКОМСТВО С РАЗРАБОТКОЙ МОБИЛЬНЫХ ПРИЛОЖЕНИЙ НА ПЛАТФОРМЕ «1С:ПРЕДПРИЯТИЕ 8»



Издание 2

+CD

Включает мобильную
платформу

1C[®]
ПАБЛИШИГ

Хрусталева Е.Ю.

Знакомство с разработкой мобильных приложений на платформе "1С:Предприятие 8". Издание 2

Электронная книга в формате pdf; ISBN 978-5-9677-2497-8.

Электронный аналог издания "Знакомство с разработкой мобильных приложений на платформе "1С:Предприятие 8". Издание 2" (ISBN 978-5-9677-2494-7, М.: ООО "1С-Паблишинг", 2015; артикул печатной книги по прайс-листу фирмы "1С": 4601546122896; по вопросам приобретения печатных изданий издательства "1С-Паблишинг" обращайтесь к партнеру "1С", обслуживающему вашу организацию, или к другим партнерам фирмы "1С").

Книга адресована начинающим разработчикам прикладных решений в системе "1С:Предприятие 8". Она поможет овладеть наиболее эффективными приемами разработки приложений.

Средства разработки "1С:Предприятия 8" позволяют решать широкий круг задач. Начинающим разработчикам бывает сложно сориентироваться в многообразии имеющихся инструментов и возможностей. Зачастую бывает так, что одна и та же задача может быть выполнена с помощью разных инструментов или разными способами. Но непонятно, какой из этих способов является предпочтительным. Именно в таких случаях поможет эта книга. Она содержит набор простых примеров, показывающих один наиболее эффективный способ решения той или иной задачи, возникающей в процессе разработки.

Примеры достаточно подробно описаны и проиллюстрированы для того, чтобы их мог воспроизвести начинающий. В то же время за более подробной информацией об используемых инструментах следует обращаться к документации "1С:Предприятие 8".

Для создания демонстрационных примеров использовалась версия 8.3.5.1443 платформы "1С:Предприятие 8".

Дополнительные материалы к книге включают эти дистрибутивы, а также несколько информационных баз. Эти материалы помогут вам самостоятельно воспроизвести примеры, описанные в книге. Скачать дополнительные материалы можно по ссылке: http://its.1c.ru/book_demo/.

Книга выпущена под редакцией Максима Радченко.

Интернет-конференция для начинающих разработчиков
<http://devtrainingforum.v8.1c.ru/forum>.

Оглавление

Введение	7
Функциональность мобильного приложения	11
Постановка задачи	11
Последовательность наших действий	13
Подготовка стационарного компьютера	15
Платформа «1С:Предприятия»	15
Глава 1. С чего начать	15
Установка веб-сервера	17
Android SDK	18
Мобильная платформа «1С:Предприятия»	22
USB-драйвер	22
Подготовка планшета	23
Настройки планшета	23
Установка мобильной платформы разработчика	25
Обновление мобильного приложения на планшете	26
Публикация мобильного приложения на веб-сервере	26
Добавление мобильного приложения на планшет	28
Изменение свойств мобильного приложения	31
Глава 2. Разработка мобильного приложения	33
Создание основных объектов конфигурации	34
Первоначальный обмен данными	37
План обмена	37

Обработка для обмена данными	41
Создание конфигурации для мобильного приложения	46
Перенос данных на планшет	53
Интерфейс мобильного приложения	60
Доработка командного интерфейса	62
Ограничения прав доступа	64
Разработка форм	68
Функциональность мобильного приложения	85
Заказы	85
Хранимые файлы	119
Клиенты	131
Список заказов	151
Обслуживание заказов	158
Отчет по данным мобильного приложения	161
Глава 3. Разработка основного приложения	175
Создание и редактирование информации для отправки курьеру	176
Заказы	176
Клиенты	189
Хранимые файлы	192
Товары	204
Отчеты	213
Приход товаров	214
Расход товаров	215
Регистры накопления	219
Движения документов при проведении	221
Остатки товаров на складах	234
Причины отказа от товаров	238
Продажи клиентам	244
Запасной способ обмена данными с основным приложением	251
Глава 4. Обмен данными	251
Обмен новыми данными	252
Обмен данными с разной структурой	255
Разрешение коллизий при обмене существующими данными	269
Передача данных из основного приложения	273
Тестирование обмена данными в обе стороны	276
Основной обмен данными через Web-сервис	292
Создание Web-сервиса в основном приложении	292
Использование Web-сервиса в мобильном приложении	312
Работа с push-уведомлениями	328
Общая информация	328

Пример работы с push-уведомлениями	329
Сборщик мобильных приложений.....	351
Глава 5. Сборка мобильного приложения	351
Настройка параметров прикладного решения	354
Настройка параметров поставщика	362
Загрузка мобильной платформы.....	365
Загрузка мобильной конфигурации.....	366
Описание параметров мобильного приложения.....	368
Сборка мобильного приложения	375

Введение

Организация бизнеса в современном мире все чаще требует наличия удаленного доступа пользователей к данным и функционалу систем по управлению хозяйственной деятельностью предприятия.

Это могут быть руководители и различные «управленцы», которым необходимо, не находясь в офисе, быстро просмотреть какие-то важные отчеты и документы (например, на смартфоне по дороге на совещание).

С другой стороны, все больше рядовых сотрудников работают с клиентами «на выезде»: курьеры интернет-магазинов, торговые агенты и т.д. Результаты своей деятельности (например, оформленные новые заказы) они должны иметь возможность оперативно внести в основную базу, чтобы в офисе не дожидались их возвращения. Из офиса они также могут получить какие-то данные (документы, отчеты), необходимые для работы.

При этом таким сотрудникам не нужен доступ к полнофункциональной версии «1С:Предприятия». Им нужно только иметь при себе мобильное устройство (смартфон или планшет), на котором будет реализован необходимый им ограниченный круг задач.

В этой книге мы рассмотрим пошагово весь процесс разработки мобильных приложений, то есть приложений, работающих на мобильных устройствах под управлением мобильной платформы «1С:Предприятия».

Мобильная платформа – это специальная технология, позволяющая создавать приложения, работающие на мобильных устройствах (смартфонах или планшетных ПК) под управлением операционных систем Android или iOS.

Мобильное приложение, установленное на устройстве, содержит мобильную платформу и информационную базу. В свою очередь, информационная база включает аналог файловой информационной базы, предназначенной для хранения данных пользователя, и мобильное приложение, то есть программный код, выполняющийся на мобильном устройстве (рис. 1.1).

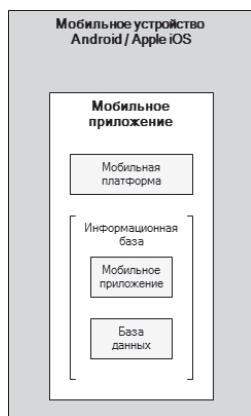


Рис. 1.1. Составные части мобильного приложения

Необходимо понимать, что мобильное приложение по сути не является мобильным клиентом, который подключается к офисной базе и имеет доступ ко всем функциям офисного приложения.

Мобильные приложения предназначены в основном для организации удаленных рабочих мест для прикладных решений, функционирующих на стационарных компьютерах. По отношению к мобильным приложениям они являются *основными приложениями*. В процессе работы они постоянно обмениваются данными с мобильными приложениями через веб-сервисы, файловый обмен или электронную почту (рис. 1.2).



Рис. 1.2. Обмен данными с мобильным приложением

Таким образом, мобильные приложения в режиме удаленного доступа решают некоторый специфический набор задач, необходимый владельцу мобильного устройства.

Мобильное приложение, разработанное нами и описанное в книге, реализовано на планшете Samsung Galaxy Tab2 (10.1) GT-P5110, работающем под управлением операционной системы Android. Но то же самое мобильное приложение можно собрать и для работы на iPhone или iPad под управлением iOS. В данной книге этот вопрос рассматриваться не будет.

ПРИМЕЧАНИЕ

Для сборки под iOS нужен специальным образом настроенный компьютер Mac, доступный по сети сборщику мобильного приложения. Кроме того, если приложение под iOS будет работать с push-уведомлениями, то необходимо получить сертификат мобильного приложения и зарегистрироваться на сервисе APNs (для этого тоже необходим компьютер Mac).

ПОДРОБНЕЕ

Документация 1С:Предприятие 8.3. Руководство разработчика. Глава 26. Разработка решений для мобильной платформы > Взаимодействие с мобильным устройством при разработке мобильного приложения > Сборка мобильного приложения перед публикацией в магазине приложений.

Документация 1С:Предприятие 8.3. Руководство разработчика. Глава 26. Разработка решений для мобильной платформы > Особенности использования > Специальные возможности > Работа с уведомлениями > Push-уведомления > Работа с сервисом APNs.

Постановка задачи

Функциональность мобильного приложения

В данной книге мы разработаем мобильное приложение, работающее на планшете, которое будет автоматизировать деятельность курьера интернет-магазина по продаже одежды. Это приложение будет обмениваться данными с основным приложением интернет-магазина, работающим на стационарном компьютере, то есть получать оттуда данные, изменять их и отсылать обратно.

Итак, реализуем типовые сценарии действий курьера интернет-магазина:

- Курьер выполняет обмен данными с основной базой интернет-магазина и получает оттуда заказы на текущую дату, которые он должен обслужить. Затем забирает заказанные товары со склада.
- Также курьер получает из интернет-магазина список заказов в порядке их обслуживания. Открывает первый заказ из этого списка и определяет местоположение клиента, сделавшего заказ, на карте.
- Звонит клиенту и сообщает о том, что он к нему едет. Возможно, он не дозванивается или задерживается, тогда курьер может послать клиенту SMS.
- Приезжает к клиенту. После знакомства с товарами клиент какие-то позиции заказа берет, а какие-то – нет.

- Курьер отмечает факт продажи или возврата товаров в заказе. При этом у возвращенных товаров указывается причина отказа (например, не подошел размер, цвет и т. п.).
- В случае несоответствия товара требованиям клиента (например, по цвету или качеству) курьер может сделать его фото- и видеосъемку, а также записать аудиоотзыв клиента о заказе.
- После этого курьер ставит отметку о выполнении заказа, и сразу же выполняется обмен с основной базой, чтобы в интернет-магазине была оперативная информация о том, что точно куплено, а что будет возвращено обратно и может быть использовано, например, для заказов на завтра и ближайшие дни.
- Курьер может добавить новый заказ этого же или нового клиента, если возникла такая необходимость (например, нужно перезаказать тот же товар, но другого размера).
- В случае необходимости курьер может создавать для себя напоминания (например, о звонке клиенту), которые будут появляться в указанное время или повторяться периодически. Кроме того, курьер может отправить клиенту письмо по электронной почте.
- При внесении курьером изменений в заказ, добавлении нового заказа, клиента и т. п. автоматически выполняется обмен данными с интернет-магазином, и эти изменения отправляются в основную базу.
- С учетом этих данных в интернет-магазине формируется отчет об остатках товаров на складе и отправляется на планшет курьеру.
- Также курьер может получить отчет о своих продажах за период, используя те данные, которые есть на планшете.
- Интернет-магазин также может посылать курьеру сообщения (например, о необходимости произвести с ним обмен данными), если в офисе изменилась важная для курьера информация.
- Затем курьер находит следующий по порядку заказ в списке, открывает данные клиента, сделавшего заказ, и т. д.

Последовательность наших действий

Надо понимать, что мобильное приложение разрабатывается в конфигураторе «1С:Предприятия» на стационарном компьютере. В процессе разработки его можно запустить в пользовательском режиме и проверить, как оно работает.

Однако на мобильном устройстве существует ряд ограничений, специфических возможностей и особенностей. Поэтому в процессе разработки мы будем запускать мобильное приложение на планшете и смотреть, как оно выглядит и реализует свои функции.

Для работы мобильного приложения на мобильном устройстве требуется мобильная платформа «1С:Предприятия». Поэтому, прежде чем начинать разрабатывать мобильное приложение, мы должны установить мобильную платформу на мобильном устройстве и выполнить ряд других подготовительных действий, речь о которых пойдет в первой главе книги.

Таким образом, в книге мы рассмотрим следующую последовательность действий:

- Сначала в первой главе («С чего начать») мы подготовим стационарный компьютер и планшет и настроим связь между ними. Эти действия необходимы для разработки любого мобильного приложения и никак не связаны с прикладной спецификой нашего примера.
- Во второй главе («Разработка мобильного приложения») мы разработаем всю функциональность мобильного приложения, описанную выше, а также реализуем простейший обмен данными, для того чтобы протестировать мобильное приложение в процессе разработки.
- В третьей главе («Разработка основного приложения») мы реализуем небольшое основное приложение для интернет-магазина, которое будет служить источником данных для работы курьера на планшете. На основе полученных от курьера данных в основном приложении будет формироваться отчет и передаваться курьеру на планшет через веб-сервис.
- В четвертой главе («Обмен данными») мы реализуем основной обмен данными с мобильным приложением через веб-сервис. Этот обмен данными предполагается использовать всегда, когда возможно интернет-соединение. А также доработаем первоначальный обмен данными так, чтобы его можно было использовать как аварийный способ обмена в том случае, если отсутствует соединение с Интернетом.
- В пятой главе («Сборка мобильного приложения») мы покажем, как собрать уже разработанное мобильное приложение в установочный файл для загрузки на планшет (для ОС Android).

Глава 1.

С чего начать

Подготовка стационарного компьютера

Платформа «1С:Предприятия»

Прежде всего для разработки мобильного приложения на стационарном компьютере должна быть установлена «боевая» или учебная версия платформы «1С:Предприятия». При написании данной книги использовалась версия платформы 8.3.6.2041.

При установке платформы должен быть установлен компонент «Модули расширения веб-сервера». Этот компонент необходим, чтобы опубликовать мобильное приложение на веб-сервере.

Если платформа уже установлена на компьютер, можно проверить, какие компоненты у нее установлены. Для этого нужно открыть список установленных на компьютере программ (Панель управления > Программы и компоненты), выделить используемую версию «1С:Предприятия» и нажать кнопку Изменить. Появится диалог обслуживания программ платформы с помеченной секцией Изменить (рис. 1.3).

В этом диалоге нужно нажать Далее. В появившемся списке компонентов платформы необходимо убедиться, что компонент Модули расширения веб-сервера установлен. В этом случае можно нажать Отмена и выйти из диалога выборочной установки. В противном случае нужно сделать компонент доступным, нажать Далее и продолжить установку платформы, как обычно (рис. 1.4).

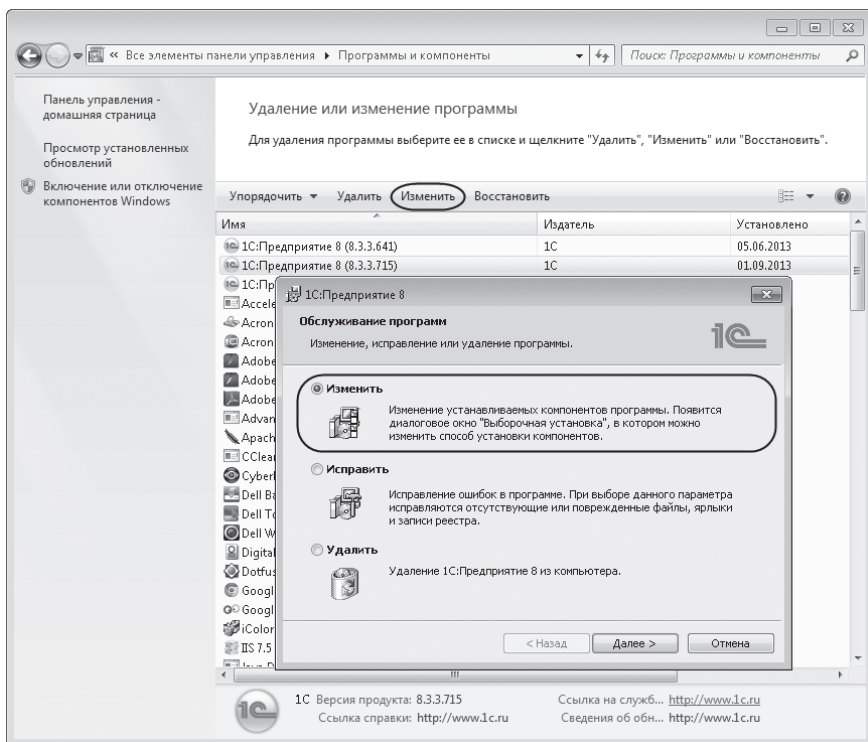


Рис. 1.3. Диалог обслуживания программ «1С:Предприятия»

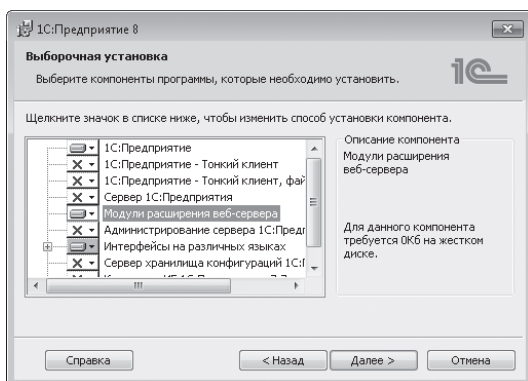


Рис. 1.4. Выборочная установка компонентов платформы «1С:Предприятия»

Установка веб-сервера

Затем на компьютер необходимо установить веб-сервер. В нашем примере мы будем использовать веб-сервер Apache.

Если на вашем компьютере уже установлен локальный веб-сервер Internet Information Services (IIS), то он может конфликтовать с веб-сервером Apache. Поэтому лучше его остановить или удалить (если он не нужен) из состава компонентов Windows (Панель управления > Программы и компоненты > Включение или отключение компонентов Windows), см. рис. 1.5.

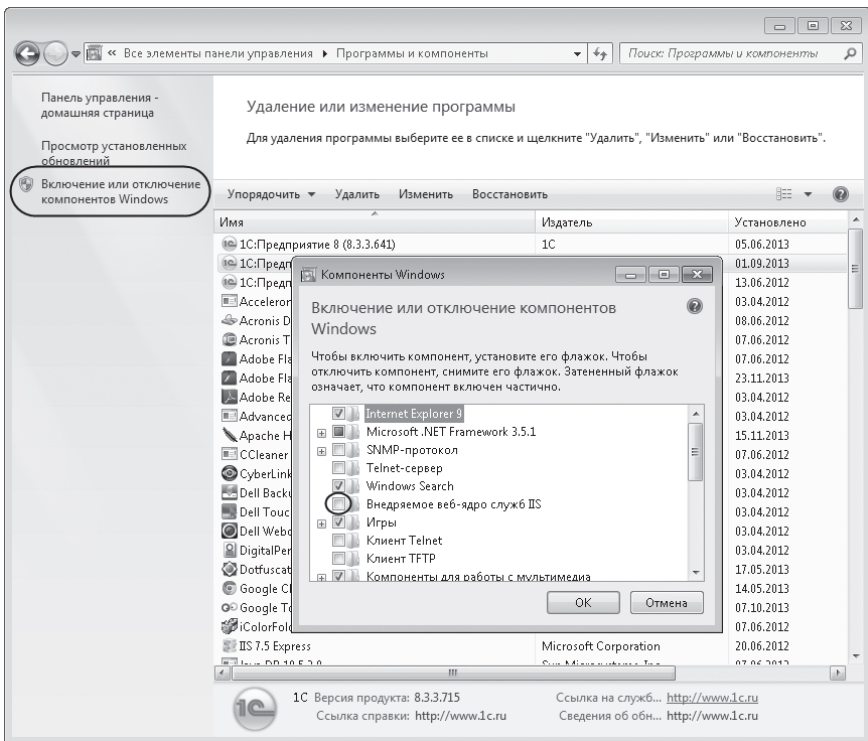


Рис. 1.5. Удаление IIS из состава компонентов Windows

Скачать установочный пакет MSI Installer для веб-сервера Apache можно по адресу: <http://archive.apache.org/dist/httpd/binaries/win32/>.

На открывшейся странице находятся ссылки на дистрибутивы Apache для Windows. Чтобы быстро найти наиболее свежую версию дистрибутива, можно щелкнуть два раза по заголовку колонки Last modified, затем выбрать и скачать дистрибутив `httpd-2.2.25-win32-x86-no_ssl` (рис. 1.6).

Name	Last modified	Size
Parent Directory		-
symbols/	2014-11-23 17:47	-
patches_applied/	2012-06-13 23:14	-
httpd-2.2.25-win32-x86-openssl-0.9.8y.msi.sha1	2013-07-10 08:06	83
httpd-2.2.25-win32-x86-openssl-0.9.8y.msi.md5	2013-07-10 08:06	76
httpd-2.2.25-win32-x86-openssl-0.9.8y.msi.asc	2013-07-10 08:06	835
httpd-2.2.25-win32-x86-openssl-0.9.8y.msi	2013-07-10 08:06	6.1M
httpd-2.2.25-win32-x86-no_ssl.msi.sha1	2013-07-10 08:06	75
httpd-2.2.25-win32-x86-no_ssl.msi.md5	2013-07-10 08:06	68
httpd-2.2.25-win32-x86-no_ssl.msi.asc	2013-07-10 08:06	835
httpd-2.2.25-win32-x86-no_ssl.msi	2013-07-10 08:06	5.5M
httpd-2.0.65-win32-x86-openssl-0.9.8y.msi.sha1	2013-07-10 08:06	83
httpd-2.0.65-win32-x86-openssl-0.9.8y.msi.md5	2013-07-10 08:06	76
httpd-2.0.65-win32-x86-openssl-0.9.8y.msi.asc	2013-07-10 08:06	835
httpd-2.0.65-win32-x86-openssl-0.9.8y.msi	2013-07-10 08:06	5.5M
httpd-2.0.65-win32-x86-no_ssl.msi.sha1	2013-07-10 08:06	75
httpd-2.0.65-win32-x86-no_ssl.msi.md5	2013-07-10 08:06	68
httpd-2.0.65-win32-x86-no_ssl.msi.asc	2013-07-10 08:06	835
httpd-2.0.65-win32-x86-no_ssl.msi	2013-07-10 08:06	4.8M

Рис. 1.6. Установочный пакет для Windows веб-сервера Apache

Затем нужно сохранить установочный пакет, запустить его и установить веб-сервер Apache на компьютер. После установки значок веб-сервера появится в нижнем правом углу монитора, в списке загруженных программ в панели задач.

Android SDK

Далее на стационарном компьютере нужно установить комплект средств разработки Android SDK. Этот комплект понадобится, чтобы установить мобильную платформу разработчика «1С:Предприятия» на планшет.

Скачать Android SDK можно по адресу: <http://developer.android.com/sdk/index.html>.

Внизу открывшейся страницы в группе SDK Tools Only нужно скачать архив `installer_r24.3.4-windows.exe (Recommended)`, см. рис. 1.7.

Download ^

Installing the SDK

Adding SDK Packages

Android Studio v

Workflow v

Tools Help v

Build System v

Performance Tools v

Testing Tools v

SDK Tools Only

If you prefer to use a different IDE or run the tools from the command line or with build scripts, you can instead download the stand-alone Android SDK Tools. These packages provide the basic SDK tools for app development, without an IDE. Also see the SDK tools release notes.

Platform	Package	Size	SHA-1 Checksum
Windows	installer_r24.3.4-windows.exe (Recommended)	139477985 bytes	094dd45f98a31f839feae898b48f23704f2878dd
	android-sdk_r24.3.4-windows.zip	187496897 bytes	4a8718fb4a2bf2128d34b92f23ddd79fc65839e7
Mac OS X	android-sdk_r24.3.4-macosx.zip	98340900 bytes	128f10fba668aa490cc94a08e505a48a608879b9
Linux	android-sdk_r24.3.4-linux.tgz	309138331 bytes	fb293d7bca42e05580be56b1adc22055d46603dd

Рис. 1.7. Получение Android SDK

На следующей странице нужно согласиться с правилами и соглашениями (установить флажок I have read and agree...) и нажать на кнопку Download installer_r24.3.4-windows.exe (рис. 1.8).

Download ^

Installing the SDK

Adding SDK Packages

Android Studio v

Workflow v

Tools Help v

Build System v

Performance Tools v

Testing Tools v

Support Library v

Data Binding Library

Revisions v

NDK

Download

Before installing Android Studio or the standalone SDK tools, you must agree to the following terms and conditions.

Terms and Conditions

This is the Android Software Development Kit License Agreement

1. Introduction

1.1 The Android Software Development Kit (referred to in this License Agreement as the "SDK" and specifically including the Android system files, packaged APIs, and Google APIs add-ons) is licensed to you subject to the terms of this License Agreement. This License Agreement forms a legally binding contract between you and Google in relation to your use of the SDK.

1.2 "Android" means the Android software stack for devices, as made available under the Android Open Source Project, which is located at the following URL: <http://source.android.com/>, as updated from time to time.

☒ I have read and agree with the above terms and conditions

DOWNLOAD INSTALLER_R24.3.4-WINDOWS.EXE

Рис. 1.8. Получение Android SDK

Затем нужно сохранить полученный инсталлятор и запустить его. После установки Android SDK Tools автоматически запускается приложение SDK Manager.

После запуска менеджер соединяется с Интернетом, чтобы определить список доступных для установки пакетов API. Это может занять длительное время; о ходе процесса сигнализирует зеленая строка индикатора, расположенная в нижней части окна. Когда все пакеты API будут загружены, внизу окна Android SDK Manager появится надпись «Done loading packages».

ВНИМАНИЕ

Нужно иметь в виду, что SDK Manager не всегда может самостоятельно соединиться с Интернетом. Если у вас возникнет такая проблема, то нужно будет открыть настройки менеджера (Tools – Options) и указать прокси-сервер, через который осуществляется выход в Интернет, и порт соединения (обычно стандартный – 8080). После этого нужно нажать Packages – Reload и ввести имя пользователя и пароль для выхода в Интернет.

Для установки мобильной платформы разработчика на мобильное устройство и последующей сборки мобильного приложения нужно поместить и установить пакеты:

- Версия Android SDK Tools – не ниже 20.0.3;
- Версия Android SDK Platform-tools – не ниже 14;
- Версия SDK Platform – API 17.

Чтобы установить отмеченные пакеты, нужно нажать кнопку Install... (рис. 1.9).

При установке пакетов нужно принять лицензионные соглашения (отметить опцию Accept License) и нажать кнопку Install (рис. 1.10).

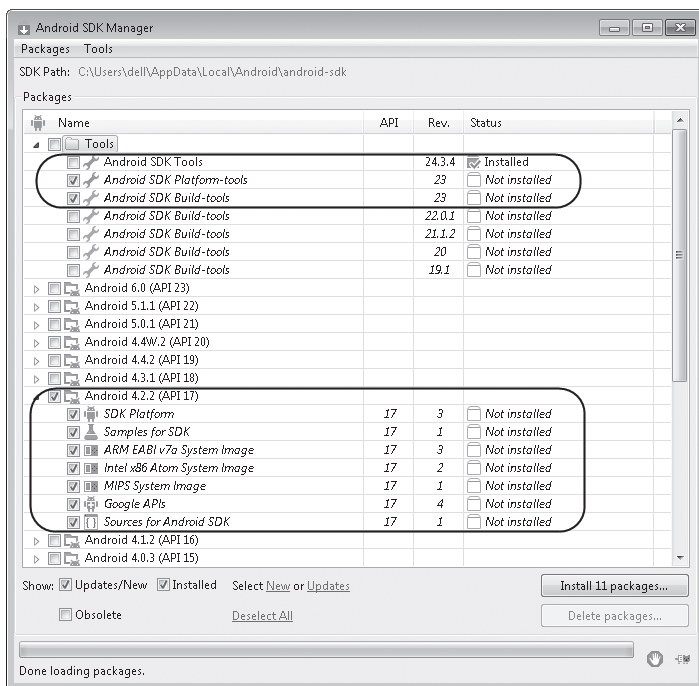


Рис. 1.9. Установка SDK Manager

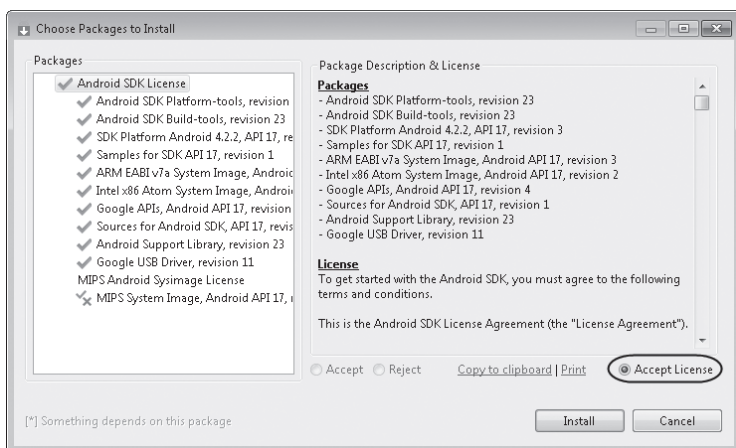


Рис. 1.10. Установка SDK Manager

Мобильная платформа «1С:Предприятия»

Далее с сайта поддержки пользователей «1С:Предприятия» (<http://users.v8.1c.ru>) нужно получить самую последнюю актуальную версию мобильной платформы (рис. 1.11).

Корпоративный инструментальный пакет 8	2.0.15.9	03.06.15		Не определена			Не определена
Менеджер лицензий аппаратной защиты NetHASP	8.31	01.01.07		Не определена			Не определена
Мобильная платформа 1С:Предприятия	8.3.6.182	01.06.15		Не определена		8.3.5.106 8.3.4.24	16.04.15 13.08.14
Обработки обслуживания торгового оборудования - для Технологической платформы 8.1				Не определена			Не определена
Обработки обслуживания торгового оборудования - для Технологической платформы 8.2				Не определена			Не определена
Расширение для карманных компьютеров	8.2.9.4	16.10.13		Не определена			Не определена
Расширение для карманных компьютеров. Базовая версия.	8.2.9.4	16.10.13		Не определена			Не определена
Система проектирования прикладных решений	1.1.12.7	08.05.15		Не определена			Не определена
Технологическая платформа 8.0	8.0.18.2	19.12.06		Не определена			Не определена
Технологическая платформа 8.1	8.1.15.14	30.10.09		Не определена			Не определена
Технологическая платформа 8.2	8.2.19.130	13.02.15		Не определена			Не определена
Технологическая платформа 8.2 (учебная версия)	8.2.19.130	13.02.15		Не определена			Не определена

Рис. 1.11. Список дистрибутивов «1С:Предприятия»

В нашей книге мы используем мобильную платформу версии 8.3.6.209.

Затем нужно сохранить полученный архив `mobile.zip` и запомнить расположение этого архива, так как в будущем он понадобится нам для сборки уже готового мобильного приложения. А также нужно распаковать архив `mobile.zip` в какую-нибудь папку, где в результате окажутся три папки с файлами: `Android`, `iOS MobileAppMaker` и служебные файлы.

USB-драйвер

Затем нужно установить на стационарный компьютер USB-драйвер для планшета (в нашем случае – планшет Samsung Galaxy Tab 2 (10.1) GT-P5110). Особенность нашего планшета заключается в том, что нужный нам драйвер находится в составе Samsung Kies – специальной программы фирмы «Самсунг» для взаимодействия планшета и настольного компьютера. Поэтому мы скачаем и установим ее всю, хотя вся она нам не нужна.

Скачать установочный пакет Samsung Kies можно по адресу: <http://www.samsung.com/ru/support/model/GT-P5110TSASER-downloads>.

Внизу открывшейся страницы в группе Загрузки нужно нажать на ссылку в разделе Программное обеспечение и скачать установочный файл для ОС Windows (рис. 1.12).

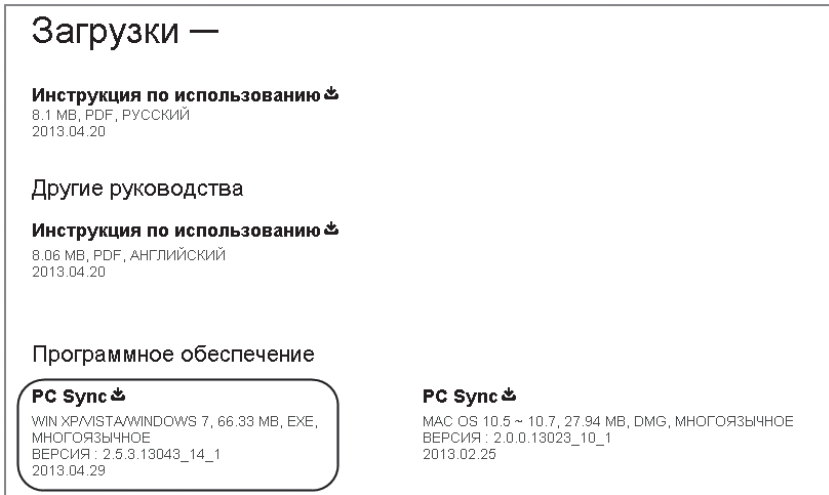


Рис. 1.12. Установочный пакет Samsung Kies

Затем нужно сохранить установочный файл, запустить его и установить Samsung Kies на компьютер.

Теперь на стационарном компьютере все готово к взаимодействию с планшетом.

Подготовка планшета

Настройки планшета

Осталось задать необходимые настройки планшета. Чтобы открыть список настроек, нужно вызвать приложение Настройки в списке приложений планшета (рис. 1.13).

Затем нужно найти в списке групп настроек (слева) группу Безопасность и отметить ее. В списке настроек безопасности (справа) нужно установить флажок у настройки Неизвестные источники (рис. 1.14).



Рис. 1.13. Вызов настроек планшета

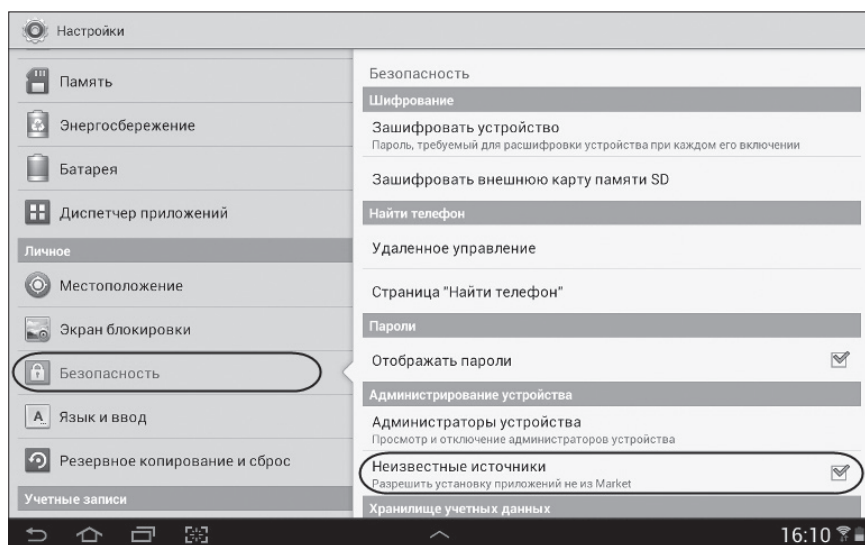


Рис. 1.14. Настройки планшета

Затем в списке групп настроек нужно отметить группу Параметры разработчика, в правом верхнем углу планшета переключить тумблер и сделать доступными параметры разработчика. В списке этих параметров нужно установить флажок у настройки Отладка USB (рис. 1.15).

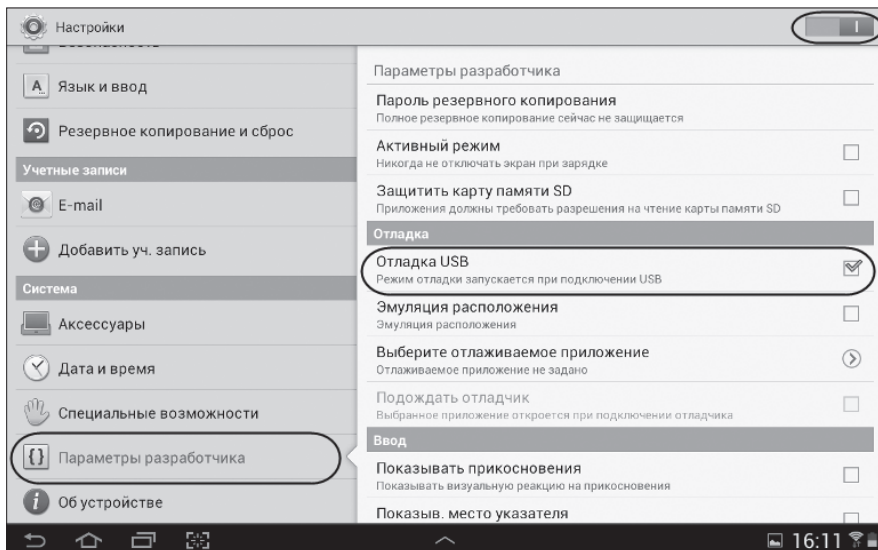


Рис. 1.15. Настройки планшета

Установка мобильной платформы разработчика

Чтобы иметь возможность протестировать работу мобильного приложения на планшете, нужно установить на него мобильную платформу разработчика «ICS:Предприятия».

Для этого нужно подключить планшет к стационарному компьютеру через установленный выше USB-драйвер. В нашем случае нужно подключить планшет к компьютеру через USB, запустить Samsung Kies и дожидаться, когда Samsung Kies увидит планшет и подключит его.

Затем с помощью интерпретатора командной строки ОС Windows выполнить команду: <каталог ANDROID_SDK>\platform-tools\adb.exe install -r <Каталог платформы>\1cem-arm.apk. Например:

```
"F:\AndroidSDK\platform-tools\adb.exe" install -r "F:\Mobile1C\Android\1cem-arm.apk"
```

Здесь:

- F:\AndroidSDK – каталог, куда мы распаковали комплект средств разработки Android SDK;
- F:\Mobile1C – каталог, куда мы распаковали мобильную платформу.

В результате мобильная платформа разработчика будет установлена на планшет и появится в списке приложений планшета (рис. 1.16).



Рис. 1.16. Мобильная платформа разработчика «1С:Предприятия»
в списке приложений планшета

Обновление мобильного приложения на планшете

Публикация мобильного приложения на веб-сервере

В процессе разработки мобильного приложения нам необходимо, чтобы при любых изменениях в конфигураторе мы сразу же могли посмотреть, как это работает на планшете. Для этого измененное мобильное приложение автоматически должно передаваться на планшет через веб-сервер (см. рис. 1.17).

Для примера создадим новую конфигурацию с именем МобильноеПриложение. Свойство конфигурации Назначение использования установим в значение Мобильное устройство (рис. 1.17).

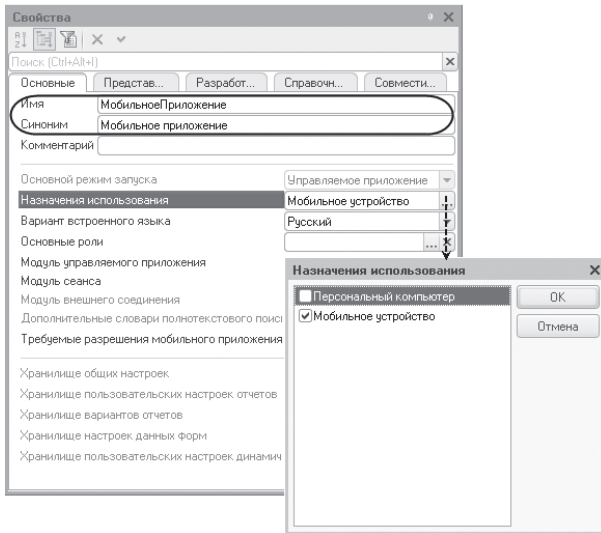


Рис. 1.17. Создание конфигурации мобильного приложения

Заметьте, что конфигурацию, разрабатываемую для мобильного устройства, мы будем называть *мобильное приложение*.

Теперь обновим конфигурацию базы данных (F7) и опубликуем это мобильное приложение на веб-сервере.

Для этого нужно выполнить команду конфигурирования Конфигурация > Мобильное приложение > Публиковать ... (рис. 1.18).

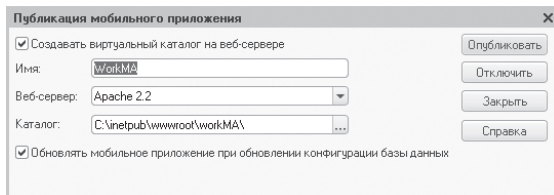


Рис. 1.18. Публикация мобильного приложения на веб-сервере

В появившемся диалоге в поле **Имя** нужно задать имя виртуального каталога на веб-сервере, в который будет выполнена публикация мобильного приложения (это имя будет использовать мобильное приложение для своего обновления).

Поле **Веб-сервер** автоматически заполнилось единственным установленным на компьютере веб-сервером Apache 2.2.

В поле **Каталог** нужно указать физический каталог компьютера, в котором будет находиться файл публикации мобильного приложения.

Затем нужно нажать кнопку **Опубликовать** и подтвердить, что нужно обновить мобильное приложение сейчас.

Обратите внимание, что флажок **Обновлять мобильное приложение при обновлении конфигурации базы данных** стандартно установлен. Это значит, что при обновлении конфигурации базы данных будет автоматически обновляться публикация мобильного приложения.

Добавление мобильного приложения на планшет

После этого можно добавить мобильное приложение на планшет. Для этого нужно найти в списке приложений планшета мобильную платформу разработчика «1С:Предприятия» и запустить ее. Платформа откроет список своих приложений, который пока пуст.

Теперь нужно добавить новое мобильное приложение, нажав на кнопку со значком «+» в правом верхнем углу экрана в строке **Приложения**. В появившемся окне в поле **Адрес** необходимо указать URL веб-сервера, на котором опубликовано мобильное приложение, и нажать кнопку **Загрузить** в правом верхнем углу экрана (рис. 1.19).

ПРИМЕЧАНИЕ

Если ваш компьютер не подключен к сети и не имеет сетевого имени, то, чтобы к нему обратиться, нужно настроить роутер так, чтобы ваш компьютер имел в беспроводной сети статический IP-адрес.

В приведенном примере на рис. 1.19 в поле **Адрес** указано: `http://<IP-адрес компьютера в беспроводной сети>/<Каталог веб-сервера, на котором опубликовано мобильное приложение>`.

Затем нужно задать имя мобильного приложения (по умолчанию это синоним мобильной конфигурации), которое будет отображаться в списке приложений мобильной платформы разработчика, включить флажок **Перезапуск из конфигуратора** (переместить тумблер направо) и нажать кнопку **ОК** в правом верхнем углу экрана (рис. 1.20).

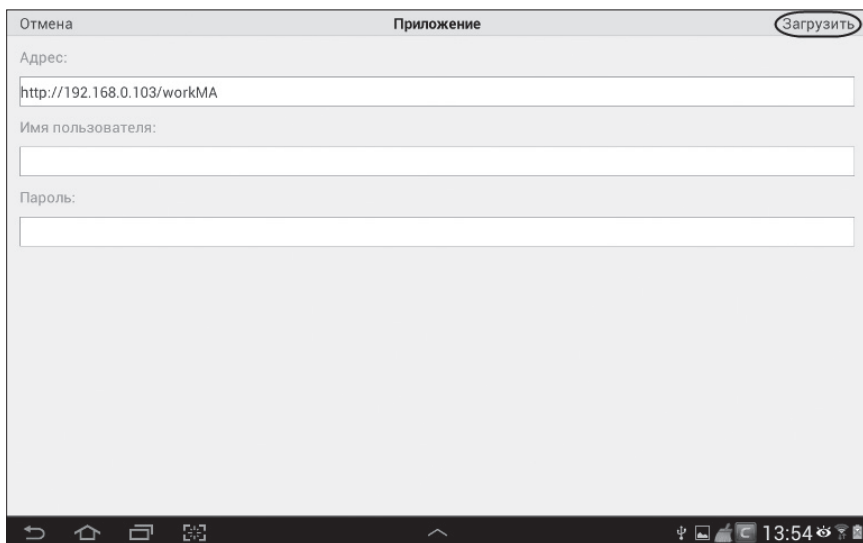


Рис. 1.19. Создание нового мобильного приложения

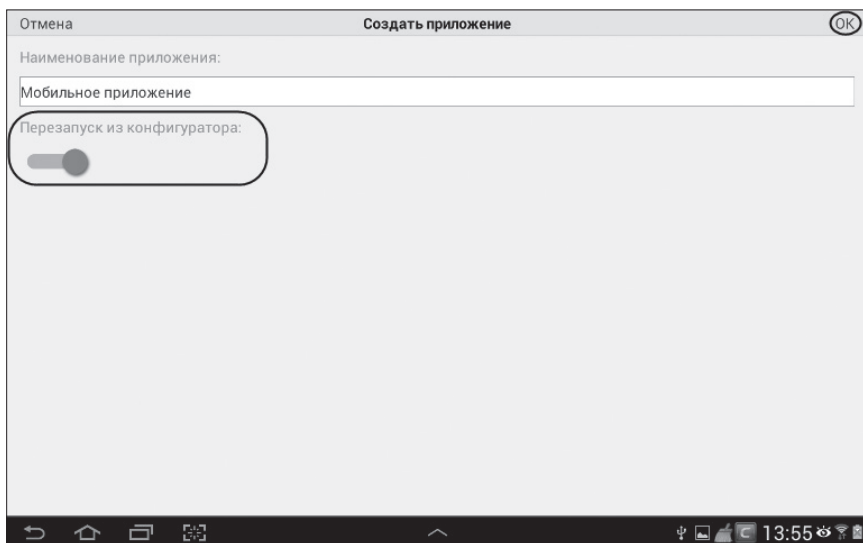


Рис. 1.20. Создание нового мобильного приложения

Мобильное приложение будет скачано с веб-сервера и готово к использованию.

При этом следует обратить внимание на состояние флажка Перезапуск из конфигуратора. Если он установлен (как в нашем случае), то при открытии мобильного приложения платформа разработчика будет выполнять поиск новой версии мобильного приложения на веб-сервере и приложение на планшете будет обновлено. Стандартно этот флажок не установлен.

После этого в списке мобильных приложений платформы разработчика появится созданное нами приложение, можно его запустить и использовать (рис. 1.21).

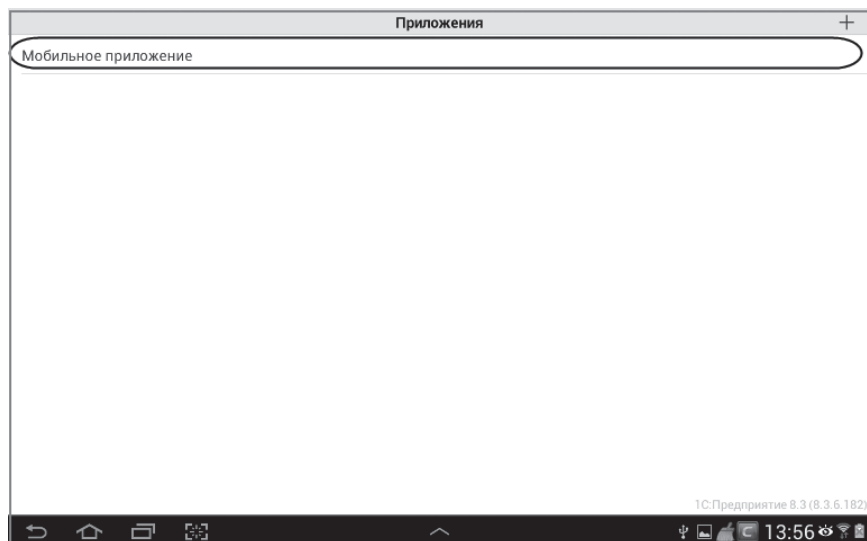


Рис. 1.21. Список мобильных приложений

Запустив наше мобильное приложение, мы увидим пока пустой экран. Слева в заголовке мобильного приложения находится кнопка главного меню приложения , нажав на которую и вызвав пункт О программе, можно посмотреть информацию о программе (рис. 1.22).

Пока в нашем тестовом мобильном приложении больше ничего нет. В следующей главе на его основе мы разработаем реальное мобильное приложение, отражающее всю функциональность нашего примера.

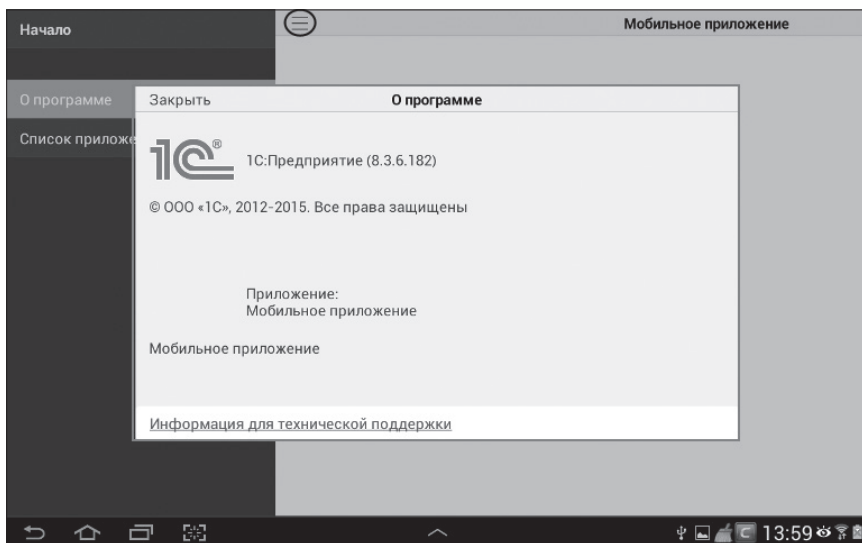


Рис. 1.22. Главное меню мобильного приложения

Изменение свойств мобильного приложения

Чтобы изменить свойства мобильного приложения, следует выполнить долгое нажатие на нужном приложении в списке приложений мобильной платформы разработчика. В открывшемся контекстном меню (справа в строке списка) нужно выбрать пункт Изменить.

В появившемся окне можно изменить наименование приложения, URL веб-сервера, на котором опубликовано мобильное приложение, имя и пароль пользователя для доступа к этому веб-серверу и состояние флажка Перезапуск из конфигулятора (рис. 1.23).

Также в этом диалоге можно запустить приложение (кнопка Открыть), проверить наличие обновлений мобильного приложения на веб-сервере (кнопка Проверить обновления) или удалить приложение (кнопка Удалить).

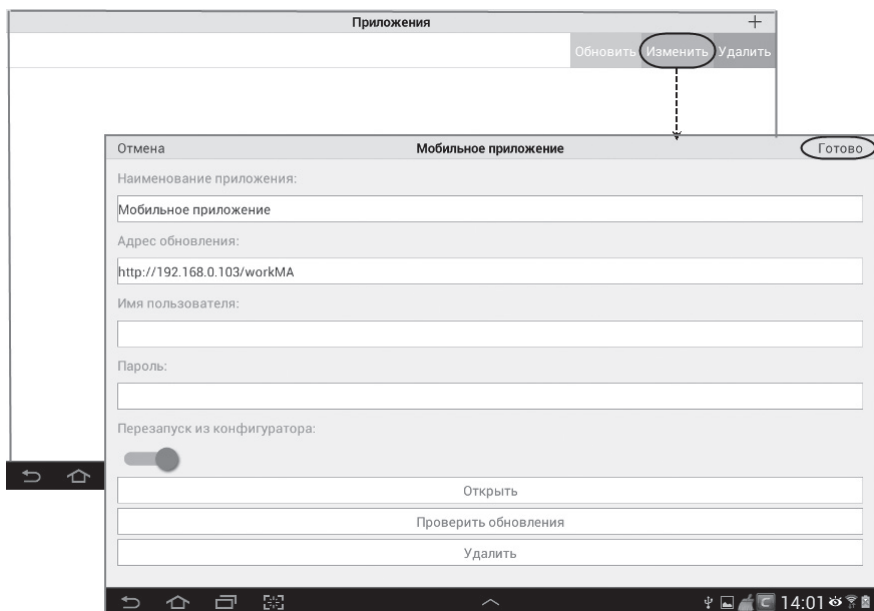


Рис. 1.23. Изменение свойств мобильного приложения

Следует иметь в виду, что после удаления мобильного приложения данные информационной базы также будут безвозвратно утеряны.

Глава 2.

Разработка мобильного приложения

Итак, в книге мы рассмотрим пошагово весь процесс разработки мобильного приложения, которое будет являться удаленным рабочим местом курьера интернет-магазина, работающего на планшете.

В данной главе мы разработаем мобильное приложение так, чтобы оно позволяло курьеру решать все возникающие перед ним задачи, описанные в разделе «Функциональность мобильного приложения» на стр. 11.

Поскольку мобильное приложение не является автономным, для его функционирования потребуется основное приложение, с которым мобильное приложение будет обмениваться данными в процессе своей работы. То есть в итоге мы разработаем две конфигурации: основную конфигурацию и мобильное приложение.

Чтобы не дублировать свои действия, разработаем сначала базовые объекты в основной конфигурации, выгрузим эту конфигурацию в файл и затем создадим из этого файла мобильное приложение. Далее мобильное приложение будет развиваться своим путем, а основное приложение – своим, с учетом специфики каждого приложения.

Чтобы протестировать работу мобильного приложения, мы будем в процессе разработки передавать его на планшет с помощью мобильной платформы разработчика через веб-сервер. Но при обновлении мобильного приложения на планшете данные туда не переносятся. А тестовые данные нам нужны, чтобы проверить работоспособность мобильного приложения на планшете.

Можно ввести необходимые тестовые данные на планшете вручную. Но проще и удобнее создать эти данные в основном приложении на компьютере, а затем с помощью механизмов обмена передать их оттуда на планшет.

Такой обмен нам потребуется выполнить только один раз, в самом начале, чтобы тестовые данные попали на планшет. После отладки всей функциональности мобильного приложения мы доработаем этот обмен так, чтобы он стал запасным способом обмена с основным приложением в случае отсутствия доступа в Интернет.

Все описанные ниже действия по разработке мобильного приложения содержатся в выгрузке информационной базы «Курьер Интернет-магазина.db».

Создание основных объектов конфигурации

Итак, создадим новую базу с пустой конфигурацией в списке информационных баз «1С:Предприятия» и запустим конфигуратор. Это будет прообраз основной конфигурации, из которой мы потом создадим мобильное приложение.

Выполним команду конфигулятора Конфигурация > Открыть конфигурацию, дадим основной конфигурации имя – ИнтернетМагазин и начнем заполнять ее объектами.

Прежде всего, для учета продаж товаров клиентам нам, конечно, понадобятся справочники товаров, складов, клиентов.

Создадим в конфигурации эти справочники со следующими свойствами:

- Справочник Товары (представление объекта – Товар):
 - Иерархия (группы сверху);
 - длина кода 9;
 - длина наименования 30;
 - реквизиты:
 - Артикул (тип Строка, длина 10);
 - Описание (тип Строка, неограниченная длина);
 - способ поиска строки – Любая часть.
- Справочник Клиенты (представление объекта – Клиент):
 - длина кода 9;
 - длина наименования 30;
 - реквизиты:
 - Страна (тип Строка, длина 20);

- Город (тип Строка, длина 20);
- Улица (тип Строка, длина 30);
- Дом (тип Строка, длина 10);
- Квартира (тип Строка, длина 5);
- Телефон (тип Строка, длина 16);
- ЭлектроннаяПочта (тип Строка, длина 30);
- Широта (тип Число, длина 10,6);
- Долгота (тип Число, длина 10,6).

■ Справочник Склады (представление объекта – Склад):

- Быстрый выбор;
- создание при вводе – Не использовать.

Также для учета продаж товаров в разрезе цветов и размеров нам понадобятся соответствующие справочники. Создадим их:

- справочник Цвета (представление объекта – Цвет, остальные свойства по умолчанию);
- справочник Размеры (представление объекта – Размер, остальные свойства по умолчанию).

В случае отказа от покупки товаров нам понадобится справочник для учета причин отказа. Создадим его.

■ Справочник ПричиныОтказа (представление объекта – Причина отказа от товара, представление списка – Причины отказа от товаров):

- длина кода 9;
- длина наименования 40.

Для идентификации курьера на планшете нам также понадобится список пользователей. Создадим его.

■ Справочник Пользователи (представление объекта – Пользователь):

- длина кода 20;
- длина наименования 40;
- Быстрый выбор.

Теперь создадим в конфигурации перечисление СтатусыЗаказов для учета состояния заказов клиентов. Это перечисление будет иметь значения:

- Открыт;
- ВРаботе;
- Выполнен;
- Закрыт.

Для определения цен заказанных товаров создадим регистр сведений для хранения цены товара на определенную дату.

- Регистр сведений **ЦеныТоваров** (представление записи **Цена товара на дату**):
 - периодичность – В пределах дня;
 - Независимый;
 - Основной отбор по периоду;
 - измерения:
 - Товар (тип СправочникСсылка.Товары, Ведущее, Основной отбор);
 - ресурсы:
 - Цена (тип Число, длина 10, 0, Неотрицательное).

И, наконец, создадим саму структуру документа, содержащего всю информацию о заказе клиента.

- Документ **Заказ** (представление списка – **Заказы**):
 - реквизиты:
 - Клиент (тип СправочникСсылка.Клиенты, Проверка заполнения – Выдавать ошибку);
 - Курьер (тип СправочникСсылка.Пользователи);
 - Склад (тип СправочникСсылка.Склады);
 - ДатаДоставки (тип Дата, Проверка заполнения – Выдавать ошибку);
 - АдресДоставки (тип Строка, длина 100, Проверка заполнения – Выдавать ошибку);
 - ДополнительнаяИнформация (тип Строка, неограниченная длина);
 - СтатусЗаказа (тип ПеречислениеСсылка.СтатусыЗаказов, Проверка заполнения – Выдавать ошибку, Значение заполнения – ПеречислениеСсылка.СтатусыЗаказов.Открыт);
 - табличная часть **Товары**:
 - Товар (тип СправочникСсылка.Товары, Проверка заполнения – Выдавать ошибку);
 - Цвет (тип СправочникСсылка.Цвета, Проверка заполнения – Выдавать ошибку);
 - Размер (тип СправочникСсылка.Размеры, Проверка заполнения – Выдавать ошибку);
 - Цена (тип Число, длина 10, 2, Неотрицательное, Проверка заполнения – Выдавать ошибку);

- Количество (тип Число, длина 5, 0);
- Сумма (тип Число, длина 10, 2);
- Отказ (тип Булево);
- ПричинаОтказа (тип СправочникСсылка.ПричиныОтказа).

А также нам понадобится документ, содержащий список заказов для обслуживания в интернет-магазине, который выдается конкретному курьеру на конкретную дату.

■ Документ ОбслуживаниеЗаказов:

- реквизиты:
 - ДатаОбслуживания (тип Дата);
 - Курьер (тип СправочникСсылка.Пользователи);
- табличная часть Заказы:
 - Заказ (тип ДокументСсылка.Заказы);
 - Склад (тип СправочникСсылка.Склады);
 - Выполнен (тип Булево).

Конечно, трудно предусмотреть все заранее, но, кажется, все основные объекты конфигурации мы создали. Изменения будем вносить по ходу разработки – отдельно в мобильное приложение и отдельно в основную конфигурацию.

Первоначальный обмен данными

План обмена

Теперь разработаем механизм для первоначального обмена данными между основным и мобильным приложениями. Для этого сначала необходимо создать план обмена, включающий все созданные нами объекты конфигурации. Добавим его в нашу конфигурацию.

■ План обмена Мобильные (синоним Мобильные устройства):

- длина кода 9;
- длина наименования 40;
- реквизиты:
 - Курьер (тип СправочникСсылка.Пользователи).

Обратите внимание, что признак Распределенная информационная база у нашего плана обмена отключен (см. рис. 2.1). Дело в том, что обмен между основным и мобильным приложениями должен быть универсальным, так

как в этом обмене будут участвовать данные с разной структурой. В нашем примере на первом этапе структура данных в обоих приложениях будет одинаковой, но это всего лишь частный случай. Позднее мы доработаем обмен данными так, чтобы мобильное и основное приложение могли бы обмениваться данными разной структуры (см. раздел «Обмен данными с разной структурой» на стр. 255).

На закладке Основные нажмем кнопку Состав и включим в состав плана обмена все объекты конфигурации, которые мы создали (рис. 2.1).

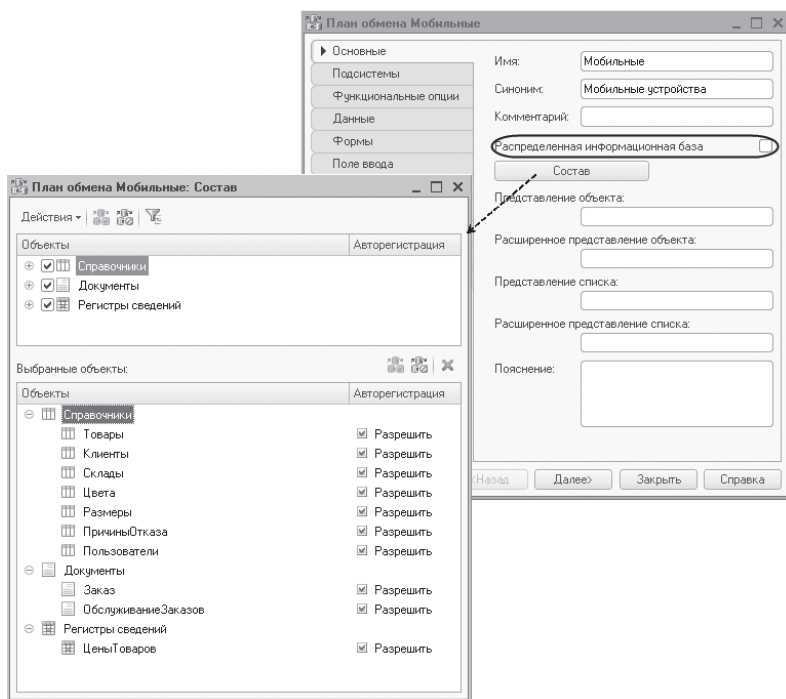


Рис. 2.1. Состав плана обмена «Мобильные»

Теперь в модуле плана обмена поместим процедуры для записи и чтения сообщений обмена, посылаемых конкретному узлу плана обмена (листинги 2.1, 2.2).

Листинг 2.1. Процедура для записи данных обмена

Процедура ЗаписатьСообщениеСИзмениями(Каталог) Экспорт

```
Сообщение = Новый СообщениеПользователю;  
Сообщение.Текст = "----- Выгрузка в узел " + Строка(ЭтотОбъект) + "-----";  
Сообщение.Сообщить();  
  
// Сформировать имя временного файла.  
ИмяФайла = Каталог + ?(Прав(Каталог, 1) = "\", "" , "\") + "Message" +  
    СокрЛП(ПланыОбмена.Мобильные.ЭтотУзел().Код) + "_" + СокрЛП(Ссылка.Код) + ".xml";  
  
// Создать объект записи XML  
// *** ЗаписьXML-документов.  
ЗаписьXML = Новый ЗаписьXML;  
ЗаписьXML.ОткрытьФайл(ИмяФайла);  
ЗаписьXML.ЗаписатьОбъявлениеXML();  
  
// *** Инфраструктура сообщений.  
ЗаписьСообщения = ПланыОбмена.СоздатьЗаписьСообщения();  
ЗаписьСообщения.НачатьЗапись(ЗаписьXML, Ссылка);  
Сообщение = Новый СообщениеПользователю;  
Сообщение.Текст = " Номер сообщения: " + ЗаписьСообщения.НомерСообщения;  
Сообщение.Сообщить();  
  
// Получить выборку измененных данных  
// *** Механизм регистрации изменений.  
ВыборкаИзменений = ПланыОбмена.ВыбратьИзменения(  
    ЗаписьСообщения.Получатель, ЗаписьСообщения.НомерСообщения);  
Пока ВыборкаИзменений.Следующий() Цикл  
    Данные = ВыборкаИзменений.Получить();  
  
    // Записать данные в сообщение *** XML-сериализация.  
    ЗаписатьXML(ЗаписьXML, Данные);  
КонецЦикла;  
  
ЗаписьСообщения.ЗакончитьЗапись();  
ЗаписьXML.Закрыть();  
  
Сообщение = Новый СообщениеПользователю;  
Сообщение.Текст = "----- Конец выгрузки -----";  
Сообщение.Сообщить();
```

КонецПроцедуры

Листинг 2.2. Процедура для чтения данных обмена

Процедура ПрочитатьСообщениеСИзмениями(Каталог) Экспорт

```
// Сформировать имя файла.  
ИмяФайла = Каталог + ?(Прав(Каталог, 1) = "\", "" , "\") + "Message" + СокрЛП(Ссылка.Код) + "_" +  
    СокрЛП(ПланыОбмена.Мобильные.ЭтотУзел().Код) + ".xml";  
  
Файл = Новый Файл(ИмяФайла);
```

```

Если Не Файл.Существует() Тогда
    Возврат;
КонецЕсли;

// *** Чтение документов XML
// Попытаться открыть файл.
ЧтениеXML = Новый ЧтениеXML;
Попытка
    ЧтениеXML.ОткрытьФайл(ИмяФайла);

Исключение
    Сообщение = Новый СообщениеПользователю;
    Сообщение.Текст = "Невозможно открыть файл обмена данными.";
    Сообщение.Сообщить();

    Возврат;

КонецПопытки;

Сообщение = Новый СообщениеПользователю;
Сообщение.Текст = "----- Загрузка из " + Строка(ЭтотОбъект) + " -----";
Сообщение.Сообщить();
Сообщение = Новый СообщениеПользователю;
Сообщение.Текст = " – Считывается файл " + ИмяФайла;
Сообщение.Сообщить();

// Загрузить из найденного файла
// *** Инфраструктура сообщений.
ЧтениеСообщения = ПланыОбмена.СоздатьЧтениеСообщения();

// Читать заголовок сообщения обмена данными – файла XML.
ЧтениеСообщения.НачатьЧтение(ЧтениеXML);

// Сообщение предназначено не для этого узла.
Если ЧтениеСообщения.Отправитель <> Ссылка Тогда
    ВызватьИсключение "Неверный узел";
КонецЕсли;

// Удаляем регистрацию изменений для узла отправителя сообщения.
// *** Служба регистрации изменений.
ПланыОбмена.УдалитьРегистрациюИзменений(
    ЧтениеСообщения.Отправитель, ЧтениеСообщения.НомерПринятого);

// Читаем данные из сообщения *** XML-сериализация.
Пока ВозможностьЧтенияXML(ЧтениеXML) Цикл

    // Читаем очередное значение.
    Данные = ПрочитатьXML(ЧтениеXML);

    // Записать полученные данные.
    Данные.ОбменДанными.Отправитель = ЧтениеСообщения.Отправитель;
    Данные.ОбменДанными.Загрузка = Истина;
    Данные.Записать();

КонецЦикла;

```

```
ЧтениеСообщения.ЗакончитьЧтение();  
ЧтениеXML.Закрыть();  
УдалитьФайлы(ИмяФайла);  
  
Сообщение = Новый СообщениеПользователю;  
Сообщение.Текст = "----- Конец загрузки -----"  
Сообщение.Сообщить();
```

КонецПроцедуры

Это стандартные процедуры выполнения обмена с помощью базовых средств чтения и записи XML-документов, которые подробно рассматривались в других книгах, например в книге «1С:Предприятие 8.3. Практическое пособие разработчика». Сейчас мы их подробно разбирать не будем, а вернемся к ним в разделе «Запасной способ обмена данными с основным приложением» (стр. 251).

Отметим только, что в эти процедуры передается каталог для чтения и записи файлов с данными обмена.

Обработка для обмена данными

В заключение создадим в нашей конфигурации обработку для выполнения обмена данными с именем *ЗапаснойОбменДанными*. На закладке *Формы* создадим форму этой обработки.

В окне редактора форм на закладке *Реквизиты* добавим реквизит формы *УзелОбмена* типа *ПланОбменаСсылка.Мобильные* и перетащим его в окно элементов формы.

Затем на закладке *Команды* создадим команды *ЗарегистрироватьИзменения*, *ВыгрузитьДанные* и *ЗагрузитьДанные* и поочередно перетащим эти команды в окно элементов формы. В палитре свойств соответствующих кнопок формы снимем флажок *Доступность*.

В результате форма обработки примет следующий вид (рис. 2.2).

Таким образом, при открытии обработки эти кнопки будут недоступны, пока не выбран узел плана обмена в поле *Узел обмена*. Эти кнопки также будут недоступны в случае выбора предопределенного узла нашей информационной базы – то есть команды обмена выполнить будет невозможно, если выбранный узел является предопределенным.

Чтобы обеспечить такое поведение кнопок, создадим в модуле формы обработки функцию, выполняющуюся на сервере и возвращающую истину, если переданный в функцию узел является предопределенным (листинг 2.3).

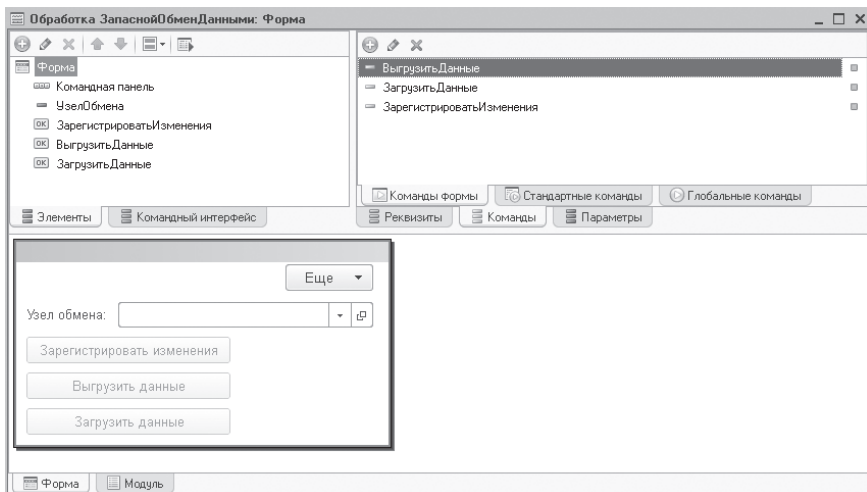


Рис. 2.2. Форма обработки «Обмен данными»

Листинг 2.3. Функция «ПредопределенныйУзел()»

```
&НаСервереБезКонтекста
Функция ПредопределенныйУзел(Узел)
    Возврат Узел = ПланыОбмена.Мобильные.ЭтотУзел();
КонецФункции
```

Затем вызовем контекстное меню элемента формы `УзелОбмена` и создадим клиентский обработчик события `ПриИзменении`. Заполним этот обработчик следующим образом (листинг 2.4).

Листинг 2.4. Процедура «УзелОбменаПриИзменении()»

```
&НаКлиенте
Процедура УзелОбменаПриИзменении(Элемент)

    Если ПредопределенныйУзел(УзелОбмена) Тогда
        Элементы.ЗарегистрироватьИзменения.Доступность = Ложь;
        Элементы.ВыгрузитьДанные.Доступность = Ложь;
        Элементы.ЗагрузитьДанные.Доступность = Ложь;
    Иначе
        Элементы.ЗарегистрироватьИзменения.Доступность = Истина;
        Элементы.ВыгрузитьДанные.Доступность = Истина;
        Элементы.ЗагрузитьДанные.Доступность = Истина;
    КонецЕсли;

КонецПроцедуры
```

В этой процедуре доступность кнопок для выполнения команд обмена определяется в зависимости от значения функции `ПредопределенныйУзел()`, в которую передается ссылка на выбранный узел.

Затем создадим обработчики каждой из команд обмена. Для этого в палитре свойств каждой команды в строке Действие нажмем кнопку открытия.

На запрос конфигуратора о типе обработчика команды выберем вариант Создать на клиенте и процедуру на сервере без контекста. В модуле формы для каждой команды будут созданы шаблоны двух процедур: клиентской процедуры, выполняющейся при нажатии на соответствующую кнопку, и серверной внеконтекстной процедуры, которая вызывается из клиентской процедуры.

Процедуру `ЗарегистрироватьИзменения()` изменим следующим образом (листинг 2.5).

Листинг 2.5. Обработчик команды «ЗарегистрироватьИзменения»

```
&НаКлиенте  
Процедура ЗарегистрироватьИзменения(Команда)  
    ЗарегистрироватьИзмененияНаСервере(УзелОбмена);  
КонечПроцедуры
```

В процедуру `ЗарегистрироватьИзмененияНаСервере()` поместим следующий текст (листинг 2.6).

Листинг 2.6. Процедура «ЗарегистрироватьИзмененияНаСервере»

```
&НаСервереБезКонтекста  
Процедура ЗарегистрироватьИзмененияНаСервере(Узел)  
    ПланыОбмена.ЗарегистрироватьИзменения(Узел);  
КонечПроцедуры
```

В этой процедуре мы обращаемся к механизму регистрации изменений, вызывая метод менеджера планов обмена – `ЗарегистрироватьИзменения()`. В этот метод передается ссылка на текущий узел плана обмена, которая содержится в реквизите `УзелОбмена`.

В результате выполнения этой процедуры в информационной базе будут созданы записи регистрации изменений, предназначенные для пересылки в выбранный нами узел, для всех объектов обмена, указанных в составе данного плана обмена.

Процедуру `ВыгрузитьДанные()` изменим следующим образом (листинг 2.7).

Листинг 2.7. Обработчик команды «ВыгрузитьДанные»

```
&НаКлиенте  
Процедура ВыгрузитьДанные(Команда)
```

```

Диалог = Новый ДиалогВыбораФайла(РежимДиалогаВыбораФайла.ВыборКаталога);
Диалог.Заголовок = "Укажите каталог информационной базы.";
Если Диалог.Выбрать() Тогда
    ВыгрузитьДанныеНаСервере(УзелОбмена, Диалог.Каталог);
КонецЕсли;

```

КонецПроцедуры

В этой процедуре мы вызываем диалог выбора каталога, в который будет записан файл с сообщением обмена, и затем вызываем процедуру `ВыгрузитьДанныеНаСервере()`, в которую передаем выбранный каталог во втором параметре. В первом параметре в процедуру передается ссылка на узел (реквизит формы `УзелОбмена`), для которого будет производиться запись изменений (листинг 2.8).

Листинг 2.8. Обработчик команды «ВыгрузитьДанныеНаСервере»

```

&НаСервереБезКонтекста
Процедура ВыгрузитьДанныеНаСервере(Узел, Каталог)

    УзелОбъект = Узел.ПолучитьОбъект();
    УзелОбъект.ЗаписатьСообщениеСИзменениями(Каталог);

```

КонецПроцедуры

В процедуре `ВыгрузитьДанныеНаСервере()` мы получаем объект от ссылки на узел плана обмена и выполняем метод `ЗаписатьСообщенияСИзменениями()`, определенный в модуле плана обмена `Мобильные`.

Процедуру `ЗагрузитьДанные()` изменим следующим образом (листинг 2.9).

Листинг 2.9. Обработчик команды «ЗагрузитьДанные»

```

&НаКлиенте
Процедура ЗагрузитьДанные(Команда)

    Диалог = Новый ДиалогВыбораФайла(РежимДиалогаВыбораФайла.ВыборКаталога);
    Диалог.Заголовок = "Укажите каталог информационной базы.";
    Если Диалог.Выбрать() Тогда
        ЗагрузитьДанныеНаСервере(УзелОбмена, Диалог.Каталог);
    КонецЕсли;

```

КонецПроцедуры

В этой процедуре мы вызываем диалог выбора каталога, из которого будет прочитан файл с сообщением обмена, и затем вызываем процедуру `ЗагрузитьДанныеНаСервере()`, в которую передаем выбранный каталог во втором параметре. В первом параметре метода передается ссылка на узел (реквизит формы `УзелОбмена`), от которого будет производиться чтение изменений (листинг 2.10).

Листинг 2.10. Обработчик команды «ЗагрузитьДанныеНаСервере»

```
&НаСервереБезКонтекста
Процедура ЗагрузитьДанныеНаСервере(Узел, Каталог)

    УзелОбъект = Узел.ПолучитьОбъект();
    УзелОбъект.ПрочитатьСообщениеСИзменениями(Каталог);

КонецПроцедуры
```

В процедуре ЗагрузитьДанныеНаСервере() мы получаем объект от ссылки на узел плана обмена и выполняем метод ПрочитатьСообщенияСИзменениями(), определенный в модуле плана обмена Мобильные.

В заключение отредактируем командный интерфейс приложения – расставим созданные нами объекты конфигурации по важности, а также включим видимость некоторых команд по созданию объектов. Выделим корень конфигурации ИнтернетМагазин, вызовем из контекстного меню команду Открыть командный интерфейс основного раздела и зададим следующий вид командного интерфейса (рис. 2.3).

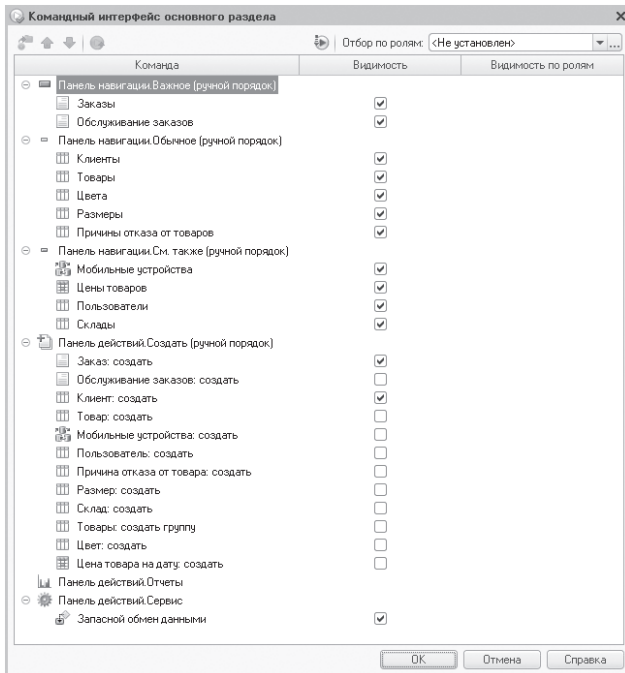


Рис. 2.3. Командный интерфейс основного раздела

Создание конфигурации для мобильного приложения

После того как созданы базовые объекты в основной конфигурации и разработан механизм обмена данными с мобильным приложением, мы можем получить основу для конфигурации мобильного приложения путем копирования в нее всего, что мы создали в основной конфигурации.

Обновим конфигурацию базы данных (F7). Затем сохраним нашу конфигурацию, выполнив команду главного меню Конфигурация > Сохранить конфигурацию в файл... Выберем каталог и зададим имя файла конфигурации.

Затем запустим «1С:Предприятие» и откроем в конфигураторе созданную нами ранее (см. раздел «Публикация мобильного приложения на веб-сервере» на стр. 26) конфигурацию с именем МобильноеПриложение. Загрузим в нее конфигурацию из сохраненного нами файла (Конфигурация > Загрузить конфигурацию из файла...).

На вопрос системы об обновлении конфигурации ответим утвердительно (два раза) и в окне изменений структуры конфигурации нажмем кнопку Принять. Мы видим, что в дереве объектов конфигурации появились все объекты, созданные нами в основной конфигурации.

Пока пропустим список ошибок, которые возникли при загрузке конфигурации из основного приложения. Немного позже мы их исправим.

Теперь доработаем полученную конфигурацию так, чтобы она работала на планшете.

Прежде всего, дадим ей имя КурьерИнтернетМагазина и установим свойство конфигурации Назначение использования в значение Мобильное устройство (рис. 2.4).

С этого «символического» момента конфигурация, разрабатываемая нами для мобильного устройства, становится «мобильным приложением», и, чтобы не путать ее с другими конфигурациями, мы будем называть ее именно так.

Платформа предупредит, что свойство Режим совместимости интерфейса будет изменено. И действительно, перейдя на закладку свойств конфигурации Совместимость, можно увидеть, что ряд свойств изменены (по сравнению с основной конфигурацией) и недоступны для редактирования. В частности, в мобильном приложении свойство Режим совместимости интерфейса установлено как Версия 8.2 (рис. 2.5), в то время как основная конфигурация стандартно разрабатывается в интерфейсе «Такси».

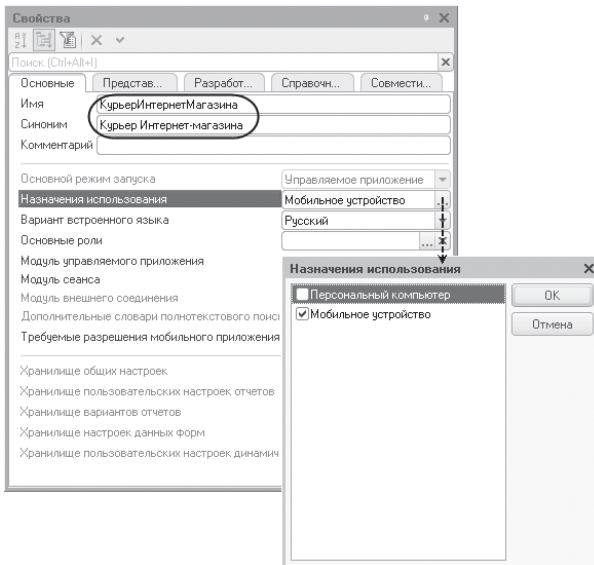


Рис. 2.4. Свойства мобильного приложения

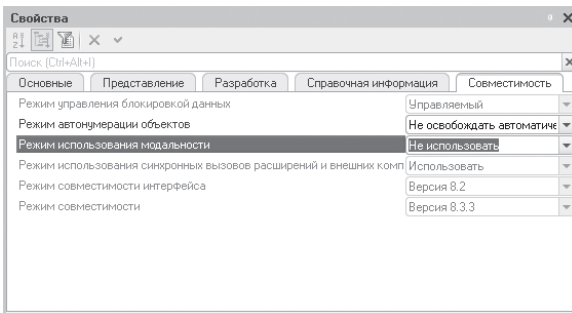


Рис. 2.5. Свойства мобильного приложения

Если посмотреть на дерево объектов конфигурации, то также становится понятно, что в мобильном приложении доступны не все объекты конфигурации (рис. 2.6).

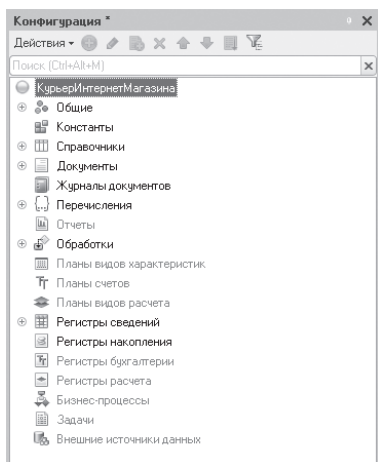


Рис. 2.6. Дерево объектов конфигурации

Таким образом, при разработке мобильного приложения необходимо учитывать ограничения, которые накладывает мобильная платформа по сравнению с платформой «1С:Предприятие» для персонального компьютера:

- ограниченный перечень доступных объектов конфигурации и механизмов;
- ограниченный набор свойств реквизитов;
- ограниченный набор элементов управляемых форм;
- упрощенная реализация некоторых механизмов (например, права доступа или начальная страница);
- отсутствие обычных форм и обычного режима запуска;
- отсутствие средств отладки мобильного приложения и др.

ПОДРОБНЕЕ

Документация 1С:Предприятие 8.3. Руководство разработчика. Глава 26.
Разработка решений для мобильной платформы > Общая информация.

Большинство этих ограничений будут рассмотрены нами далее по ходу разработки мобильного приложения.

Пока, чтобы заполнить данными мобильное приложение, нужно немного доработать обработку и процедуры обмена данными нашего мобильного приложения.

Откроем форму обработки `ЗапаснойОбменДанными`.

В мобильном приложении недоступен выбор каталога для чтения и записи файлов с сообщениями обмена. Поэтому при выгрузке или загрузке данных мы будем использовать для обмена корень планшета ("/storage/sdcard0/") и явно укажем этот каталог в коде обработчиков команд для обмена данными (листинги 2.11, 2.12).

Листинг 2.11. Обработчик команды «ВыгрузитьДанные»

```
&НаКлиенте
Процедура ВыгрузитьДанные(Команда)
    ВыгрузитьДанныеНаСервере(УзелОбмена, "/storage/sdcard0/");
КонечПроцедуры
```

Листинг 2.12. Обработчик команды «ЗагрузитьДанные»

```
&НаКлиенте
Процедура ЗагрузитьДанные(Команда)
    ЗагрузитьДанныеНаСервере(УзелОбмена, "/storage/sdcard0/");
КонечПроцедуры
```

Теперь откроем модуль плана обмена Мобильные и изменим сами процедуры записи и чтения сообщений обмена. Удалим из пути к файлу обмена обратные слэши (листинги 2.13, 2.14).

Листинг 2.13. Процедура для записи данных обмена

```
Процедура ЗаписатьСообщениеСИзменениями(Каталог) Экспорт
...
    // Сформировать имя временного файла.
    ИмяФайла = Каталог + "Message" + СокрЛП(ПланыОбмена.Мобильные.ЭтотУзел().Код) + "_" +
                                                СокрЛП(Ссылка.Код) + ".xml";
...
КонечПроцедуры
```

Листинг 2.14. Процедура для чтения данных обмена

```
Процедура ПрочитатьСообщениеСИзменениями(Каталог) Экспорт
...
    // Сформировать имя файла.
    ИмяФайла = Каталог + "Message" + СокрЛП(Ссылка.Код) + "_" +
                                                СокрЛП(ПланыОбмена.Мобильные.ЭтотУзел().Код) + ".xml";
...
КонечПроцедуры
```

Теперь все готово к тестированию мобильного приложения на планшете. Остался последний шаг – нужно опубликовать мобильное приложение на веб-сервере.

Для этого обновим конфигурацию базы данных (F7) и выполним команду конфигуратора Конфигурация > Мобильное приложение > Опубликовать ...

В появившемся диалоге в поле **Имя** зададим имя виртуального каталога на веб-сервере, в который будет выполнена публикация мобильного приложения (см. рис. 2.7).

В поле **Каталог** укажем физический каталог компьютера, в котором будет находиться файл публикации мобильного приложения (см. рис. 2.7).

Поскольку флажок **Обновлять мобильное приложение при обновлении конфигурации базы данных** установлен, то при обновлении конфигурации базы данных будет автоматически обновляться публикация мобильного приложения на веб-сервере.

Затем нажмем кнопку **Опубликовать** и подтвердим, что нужно обновить мобильное приложение сейчас (рис. 2.7).

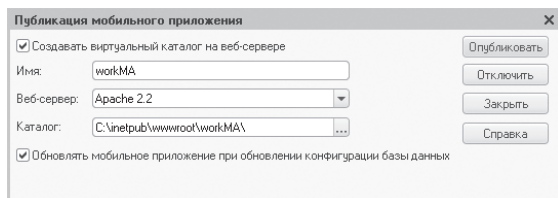


Рис. 2.7. Публикация мобильного приложения на веб-сервере

В первой главе, в разделе «Добавление мобильного приложения на планшет» на стр. 28, мы уже добавили на планшет тестовое мобильное приложение с именем **Мобильное приложение**, которое ссылалось на тот же каталог публикации (**workMA**).

Теперь запустим из списка приложений на планшете мобильную платформу разработчика «1С:Предприятия» (см. рис. 2.8).

В открывшемся списке приложений мобильной платформы выполним долгое нажатие на строке **Мобильное приложение**, в появившемся контекстном меню нажмем кнопку **Изменить** и изменим имя приложения на **Курьер Интернет-магазина**. Адрес обновления мобильного приложения на веб-сервере оставим без изменений. Затем нажмем кнопку **Готово** (рис. 2.8).

ПРИМЕЧАНИЕ

Вызвать контекстное меню в строке списка приложений можно также жестом пролистывания с правой стороны экрана.

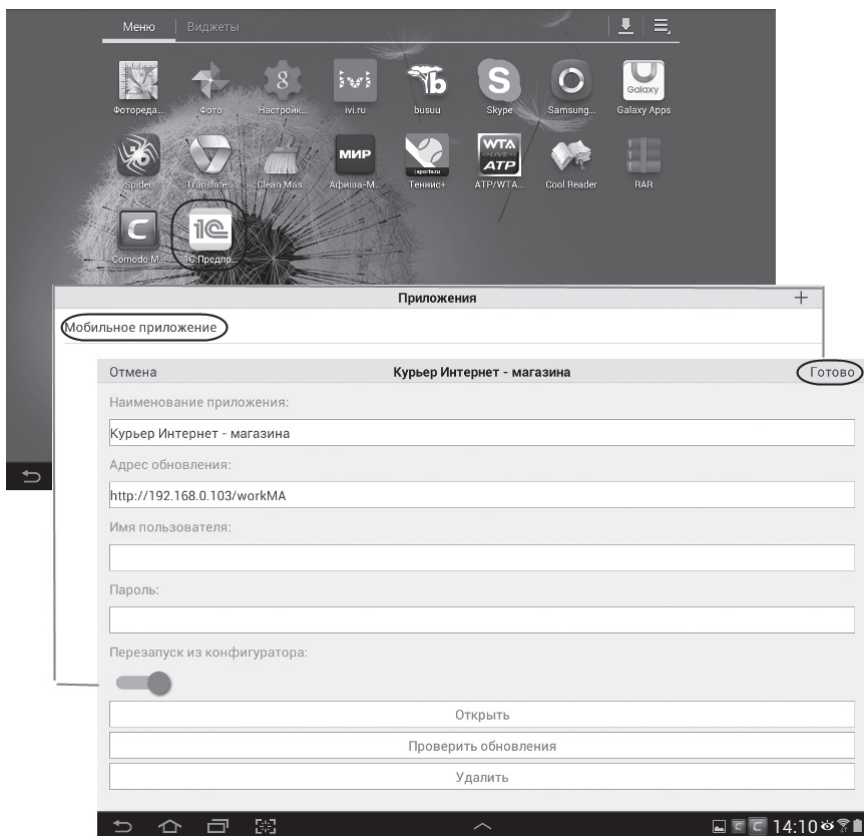


Рис. 2.8. Запуск мобильного приложения «Курьер Интернет-магазина»

Теперь нажмем на имя приложения Курьер Интернет-магазина в списке приложений мобильной платформы разработчика и запустим его. При открытии мобильное приложение будет обновлено, так как флажок Перезапуск из конфигулятора установлен.

На экране планшета мы увидим команды для открытия списков справочников, документов и т.п. в том виде, который мы задали при редактировании командного интерфейса основного раздела основного приложения, кроме группы См. также (см. рис. 2.3).

Все команды отображаются в виде матрицы кнопок со стандартными рисунками мобильной платформы «1С:Предприятия» (см. рис. 2.9). Нажав

на кнопку  в верхнем левом углу экрана, мы можем вызвать главное меню мобильного приложения (рис. 2.9).

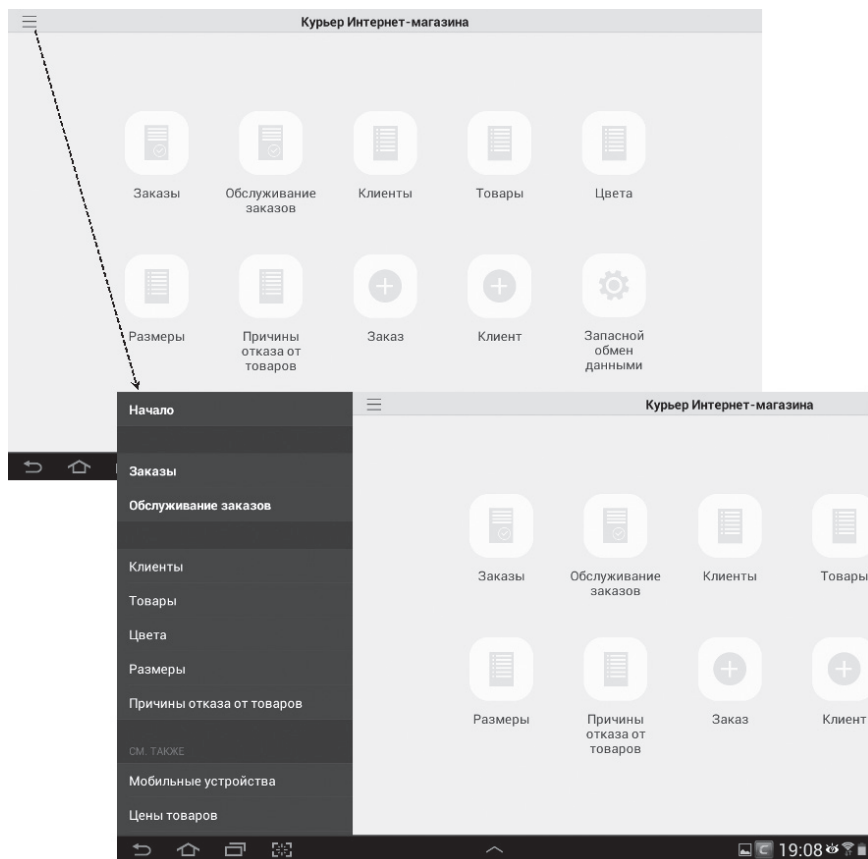


Рис. 2.9. Главное меню мобильного приложения «Курьер Интернет-магазина»

ПРИМЕЧАНИЕ

Вызвать главное меню приложения можно также жестом пролистывания с левой стороны экрана.

Но если открыть любой справочник или документ, то мы увидим, что никаких данных там нет. Поэтому нужно заполнить данными основное приложение, выгрузить эти данные в файл обмена, а на планшете загрузить данные из этого файла.

Перенос данных на планшет

Запустим основное приложение Интернет-магазин в режиме 1С:Предприятие и заполним его данными, для того чтобы можно было протестировать мобильное приложение на планшете.

При этом можно взять уже готовую выгрузку с данными («Первоначальный обмен.db»), которая находится на диске, прилагающемся к книге, или можно создать собственные тестовые данные, следуя дальнейшим указаниям.

Сначала заполним справочники товаров, складов, клиентов, которые нам понадобятся для учета продаж товаров клиентам.

Поскольку справочник Товары – иерархический, добавим две группы товаров, и в каждую из групп внесем по два-три товара.

Затем добавим двух-трех клиентов в справочник Клиенты и заполним их адресные данные и телефон.

В справочник Склады добавим какой-нибудь склад.

Для учета продаж товаров в разрезе цветов и размеров добавим три-четыре записи в справочники Цвета и Размеры.

Для учета причин отказа от заказанных товаров добавим две-три записи в справочник Причины отказа.

Также добавим двух-трех пользователей в справочник Пользователи, который понадобится нам для идентификации курьера на планшете.

Для определения цен заказанных товаров заполним регистр сведений Цены товаров, в котором будут храниться цены товаров на определенную дату. Укажем хотя бы одну цену для каждого товара, а для некоторых товаров внесем несколько цен на разные даты.

После этого создадим три-четыре документа Заказ, в которых заполним всю информацию о заказе клиента, используя перечисленные выше данные.

В шапке заказа в полях Клиент, Курьер и Склад выберем записи справочников Клиенты, Пользователи, Склады, на которые ссылаются соответствующие реквизиты документа. Заполним вручную поля Дата доставки, Адрес доставки, Дополнительная информация.

В табличной части каждого заказа добавим по две-три строки. В каждой строке списка заказанных товаров в полях Товар, Цвет и Размер выберем записи справочников Товары, Цвета, Размеры, на которые ссылаются соответствующие реквизиты табличной части документа. Колонки Цена, Количество и Сумма заполним вручную.

В заключение создадим два документа **Обслуживание заказов**, содержащих список заказов для обслуживания на конкретную дату, который получает курьер в интернет-магазине.

В шапке документа в поле **Курьер** выберем записи справочника **Пользователи**, на которые ссылается соответствующий реквизит документа. Заполним поле **Дата обслуживания**.

В табличной части каждого документа добавим по две-три строки. В каждой строке списка заказов для обслуживания в полях **Заказ** и **Склад** выберем записи из списка заказов и складов, на которые ссылаются соответствующие реквизиты табличной части документа.

Теперь перенесем эти тестовые данные из основного приложения на планшет. Для этого откроем план обмена **Мобильные устройства**. В списке узлов плана обмена уже присутствует одна запись с пиктограммой – **предопределенный узел**. Установим для нее значения полей: **Код** – ЦО, **Наименование** – **Центральный офис**.

Затем добавим еще один узел плана обмена со значениями полей: **Код** – МПП, **Наименование** – **Мобильное приложение – планшет** (рис. 2.10).

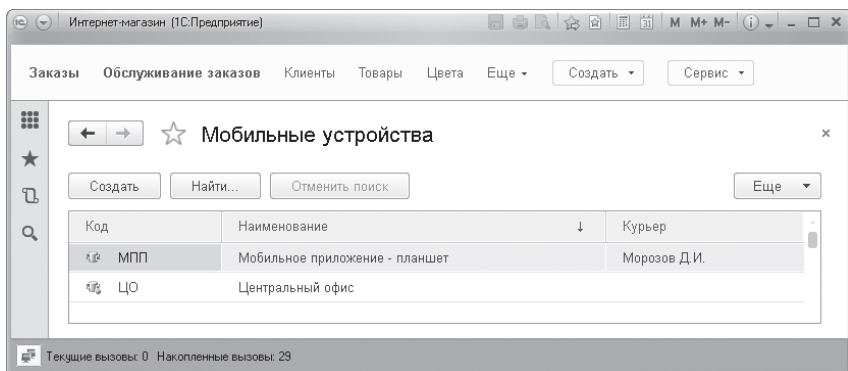


Рис. 2.10. Список узлов плана обмена «Мобильные устройства»

Значение поля **Курьер**, ссылающегося на справочник **Пользователи**, при первоначальном обмене роли не играет – можно его не задавать.

Далее откроем обработку **Запасной обмен данными**, выберем в поле **Узел обмена** узел **Мобильное приложение – планшет** и нажмем кнопку **Зарегистрировать изменения** (рис. 2.11).

Таким образом, мы сформировали для планшета все необходимые записи регистрации изменений для всех объектов конфигурации, входящих

в данный план обмена. Это – своего рода начальная синхронизация узла обмена всеми данными обмена.

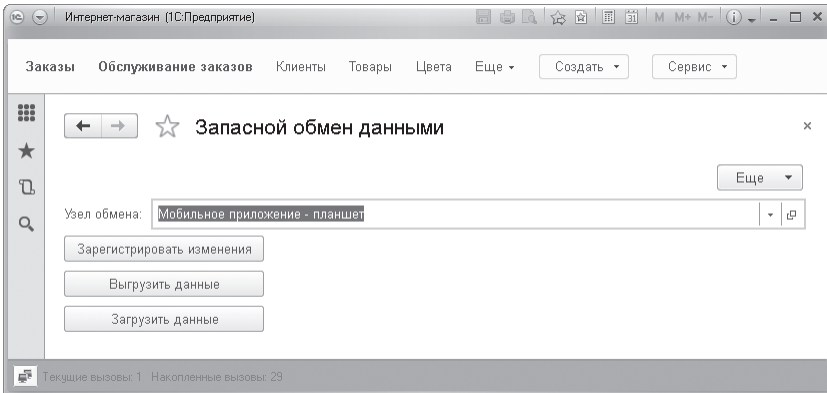


Рис. 2.11. Регистрация всех изменений для узла «Мобильное приложение – планшет»

Теперь нажмем кнопку **Выгрузить данные** и выберем каталог для записи XML-файла с сообщением обмена (рис. 2.12). Заметьте, что в поле **Узел обмена** выбран узел **Мобильное приложение – планшет** – это узел, в который мы хотим передать данные.

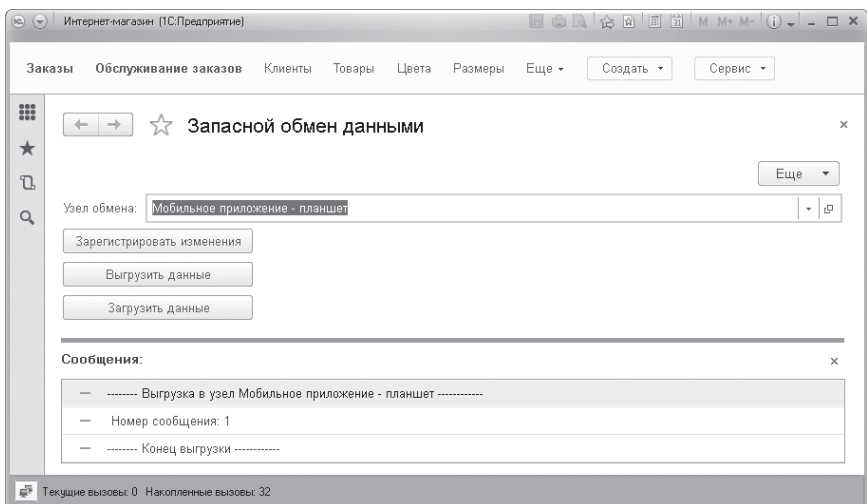


Рис. 2.12. Выгрузка данных для мобильного приложения

После этого надо подсоединить планшет через USB-кабель к компьютеру и вручную скопировать файл с именем MessageЦО_МПП из выбранного каталога (в нашем случае Компьютер > Data(E) > Mobile1C) в корень планшета – Компьютер > GT-P5110 > Tablet (рис. 2.13).

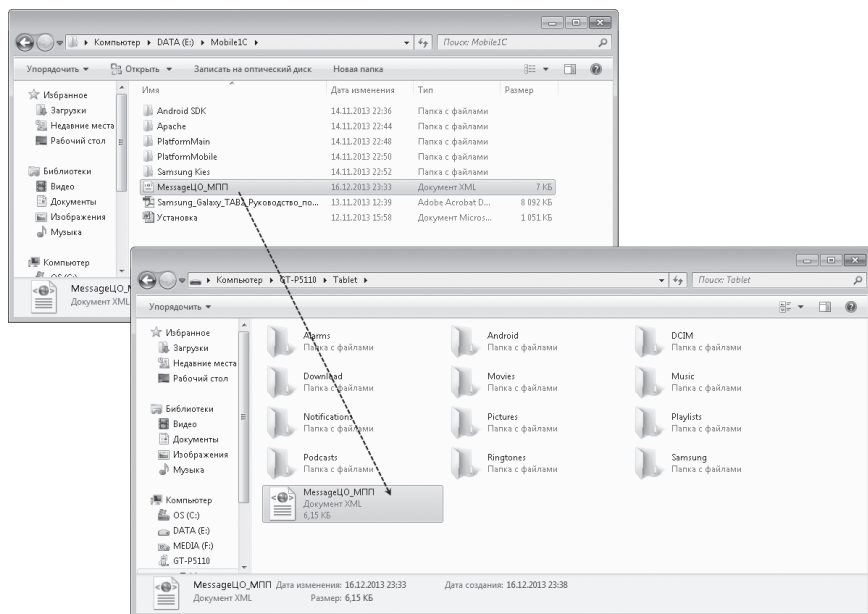


Рис. 2.13. Копирование файла обмена на планшет

Теперь откроем на планшете наше мобильное приложение Курьер Интернет-магазина (список приложений планшета > Мобильная платформа разработчика «1С:Предприятие» > приложение «Курьер Интернет-магазина»).

Затем откроем меню функций приложения (жестом пролистывания с левой стороны экрана), найдем команду для открытия списка узлов плана обмена Мобильные устройства и выполним ее.

В списке узлов плана обмена уже присутствует одна запись с пиктограммой – предопределенный узел. Установим для нее значения полей: Код – МПП, Наименование – Планшет – Samsung. Нажмем кнопку Готово в правом верхнем углу экрана.

Затем в списке узлов плана обмена нажмем кнопку + в правом верхнем углу экрана и добавим еще один узел плана обмена со значениями полей: Код – ЦО, Наименование – Центральный офис (рис. 2.14).

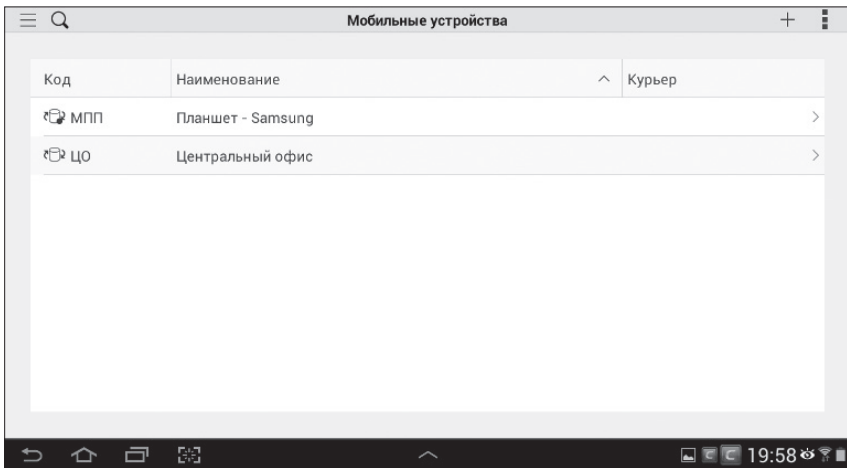


Рис. 2.14. Список узлов плана обмена «Мобильные устройства» на планшете

То есть по отношению к узлам плана обмена основного приложения предопределенный и подчиненный узлы поменялись местами. Но будьте внимательны: коды узлов обмена в обоих приложениях должны совпадать.

Далее на рабочей области нашего мобильного приложения нажмем кнопку для открытия обработки Запасной обмен данными (рис. 2.15).

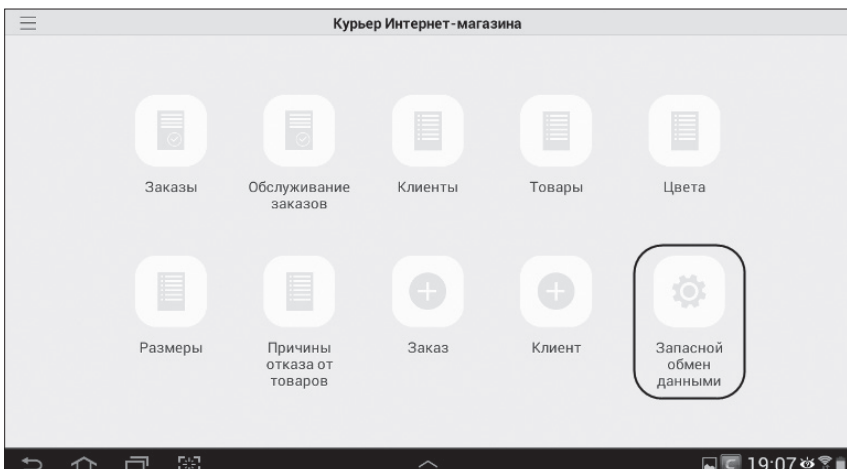


Рис. 2.15. Рабочая область мобильного приложения

Выберем в поле Узел обмена узел Центральный офис (узел, от которого мы принимаем данные) и нажмем кнопку Загрузить данные (рис. 2.16, 2.17).

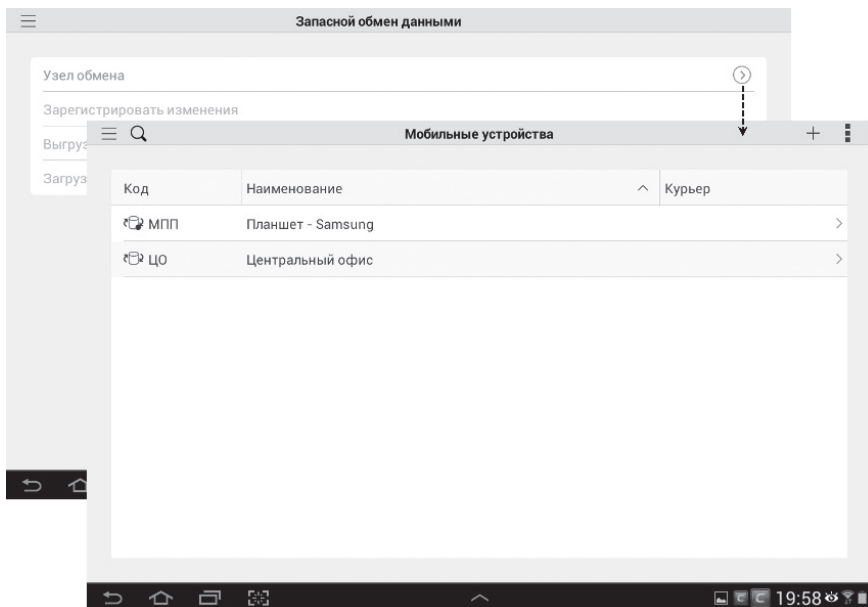


Рис. 2.16. Загрузка данных на планшет

Заметьте, что если узел обмена не выбран или является predetermined узлом информационной базы, то команды для выполнения обмена данными недоступны.

Путь к файлу с сообщением обмена мы прописали явно в процедурах обработки Запасной обмен данными, поэтому загрузка данных произойдет непосредственно из файла `/storage/sdcard0/MessageЦО_МПП.xml`, расположенного в корне планшета.

После этого можно открыть любой документ или справочник нашего мобильного приложения и увидеть, что все введенные нами данные перенесены из основного приложения (рис. 2.18).

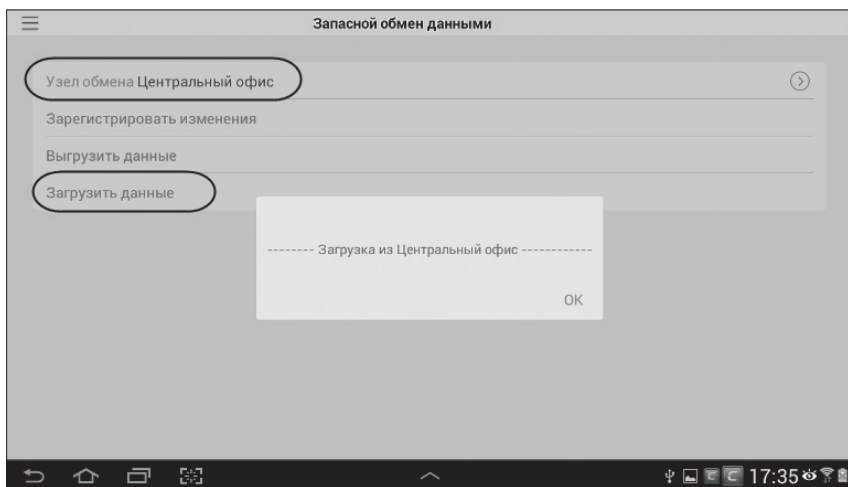


Рис. 2.17. Загрузка данных на планшет

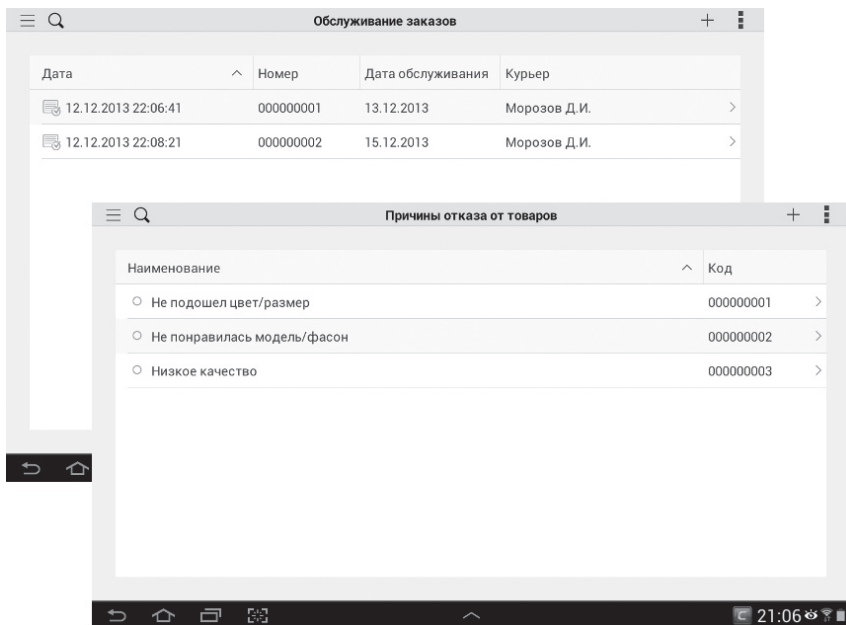


Рис. 2.18. Список документов «Обслуживание заказов»
и элементов справочника «Причины отказа от товаров»

Интерфейс мобильного приложения

В данном разделе мы рассмотрим особенности разработки интерфейса мобильного приложения, работающего под управлением мобильной платформы «1С:Предприятия».

Интерфейс приложения является аналогом интерфейса «Такси», с возможностью переключения между окнами с помощью главного меню приложения.

Интерфейс мобильного приложения устроен так, что в один момент времени на экране отображается одна форма. Под нее освобождается максимум свободного места. Поэтому команды, которые на персональном компьютере расположены в панели команд разделов, собраны в главное меню приложения.

Также это меню содержит команду возврата на начальную страницу, список разделов (команды каждого раздела отображаются в виде подменю), а также команды отображения информации о программе и перехода к списку приложений. Вызов главного меню приложения осуществляется жестом пролистывания с левой стороны экрана или с помощью кнопки , расположенной в левой части заголовка окна.

Также у мобильного приложения есть начальная страница, на которую можно поместить какую-нибудь форму. В нашем случае это может быть форма обработки, показывающая продажи курьера за период, или список заказов, которые нужно обслужить на текущую дату. Позже мы разработаем такую форму и поместим ее на начальную страницу приложения. Кроме формы на начальной странице может располагаться командный интерфейс основного раздела, который мы сейчас и видим в рабочей области окна приложения (см. рис. 2.15).

Навигация по приложению возможна с помощью главного меню. Если открыта форма объекта, то вместо кнопки  в левой части заголовка окна формы появляется кнопка  для возврата в предыдущее окно. При ее нажатии текущее окно будет закрыто и вы вернетесь в то место, откуда была вызвана закрываемая форма. Чтобы попасть на начальную страницу, нужно вызвать главное меню приложения и нажать на строку Начало.

Например, чтобы открыть один из документов Обслуживание заказов, запустим наше мобильное приложение на планшете. На начальной странице или в главном меню приложения найдем соответствующую команду и выполним ее. В открывшемся списке документов Обслуживание заказов нажмем на конкретный документ и откроем его (рис. 2.19).

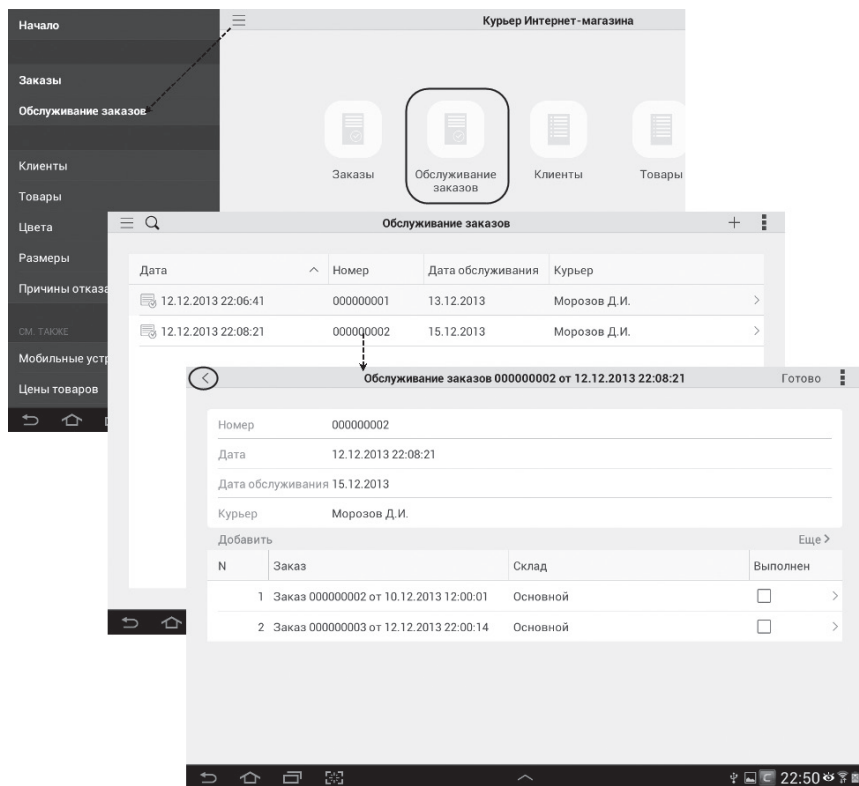


Рис. 2.19. Открытие документа «Обслуживание заказов»

Чтобы сохранить внесенные в заказ изменения и закрыть форму заказа, нужно нажать кнопку Готово (это аналог кнопки Сохранить и закрыть в обычном приложении).

Для возврата из формы документа в список документов без сохранения изменений нужно нажать кнопку < слева в заголовке формы документа или же нажать значок в системной панели планшета. При этом форма документа будет закрыта.

Главное меню приложения можно вызвать из формы документа, или формы элемента справочника, или другой формы объекта при помощи жеста пролистывания с левой стороны экрана.

Надо признать, что тот интерфейс, который мы сейчас видим на экране нашего планшета, – хаотичен, перенасыщен командами и объектами и не адаптирован к работе на планшете. Формы объектов автоматически генерируются платформой с учетом специфики мобильного приложения, но они также далеки от идеала.

Этого следовало ожидать, так как все объекты и командный интерфейс мобильного приложения просто скопированы из основной конфигурации, интерфейсом которой мы попросту еще не занимались.

При разработке интерфейса мобильного приложения прежде всего надо понимать, что на мобильном устройстве (на планшете и в особенности на смартфоне) мы постоянно сталкиваемся с дефицитом рабочего места. Поэтому интерфейс мобильного приложения должен быть максимально лаконичен, удобен и интуитивно понятен.

Формы должны содержать только самое нужное. Размеры элементов форм должны автоматически подстраиваться к размеру мобильного устройства. По расположению элементов формы должны быть вытянуты в длину и ограничены по ширине, так как вертикальная прокрутка форм вполне ожидаема и привычна, в отличие от горизонтальной.

В свете этих рекомендаций попробуем улучшить интерфейс нашего мобильного приложения.

Доработка командного интерфейса

Прежде всего ограничим состав командного интерфейса нашего мобильного приложения, чтобы пользователь мог лучше и быстрее ориентироваться в главном меню.

Первое, что приходит в голову: нам не нужны команды для открытия списков всех объектов, которые присутствуют в мобильном приложении.

Для решения ограниченного круга своих задач курьеру может понадобиться работать со следующими объектами мобильного приложения:

- справочники: Клиенты, Товары (только просмотр), Причины отказа от товаров;
- документы: Заказы, Обслуживание заказов (только просмотр);
- план обмена Мобильные устройства;
- обработка Запасной обмен данными.

То есть курьер может изменять и создавать новые заказы, редактировать информацию о клиентах, создавать новых клиентов, изменять и создавать новые причины отказа от товаров.

Также курьер должен иметь возможность просматривать информацию о товарах и список заказов для обслуживания без возможности их изменения.

Чтобы выполнять обмен данными с интернет-магазином, курьеру понадобится список узлов плана обмена и обработка для обмена данными.

Остальные справочники (Склады, Пользователи, Цвета, Размеры) нужны только для выбора из них значений в соответствующих ссылочных полях.

Запустим наше мобильное приложение в конфигураторе и в окне редактирования вышеперечисленных объектов на закладке Команды снимем флажок Использовать стандартные команды. Тогда никакие команды, связанные с этими объектами, не попадут в соответствующие фрагменты командного интерфейса (рис. 2.20).

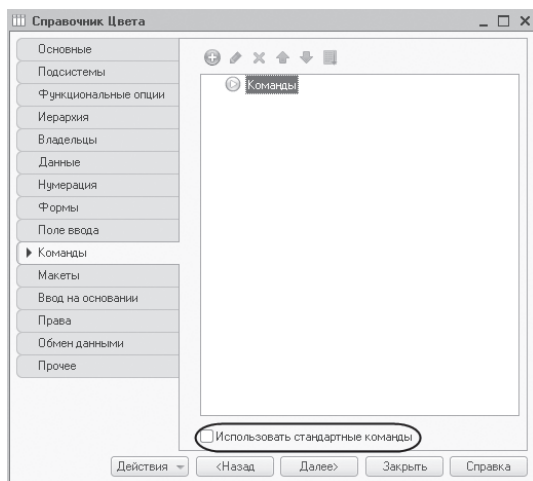


Рис. 2.20. Отключение свойства «Использовать стандартные команды»

Проверим сделанные изменения. Для этого обновим конфигурацию базы данных (F7), при этом автоматически обновится версия мобильного приложения на веб-сервере.

Запустим наше мобильное приложение на планшете. В главном меню приложения мы увидим теперь ограниченный набор команд интерфейса, как мы и хотели (рис. 2.21).

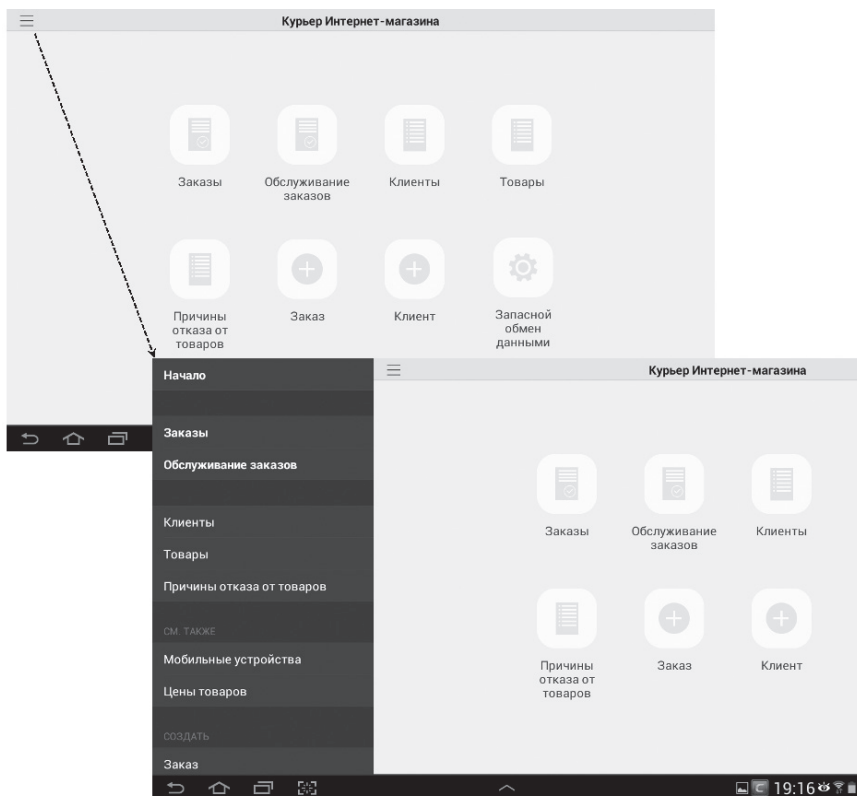


Рис. 2.21. Главное меню мобильного приложения

Помимо команд открытия различных списков мы видим здесь также команды для создания новых заказов и клиентов – в соответствии с постановкой нашей задачи. Эти команды унаследованы из основной конфигурации, и они понадобятся курьеру в его работе.

Ограничения прав доступа

Для того чтобы ограничить возможность изменения данных в соответствии с вышеописанными задачами, мы будем использовать свойство мобильного приложения Основные роли. В нем нужно указать список ролей, которые будут использоваться для ограничения доступа к данным, когда пользователь в мобильном приложении отсутствует (как в нашем случае).

Но сначала нужно создать эту роль и установить для нее нужные права. Для этого перейдем опять в конфигуратор.

В дереве конфигурации в ветви Общие > Роли добавим роль с именем Основная. Для корня приложения установим все права и включим флажок Устанавливать права для новых объектов (рис. 2.22).

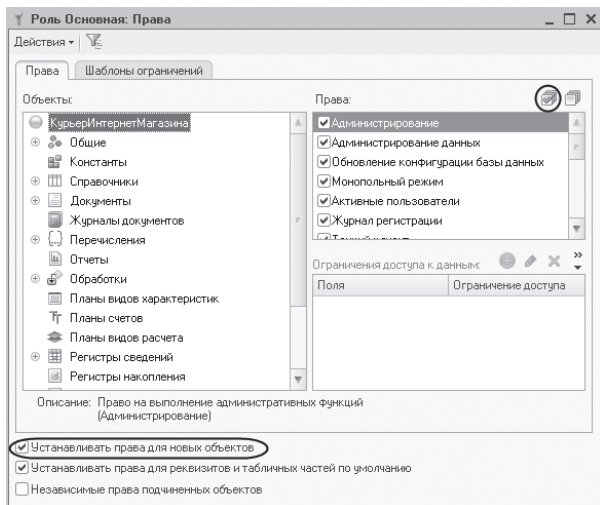


Рис. 2.22. Установка прав для роли «Основная»

Затем выделим ветвь объектов Справочники и отметим право Ввод по строке. При этом права Чтение и Просмотр будут установлены автоматически (рис. 2.23).

Эти права будут даны сразу всем справочникам. Для справочников Клиенты и Причины отказа, которые курьер может редактировать, установим все права, кроме права Интерактивное удаление (рис. 2.24).

Так же поступим и с документами. Для документа Заказ установим все права, кроме права Интерактивное удаление, для документа Обслуживание заказов – только права Ввод по строке, Чтение и Просмотр.

Для плана обмена Мобильные также установим все права, кроме права Интерактивное удаление.

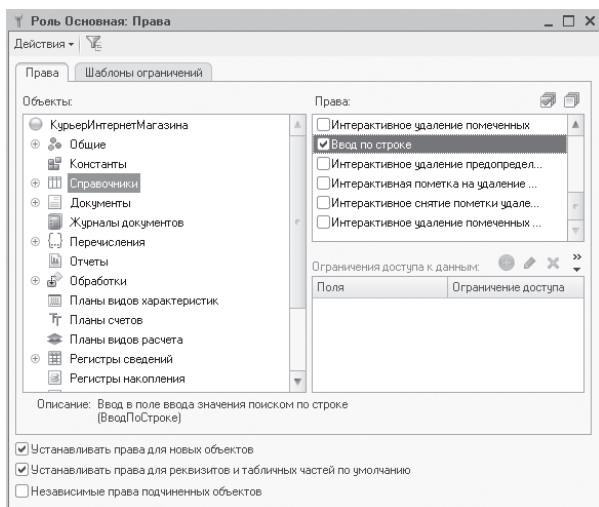


Рис. 2.23. Установка прав для роли «Основная»

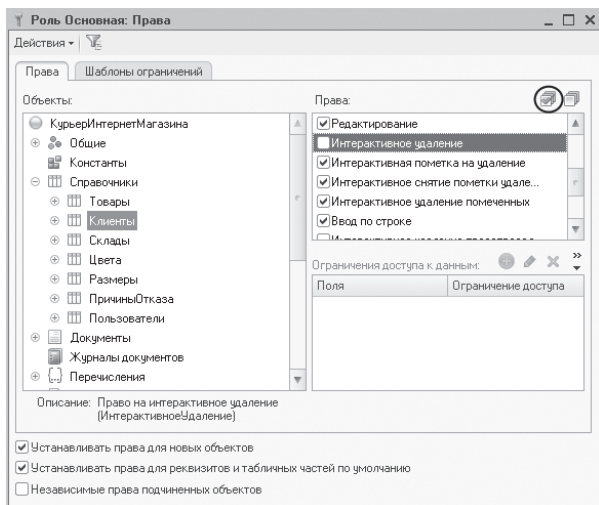


Рис. 2.24. Установка прав для роли «Основная»

Выделим ветвь обработки и установим права Использование и Просмотр (рис. 2.25).

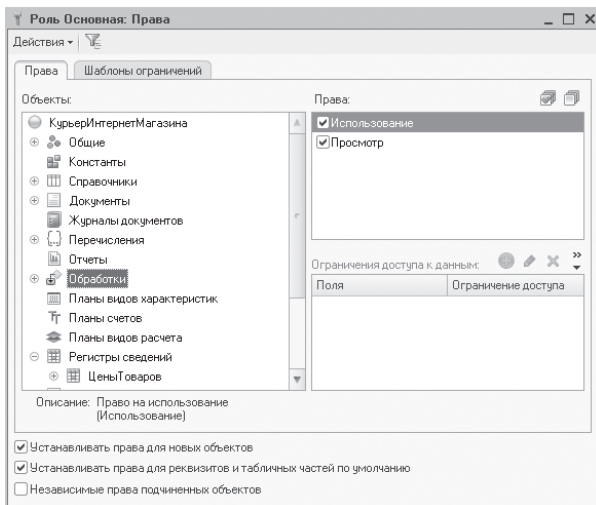


Рис. 2.25. Установка прав для роли «Основная»

Для регистра сведений Цены товаров установим права Чтение и Просмотр (рис. 2.26).

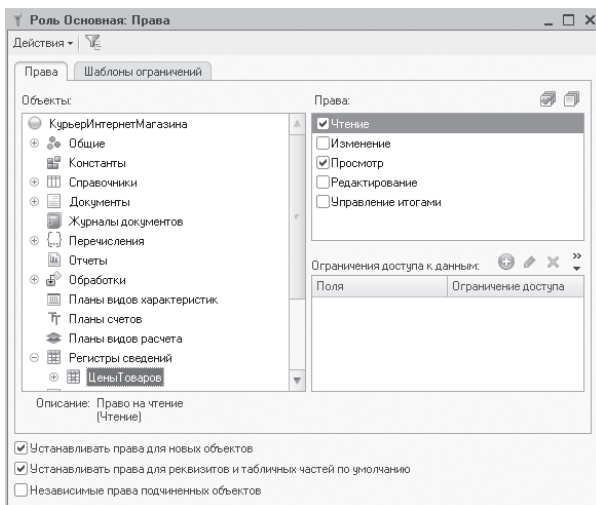


Рис. 2.26. Установка прав для роли «Основная»

После того как мы установили все необходимые права на доступ к данным для роли Основная, выберем ее в списке свойства Основные роли нашего мобильного приложения (рис. 2.27).

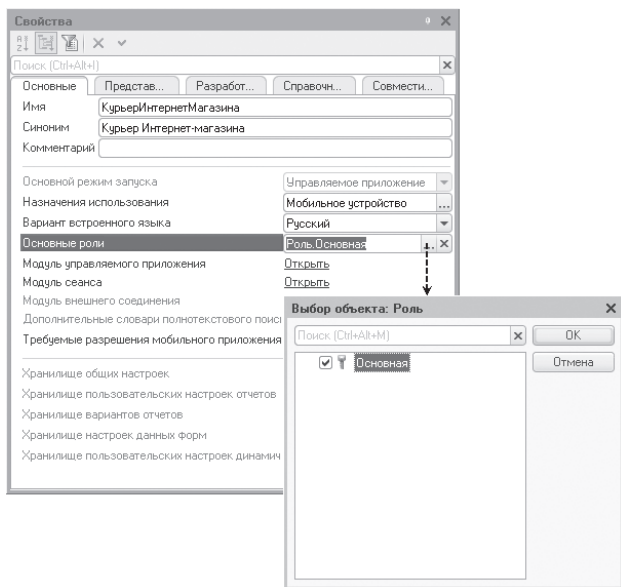


Рис. 2.27. Установка свойства мобильного приложения «Основные роли»

Таким образом, при формировании командного интерфейса приложения мобильная платформа автоматически скроет от пользователя все недоступные команды в соответствии с правами, установленными для роли Основная.

Разработка форм

Теперь пришло время заняться разработкой форм нашего мобильного приложения. Надо заметить, что у справочников и документов в мобильном приложении вообще еще нет никаких форм. Поэтому мобильная платформа автоматически генерирует формы объектов, которые мы видим в интерфейсе мобильного приложения.

Выше мы задали ограничения прав доступа, основанные на основной роли, поэтому командный интерфейс форм будет автоматически учитывать эти ограничения.

Но некоторые формы, которые нас не устраивают своим внешним видом, мы должны создать конструктором форм и изменить с учетом вышеперечисленных требований к формам мобильного приложения (минимализм, удобство, вертикальная ориентация и т. п.).

Справочники для выбора ссылочных значений

Сначала разберемся с формами справочников, которые потребуются только для выбора из них значений в полях ссылочного типа.

Поскольку у справочников Склады и Пользователи мы установили свойство Быстрый выбор, то выбор из них будет производиться в выпадающем списке под ссылочным полем. И поскольку для этих объектов не используются стандартные команды, то мы вообще не увидим нигде в интерфейсе форм этих объектов и, соответственно, команд для их изменения.

У справочников Цвета и Размеры свойство Быстрый выбор мы не устанавливали, так как записей в них может быть достаточно много. Сделаем формы выбора этих справочников более лаконичными.

Создадим форму выбора для справочника Цвета. Так же как и формы списка, формы выбора строятся на основе динамического списка, источником которого служит основная таблица справочника. Платформа автоматически добавляет в таблицу списка поля справочника Наименование и Код.

Так как мы стремимся убрать из формы все лишнее, то удалим поле Код из дерева элементов формы – при выборе элементов справочника это ненужная информация.

Поскольку в списке элементов справочника теперь содержится только одно поле – Наименование, для экономии места в форме уберем отображение шапки в таблице. Для этого сбросим флажок Шапка в свойствах таблицы Список.

Также удалим автоматически добавленную платформой группу СписокКомпонавшиКНастроекПользовательскиеНастройки, так как настройка списка пользователем в мобильном приложении недоступна.

В результате форма выбора справочника Цвета будет выглядеть следующим образом (рис. 2.28).

Сохраним конфигурацию базы данных (F7) и запустим мобильное приложение на планшете. При выборе цвета в мобильном приложении будет открываться форма выбора, содержащая только список названий цветов. В этой форме невозможно изменять данные, что нам и нужно (рис. 2.29).

Затем сделаем то же самое с формой выбора для справочника Размеры.

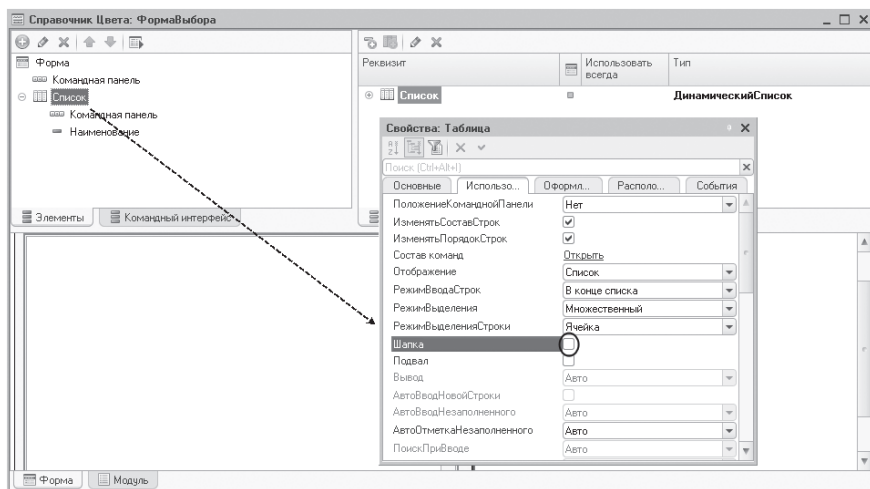


Рис. 2.28. Форма выбора справочника «Цвета»

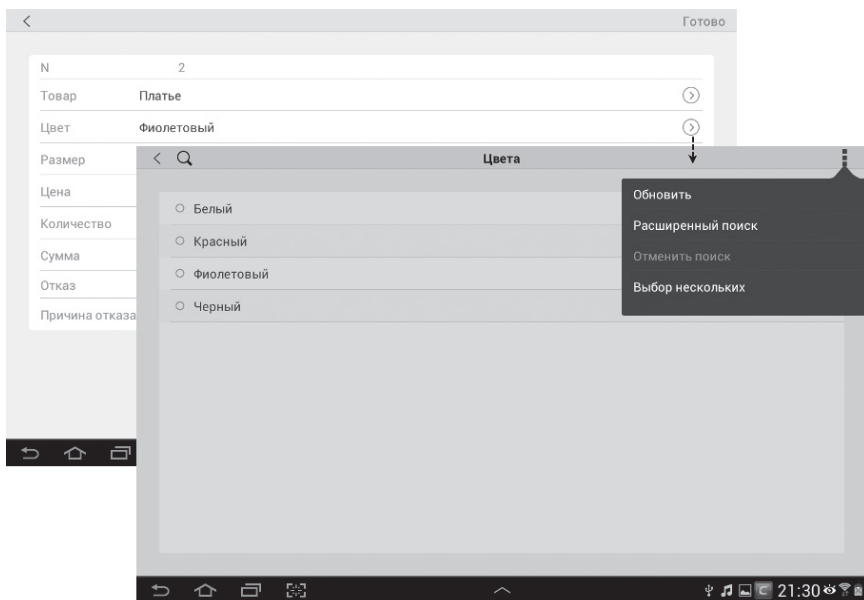


Рис. 2.29. Выбор цвета в мобильном приложении

Нередактируемые справочники

Теперь займемся справочником Товары. Мы исходим из предположения, что курьеру может понадобиться не только выбирать товары в заказах клиентов, но и просматривать весь список и отдельные свойства товаров.

Поэтому справочник товаров должен присутствовать в командном интерфейсе мобильного приложения (флажок Использовать стандартные команды установлен). Но курьер по-прежнему не должен иметь возможность изменять информацию о товарах, удалять или добавлять новые товары.

Сначала, поскольку справочник иерархический, зададим более понятное представление в интерфейсе для поля Родитель. Для этого на закладке Данные окна редактирования справочника Товары нажмем кнопку Стандартные реквизиты и установим для поля Родитель синонимом Группа (рис. 2.30).

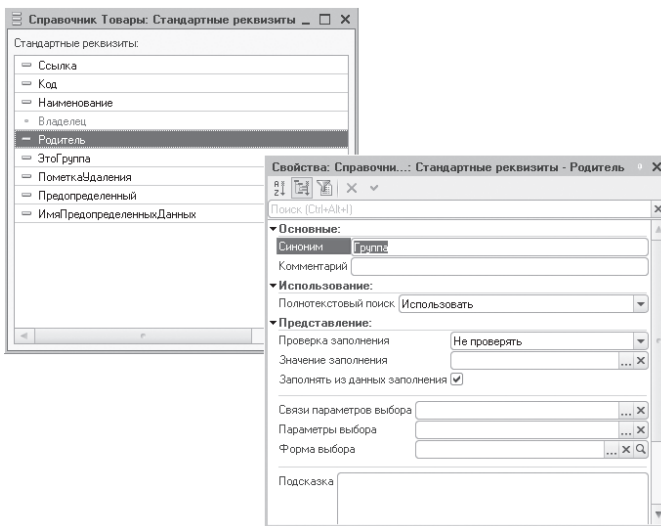


Рис. 2.30. Синоним стандартного реквизита «Родитель»

Теперь создадим форму выбора и форму списка справочника Товары. Удалим из дерева элементов этих форм группу СписокКомпоновщикНастроекПользовательскиеНастройки, поле Код из таблицы списка и снимем флажок Шапка в свойствах таблицы.

В результате при выборе товара будет открываться следующая форма (рис. 2.31).

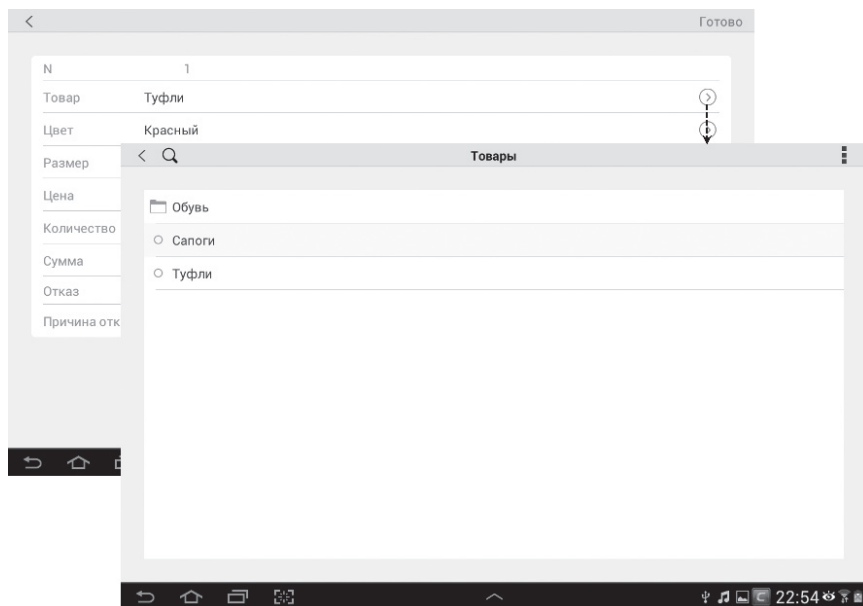


Рис. 2.31. Форма выбора справочника «Товары»

Кроме того, в форме списка поместим прямо над таблицей товаров строку поиска, в которой можно набрать искомые символы, после чего содержимое списка будет соответствующим образом отфильтровано. Для этого выделим в дереве элементов формы таблицу Список и установим свойство ПоложениеСтрокиПоиска в значение Верх (рис. 2.32).

В результате поиск товаров с помощью строки поиска будет выглядеть следующим образом (рис. 2.33).

Форму элемента справочника Товары мы пока создавать не будем, так как автоматически генерируемая платформой форма в целом устраивает нас своим внешним видом (вместо названия поля Родитель там показывается его синоним – Группа). В дальнейшем мы доработаем форму товара. Например, поместим в форму картинку товара. Но для этого нужно сначала изменить основную конфигурацию, чтобы реализовать там возможность выбора картинки товара и сохранения ее в справочнике хранимых файлов.

Но сейчас мы просто хотим предоставить курьеру возможность редактировать или создавать новые заказы, поэтому пока не будем отвлекаться от нашей основной задачи.

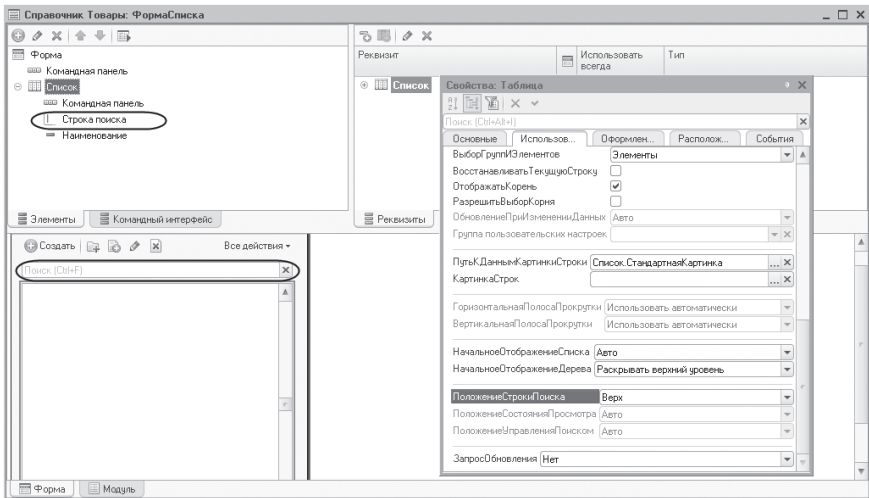


Рис. 2.32. Отображение строки поиска над таблицей динамического списка

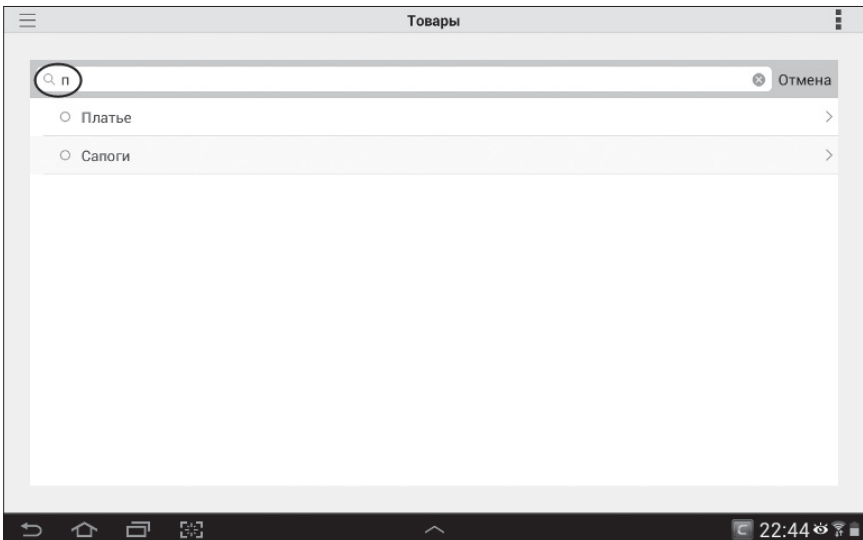


Рис. 2.33. Строка поиска в форме списка товаров

Редактируемые справочники

Осталось только два справочника: Клиенты и Причины отказа от товаров, информацию в которых курьер может редактировать, а также создавать новые и удалять (только помечать на удаление) записи в этих справочниках.

Для справочника Причины отказа от товаров мы вообще не будем создавать никаких форм. Все формы этого справочника (форма выбора, форма списка и форма элемента), генерируемые мобильной платформой, позволяют редактировать информацию в этом справочнике и вполне устраивают нас своим внешним видом.

Что касается справочника Клиенты, то сейчас мобильная платформа показывает автоматически сгенерированный список клиентов, содержащий все реквизиты справочника Клиенты. Этих реквизитов много (Город, Улица, Телефон и т.д.) и они располагаются по горизонтали, что неприемлемо.

Создадим форму списка справочника Клиенты и ограничим состав полей списка двумя полями: Код и Наименование (именно эти поля и добавляет стандартно в форму списка редактор форм).

Поле Код тоже можно было бы удалить, но, на наш взгляд, по нему будет удобно различать, какие записи в справочник внесены курьером, а какие — получены из интернет-магазина (т.к. при обмене код справочника будет иметь соответствующий префикс).

По этой причине (наличие в списке нескольких полей) шапку у таблицы списка тоже оставим. Удалим только группу СписокКомпоновщикНастроекПользовательскиеНастройки из дерева элементов формы.

Сгенерированная платформой форма элемента справочника Клиенты в принципе нас тоже устраивает, но раз уж сейчас мы занимаемся улучшением внешнего вида форм нашего мобильного приложения, то немного усовершенствуем и ее.

Создадим форму элемента справочника Клиенты. Мы видим, что все элементы формы конструктор форм стандартно расположил друг под другом. Это нас устраивает, так как на мобильных устройствах принята вертикальная прокрутка содержимого экрана. Но поскольку элементов в форме клиента много, хорошо бы их зрительно выделить, объединив в группы.

В дереве формы создадим группы (вида Обычная группа) Адрес, Контакты, Координаты и перетащим в эти группы элементы формы, отображающие соответствующую информацию о клиенте (рис. 2.34).

Рис. 2.34. Форма элемента справочника «Клиенты»

В результате на планшете форма клиента будет выглядеть следующим образом (рис. 2.35).

Таким образом, информация о клиенте более структурирована и легче воспринимается. Позднее мы добавим в форму клиента команды для выполнения действий, специфических для мобильного устройства, с помощью которых можно позвонить клиенту, отправить СМС, определить его текущее местоположение, проложить до него маршрут, отправить ему письмо по электронной почте и т. п.

Шувалова Е.М. (Клиент) Готово

Код 000000003

Наименование Шувалова Е.М.

АДРЕС

Страна Россия

Город Москва

Улица ул. Грибоедова

Дом 51

Квартира 140

КОНТАКТЫ

Телефон +7(929)236-45-71

Электронная почта

КООРДИНАТЫ

Широта	0,000000
Долгота	0,000000

13:26

Рис. 2.35. Форма клиента на планшете

Документ «Обслуживание заказов»

Создадим форму списка документа `ОбслуживаниеЗаказов` и удалим из дерева элементов формы группу `СписокКомпоновщикНастроекПользовательскиеНастройки`, а также поля `Дата` и `Номер` из таблицы списка. Раскроем основной реквизит формы `Список` и добавим в таблицу списка поле `ДатаОбслуживания`.

Документ «Заказ»

По условию нашей задачи курьер может создавать новые заказы и редактировать уже существующие – например, проставлять причину отказа от товаров, покупать которые клиент не хочет. В соответствии с этим внесем изменения в интерфейс форм заказов.

Создадим форму списка для документа Заказ и ограничим состав полей списка двумя полями: ДатаДоставки и Номер. А также удалим группу СписокКомпоновщикНастроекПользовательскиеНастройки из дерева элементов формы.

Теперь создадим форму документа Заказ и оценим ее внешний вид, созданный конструктором форм (рис. 2.36).

N	Товар	Цвет	Размер	Цена	Количество	C
---	-------	------	--------	------	------------	---

Рис. 2.36. Форма документа «Заказ», созданная конструктором форм, в конфигураторе

Сначала друг за другом вертикально располагаются элементы, отображающие значения реквизитов документа Клиент, Курьер, Склад и т.д. Их достаточно много, и для лучшей читабельности можно объединить их в группы. Но это дело вкуса. Мы пока делать этого не будем.

Но для поля Клиент сделаем видимой кнопку открытия значения (по умолчанию она не видна). Для этого выделим поле Клиент в дереве элементов формы и установим свойство КнопкаОткрытия в значение Да. Теперь прямо из формы заказа мы можем нажать на значок с информацией («i») в этом поле и просмотреть или изменить информацию о клиенте (рис. 2.37).

The image shows two overlapping mobile application screens. The background screen is titled 'Заказ 000000003 от 12.12.2013 22:00:14' and 'Готово'. It contains a form with the following fields:

Номер	000000003
Дата	12.12.2013 22:00:14
Клиент	Новикова В.С.
Курьер	
Склад	
Дата доставки	
Адрес доставки	
Дополнительная информация	
Статус заказа	
Добавить	
Цвет	
Цена	
Туфли Черный	5 000,00
Платье Фиолетовый	6 000,00

The foreground screen is titled 'Новикова В.С. (Клиент)' and 'Готово'. It displays detailed information about the client:

Код	000000001
Наименование	Новикова В.С.
АДРЕС	
Страна	Россия
Город	Москва
Улица	Гоголевский бульвар
Дом	23
Квартира	77
КОНТАКТЫ	
Телефон	+7(916)655-77-22
Электронная почта	
КООРДИНАТЫ	
Широта	0,000000
Долгота	0,000000

Рис. 2.37. Просмотр информации о клиенте из формы заказа

После реквизитов документа в форме отображается таблица с данными табличной части документа **Товары**. В ней присутствует довольно много реквизитов: ссылка на заказанный товар, цвет и размер товара (чтобы учитывать расход товара в разрезе цветов и размеров), цена, количество и сумма по товару, а также признак отказа от товара и причина отказа (в случае если товар почему-либо не подошел).

Расположение этих полей стандартным образом (в колонках таблицы, т.е. по горизонтали) неприемлемо, так как пользователь просто не увидит всех колонок на экране мобильного устройства. Выходом из положения могут

служить группы колонок, связанные друг с другом и расположенные вертикально, друг под другом.

Кроме того, информацию об отказе от товаров хотелось бы отделить от информации о списке заказанных товаров, их цены, количества и т.п. Для этого нам понадобятся группы страниц, содержащие таблицы с различной информацией о товарах.

Сейчас в дереве элементов формы документа, созданной конструктором форм, мы видим таблицу Товары, отражающую информацию из табличной части документа (см. рис. 2.36). Добавим над ней группу ГруппаТовары вида Страницы и установим свойство этой группы ОтображениеСтраниц в значение Пролистывание (сенсорный экран). Затем вложим в нее две группы вида Страница: СписокТоваров и ОтказОтТоваров (см. рис. 2.38).

В группу СписокТоваров перетащим таблицу Товары и удалим из нее поля ТоварыНомерСтроки, ТоварыОтказ и ТоварыПричинаОтказа. Затем добавим под командной панелью таблицы группу колонок ТоварыГруппа1 с вертикальной группировкой и перетащим в нее поле ТоварыТовар. В свойствах этого поля снимем флажок ОтображатьВШапке.

Далее вложим в эту группу две группы колонок ТоварыГруппа2, ТоварыГруппа3 с горизонтальной группировкой и перетащим в них соответственно поля ТоварыЦвет, ТоварыРазмер и ТоварыЦена, ТоварыКоличество, ТоварыСумма. Первая страница с данными табличной части готова (см. рис. 2.38).

По аналогии создадим вторую страницу. Сначала раскроем основной реквизит формы Объект и перетащим в группу ОтказОтТоваров табличную часть документа Товары. Согласимся с созданием в форме колонок таблицы (по умолчанию она будет названа Товары1). Затем удалим из этой таблицы поля Товары1НомерСтроки, Товары1Цвет, Товары1Размер, Товары1Цена, Товары1Количество, Товары1Сумма.

Затем добавим под командной панелью таблицы группу колонок Товары1Группа1 с вертикальной группировкой и перетащим в нее поле Товары1Товар. В свойствах этого поля снимем флажок ОтображатьВШапке. Далее вложим в эту группу еще одну группу колонок Товары1Группа2 с горизонтальной группировкой и перетащим в нее поля Товары1Отказ и Товары1ПричинаОтказа.

После этого дерево элементов формы документа в редакторе формы будет выглядеть следующим образом (рис. 2.38).

В окне предварительного просмотра редактора формы документа мы можем оценить ее внешний вид (рис. 2.39).



Однако надо понимать, что в редакторе формы на стационарном компьютере форма отображается в интерфейсе 8.2, на планшете же вид формы будет значительно отличаться, т.к. интерфейс мобильного приложения адаптирован под мобильное устройство и приближен к интерфейсу «Такси».

Итак, мы разделили информацию из табличной части документа на две смысловые группы, которые отображаются на различных страницах в двух таблицах, источником которых служит эта табличная часть.

Теперь нажмем F7, запустим наше мобильное приложение и посмотрим, как будет выглядеть форма заказа на планшете (рис. 2.40).

Заказ 000000003 от 12.12.2013 22:00:14 Готово

Номер	000000003
Дата	12.12.2013 22:00:14
Клиент	Новикова В.С. ⓘ ➤
Курьер	Морозов Д.И.
Склад	Основной
Дата доставки	15.12.2013
Адрес доставки	Москва, Гоголевский б-р 23 кв....
Дополнительная информация доставка до 13-00	
Статус заказа	В работе

Добавить Еще ➤

Цвет	Размер	Количество	Сумма
Туфли Черный	38	1	5 000,00
Платье Фиолетовый	46	1	6 000,00

Рис. 2.40. Форма заказа на планшете в вертикальной ориентации

Под основными реквизитами товара мы видим таблицу со списком заказанных товаров. При этом одна строка табличной части отображается в трех подстроках таблицы на экране планшета. В первой – сам товар, во второй – его цвет и размер и в третьей – цена, количество и сумма по товару. При выделении любой из них выделяются все три подстроки, так как соответствующие группы колонок связаны друг с другом (вторая и третья группа колонок вложены в первую – см. рис. 2.38).

Для редактирования информации из таблицы мобильная платформа открывает специальную системную форму, в которой колонки таблицы следуют друг за другом. На вид этой формы мы никак не можем повлиять – ни из конфигуратора, ни из встроенного языка.

Но для числового поля ТоварыКоличество установим свойство КнопкаРегулирования в значение Да, чтобы было удобнее увеличивать/уменьшать количество заказанных товаров.

Проверим изменения. При нажатии на строку таблицы с первой страницы открывается следующая форма (рис. 2.41).

Товар	Платье	>
Цвет	Фиолетовый	>
Размер	46	>
Цена	6 000,00	
Количество	1 - +	
Сумма	6 000,00	

Рис. 2.41. Форма редактирования данных табличной части

Внизу под таблицей со списком заказов (см. рис. 2.40, 2.42) находятся точки, которые показывают количество страниц с информацией. Пролистнув таблицу вправо, мы увидим данные об отказе от товаров (рис. 2.42).

При нажатии на строку таблицы курьер может отметить, от каких товаров клиент отказался, и указать причину отказа. Для поля Товары1ПричинаОтказа установим свойство КнопкаОткрытия в значение Да, чтобы курьер мог сразу же из заказа просмотреть или изменить информацию о причине отказа (рис. 2.43).

Заказ 000000003 от 12.12.2013 22:00:14 * Готово

Номер	000000003
Дата	12.12.2013 22:00:14
Клиент	Новикова В.С. (i) (>)
Курьер	Морозов Д.И.
Склад	Основной
Дата доставки	15.12.2013
Адрес доставки	Москва, Гоголевский б-р 23 кв....
Дополнительная информация	доставка до 13-00
Статус заказа	В работе

Добавить Еще >

Отказ	Причина отказа
☒ Туфли	Не подошел цвет/размер (>)
☐ Платье	(>)

Рис. 2.42. Форма заказа на планшете в вертикальной ориентации

Готово

Товар	Туфли (>)
Отказ	☒
Причина отказа	Не подошел цвет/размер (i) (>)

Рис. 2.43. Форма редактирования данных табличной части

При выборе причин отказа можно нажать кнопку ⓘ (при этом появится форма выбора), а можно просто, нажав на эту строку, начать вводить нужное значение, и найденные соответствия из элементов справочника Причины отказа тут же будут предложены для выбора в выпадающем окне под строкой ввода. В этом же окне можно добавить (кнопка «+») элемент справочника, если в справочнике нет нужного значения (рис. 2.44).

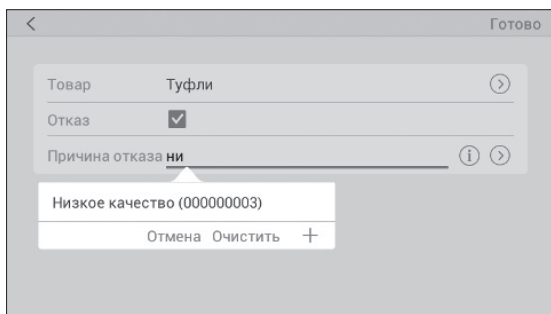


Рис. 2.44. Выбор и создание элемента справочника «Причины отказа» при вводе по строке

Аналогично при вводе по строке можно выбирать и добавлять клиентов. Товары, цвета и размеры таким образом можно только выбирать: добавление элементов в эти справочники недоступно в соответствии с правами доступа (кнопка «+» отсутствует).

Заключение

На этом первоначальную разработку форм объектов мобильного приложения можно считать завершенной. Пока мы занимались только внешним видом и командным интерфейсом этих форм.

Заметьте, что при разработке форм мы нигде не задавали явно ширину и высоту элементов, содержащихся в формах. На экране мобильного устройства стандартно располагается только одна форма, и она растягивается на весь экран. В управляемых формах, которые используются в мобильном приложении, свойства Ширина и Высота конкретных элементов стандартно установлены в значение 0, а свойства РастягиватьПоВертикали и РастягиватьПоГоризонтали включены.

Это значит, что размеры элементов определяются автоматически в зависимости от отображаемых в них данных и автоматически подстраиваются под размеры формы, а значит, и под размеры мобильного устройства.

Например, на рис. 2.40, 2.42 планшет имеет вертикальную ориентацию, но внешний вид форм приложения от этого ничуть не страдает.

Кроме того, все элементы формы стандартно располагаются друг под другом, что также нас устраивает, так как прокрутка форм мобильного приложения должна быть только вертикальной.

Для более точной адаптации к размерам мобильного устройства можно разрабатывать несколько комплектов форм под каждый размер устройства и затем программно выбирать, какую из форм открывать. Или же можно программно скрывать/отображать элементы формы, устанавливать свойства, определяющие их отображение в форме, в зависимости от размеров используемого мобильного устройства.

Но, на наш взгляд, если и стоит этим заниматься, то уже после того, как будет реализована основная задача мобильного приложения, описанная в начале этой книги. В данном случае мы пошли по наиболее простому пути. При желании после усвоения всего материала книги вы можете самостоятельно поэкспериментировать в этом направлении.

Функциональность мобильного приложения

Теперь, собственно, мы можем заняться «программной начинкой» форм, с помощью которой курьер сможет решать все возникающие перед ним задачи, описанные нами в разделе «Функциональность мобильного приложения» на стр. 11.

Заказы

Сначала реализуем работу с заказами. Откроем форму документа Заказ.

Работа со списком товаров

Расчет суммы по товару

Прежде всего реализуем алгоритм расчета суммы по товару как произведения его цены на количество.

Цена товара на определенную дату хранится в периодическом регистре сведений Цены товаров. Данные о ценах товаров заполняются в интернет-магазине и передаются курьеру на планшет при обмене данными.

При изменении или добавлении товаров в заказ цена товара из регистра сведений должна умножаться на его количество, введенное вручную,

и записываться в колонку Сумма в измененной строке таблицы заказанных товаров. Для этого в обработчиках события ПриИзменении этих полей таблицы мы должны задать необходимый алгоритм расчета.

Итак, создадим обработчик события ПриИзменении для поля таблицы ТоварыЦена. Укажем, что обработчик события будет выполняться на клиенте, и заполним его следующим образом (листинг 2.15).

Листинг 2.15. Обработчик события «ПриИзменении» для поля «ТоварыЦена»

```
&НаКлиенте  
Процедура ТоварыЦенаПриИзменении(Элемент)
```

```
    СтрокаТабличнойЧасти = Элементы.Товары.ТекущиеДанные;  
    СтрокаТабличнойЧасти.Сумма = СтрокаТабличнойЧасти.Количество * СтрокаТабличнойЧасти.Цена;
```

```
КонецПроцедуры
```

В первой строке обработчика в переменной СтрокаТабличнойЧасти мы получаем объект, содержащий данные, находящиеся в текущей строке табличной части. Теперь, используя эту переменную, мы можем получить доступ к данным колонок табличной части и рассчитать сумму как произведение цены на количество.

Затем создадим клиентский обработчик события ПриИзменении для поля табличной части ТоварыКоличество и заполним его аналогичным образом (см. листинг 2.15).

А для поля ТоварыСумма в палитре свойств установим свойство ТолькоПросмотр в значение Истина, так как это поле будет расчетным и не должно изменяться вручную.

Сохраним конфигурацию базы данных, запустим мобильное приложение на планшете и проверим изменения. Откроем любой заказ и вызовем форму для редактирования строки таблицы со списком товаров (см. рис. 2.41). Здесь мы можем увеличивать/уменьшать количество товара с помощью кнопки регулирования (+/-) или вручную изменять его цену. При этом соответственно будет пересчитана сумма – как произведение цены товара на его количество.

Получение актуальной цены товара

Однако до сих пор мы просто указывали произвольную цену товара. На самом деле цена товара на определенную дату хранится в периодическом регистре сведений Цены товаров. Данные о ценах товаров заполняются в интернет-магазине и передаются курьеру на планшет при обмене данными. Курьер не может ни изменять, ни даже просматривать

эти данные. Поэтому мы не видим нигде в мобильном приложении команд, связанных с регистром.

Единственно, для чего он понадобится: в случае если курьер создает заказ, нужно, чтобы при выборе товара в строке таблицы автоматически подставлялась бы его цена из регистра сведений. Цена товара должна быть актуальной на дату заказа. И после этого должна пересчитаться сумма по товару с учетом новой цены.

Для этого сначала создадим в модуле формы функцию, которая будет возвращать нам актуальную цену товара на определенную дату. Эту дату и ссылку на товар мы будем передавать в функцию в качестве параметров (листинг 2.16).

Листинг 2.16. Функция для получения актуальной цены товара

```
&НаСервереБезКонтекста
Функция ПолучитьЦенуТовара(Дата, Товар)

    Отбор = Новый Структура("Товар", Товар);
    ЦенаТовара = РегистрыСведений.ЦеныТоваров.ПолучитьПоследнее(Дата, Отбор);
    Возврат ЦенаТовара.Цена;

КонецФункции
```

Функция выполняется на сервере без контекста формы, так как в ней требуется прочитать последнее значение регистра сведений для определенного товара, что возможно только на сервере. А контекст формы в этой функции не нужен, так как необходимые данные формы передаются в функцию в качестве параметров.

В теле функции мы сначала создаем структуру `Отбор`, содержащую отбор по измерениям регистра. С ее помощью определяем, что нас будут интересовать записи регистра, в которых измерение регистра `Товар` равно переданной в функцию ссылке на элемент справочника товаров.

Затем мы выполняем метод менеджера регистра сведений `ЦеныТоваров.ПолучитьПоследнее()`, который возвращает нам значения ресурсов самой поздней записи регистра, соответствующей переданной в функцию дате и значениям измерений регистра.

Поскольку метод `ПолучитьПоследнее` возвращает структуру, содержащую значения всех ресурсов регистра (в общем случае у регистра может быть несколько ресурсов), то в следующей строке мы возвращаем цену товара, просто указав имя нужного нам ресурса через точку.

Теперь эту функцию нужно вызвать при выборе товара в табличной части заказа.

Для этого создадим клиентский обработчик события ПриИзменении для поля табличной части ТоварыТовар и заполним его следующим образом (листинг 2.17).

Листинг 2.17. Обработчик события «ПриИзменении» для поля «ТоварыТовар»

```
&НаКлиенте
Процедура ТоварыТоварПриИзменении(Элемент)

    СтрокаТабличнойЧасти = Элементы.Товары.ТекущиеДанные;

    СтрокаТабличнойЧасти.Цена = ПолучитьЦенуТовара(Объект.Дата, СтрокаТабличнойЧасти.Товар);
    СтрокаТабличнойЧасти.Количество = ?(СтрокаТабличнойЧасти.Количество = 0, 1,
                                           СтрокаТабличнойЧасти.Количество);

    СтрокаТабличнойЧасти.Сумма = СтрокаТабличнойЧасти.Количество * СтрокаТабличнойЧасти.Цена;

КонецПроцедуры
```

Сначала в переменной СтрокаТабличнойЧасти мы получаем объект, содержащий данные, находящиеся в текущей строке табличной части. Эта строка обработчика нам уже знакома по процедурам ТоварыКоличествоПриИзменении и ТоварыЦенаПриИзменении.

Используя эту переменную, мы передаем значение, содержащееся в колонке Товар (ссылку на элемент справочника Товары), вторым параметром в функцию ПолучитьЦенуТовара(), определенную нами выше (см. листинг 2.16).

Первым параметром мы передаем в функцию дату документа, на которую нужно получить цену товара. Дату документа мы получаем из основного реквизита формы (Объект.Дата).

Возвращенное функцией последнее значение цены товара мы присваиваем полю Цена в текущей строке табличной части документа. Если строка создается заново, то полю Количество присваивается единица.

Затем так же, как и в процедурах ТоварыКоличествоПриИзменении и ТоварыЦенаПриИзменении, сумма в текущей строке табличной части документа пересчитывается как произведение цены товара на его количество.

Отказ от товаров

Теперь настроим нужное поведение второй таблицы формы Товары1 с данными об отказе от товаров. Прежде всего сделаем так, чтобы курьер не смог бы ни добавлять, ни удалять информацию в этой таблице, а также изменять список заказанных товаров.

Чтобы обеспечить такое поведение, в палитре свойств таблицы Товары1 установим свойство ПоложениеКоманднойПанели в значение Нет и отключим свойство ИзменятьСоставСтрок (рис. 2.45).

У поля этой таблицы Товары1Товар установим свойство ТолькоПросмотр (рис. 2.46).

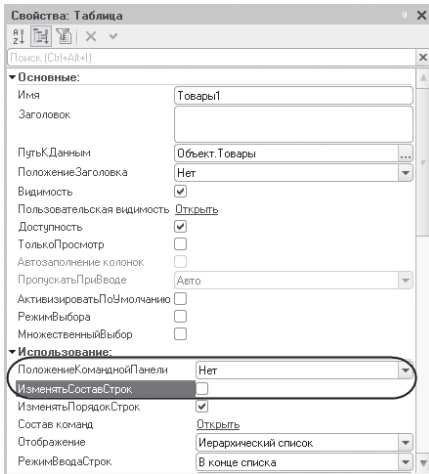


Рис. 2.45. Палитра свойств таблицы формы «Товары1»

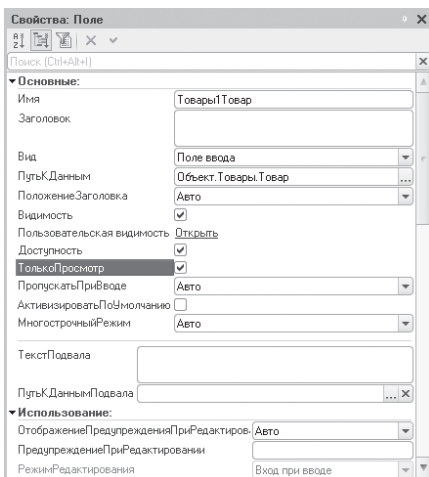


Рис. 2.46. Палитра свойств поля «Товар» таблицы «Товары1»

Таким образом, у таблицы, содержащей информацию об отказе от товаров, будет отсутствовать командная панель и возможность изменения состава строк. Соответственно, курьер не сможет добавлять или удалять строки

табличной части на странице с данными об отказе от товаров. А также курьер сможет только проставлять признак отказа от товара и указывать его причину, но не сможет изменять сам товар (см. рис. 2.47).

Затем сделаем так, чтобы в форме редактирования строки табличной части при отметке флажка Отказ сразу же вызывалась бы форма выбора причины отказа от товара, и после выбора соответствующее значение устанавливалось бы в поле Причина отказа. А при снятии флажка Отказ очищалась бы и причина отказа (рис. 2.47).

Для этого создадим клиентский обработчик события ПриИзменении для поля табличной части Товары1Отказ и заполним его следующим образом (листинг 2.18).

Листинг 2.18. Обработчик события «ПриИзменении» для поля «Товары1Отказ»

```
&НаКлиенте
Процедура Товары1ОтказПриИзменении(Элемент)

    Если Элементы.Товары1.ТекущиеДанные.Отказ = Истина Тогда
        ОткрытьФорму("Справочник.ПричиныОтказа.ФормаВыбора", , Элементы.Товары1);
    Иначе
        Элементы.Товары1.ТекущиеДанные.ПричинаОтказа = ОчиститьПричинуОтказа();
    КонецЕсли;

КонецПроцедуры
```

В этом обработчике в случае отметки поля таблицы Отказ мы открываем форму выбора справочника ПричиныОтказа. При открытии этой формы в третьем параметре мы указываем владельца формы – таблицу Товары1 (Элементы.Товары1). Соответственно, выбранное значение мы сможем получить в обработчике события ОбработкаВыбора этой таблицы.

Создадим клиентский обработчик события ОбработкаВыбора для таблицы формы Товары1 и заполним его следующим образом (листинг 2.19).

Листинг 2.19. Обработчик события «ОбработкаВыбора» для таблицы «Товары1»

```
&НаКлиенте
Процедура Товары1ОбработкаВыбора(Элемент, ВыбранноеЗначение, СтандартнаяОбработка)

    Элементы.Товары1.ТекущиеДанные.ПричинаОтказа = ВыбранноеЗначение;

КонецПроцедуры
```

В этом обработчике мы присваиваем выбранное значение полю ПричинаОтказа в текущей строке табличной части.

В случае снятия отметки в поле Отказ в обработчике события ПриИзменении этого поля (см. листинг 2.18) мы присваиваем полю ПричинаОтказа значение пустой ссылки на соответствующий справочник.

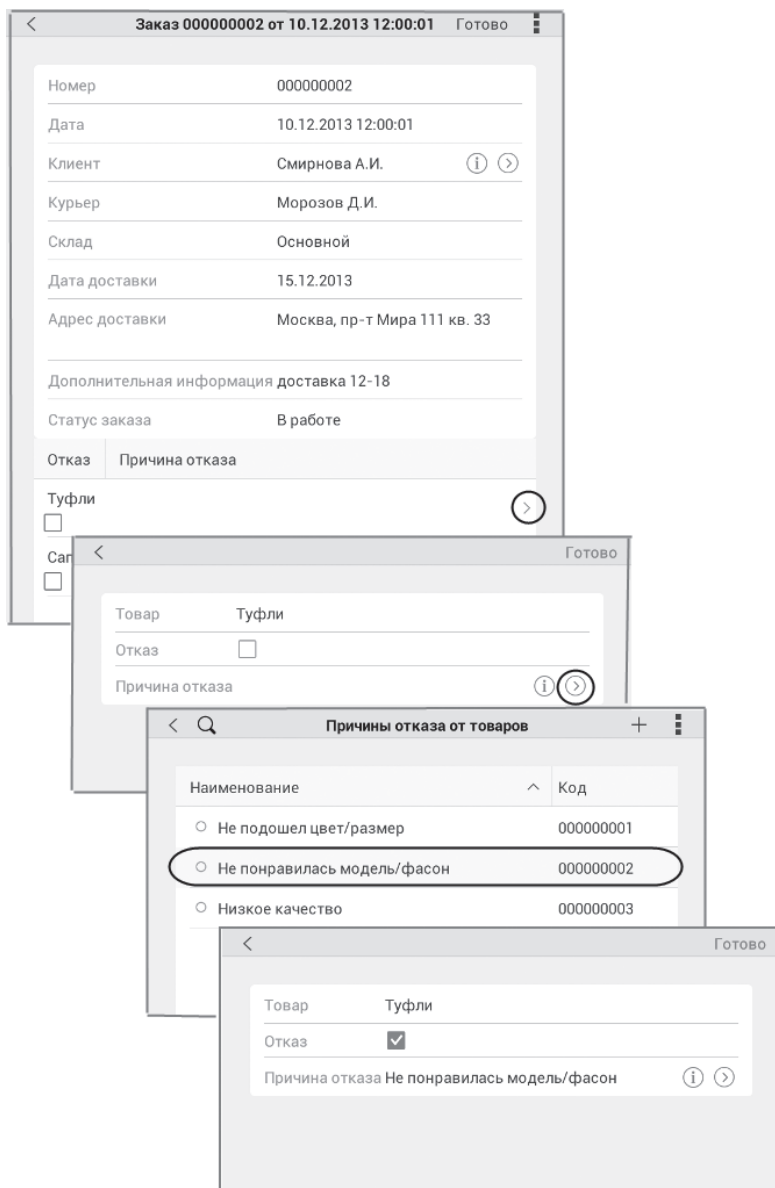


Рис. 2.47. Установка причины отказа от товаров

Для этого в модуле формы мы создаем серверную внеконтекстную функцию `ОчиститьПричинуОтказа()`, которая возвращает пустую ссылку на справочник `ПричиныОтказа` (листинг 2.20).

Листинг 2.20. Функция «`ОчиститьПричинуОтказа()`»

```
&НаСервереБезКонтекста
Функция ОчиститьПричинуОтказа()

    Возрат Справочники.ПричиныОтказа.ПустаяСсылка();

КонецФункции
```

Итоговые суммы заказанных товаров

Теперь улучшим внешний вид таблицы, содержащей список заказанных товаров. Под таблицей хорошо бы видеть общую сумму и количество товаров.

Для этого можно было бы просто отобразить в форме стандартные итоговые поля табличной части из основного реквизита формы. Но в данном случае нам это не подходит, так как мы хотим видеть итоговые суммы заказанных товаров без учета товаров, от которых клиент отказался.

Поэтому создадим реквизиты формы `КоличествоИтог` (типа `Число 10`) и `СуммаИтог` (типа `Число 10,2`) и перетащим их в дерево элементов формы в группу `СписокТоваров` после таблицы `Товары`. Установим для соответствующих элементов формы свойство `Вид` в значение `Поле надписи` (рис. 2.48).

Для расчета итоговых сумм по табличной части документа мы создадим следующую процедуру (листинг 2.21).

Листинг 2.21. Процедура «`ПересчитатьИтоговуюСумму()`»

```
&НаСервере
Процедура ПересчитатьИтоговуюСумму()

    СуммаИтог = 0;
    КоличествоИтог = 0;
    Для Каждого ТекущаяСтрока Из Объект.Товары Цикл
        Если ТекущаяСтрока.Отказ = Ложь Тогда
            СуммаИтог = СуммаИтог + ТекущаяСтрока.Сумма;
            КоличествоИтог = КоличествоИтог + ТекущаяСтрока.Количество;
        КонецЕсли;
    КонецЦикла;

КонецПроцедуры
```

В данной процедуре мы обходим в цикле табличную часть документа (`Объект.Товары`) и накапливаем итоговые значения в реквизитах формы `КоличествоИтог` и `СуммаИтог` в том случае, если у товара нет отметки об отказе.

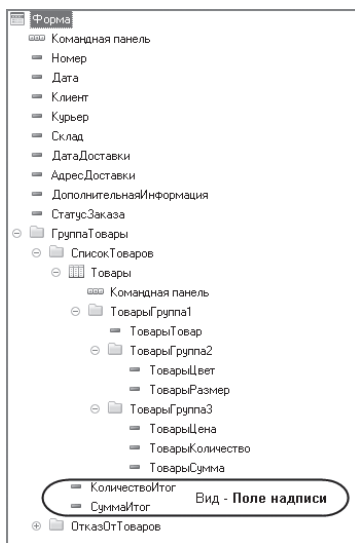


Рис. 2.48. Добавление итоговых сумм в форму

Процедуру для пересчета итоговых сумм табличной части будем вызывать каждый раз при изменении цены и количества товара, при перевыборе товара, а также при изменении информации об отказе от товара.

Соответственно в процедуры `ТоварыКоличествоПриИзменении()`, `ТоварыЦенаПриИзменении()`, `ТоварыТоварПриИзменении()`, `Товары1ОтказПриИзменении()` последней строкой нужно добавить процедуру `ПересчитатьИтоговуюСумму()`.

Кроме того, создадим обработчик события формы `ПриСозданииНаСервере` и также поместим туда процедуру для пересчета итогов (листинг 2.22).

Листинг 2.22. Обработчик события «ПриСозданииНаСервере»

```
&НаСервере
Процедура ПриСозданииНаСервере(Отказ, СтандартнаяОбработка)

    ПересчитатьИтоговуюСумму();

КонецПроцедуры
```

В результате при открытии заказа под списком заказанных товаров курьер будет видеть итоговое количество и сумму без учета товаров, от которых клиент отказался (рис. 2.49). И при любых изменениях в табличной части эти итоги будут пересчитываться.

Заказ 000000003 от 12.12.2013 22:00:14 Готово

Номер	000000003		
Дата	12.12.2013 22:00:14		
Клиент	Новикова В.С.		(i) (>)
Курьер	Морозов Д.И.		
Склад	Основной		
Дата доставки	15.12.2013		
Адрес доставки	Москва, Гоголевский б-р 23 кв....		
Дополнительная информация доставка до 13-00			
Статус заказа	В работе		

Добавить Еще >

Цвет	Размер	Количество	Сумма
Туфли Черный	38	1	5 000,00
Платье Фиолетовый	46	1	6 000,00

Количество итог 2

Сумма итог 11 000,00

Рис. 2.49. Список заказанных товаров с итоговыми суммами

При подсчете итоговых сумм в заказе для упрощения сделано одно допущение. Если клиент отказывается от товара, то считается, что он не берет сразу все количество товара, которое заказано.

Работа с клиентами

Теперь реализуем необходимые функции для работы с клиентами непосредственно из формы заказа. Было бы удобно, чтобы, не закрывая заказа, курьер мог добавить нового клиента, для которого создается заказ, позвонить или отправить СМС клиенту.

Чтобы реализовать эти функции в форме заказа, на закладке Команды добавим команды Позвонить и ОтправитьСМС. Что касается добавления клиента, то это стандартная функциональность мобильной платформы, и здесь нам ничего делать не придется, за исключением изменения адреса доставки.

Добавление нового клиента

Мобильная платформа предоставляет удобную возможность добавления новых элементов справочников при вводе по строке в соответствующих ссылочных полях.

Поэтому как только пользователь начинает вводить наименование клиента в поле заказа Клиент, под ним открывается выпадающий список вариантов ввода из элементов справочника Клиенты. Если подходящего наименования в справочнике нет, то можно нажать на кнопку создания (со значком «+») в правом нижнем углу выпадающего списка (рис. 2.50).

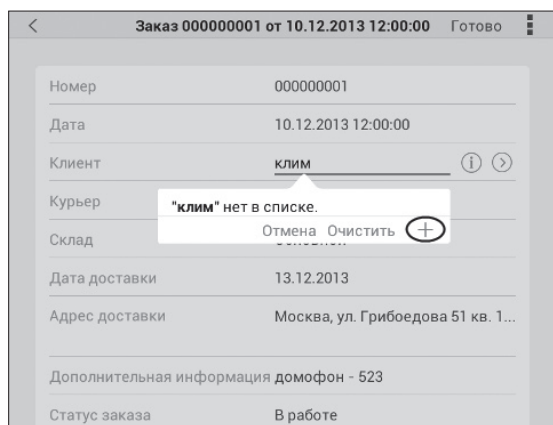


Рис. 2.50. Добавление нового клиента из текущего заказа

При этом откроется форма для добавления нового клиента, а ссылка на этого клиента автоматически попадет в заказ. Было бы удобно, чтобы при добавлении нового клиента в заказ (или изменении уже существующего клиента) сразу же заполнялся адрес доставки исходя из адресных реквизитов клиента. Чтобы предоставить эту возможность, создадим обработчик события ПриИзменении для поля формы Клиент и поместим туда вызов функции для получения адреса доставки клиента (листинг 2.23).

Листинг 2.23. Обработчик события «ПриИзменении» для поля «Клиент»

```

&НаКлиенте
Процедура КлиентПриИзменении(Элемент)

    Объект.АдресДоставки = ПолучитьАдресДоставки(Объект.Клиент);

КонецПроцедуры

```

В этом обработчике мы вызываем серверную внеконтекстную функцию `ПолучитьАдресДоставки()`, которая возвращает адресные реквизиты клиента (Город, Улица, Дом и т.д.), перечисленные через запятую в одной строке, и заполняем ими реквизит `АдресДоставки` (листинг 2.24).

Листинг 2.24. Функция для получения адреса доставки для клиента

```

&НаСервереБезКонтекста
Функция ПолучитьАдресДоставки(Клиент)

    АдресДоставки = СокрЛП(Клиент.Страна) + ", " + СокрЛП(Клиент.Город) + ", " +
        СокрЛП(Клиент.Улица) + ", " + СокрЛП(Клиент.Дом) + ", " +
        СокрЛП(Клиент.Квартира);

    Возврат АдресДоставки;

КонецФункции

```

Таким образом, при добавлении нового клиента в заказ курьер сразу же получает его адрес доставки, заполненный исходя из адресных реквизитов клиента. После этого он может вручную откорректировать это поле в заказе.

Для примера рассмотрим следующую ситуацию. Курьер доставил заказ клиенту. Родственник, сослуживец или знакомый клиента решил заказать себе те же самые товары.

Тогда курьер выполняет команду **Скопировать** из меню функций в правом верхнем углу формы заказа (кнопка ). Содержимое заказа копируется в новый заказ. Далее курьеру нужно добавить нового клиента, для которого создается заказ. Для этого курьер нажимает на наименование клиента и затем на кнопку со значком «+» в появившемся окне под этим полем.

В появившейся форме создания клиента он заполняет данные на нового клиента и нажимает кнопку **Готово** в правом верхнем углу формы (рис. 2.51).

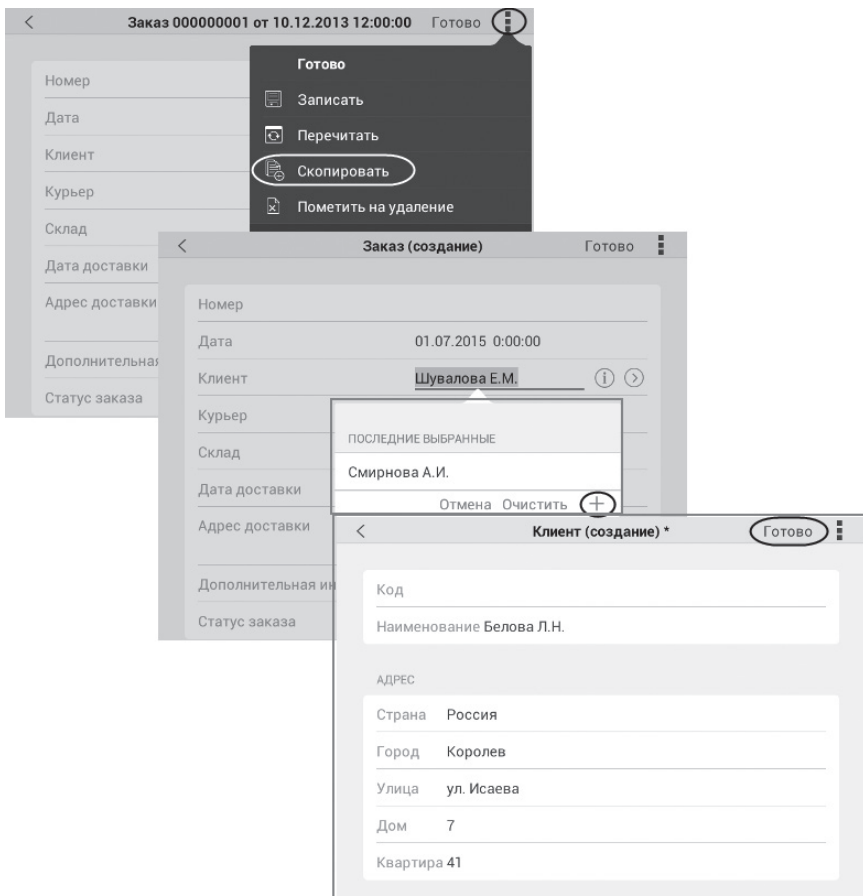


Рис. 2.51. Добавление нового заказа и нового клиента из текущего заказа

После этого новый клиент и адрес доставки, составленный из адресных реквизитов клиента, попадают в заказ. Курьеру остается только изменить дату доставки, цвет или размер товаров, если потребуется (рис. 2.52).

Заказ (создание) * Готово

Номер

Дата 01.07.2015 0:00:00

Клиент Белова Л.Н. (i) (>)

Курьер Морозов Д.И.

Склад Основной

Дата доставки 13.12.2013

Адрес доставки Россия, Королев, ул. Исаева, 7, ...

Дополнительная информация домофон - 523

Статус заказа В работе

Добавить Еще >

Цвет	Размер	Количество	Сумма
Джинсы Черный	44	1	3 000,00
Сапоги Черный	37	1	7 000,00
Количество итог		2	
Сумма итог			10 000,00

16:39

Рис. 2.52. Добавление нового заказа и нового клиента из текущего заказа

Вот так, быстро и удобно, курьер может добавить и новый заказ, и нового клиента, не закрывая текущего заказа.

Связь с клиентом

Теперь добавим в форму заказа команды, с помощью которых можно позвонить клиенту или написать ему сообщение. Для этого используются возможности сотовой связи, которые поддерживаются не всеми мобильными устройствами. В частности, у нашего планшета такой возможности нет, однако соответствующие функции предусмотрены в нашем мобильном приложении.

Перетащим команды Позвонить и ОтправитьСМС в командную панель формы. В командной панели формы появятся соответствующие кнопки для выполнения этих команд.

Здесь также будем действовать по принципу минимизации командного интерфейса формы. Не стоит нагружать форму командами, которые могут и не понадобиться. Поэтому сделаем так, чтобы команда Отправить СМС была видна только в меню функций, вызываемом кнопкой  в правом верхнем углу заказа. Для этого установим в палитре свойств кнопки командной панели ФормаОтправитьСМС свойство ТолькоВоВсехДействиях в значение Да (рис. 2.53).

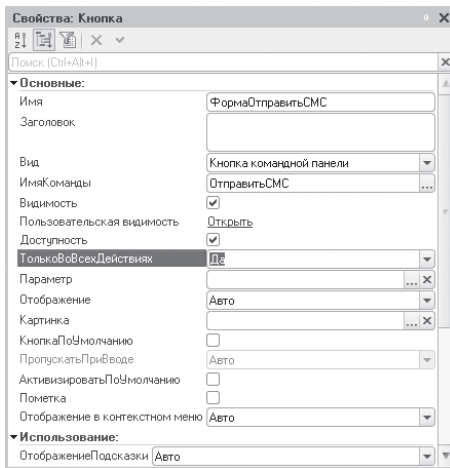


Рис. 2.53. Палитра свойств кнопки «ФормаОтправитьСМС»

Команда Позвонить будет по умолчанию видна в командной панели формы. Чтобы сделать ее более компактной и «симпатичной», зададим ее отображение в виде картинки со значком телефонной трубки. Поскольку подходящей картинки в библиотеке стандартных картинок платформы нет, добавим соответствующую картинку в общие картинки нашего мобильного приложения (рис. 2.54).

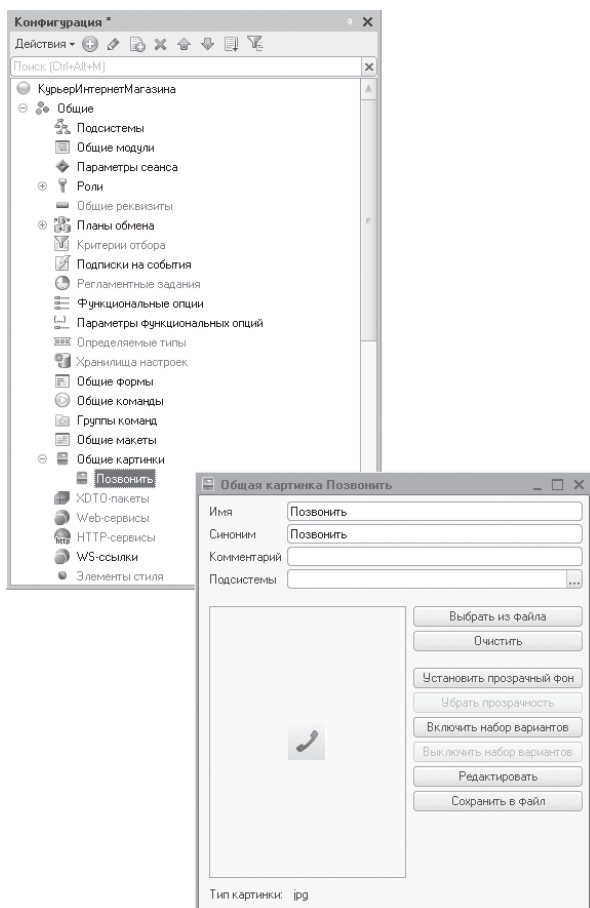


Рис. 2.54. Добавление картинки «Позвонить» в общие картинки конфигурации

После этого в палитре свойств команды Позвонить установим эту картинку в поле Картинка (рис. 2.55).

В результате форма заказа в редакторе формы примет следующий вид (рис. 2.56).

Затем создадим клиентские обработчики для команд Позвонить и ОтправитьСМС и заполним их следующим образом (листинги 2.25, 2.26).

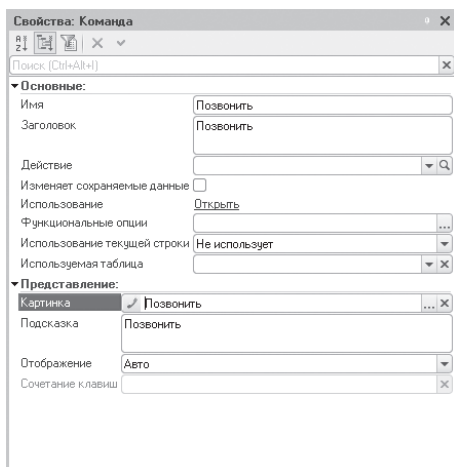


Рис. 2.55. Палитра свойств команды «Позвонить»

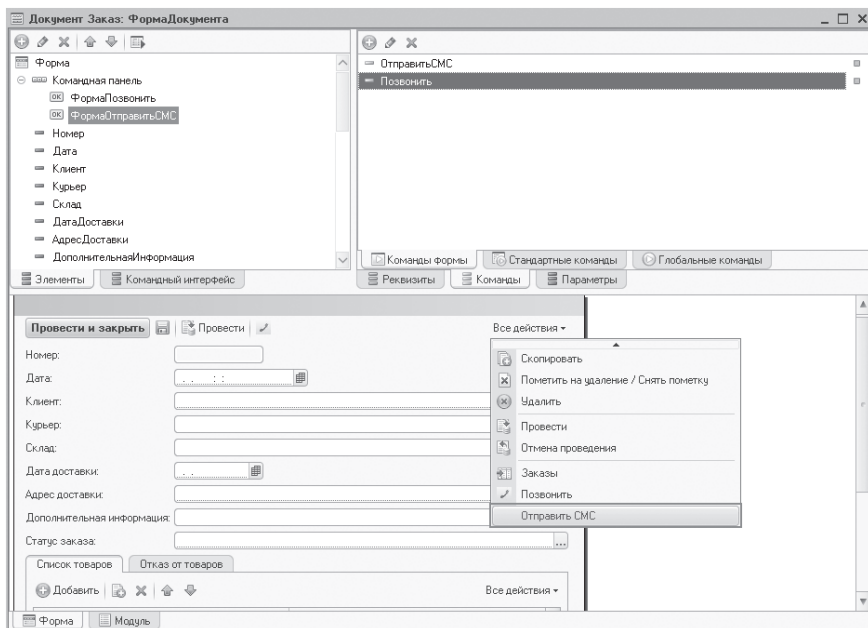


Рис. 2.56. Форма заказа в редакторе формы

Листинг 2.25. Обработчик команды «Позвонить»

&НаКлиенте

Процедура Позвонить(Команда)

```
Если ЗначениеЗаполнено(Объект.Клиент) Тогда
    Телефон = ПолучитьТелефон(Объект.Клиент);
```

```
Если ЗначениеЗаполнено(Телефон) Тогда
    СредстваТелефонии.НабратьНомер(Телефон, Ложь);
```

Иначе

```
Сообщение = Новый СообщениеПользователю();
Сообщение.Текст = "Не указан телефон клиента!";
Сообщение.Поле = "Объект.Клиент";
Сообщение.Сообщить();
```

КонецЕсли

Иначе

```
Сообщение = Новый СообщениеПользователю();
Сообщение.Текст = "Клиент не выбран!";
Сообщение.Поле = "Объект.Клиент";
Сообщение.Сообщить();
```

КонецЕсли

КонецПроцедуры

Листинг 2.26. Обработчик команды «ОтправитьСМС»

&НаКлиенте

Процедура ОтправитьСМС(Команда)

```
Если ЗначениеЗаполнено(Объект.Клиент) Тогда
    Телефон = ПолучитьТелефон(Объект.Клиент);
```

```
Если ЗначениеЗаполнено(Телефон) Тогда
```

```
    Сообщение = Новый SMSСообщение();
    Сообщение.Получатели.Добавить(Телефон);
    СредстваТелефонии.ПослатьSMS(Сообщение, Истина);
```

Иначе

```
    Сообщение = Новый СообщениеПользователю();
    Сообщение.Текст = "Не указан телефон клиента";
    Сообщение.Поле = "Объект.Клиент";
    Сообщение.Сообщить();
```

КонецЕсли

Иначе

```
    Сообщение = Новый СообщениеПользователю();
    Сообщение.Текст = "Клиент не выбран!";
    Сообщение.Поле = "Объект.Клиент";
    Сообщение.Сообщить();
```

КонецЕсли

КонецПроцедуры

В обработчиках обеих команд, в случае если поле заказа Клиент заполнено, вызывается серверная внеконтекстная функция ПолучитьТелефон() для получения телефона клиента (листинг 2.27).

Листинг 2.27. Функция для получения телефона клиента

```
&НаСервереБезКонтекста  
Функция ПолучитьТелефон(Клиент)
```

```
    Возврат Клиент.Телефон;
```

```
КонецФункции
```

Если телефон клиента заполнен, то выполняется звонок или отправка сообщения клиенту с помощью объекта глобального контекста СредстваТелефонии, который предоставляет доступ к средствам телефонии на мобильном устройстве.

Для звонка клиенту в процедуре Позвонить() используется метод средств телефонии НабратьНомер, в который передается телефон клиента. Поскольку мы передаем во втором параметре метода Ложь, при выполнении метода будет открыто приложение работы со звонками, в котором уже установлен заданный номер. Курьеру нужно будет только нажать кнопку начала вызова.

Для отправки СМС клиенту в процедуре ОтправитьСМС() используется метод средств телефонии ПослатьSMS. Сначала создается объект SMSСообщение, затем в свойство Получатели этого объекта добавляется телефон клиента. После этого вызывается метод средств телефонии ПослатьSMS, в который первым параметром передается объект SMSСообщение, а вторым Истина. При этом будет запущено системное приложение для отправки SMS-сообщений. Курьеру остается ввести текст сообщения и нажать кнопку отправки.

Следует иметь в виду, что объекты СредстваТелефонии и SMSСообщение доступны только в мобильном приложении. Поэтому при синтаксической проверке текста модуля формы может возникнуть ошибка. Но поскольку мы указали в свойстве конфигурации Назначение использования значение Мобильное устройство, то параметры проверки модулей (Сервис > Параметры > Модули > Проверка) уже стандартно включены только для мобильного приложения (рис. 2.57).

При этом надо учитывать, что вы сможете без ошибок сохранять конфигурацию базы данных и обновлять мобильное приложение, но работать это будет только на планшете, так как работа со средствами телефонии (а также с данными мультимедиа, средствами геопозиционирования и средствами почты, которые будут показаны дальше) возможна только на мобильном устройстве.

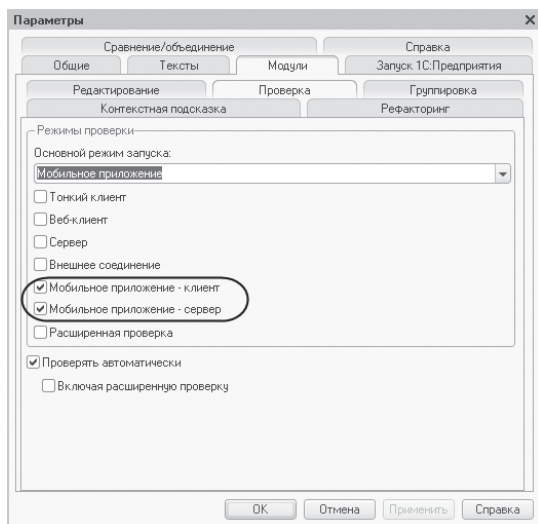


Рис. 2.57. Параметры проверки текстов модулей

Если же вам хочется, чтобы мобильное приложение работало и на стационарном компьютере, то все части кода, где происходит обращение к средствам телефонии и т.п., нужно обрамлять с помощью директив препроцессору, указывающих, что выполнять компиляцию этих частей кода нужно только на мобильном клиенте. Например (листинг 2.28):

Листинг 2.28. Фрагмент обработчика команды «Позвонить»

```
&НаКлиенте
Процедура Позвонить(Команда)
...
    Если ЗначениеЗаполнено(Телефон) Тогда
#Если МобильноеПриложениеКлиент Тогда
        СредстваТелефонии.НабратьНомер(Телефон, Ложь);
#КонецЕсли
    Иначе
        ...
КонецПроцедуры
```

ПРИМЕЧАНИЕ

В тексте листингов этой книги директивы препроцессору отсутствуют, а в выгрузке демонстрационной базы «Курьер Интернет-магазина.dbt» они присутствуют, но в закомментированном виде.

Как уже говорилось, возможности средств телефонии поддерживаются не на всех мобильных устройствах. Поэтому при открытии формы заказа установим доступность команд для работы со средствами телефонии только в том случае, если мобильное устройство поддерживает такие возможности. Для этого создадим обработчик события формы ПриОткрытии и заполним его следующим образом (листинг 2.29).

Листинг 2.29. Обработчик события формы «ПриОткрытии»

```
&НаКлиенте  
Процедура ПриОткрытии(Отказ)
```

```
Элементы.ФормаПозвонить.Доступность = СредстваТелефонии.ПоддерживаетсяНаборНомера();  
Элементы.ФормаОтправитьСМС.Доступность = СредстваТелефонии.ПоддерживаетсяОтправкаSMS(Истина);
```

```
КонецПроцедуры
```

В заключение установим в свойстве Требуемые разрешения мобильного приложения необходимые разрешения для работы со средствами телефонии (а также средствами мультимедиа, геопозиционирования, Push- и локальными уведомлениями, которые мы рассмотрим позже) на мобильном устройстве (рис. 2.58).

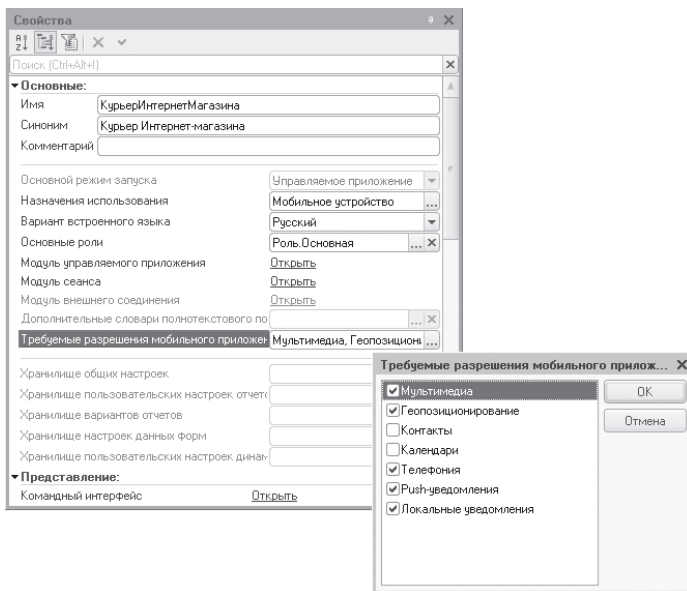


Рис. 2.58. Требуемые разрешения мобильного приложения

Дело в том, что мобильная платформа разработчика уже содержит все необходимые разрешения. Но когда мы будем собирать (см. гл. 5 «Сборка мобильного приложения» на стр. 351) готовое мобильное приложение, эти разрешения необходимы, чтобы вся указанная в них специфика мобильных устройств могла работать и без мобильной платформы разработчика.

Ввод на основании

Теперь предоставим курьеру еще одну удобную возможность — ввода новых заказов на основании отдельных товаров и клиентов.

Для этого в окне редактирования объекта конфигурации Документ Заказ на закладке Ввод на основании добавим справочники Товары и Клиенты в список Вводится на основании (рис. 2.59).

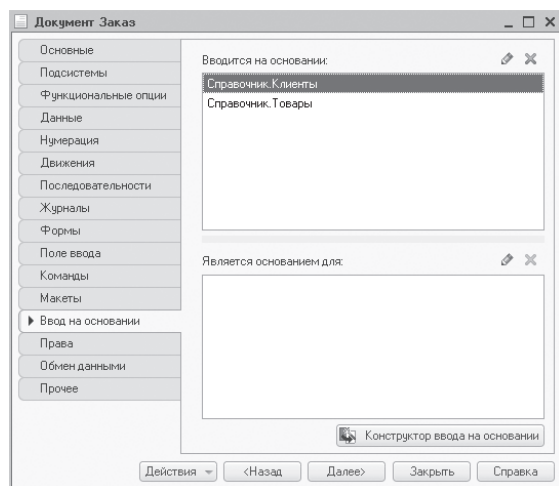


Рис. 2.59. Список объектов конфигурации, на основании которых вводится заказ

Это значит, что новый заказ можно будет ввести на основании элементов справочников товаров или клиентов. После этого в форме списка (в контекстном меню строки списка) и в форме элемента этих справочников станет доступна команда Создать на основании > Заказ. При выполнении этой команды нам нужно обеспечить следующее заполнение заказа:

- Если заказ вводится на основании клиента, ссылка на этого клиента должна попасть в поле Клиент нового заказа, а поле Адрес доставки должно быть заполнено исходя из адресных реквизитов клиента.

- Если заказ вводится на основании товара, то в табличную часть нового документа должна быть добавлена строка, содержащая ссылку на этот товар. Причем у добавленного товара должна быть получена актуальная цена из регистра сведений и подставлена в строку табличной части (вместе с одинарным количеством заказанного товара). С учетом цены и количества товара должна быть посчитана сумма по товару, на основании которого вводился заказ.

Чтобы обеспечить требуемое поведение мобильного приложения, мы должны прописать данный алгоритм в обработчике события `ОбработкаЗаполнения`, которое описывается в модуле документа. Этот обработчик вызывается при вводе документа на основании, при выполнении метода `Заполнить()`, а также при интерактивном вводе нового документа.

Итак, откроем модуль документа `Заказ` и поместим в него следующую процедуру (листинг 2.30).

Листинг 2.30. Обработчик события документа «ОбработкаЗаполнения»

Процедура `ОбработкаЗаполнения`(`ДанныеЗаполнения`, `ТекстЗаполнения`, `СтандартнаяОбработка`)

```
Если ТипЗнч(ДанныеЗаполнения) = Тип("СправочникСсылка.Клиенты") Тогда
    // Заполнение шапки
    Клиент = ДанныеЗаполнения.Ссылка;
    АдресДоставки = СокрЛП(Клиент.Страна) + ", " + СокрЛП(Клиент.Город) + ", " +
        СокрЛП(Клиент.Улица) + ", " +
        СокрЛП(Клиент.Дом) + ", " + СокрЛП(Клиент.Квартира);

ИначеЕсли ТипЗнч(ДанныеЗаполнения) = Тип("СправочникСсылка.Товары") Тогда
    // Заполнение табличной части
    НоваяСтрока = Товары.Добавить();
    НоваяСтрока.Товар = ДанныеЗаполнения.Ссылка;
    НоваяСтрока.Количество = 1;

    ЦенаТовара = РегистрыСведений.ЦеныТоваров.ПолучитьПоследнее(Дата,
        Новый Структура("Товар", ДанныеЗаполнения.Ссылка));
    НоваяСтрока.Цена = ЦенаТовара.Цена;
    НоваяСтрока.Сумма = НоваяСтрока.Цена;
КонецЕсли;
```

КонецПроцедуры

В параметре `ДанныеЗаполнения` этого обработчика нам доступен объект-основание. В зависимости от типа этого объекта мы реализуем нужный алгоритм действий. Для получения актуальной цены товара и адреса доставки клиента здесь используются фрагменты функций `ПолучитьЦенуТовара()` и `ПолучитьАдресДоставки()`, которые рассматривались ранее.

Также при создании нового заказа автоматически установим дату доставки на следующий день после даты документа. Для этого откроем обработчик события формы ПриОткрытии и изменим его следующим образом (листинг 2.31).

Листинг 2.31. Обработчик события формы «ПриОткрытии»

```
&НаКлиенте  
Процедура ПриОткрытии(Отказ)  
  
    Элементы.ФормаПозвонить.Доступность = СредстваТелефонии.ПоддерживаетсяНаборНомера();  
    Элементы.ФормаОтправитьСМС.Доступность  
        = СредстваТелефонии.ПоддерживаетсяОтправкаSMS(Истина);  
  
    Если Параметры.Ключ.Пустая() Тогда  
        Объект.ДатаДоставки = Объект.Дата + 24*60*60;  
    КонецЕсли;  
  
КонецПроцедуры
```

При открытии формы в параметре формы Ключ мы определяем, что создается новый документ, если ссылка на создаваемый объект пустая. В этом случае мы устанавливаем дату доставки следующей датой после даты документа.

Для примера рассмотрим следующую ситуацию. Курьер открывает форму товара. Из меню функций в правом верхнем углу формы курьер может вызвать команду Создать на основании, выбрать из подменю команду Заказ и создать новый заказ на основании открытого товара.

В список товаров нового заказа будет добавлена строка, содержащая данный товар, его цену, одинарное количество и посчитанную сумму по товару. А также будет заполнена дата доставки заказа – следующим числом после даты создания заказа. Кроме того, статус заказа будет установлен в значение Открыт перечисления СтатусыЗаказов в соответствии со свойством Значение заполнения для одноименного реквизита документа (рис. 2.60).

Или же, например, в форме списка клиентов курьер может вызвать контекстное меню у конкретного клиента. Для этого ему нужно выполнить жест пролистывания с правой стороны экрана у нужной строки списка клиентов, вызвать команду Заказ: создать на основании. В результате будет создан новый заказ на основании текущего клиента.

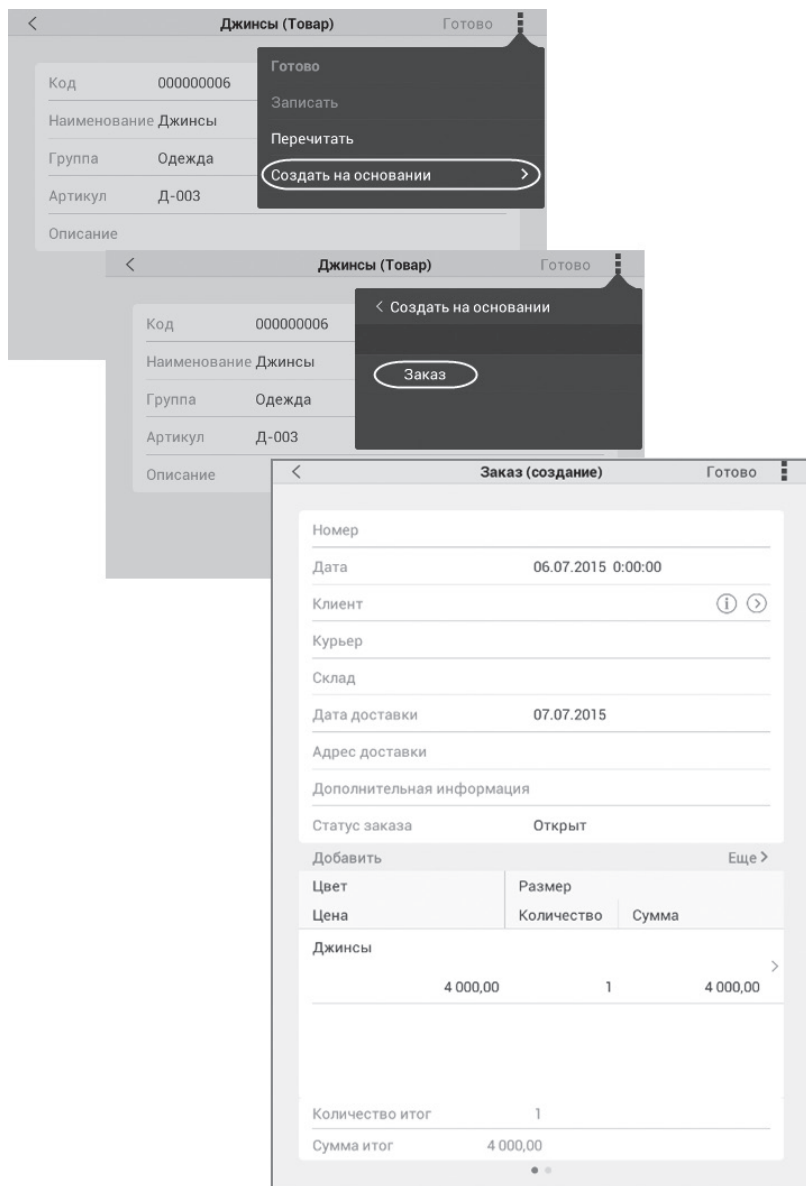


Рис. 2.60. Создание заказа на основании товара

В поле Клиент нового заказа попадет ссылка на клиента, на основании которого создается заказ, а также будет заполнено поле Адрес доставки исходя из адресных реквизитов клиента. Дата доставки заказа будет заполнена следующим числом после даты создания заказа. Кроме того, статус заказа будет установлен в значение Открыт перечисления СтатусыЗаказов в соответствии со свойством Значение заполнения для одноименного реквизита документа (рис. 2.61).

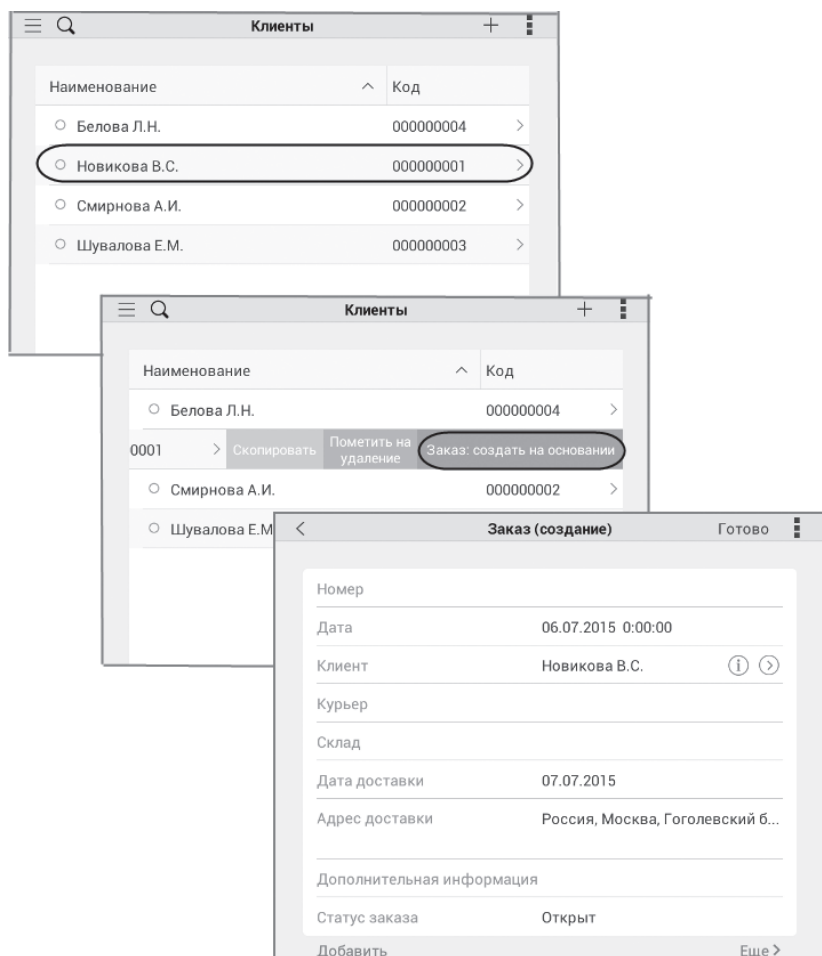


Рис. 2.61. Создание заказа на основании клиента

Редактирование уже существующего заказа

Сделаем еще одно небольшое, но важное изменение в форме заказа – ограничим возможность изменения информации в заказе, который находится в работе у курьера, так, чтобы курьер мог вводить только информацию об отказе от товаров и изменять статус заказа. А если заказ уже закрыт или выполнен, нужно, чтобы курьер мог только его просматривать.

Для этого создадим обработчик события формы ПриЧтенииНаСервере, который выполняется только для уже существующих в информационной базе объектов, и заполним его следующим образом (листинг 2.32).

Листинг 2.32. Обработчик события формы документа «ПриЧтенииНаСервере»

```
&НаСервере
Процедура ПриЧтенииНаСервере(ТекущийОбъект)

    Если ТекущийОбъект.СтатусЗаказа = Перечисления.СтатусыЗаказов.Закрыт ИЛИ
        ТекущийОбъект.СтатусЗаказа = Перечисления.СтатусыЗаказов.Выполнен Тогда
        ЭтаФорма.ТолькоПросмотр = Истина;
    ИначеЕсли ТекущийОбъект.СтатусЗаказа = Перечисления.СтатусыЗаказов.ВРаботе Тогда
        Элементы.Номер.ТолькоПросмотр = Истина;
        Элементы.Дата.ТолькоПросмотр = Истина;
        Элементы.ДатаДоставки.ТолькоПросмотр = Истина;
        Элементы.АдресДоставки.ТолькоПросмотр = Истина;
        Элементы.Клиент.ТолькоПросмотр = Истина;
        Элементы.Курьер.ТолькоПросмотр = Истина;
        Элементы.Склад.ТолькоПросмотр = Истина;
        Элементы.ДополнительнаяИнформация.ТолькоПросмотр = Истина;
        Элементы.Товары.ТолькоПросмотр = Истина;
    КонецЕсли;

КонецПроцедуры
```

При создании новых заказов данный обработчик вызываться не будет, и новые заказы курьер сможет редактировать, как и ранее созданные заказы со статусом Открыт.

Оповещение об изменении данных

По условию нашей задачи нужно, чтобы после изменения данных в заказе форма оповещала бы об этом мобильное приложение. Для этого создадим клиентский обработчик события формы ПослеЗаписи и заполним его следующим образом (листинг 2.33).

Листинг 2.33. Обработчик события формы документа «ПослеЗаписи»

```
&НаКлиенте
Процедура ПослеЗаписи(ПараметрыЗаписи)

    Если ПараметрыЗаписи.РежимЗаписи = РежимЗаписиДокумента.Проведение Тогда
```

Оповестить("ИзменилсяЗаказ");
КонецЕсли;

КонецПроцедуры

В этом обработчике, если заказ записывается с проведением, то об этом оповещается мобильное приложение.

Позднее, при реализации регулярного обмена данными с основным приложением (см. раздел «Автоматическая синхронизация данных» на стр. 326), мы поместим в мобильное приложение обработчик оповещения о том, что данные объектов изменились и надо выполнить обмен.

Работа со средствами мультимедиа

Поскольку мобильные устройства обладают различными мультимедийными возможностями, то было бы удобно, чтобы курьер мог воспользоваться этими возможностями прямо в заказе. Например, мог бы сделать фотоснимки товаров, не понравившихся клиенту, или видео/аудиозапись с отзывом клиента о заказе.

Справочник хранимых файлов

Для этого нам понадобится справочник, в котором будут храниться файлы мультимедиа, сделанные курьером. Создадим в нашей мобильной конфигурации справочник с именем `ХранимыеФайлы`, представлением объекта `Файл` и представлением списка `Файлы`.

Для хранения имени файла и собственно мультимедийного содержимого создадим у этого справочника два реквизита: `ИмяФайла` (типа `Строка` неограниченной длины) и `ДанныеФайла` (типа `ХранилищеЗначения`). А также увеличим длину основного реквизита справочника `Наименование` до 50 символов.

Кроме фотоснимков, видео- и аудиозаписей, сделанных курьером, в этом справочнике будут храниться также картинки товаров, которые будут записываться в основном приложении и в процессе обмена данными передаваться на планшет. В следующей главе (см. раздел «Хранимые файлы» на стр. 192) мы создадим такой же справочник в основном приложении и покажем возможность выбора картинки товара и ее сохранения в справочнике `ХранимыеФайлы`.

Пока в мобильном приложении предусмотрим, что у справочника `ХранимыеФайлы` могут быть два владельца: справочник `Товары` (для просмотра картинок, относящихся к конкретному товару) и справочник `Клиенты` (для просмотра файлов, сделанных курьером у конкретного клиента), см. рис. 2.62.

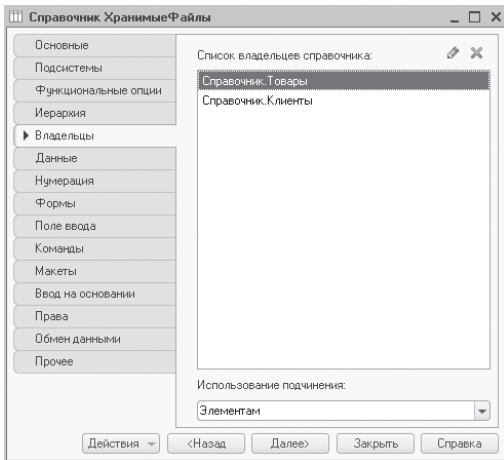


Рис. 2.62. Список владельцев справочника «ХранимыеФайлы»

Чтобы выполнять обмен данными справочника между планшетом и основным приложением, на закладке Обмен данными окна редактирования справочника ХранимыеФайлы включим его в план обмена Мобильные (рис. 2.63).

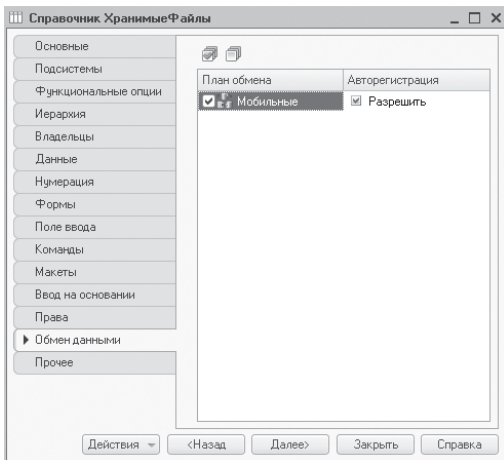


Рис. 2.63. Принадлежность справочника к плану обмена «Мобильные»

Фото-, видео- и аудиосъемка

Чтобы реализовать мультимедийные возможности, в форме заказа на закладке Команды добавим команды СделатьФотоснимок, СделатьВидеозапись и СделатьАудиоЗапись и перетащим их в командную панель формы.

Сделаем так, чтобы команды для работы со средствами мультимедиа были видны только в меню функций, вызываемом кнопкой  в правом верхнем углу заказа. Для этого установим в палитре свойств соответствующих кнопок командной панели свойство ТолькоВоВсехДействиях в значение Да.

В результате форма заказа в редакторе формы примет следующий вид (рис. 2.64).

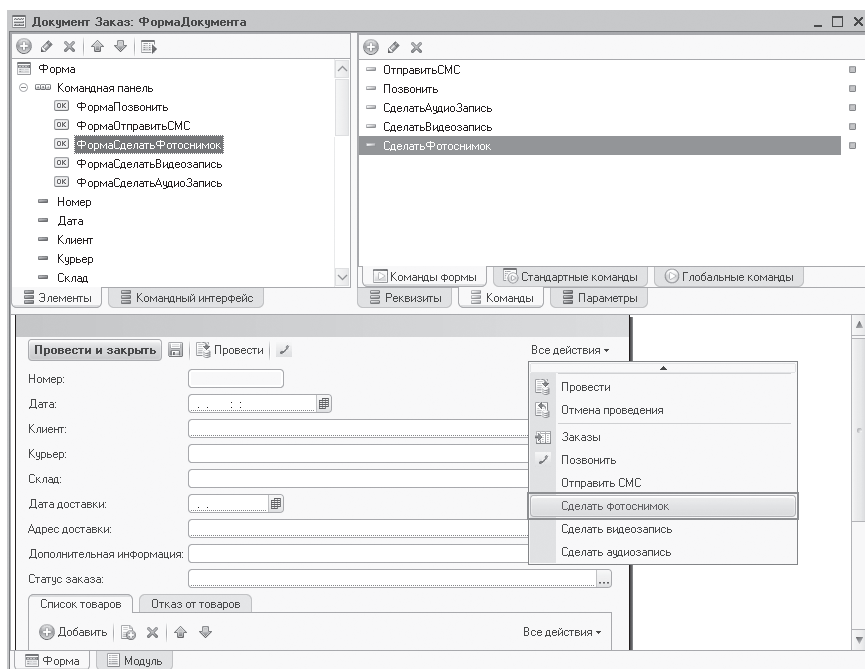


Рис. 2.64. Форма заказа в редакторе формы

Затем создадим клиентские обработчики для команд СделатьФотоснимок, СделатьВидеозапись и СделатьАудиоЗапись и заполним их следующим образом (листинги 2.34, 2.35, 2.36).

Листинг 2.34. Обработчик команды «СделатьФотоснимок»

```
&НаКлиенте
Процедура СделатьФотоснимок(Команда)

    Если Объект.Ссылка.Пустая() Тогда
        ПоказатьПредупреждение(, "Данные не записаны!");
        Возврат;
    КонецЕсли;

    ДанныеМультимедиа = СредстваМультимедиа.СделатьФотоснимок();
    Если ДанныеМультимедиа <> Неопределено Тогда
        СоздатьНовыйФайл(ДанныеМультимедиа.ПолучитьДвоичныеДанные(),
            ДанныеМультимедиа.РасширениеФайла, ДанныеМультимедиа.ТипСодержимого);
    КонецЕсли;

КонецПроцедуры
```

Листинг 2.35. Обработчик команды «СделатьВидеозапись»

```
&НаКлиенте
Процедура СделатьВидеозапись(Команда)

    Если Объект.Ссылка.Пустая() Тогда
        ПоказатьПредупреждение(, "Данные не записаны!");
        Возврат;
    КонецЕсли;

    ДанныеМультимедиа = СредстваМультимедиа.СделатьВидеозапись();
    Если ДанныеМультимедиа <> Неопределено Тогда
        СоздатьНовыйФайл(ДанныеМультимедиа.ПолучитьДвоичныеДанные(),
            ДанныеМультимедиа.РасширениеФайла, ДанныеМультимедиа.ТипСодержимого);
    КонецЕсли;

КонецПроцедуры
```

Листинг 2.36. Обработчик команды «СделатьАудиозапись»

```
&НаКлиенте
Процедура СделатьАудиозапись(Команда)

    Если Объект.Ссылка.Пустая() Тогда
        ПоказатьПредупреждение(, "Данные не записаны!");
        Возврат;
    КонецЕсли;

    ДанныеМультимедиа = СредстваМультимедиа.СделатьАудиозапись();
    Если ДанныеМультимедиа <> Неопределено Тогда
        СоздатьНовыйФайл(ДанныеМультимедиа.ПолучитьДвоичныеДанные(),
            ДанныеМультимедиа.РасширениеФайла, ДанныеМультимедиа.ТипСодержимого);
    КонецЕсли;

КонецПроцедуры
```

В этих обработчиках сначала мы должны убедиться, что данные заказа записаны, так как владельцем данных мультимедиа будет клиент, указанный в заказе.

Затем мы выполняем требуемый метод (СделатьФотоснимок(), СделатьВидеозапись(), СделатьАудиозапись()) средств мультимедиа, доступ к которым мы получаем с помощью свойства глобального контекста СредстваМультимедиа.

В результате выполнения этих методов мы получаем объект ДанныеМультимедиа. Из этого объекта мы получаем содержимое данных мультимедиа в виде двоичных данных, расширение при сохранении этих данных в файл, тип содержимого и передаем в серверную процедуру СоздатьНовыйФайл() для сохранения данных мультимедиа в справочнике ХранимыеФайлы (листинг 2.37).

Листинг 2.37. Процедура для сохранения данных мультимедиа
в справочнике «ХранимыеФайлы»

```
&НаСервере
Процедура СоздатьНовыйФайл(Данные, Расширение, Тип)

    ТипСодержимого = Тип;
    Номер = Найти(ТипСодержимого, "");
    Если Номер > 0 Тогда
        ТипСодержимого = Лев(ТипСодержимого, Номер - 1);
    КонецЕсли;
    Файл = Новый Файл(СтрЗаменить(Строка(ТекущаяДата()), ":", "_") + "." + Расширение);

    ХранимыйФайл = Справочники.ХранимыеФайлы.СоздатьЭлемент();
    ХранимыйФайл.Владелец = Объект.Клиент;
    ХранимыйФайл.Наименование = "Заказ № " + Объект.Номер + " " + ТипСодержимого + " " +
        Строка(ТекущаяДата());
    ХранимыйФайл.ИмяФайла = Файл.Имя;
    ХранимыйФайл.ДанныеФайла = Новый ХранилищеЗначения(Данные, Новый СжатиеДанных());
    ХранимыйФайл.Записать();

КонецПроцедуры
```

В данной процедуре мы создаем объект Файл с именем, образованным из текущей даты и расширения файла (jpg/mp4/3gp), переданного в процедуру. Затем добавляем новую запись в справочник ХранимыеФайлы. Имя файла записываем в соответствующий реквизит справочника ИмяФайла, а данные мультимедиа помещаем в хранилище значения и сохраняем в реквизите ДанныеФайла.

Стандартному реквизиту справочника Владелец присваиваем значение ссылки на клиента (Объект.Клиент). Таким образом, конкретные файлы мультимедиа будут относиться к клиенту, указанному в заказе. Реквизиту Наименование присваиваем строку, состоящую из номера заказа, типа содержимого мультимедиа (image/video/audio) и текущей даты.

После этого записываем созданный элемент справочника `ХранимыеФайлы`. При этом надо понимать, что сам файл физически на планшете не создается, но данные мультимедиа в двоичном виде и имя, связанное с этими данными, сохраняются в справочнике хранимых файлов.

При открытии формы заказа установим видимость команд для работы со средствами мультимедиа только в том случае, если мобильное устройство поддерживает такие возможности. Для этого добавим в обработчик события формы `ПриОткрытии` следующий фрагмент текста (листинг 2.38).

Листинг 2.38. Фрагмент обработчика события формы «`ПриОткрытии`»

```
&НаКлиенте
Процедура ПриОткрытии(Отказ)
...
    Элементы.ФормаСделатьАудиозапись.Доступность =
        СредстваМультимедиа.ПоддерживаетсяАудиозапись();
    Элементы.ФормаСделатьВидеозапись.Доступность =
        СредстваМультимедиа.ПоддерживаетсяВидеозапись();
    Элементы.ФормаСделатьФотоснимок.Доступность =
        СредстваМультимедиа.ПоддерживаетсяФотоснимок();
...
КонецПроцедуры
```

В завершение откроем командный интерфейс нашего мобильного приложения и перетащим команду для открытия справочника `ХранимыеФайлы` (эта команда автоматически добавилась в командный интерфейс после создания справочника) из группы `Панель навигации.Обычное` в группу `Панель навигации`. См. также (рис. 2.65).

Таким образом, мы увидим команду для открытия списка этого справочника только в главном меню приложения, что нам и нужно. Поскольку вызывать этот список отдельно, без привязки к конкретным товарам и клиентам, вряд ли потребуется, данная команда не будет отображаться в списке команд основного раздела на мобильном устройстве.

Проверим работу со средствами мультимедиа на планшете. Выполним из меню функций заказа команды `СделатьФотоснимок`, `СделатьВидеозапись` и `СделатьАудиоЗапись`.

Затем из главного меню приложения с помощью значка  в левом верхнем углу приложения (или жеста пролистывания с левой стороны экрана) вызовем список хранимых файлов, выполнив команду `Файлы` из группы команд `См. также`.

В результате в списке хранимых файлов появятся три новые записи (рис. 2.66).

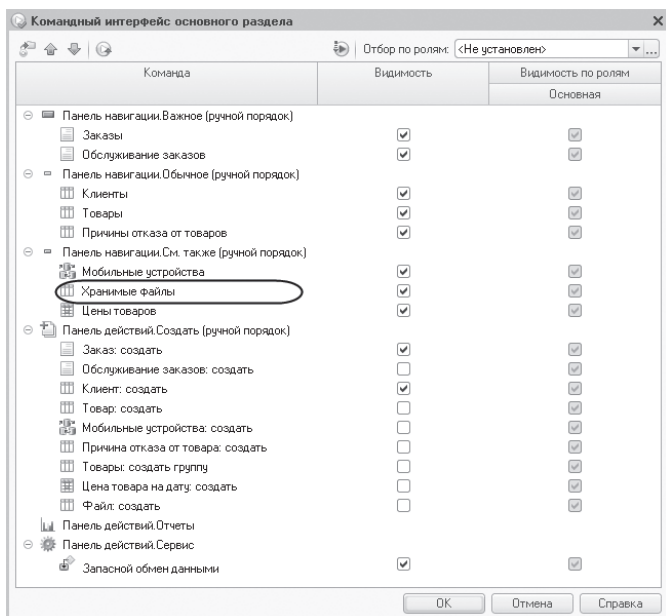


Рис. 2.65. Командный интерфейс основного раздела

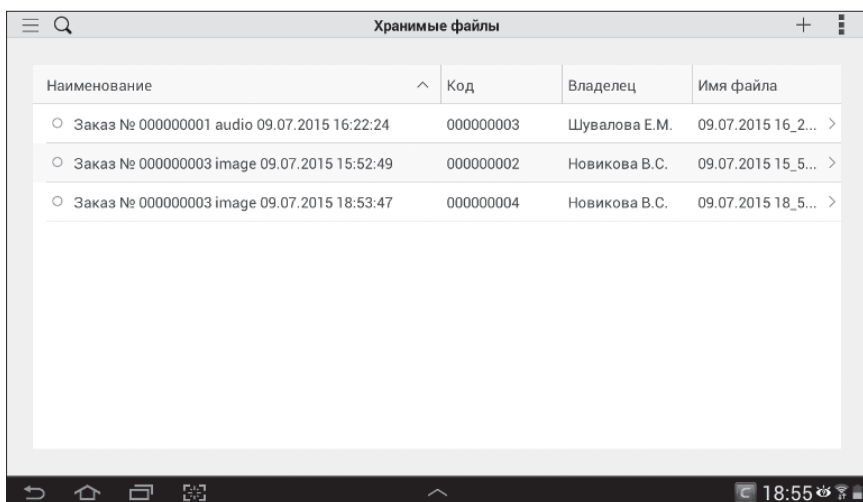


Рис. 2.66. Список хранимых файлов

Как мы видим, форма списка файлов нуждается в доработке. Внешне – нужно минимизировать состав ее полей, а функционально – нужно, чтобы при выборе строки списка открывалась бы не форма элемента справочника хранимых файлов, а соответствующий файл мультимедиа.

Этим мы и займемся в следующем разделе, а также создадим форму элемента справочника хранимых файлов и поместим в ней команды для создания и открытия файлов мультимедиа.

Хранимые файлы

Справочник ХранимыеФайлы будет редактируемым, так как курьер может интерактивно создавать и добавлять в состав хранимых файлов файлы мультимедиа. Поэтому для роли Основная, с помощью которой определяются права доступа к данным в нашем мобильном приложении, установим все права для этого справочника, кроме права Интерактивное удаление (рис. 2.67).

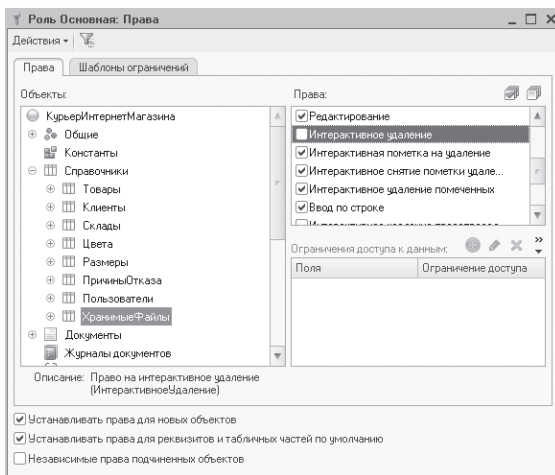


Рис. 2.67. Установка прав для роли «Основная»

Список файлов

Создадим форму списка справочника ХранимыеФайлы и ограничим состав полей списка двумя полями: Код и Наименование (именно эти поля и добавляет стандартно в форму списка редактор форм). А также удалим группу СписокКомпоновщикНастроекПользовательскиеНастройки из дерева элементов формы.

Чтобы курьер мог открыть файл мультимедиа, выделенный в списке, на закладке Команды добавим в форму списка команду ОткрытьФайл и перетащим ее в командную панель формы. В палитре свойств этой команды свойство Использование текущей строки установим в значение Использует и в свойстве Используемая таблица выберем таблицу динамического списка Список (рис. 2.68).

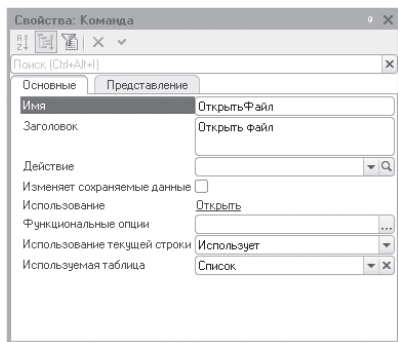


Рис. 2.68. Свойства команды «ОткрытьФайл»

С помощью этих свойств мы показываем, что команда будет использовать данные текущей строки таблицы Список. Дело в том, что данные текущей строки таблицы в мобильном приложении доступны только во время обработчика контекстной команды, событий активизации строки или ячейки или событий редактирования строки. А эти данные нам нужны для получения ссылки на хранимый файл, который мы хотим открыть.

Затем создадим клиентский обработчик этой команды и заполним его следующим образом (листинг 2.39).

Листинг 2.39. Обработчик команды «ОткрытьФайл»

```
&НаКлиенте
Процедура ОткрытьФайл(Команда)

    ХранимыйФайл = Элементы.Список.ТекущиеДанные;
    ОткрытьХранимыйФайл(ХранимыйФайл);

КонецПроцедуры
```

В этом обработчике мы используем таблицу формы Список, которая отображает данные динамического списка, полученного на основе справочника ХранимыеФайлы. Используя свойство ТекущиеДанные этой таблицы, в переменной ХранимыйФайл мы получаем объект, содержащий данные, находя-

щиеся в текущей строке таблицы. Этот объект мы передаем в клиентскую процедуру ОткрытьХранимыйФайл(), где, собственно, и происходит открытие файла мультимедиа соответствующим ему приложением (листинг 2.40).

Листинг 2.40. Процедура для открытия файла мультимедиа

&НаКлиенте

Процедура ОткрытьХранимыйФайл(ХранимыйФайл)

```
Файл = Новый Файл(ХранимыйФайл.ИмяФайла);  
ИмяФайла = ПолучитьИмяВременногоФайла(Файл.Расширение);  
Адрес = ПолучитьНавигационнуюСсылку(ХранимыйФайл.Ссылка, "ДанныеФайла");  
ПолучитьФайл(Адрес, ИмяФайла, Ложь);  
ЗапуститьПриложение(ИмяФайла);
```

КонецПроцедуры

В данной процедуре сначала мы создаем объект Файл на основе значения реквизита ИмяФайла элемента справочника хранимых файлов, переданного в процедуру в параметре ХранимыйФайл.

Затем, для того чтобы записать данные мультимедиа во временный файл, методом глобального контекста ПолучитьИмяВременногоФайла() мы получаем уникальное имя временного файла с желаемым расширением, полученным из созданного нами объекта Файл.

Затем с помощью метода ПолучитьНавигационнуюСсылку() переменной Адрес мы присваиваем навигационную ссылку на реквизит ДанныеФайла объекта, переданного в процедуру в параметре ХранимыйФайл. В первом параметре метода ПолучитьНавигационнуюСсылку() передается ссылка на объект – ХранимыйФайл.Ссылка, а во втором – имя реквизита.

Далее с помощью метода ПолучитьФайл() на основе навигационной ссылки, сохраненной в переменной Адрес, мы получаем файл с данными мультимедиа и сохраняем его на планшете под именем временного файла, записанным в переменной ИмяФайла.

И после этого с помощью метода глобального контекста ЗапуститьПриложение() выполняется открытие этого временного файла с помощью ассоциированного с ним приложения (например, при открытии видео запускается видеоплеер и т.п.).

Для того чтобы в процедуре ОткрытьХранимыйФайл() нам были доступны реквизиты ИмяФайла и Ссылка справочника ХранимыеФайлы, нужно установить флажок Использовать всегда для этих полей в свойствах основного реквизита формы Список (рис. 2.69).

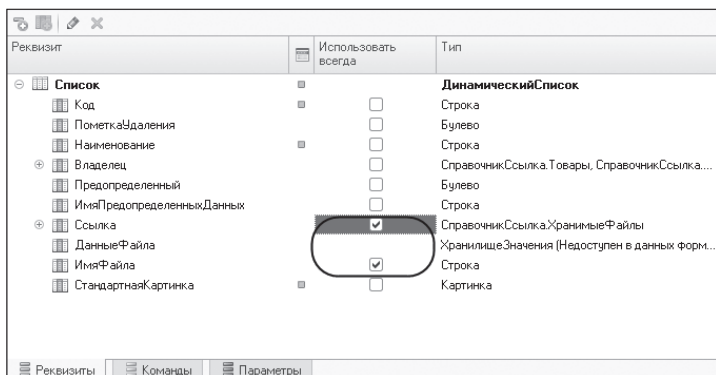


Рис. 2.69. Список полей динамического списка

Дело в том, что по умолчанию в форме на клиенте доступны данные только тех полей динамического списка, которые отображаются в форме. Но при установке флажка **Использовать всегда** данные этих полей будут передаваться на клиента даже несмотря на то, что в форме они нигде не отображаются.

Таким образом, с помощью жеста пролистывания с правой стороны экрана нужной строки списка хранимых файлов курсор может вызвать контекстное меню этого файла и выполнить команду **Открыть файл**. При этом текущий файл с данными мультимедиа будет открыт ассоциированным с ним приложением (рис. 2.70).

Теперь сделаем так, чтобы при выборе строки списка открывалась бы не форма элемента справочника хранимых файлов, как происходит по умолчанию, а соответствующий файл мультимедиа.

Для этого создадим обработчик события **Выбор** для таблицы формы **Список** и заполним его следующим образом (листинг 2.41).

Листинг 2.41. Обработчик события «Выбор» для таблицы «Список»

```
&НаКлиенте
Процедура СписокВыбор(Элемент, ВыбраннаяСтрока, Поле, СтандартнаяОбработка)

    СтандартнаяОбработка = Ложь;
    ОткрытьХранимыйФайл(Элемент.ТекущиеДанные);

КонецПроцедуры
```

В этом обработчике мы отменяем стандартную обработку события, при котором открывается форма элемента справочника, выбранного в списке.



Рис. 2.70. Открытие файла из списка хранимых файлов

В параметре обработчика Элемент нам доступна таблица формы Список, которая отображает данные динамического списка, полученного на основе справочника ХранимыеФайлы. Используя свойство ТекущиеДанные этой таблицы, мы получаем объект, содержащий данные, находящиеся в текущей строке таблицы. Этот объект мы передаем в клиентскую процедуру ОткрытьХранимыйФайл(), где, собственно, и происходит открытие файла мультимедиа соответствующим ему приложением (см. листинг 2.40).

При разработке справочника ХранимыеФайлы мы указали, что этот справочник будет подчинен справочникам Товары и Клиенты. При создании файлов мультимедиа в форме заказа мы устанавливали в качестве владельца этих файлов клиента, указанного в заказе (см. листинг 2.37).

Таким образом, открыв форму клиента и активизировав закладку Хранимые файлы в панели навигации внизу формы, курьер может просмотреть файлы, связанные с этим клиентом (рис. 2.71).

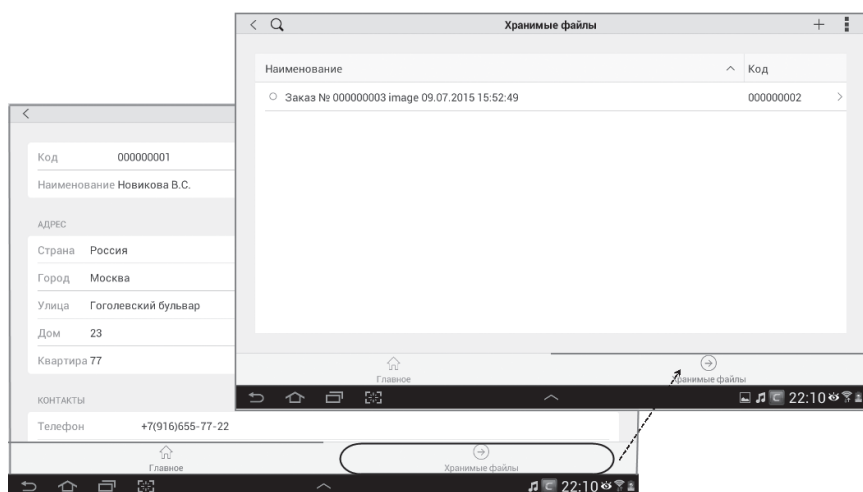


Рис. 2.71. Список хранимых файлов, связанных с конкретным клиентом

Создание и редактирование файлов мультимедиа

Создадим форму элемента справочника ХранимыеФайлы и сделаем так, чтобы курьер мог создавать новые и редактировать существующие файлы мультимедиа в этой форме, а также открывать эти файлы соответствующим приложением.

Для этого на закладке Команды добавим команды СделатьФотоснимкиИЗаписать, СделатьВидеозаписьИЗаписать, СделатьАудиоЗаписиИЗаписать и ОткрытьФайл и перетащим их в форму.

В результате форма элемента справочника в редакторе формы примет следующий вид (рис. 2.72).

Затем создадим клиентский обработчик для команды ОткрытьФайл и заполним его следующим образом (листинг 2.42).

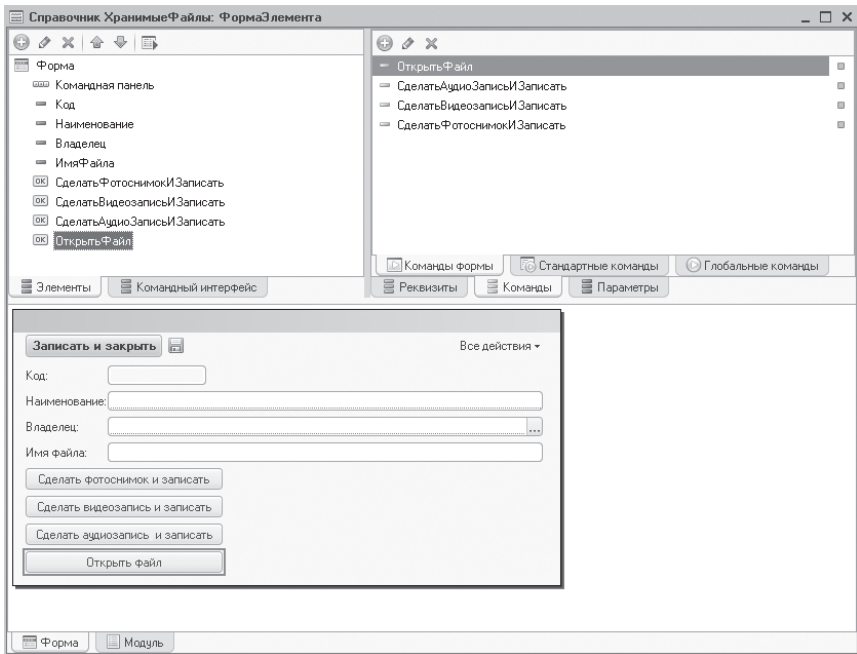


Рис. 2.72. Форма хранимого файла в редакторе формы

Листинг 2.42. Обработчик команды «ОткрытьФайл»

&НаКлиенте

Процедура ОткрытьФайл(Команда)

```

Если Объект.Ссылка.Пустая() Тогда
    ПоказатьПредупреждение(, "Данные не записаны!");
    Возврат;
КонецЕсли;

Если ПустаяСтрока(Объект.ИмяФайла) Тогда
    ПоказатьПредупреждение(, "Имя не задано!");
    Возврат;
КонецЕсли;

Файл = Новый Файл(Объект.ИмяФайла);
ИмяФайла = ПолучитьИмяВременногоФайла(Файл.Расширение);
Адрес = ПолучитьНавигационнуюСсылку(Объект.Ссылка, "ДанныеФайла");
ПолучитьФайл(Адрес, ИмяФайла, Ложь);
ЗапуститьПриложение(ИмяФайла);
    
```

КонецПроцедуры

В данном обработчике мы выполняем открытие файла мультимедиа только в случае, если данные объекта записаны и имя файла не пустое. Дальнейшие действия аналогичны действиям при открытии файла мультимедиа в форме списка хранимых файлов (см. листинг 2.40).

Сначала мы создаем объект Файл на основе имени файла основного реквизита формы (Объект.ИмяФайла).

Затем, для того чтобы записать данные мультимедиа во временный файл, методом глобального контекста ПолучитьИмяВременногоФайла() мы получаем уникальное имя временного файла с желаемым расширением, полученным из созданного нами объекта Файл.

Затем с помощью метода ПолучитьНавигационнуюСсылку() переменной Адрес мы присваиваем навигационную ссылку на реквизит объекта ДанныеФайла. В первом параметре метода ПолучитьНавигационнуюСсылку() передается ссылка на объект – Объект.Ссылка, а во втором – имя реквизита.

Далее с помощью метода ПолучитьФайл() на основе навигационной ссылки, сохраненной в переменной Адрес, мы получаем файл с данными мультимедиа и сохраняем его на планшете под именем временного файла, записанным в переменной ИмяФайла.

И после этого с помощью метода глобального контекста ЗапуститьПриложение() выполняется открытие этого временного файла с помощью ассоциированного с ним приложения.

Теперь создадим клиентские обработчики для команд СделатьФотоснимокИЗаписать, СделатьВидеозаписьИЗаписать и СделатьАудиоЗаписьИЗаписать и заполним их следующим образом (листинги 2.43, 2.44, 2.45).

Листинг 2.43. Обработчик команды «СделатьФотоснимок»

```
&НаКлиенте  
Процедура СделатьФотоснимок(Команда)  
  
    ДанныеМультимедиа = СредстваМультимедиа.СделатьФотоснимок();  
    ПоместитьМультимедиа(ДанныеМультимедиа);  
  
КонецПроцедуры
```

Листинг 2.44. Обработчик команды «СделатьВидеозапись»

```
&НаКлиенте  
Процедура СделатьВидеозапись(Команда)  
  
    ДанныеМультимедиа = СредстваМультимедиа.СделатьВидеозапись();  
    ПоместитьМультимедиа(ДанныеМультимедиа);  
  
КонецПроцедуры
```

Листинг 2.45. Обработчик команды «СделатьАудиозапись»

```
&НаКлиенте
Процедура СделатьАудиозапись(Команда)

    ДанныеМультимедиа = СредстваМультимедиа.СделатьАудиозапись();
    ПоместитьМультимедиа(ДанныеМультимедиа);

КонецПроцедуры
```

В этих обработчиках мы выполняем соответствующий метод (СделатьФотоснимок(), СделатьВидеозапись(), СделатьАудиозапись()) средств мультимедиа, доступ к которым мы получаем с помощью свойства глобального контекста СредстваМультимедиа.

В результате выполнения этих методов мы получаем объект ДанныеМультимедиа. Этот объект мы передаем в клиентскую процедуру ПоместитьМультимедиа() (листинг 2.46).

Листинг 2.46. Процедура «ПоместитьМультимедиа()»

```
&НаКлиенте
Процедура ПоместитьМультимедиа(ДанныеМультимедиа)

    Если ДанныеМультимедиа <> Неопределено Тогда
        АдресВременногоХранилища =
            ПоместитьВоВременноеХранилище(ДанныеМультимедиа.ПолучитьДвоичныеДанные(),
            УникальныйИдентификатор);

        ТипСодержимого = ДанныеМультимедиа.ТипСодержимого;
        Номер = Найти(ТипСодержимого, "");
        Если Номер > 0 Тогда
            ТипСодержимого = Лев(ТипСодержимого, Номер - 1);
        КонецЕсли;
        Объект.Наименование = ТипСодержимого + " " + Строка(ТекущаяДата());
        Объект.ИмяФайла = СтрЗаменить(Строка(ТекущаяДата()), ":", "_") + "..." +
            ДанныеМультимедиа.РасширениеФайла;

        ПоместитьФайлОбъекта(АдресВременногоХранилища);
    КонецЕсли;

КонецПроцедуры
```

В этой процедуре из параметра ДанныеМультимедиа мы получаем содержимое данных мультимедиа в виде двоичных данных и помещаем его во временное хранилище методом глобального контекста ПоместитьВоВременноеХранилище().

Затем получаем тип содержимого данных мультимедиа (image/video/audio) и заполняем наименование объекта (храняемого файла) на основе этого типа и текущей даты. Реквизиту объекта ИмяФайла присваиваем строку,

состоящую из текущей даты и расширения файла (jpg/mp4/3gp), полученного из параметра ДанныеМультимедиа.

Затем вызываем серверную процедуру ПоместитьФайлОбъекта() для сохранения данных мультимедиа в справочнике ХранимыеФайлы и передаем в нее адрес временного хранилища, в которое мы поместили содержимое мультимедиа в виде двоичных данных (листинг 2.47).

Листинг 2.47. Процедура для сохранения данных мультимедиа в справочнике «ХранимыеФайлы»

&НаСервере

Процедура ПоместитьФайлОбъекта(АдресВременногоХранилища)

```
ЭлементСправочника = РеквизитФормыВЗначение("Объект");  
ДвоичныеДанные = ПолучитьИзВременногоХранилища(АдресВременногоХранилища);  
  
ЭлементСправочника.ДанныеФайла = Новый ХранилищеЗначения(ДвоичныеДанные,  
Новый СжатиеДанных());  
  
Файл = Новый Файл(ЭлементСправочника.ИмяФайла);  
ЭлементСправочника.ИмяФайла = Файл.Имя;  
ЭлементСправочника.Записать();  
  
Модифицированность = Ложь;  
УдалитьИзВременногоХранилища(АдресВременногоХранилища);  
ЗначениеВРеквизитФормы(ЭлементСправочника, "Объект");
```

КонецПроцедуры

В данной процедуре мы преобразуем основной реквизит формы в объект – элемент справочника ХранимыеФайлы.

Затем получаем из временного хранилища двоичные данные мультимедиа по адресу временного хранилища, переданному в процедуру. На основе этих данных мы создаем объект ХранилищеЗначения и помещаем его в реквизит объекта ДанныеФайла.

Затем создаем объект Файл на основе значения реквизита элемента справочника ИмяФайла. Этот объект мы создаем для того, чтобы простым способом получить короткое имя файла из его полного имени. Это короткое имя мы записываем в соответствующий реквизит элемента справочника ИмяФайла. После этого сохраняем созданный элемент справочника ХранимыеФайлы.

В заключение удаляем данные мультимедиа из временного хранилища и преобразуем данные объекта обратно в основной реквизит формы Объект. А также свойству Модифицированность формы присваиваем значение Ложь, так как данные элемента справочника уже записаны.

При создании формы элемента справочника хранимых файлов предусмотрим два момента. Если новый элемент справочника создается путем

копирования другого элемента, то нужно очистить имя файла, чтобы не возникало иллюзии, что содержимое файла тоже копируется. А также в качестве владельца нового элемента справочника (если он не определен) нужно установить пустую ссылку на справочник Клиенты.

Если владелец файла – элемент справочника Товары, то форма должна быть открыта только в режиме просмотра, так как курьер на планшете не может редактировать информацию, связанную с товарами.

Для этого создадим обработчик события формы ПриСозданииНаСервере и заполним его следующим образом (листинг 2.48).

Листинг 2.48. Обработчик события формы «ПриСозданииНаСервере»

```
&НаСервере
Процедура ПриСозданииНаСервере(Отказ, СтандартнаяОбработка)

    Если Параметры.Ключ.Пустая() Тогда
        Если НЕ Параметры.ЗначениеКопирования.Пустая() Тогда
            // при копировании очищаем имя файла, чтобы не возникало иллюзии, что содержимое
            // файла тоже копируется
            Объект.ИмяФайла = "";
        КонецЕсли;
        Если Объект.Владелец = Неопределено Тогда
            Объект.Владелец = Справочники.Клиенты.ПустаяСсылка();
        КонецЕсли;
    КонецЕсли;

    Если ТипЗнч(Объект.Владелец) = Тип("СправочникСсылка.Товары") Тогда
        // Товары в этом приложении не редактируются
        ТолькоПросмотр = Истина;
    КонецЕсли;

КонецПроцедуры
```

При открытии формы хранимого файла установим видимость команд для работы со средствами мультимедиа только в том случае, если данные в форме можно редактировать и если мобильное устройство поддерживает мультимедийные возможности. Для этого создадим обработчик события формы ПриОткрытии и заполним его следующим образом (листинг 2.49).

Листинг 2.49. Обработчик события формы «ПриОткрытии»

```
&НаКлиенте
Процедура ПриОткрытии(Отказ)

    Если ТолькоПросмотр = Ложь Тогда
        Элементы.СделатьАудиозаписьИЗаписать.Доступность =
            СредстваМультимедиа.ПоддерживаетсяАудиозапись();
        Элементы.СделатьВидеозаписьИЗаписать.Доступность =
            СредстваМультимедиа.ПоддерживаетсяВидеозапись();
    КонецЕсли;

КонецПроцедуры
```

```

        Элементы.СделатьФотоснимокИЗаписать.Доступность =
            СредстваМультимедиа.ПоддерживаетсяФотоснимок();
    Иначе
        Элементы.СделатьАудиозаписьИЗаписать.Доступность = Ложь;
        Элементы.СделатьВидеозаписьИЗаписать.Доступность = Ложь;
        Элементы.СделатьФотоснимокИЗаписать.Доступность = Ложь;
    КонецЕсли;
КонецПроцедуры

```

В заключение перед записью данных из формы убедимся, что имя файла заполнено. Иначе запись данных не должна выполняться. Для этого создадим клиентский обработчик события формы ПередЗаписью и заполним его следующим образом (листинг 2.50).

Листинг 2.50. Обработчик события формы «ПередЗаписью»

```

&НаКлиенте
Процедура ПередЗаписью(Отказ, ПараметрыЗаписи)

    Если Объект.ИмяФайла = "" Тогда
        ПоказатьПредупреждение(, "Не выбран файл!");
        Отказ = Истина;
    КонецЕсли;
КонецПроцедуры

```

Ранее в форме списка хранимых файлов мы создали обработчик события Выбор для таблицы Список, в котором при нажатии на строку списка открывается файл мультимедиа, а не форма элемента справочника (как происходит по умолчанию). Поэтому теперь команда Изменить для открытия формы файла располагается в контекстном меню, вызываемом с помощью жеста пролистывания с правой стороны экрана у конкретной строки списка хранимых файлов. При помощи соответствующих команд в этой форме можно сделать фотоснимок, видео- или аудиозапись или открыть файл с данными мультимедиа (рис. 2.73).

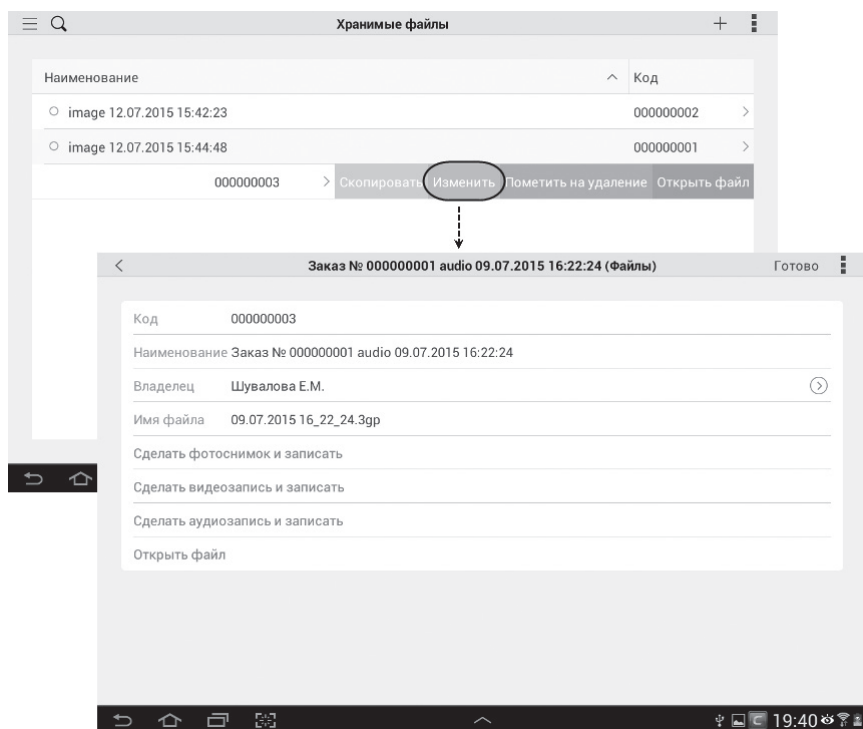


Рис. 2.73. Открытие формы хранимого файла

Клиенты

Теперь доработаем форму элемента справочника Клиенты. Ранее мы уже создавали эту форму, но занимались только ее внешним видом (см. раздел «Редактируемые справочники» на стр. 74). Сейчас наполним форму клиента той функциональностью, которая может понадобиться курьеру при доставке заказов клиенту.

Прежде всего курьер должен определить местоположение клиента на карте по его адресу, чтобы понять, как до него добраться. Для этого используются возможности геопозиционирования, позволяющие определять географические координаты по адресу.

Возможна и обратная операция – определение адреса по географическим координатам, исходя из текущего местоположения мобильного устройства.

Практически в нашем примере эта возможность вряд ли понадобится, но на всякий случай мы ее покажем.

Кроме того, предоставим курьеру удобную возможность проложить маршрут до клиента, используя сервис карт Google Maps.

Также курьер должен иметь возможность связаться с клиентом (позвонить или отправить СМС), чтобы уточнить время или адрес доставки. Для этого используются возможности сотовой связи, которые поддерживаются не всеми мобильными устройствами. В частности, у нашего планшета такой возможности нет, однако соответствующие функции предусмотрены в нашем мобильном приложении.

Кроме того, предоставим курьеру удобную возможность создать для себя напоминание – например, о том, что ему нужно перезвонить клиенту в определенное время. Это локальное напоминание должно сработать в указанные дату и время, – в результате должна автоматически появиться форма звонка с заполненным телефоном клиента и текстом напоминания.

А также курьер должен иметь возможность послать клиенту письмо по электронной почте, используя его электронный адрес.

Команды звонка и отправки СМС клиенту уже реализованы нами в форме заказа. Здесь мы их просто продублируем, а откуда их вызывать – дело вкуса и привычки конкретного пользователя мобильного приложения. Команды для определения местоположения клиента на карте также можно было бы поместить в форму заказа, но, на наш взгляд, не нужно перенасыщать заказ командами, иначе в них будет трудно потом разобраться.

Для решения поставленных задач откроем форму элемента справочника Клиенты и на закладке Команды добавим команды ПоказатьНаКарте, ИспользоватьТекущееМестоположение, ПроложитьМаршрут, Позвонить, ОтправитьСМС, НапомнитьОЗвонке и ОтправитьПисьмо.

В палитре свойств команды Позвонить в поле Картинка выберем картинку Позвонить из общих картинок нашего мобильного приложения.

Затем добавим в командную панель формы две группы вида Подменю с заголовками Телефония и Геопозиционирование и перетащим в них соответствующие команды. Команду ОтправитьПисьмо перетащим не в подменю, а просто в командную панель формы.

В результате форма элемента справочника в редакторе формы примет следующий вид (рис. 2.74).

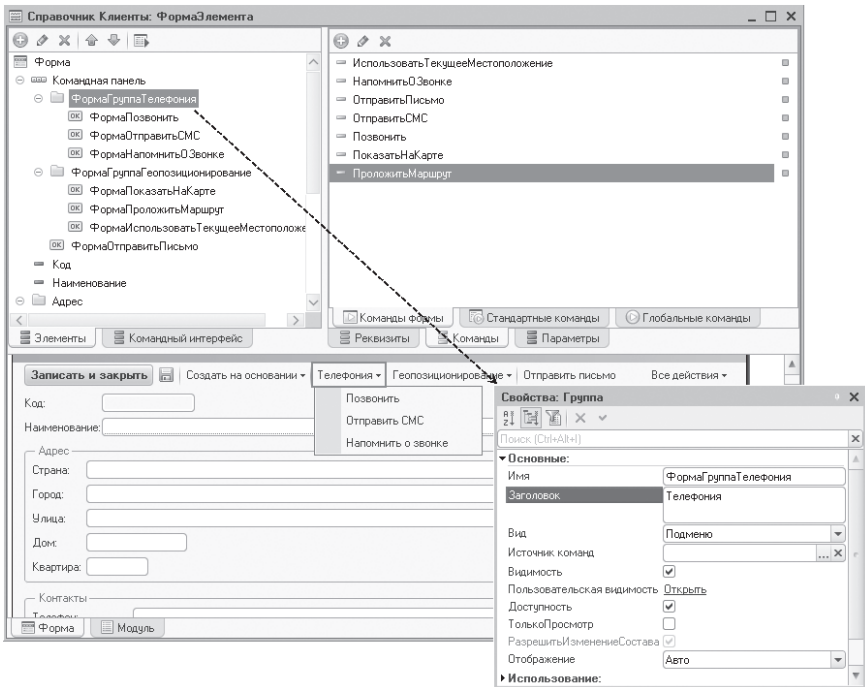


Рис. 2.74. Форма клиента в редакторе формы

Геопозиционирование

Сначала реализуем в форме клиента команды, связанные с геопозиционированием. Чтобы показать на карте местоположение клиента по его адресу, создадим клиентский обработчик для команды **ПоказатьНаКарте** и заполним его следующим образом (листинг 2.51).

Листинг 2.51. Обработчик команды «ПоказатьНаКарте»

```
&НаКлиенте
Процедура ПоказатьКарту(Команда)

    Координаты = ПолучитьКоординатыКлиента();

    Если Координаты <> Неопределено Тогда
        ПоказатьНаКарте(Координаты);
    Иначе
        // Сообщим пользователю о том, что информация не консистентна
        Сообщение = Новый СообщениеПользователю();
        Сообщение.Текст = "Не заполнены поля, описывающие адрес клиента!";
```

```
Сообщение.Поле = "Объект.Страна";
Сообщение.Сообщить();
КонецЕсли;
```

```
КонецПроцедуры
```

В этом обработчике мы сначала вызываем функцию ПолучитьКоординатыКлиента для получения географических координат местоположения клиента (см. листинг 2.52).

После этого выполняем метод ПоказатьНаКарте(), в который передаем полученные координаты. Для отображения местоположения клиента по его адресу используются мобильные карты Google Maps. С картами Google Maps мобильная платформа работает «напрямую», установка их в виде отдельного приложения не требуется.

Если координаты не определены, значит, адресные реквизиты клиента не заполнены. В этом случае выводим сообщение об ошибке.

Листинг 2.52. Функция «ПолучитьКоординатыКлиента»

```
&НаКлиенте
Функция ПолучитьКоординатыКлиента()

    Координаты = Неопределено;
    Если Объект.Широта <> 0 ИЛИ Объект.Долгота <> 0 Тогда
        Координаты = Новый ГеографическиеКоординаты(Объект.Широта, Объект.Долгота);
    Иначе
        СтруктураДанныхАдреса = Новый Структура();
        СтруктураДанныхАдреса.Вставить("Страна", Объект.Страна);
        СтруктураДанныхАдреса.Вставить("Город", Объект.Город);
        СтруктураДанныхАдреса.Вставить("Улица", Объект.Улица);
        СтруктураДанныхАдреса.Вставить("Дом", Объект.Дом);
        ДанныеАдреса = Новый ДанныеАдреса(СтруктураДанныхАдреса);
        Координаты = ПолучитьМестоположениеПоАдресу(ДанныеАдреса);
    КонецЕсли;
    Возврат Координаты;

КонецФункции
```

В функции для получения координат клиента, если реквизиты клиента Широта и Долгота заполнены, на их основе мы создаем объект ГеографическиеКоординаты для описания координат местоположения.

Если координаты не заполнены, то мы создаем структуру с полями Страна, Город, Улица, Дом и заполняем ее значениями соответствующих строковых реквизитов адреса клиента. Затем на основе этой структуры создаем объект ДанныеАдреса для описания адреса клиента. Для более точного описания адреса можно добавить в структуру поля Регион и Индекс, но в нашем мобильном приложении у клиента этих реквизитов нет. Затем с помощью

метода `ПолучитьМестоположениеПоАдресу()` мы получаем географические координаты клиента по его адресу, описанному в объекте `ДанныеАдреса`.

Проверим, как это работает на планшете. Откроем форму клиента, из меню функций откроем группу команд `Геопозиционирование` и вызовем команду `Показать на карте`. При выполнении данной команды местоположение клиента, соответствующее его адресу, показывается на карте (рис. 2.75).

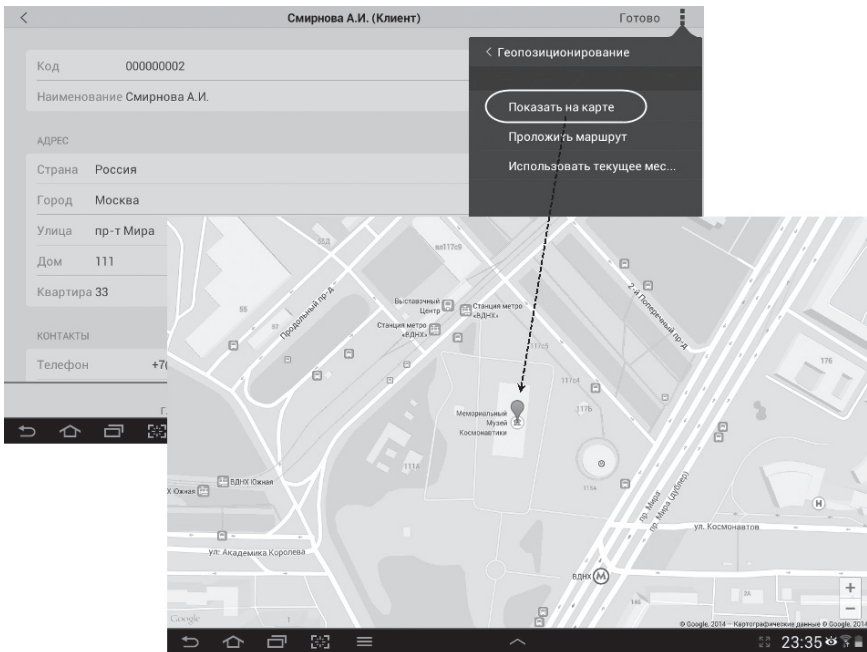


Рис. 2.75. Отображение адреса клиента на карте

В приведенном выше примере у клиента заполнены адресные реквизиты, но не заполнены координаты. Если координаты клиента заполнены, то отображаться на карте будут именно они, а не координаты, полученные по адресу.

При выполнении команды `Показать на карте` может появиться сообщение, предлагающее обновить приложение Google Play. Для этого нужно добавить существующий аккаунт Google или создать новый.

Также для выполнения команд геопозиционирования нужно, чтобы в настройках планшета в разделе `Местоположение` были включены соответствующие настройки (рис. 2.76).

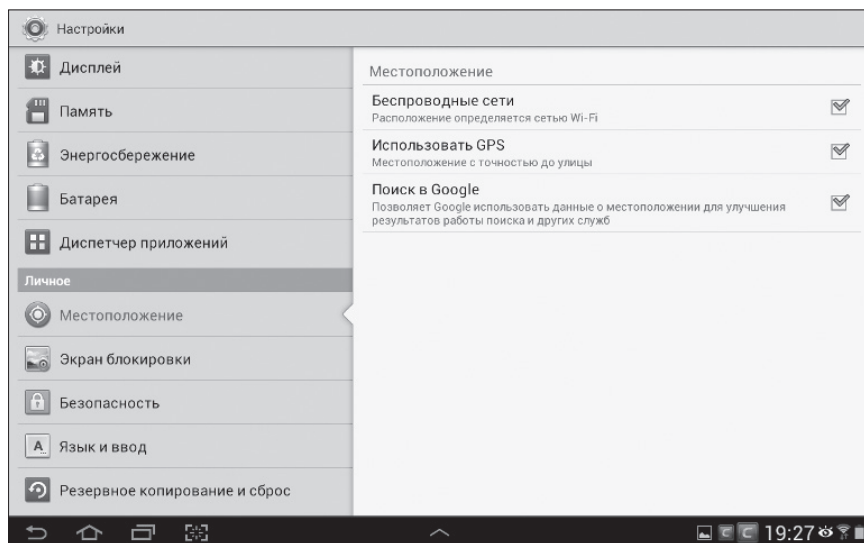


Рис. 2.76. Настройки геопозиционирования

СОВЕТ

Если вам не удалось получить координаты местоположения клиента по его адресу или координаты адреса по местоположению, хотя у клиента заполнены все адресные данные, то просто перезагрузите ваше мобильное устройство. И все должно заработать.

Теперь реализуем команду для определения адреса клиента по текущему местоположению мобильного устройства. Для этого создадим клиентский обработчик для команды `ИспользоватьТекущееМестоположение` и заполним его следующим образом (листинг 2.53).

Листинг 2.53. Обработчик команды «ИспользоватьТекущееМестоположение»

```
&НаКлиенте
Процедура ИспользоватьТекущееМестоположение(Команда)

    Провайдер = СредстваГеопозиционирования.ПолучитьСамогоЭнергоЭкономичногоПровайдера();

    Если СредстваГеопозиционирования.ОбновитьМестоположение(Провайдер.Имя, 60) Тогда
        ДанныеМестоположения =
            СредстваГеопозиционирования.ПолучитьПоследнееМестоположение(Провайдер.Имя);
        ДанныеАдреса = ПолучитьАдресПоМестоположению(ДанныеМестоположения.Координаты);

        Если ДанныеАдреса <> Неопределено Тогда
            Объект.Страна = ДанныеАдреса.Страна;
            Объект.Город = ДанныеАдреса.Город;
```

```
Объект.Улица = ДанныеАдреса.Улица;  
Объект.Дом = ДанныеАдреса.Дом;  
Объект.Широта = ДанныеМестоположения.Координаты.Широта;  
Объект.Долгота = ДанныеМестоположения.Координаты.Долгота;  
Модифицированность = Истина;  
Иначе  
    Сообщение = Новый СообщениеПользователю();  
    Сообщение.Текст = "Не удалось установить адрес по местоположению!";  
    Сообщение.Сообщить();  
КонецЕсли;  
Иначе  
    Сообщение = Новый СообщениеПользователю();  
    Сообщение.Текст = "Не удалось получить данные от провайдера геопозиционирования!";  
    Сообщение.Сообщить();  
КонецЕсли;  
КонецПроцедуры
```

В этом обработчике с помощью свойства глобального контекста СредстваГеопозиционирования мы получаем доступ к средствам геопозиционирования, которые определены только на мобильном клиенте. Для определения текущего местоположения мобильного устройства предназначен провайдер геопозиционирования.

С помощью метода средств геопозиционирования ПолучитьСамоеЭнергоЭкономичноеПровайдера() мы получаем информацию о провайдере, который обеспечивает минимальное потребление энергии.

Затем мы выполняем метод средств геопозиционирования ОбновитьМестоположение() и передаем туда имя полученного провайдера геопозиционирования (Провайдер.Имя), для того чтобы провайдер определил актуальные данные о местоположении.

Во втором параметре метода мы назначаем тайм-аут на определение местоположения – 60 секунд. В случае если метод завершен по тайм-ауту, возвращается Ложь и пользователю выводится сообщение об ошибке.

Если метод ОбновитьМестоположение() завершен успешно, возвращается Истина. В этом случае мы выполняем метод средств геопозиционирования ПолучитьПоследнееМестоположение(). Этот метод возвращает последние данные о местоположении, полученные переданным провайдером геопозиционирования, описанные в объекте ДанныеМестоположения. В свойстве этого объекта Координаты содержатся географические координаты текущего местоположения.

На основе этих координат с помощью метода ПолучитьАдресПоМестоположению() мы получаем адрес в виде объекта ДанныеАдреса, который описывает адрес в виде структуры полей Страна, Город, Улица, Дом, Регион, Индекс.

Затем мы заполняем адресные реквизиты клиента значениями соответствующих полей структуры. Реквизиты Широта и Долгота заполняем аналогичными данными объекта ДанныеМестоположения. А также свойству Модифицированность формы присваиваем значение Истина, так как в форме изменились данные адреса клиента.

Если структура ДанныеАдреса не определена, то выводим сообщение об ошибке.

Теперь реализуем команду, которая с помощью сервиса карт Google Maps позволяет проложить маршрут от текущего местоположения курьера (то есть мобильного устройства) до местонахождения клиента. Для этого создадим клиентский обработчик для команды ПроложитьМаршрут и заполним его следующим образом (листинг 2.54).

Листинг 2.54. Обработчик команды «ПроложитьМаршрут»

```
&НаКлиенте
Процедура ПроложитьМаршрут(Команда)

    ДанныеМестоположения = Неопределено;
    Провайдер = СредстваГеоопозиционирования.ПолучитьСамогоЭнергоЭкономичногоПровайдера();

    Если СредстваГеоопозиционирования.ОбновитьМестоположение(Провайдер.Имя, 60) Тогда
        ДанныеМестоположения =
            СредстваГеоопозиционирования.ПолучитьПоследнееМестоположение(Провайдер.Имя);
    КонецЕсли;

    Если ДанныеМестоположения = Неопределено Тогда
        Сообщение = Новый СообщениеПользователю();
        Сообщение.Текст = "Не удалось установить текущее местоположение!";
        Сообщение.Сообщить();
        Возврат;
    КонецЕсли;

    КоординатыКлиента = ПолучитьКоординатыКлиента();
    Если КоординатыКлиента = Неопределено Тогда
        Сообщение = Новый СообщениеПользователю();
        Сообщение.Текст = "Не удалось установить расположение клиента!";
        Сообщение.Сообщить();
        Возврат;
    КонецЕсли;

    Запуск = Новый ЗапускПриложенияМобильногоУстройства("android.intent.action.VIEW",
        "http://maps.google.com/maps?saddr="
        + XMLСтрока(ДанныеМестоположения.Координаты.Широта) + "," +
        XMLСтрока(ДанныеМестоположения.Координаты.Долгота)
        + "&daddr="
        + XMLСтрока(КоординатыКлиента.Широта) + "," +
        XMLСтрока(КоординатыКлиента.Долгота));
    Запуск.Запустить(Ложь);

КонецПроцедуры
```

В этом обработчике мы так же, как и в предыдущем примере (см. листинг 2.53), сначала получаем самого энергоэкономичного провайдера геопозиционирования. Затем мы выполняем метод средств геопозиционирования `ОбновитьМестоположение()` и передаем туда имя полученного провайдера (`Провайдер.Имя`), для того чтобы провайдер определил актуальные данные о местоположении.

Если данные о местоположении обновлены успешно, мы выполняем метод средств геопозиционирования `ПолучитьПоследнееМестоположение()`. Этот метод в объекте `ДанныеМестоположения` возвращает последние данные о местоположении, полученные провайдером геопозиционирования. В свойстве этого объекта `Координаты` содержатся географические координаты текущего местоположения.

Затем мы вызываем функцию `ПолучитьКоординатыКлиента` для получения географических координат местоположения клиента (см. листинг 2.52).

Если координаты текущего местоположения или координаты клиента не определены, то выводим сообщение об ошибке.

После этого мы запускаем сервис карт Google Maps, в который мы передаем координаты текущего местоположения мобильного устройства и координаты местонахождения клиента для построения маршрута движения между ними.

Проверим, как это работает на планшете. Откроем форму клиента, из меню функций откроем группу команд Геопозиционирование и вызовем команду Проложить маршрут. При выполнении данной команды на карте прокладывается маршрут движения от местоположения курьера до клиента (рис. 2.77).

Следует иметь в виду, что для работы методов преобразования координат в адрес и обратно на мобильном устройстве должен быть доступ в Интернет.

В качестве общей рекомендации по решению задач геопозиционирования можно посоветовать выбирать того провайдера, который максимально подходит для реализации поставленной задачи. Для этого следует проанализировать свойства используемого провайдера геопозиционирования: является ли он платным, использует ли сотовую сеть, использует ли спутник и т.п. В общем случае стоит выбирать провайдера, который обеспечивает минимальное энергопотребление и максимальную точность определения координат.

ПОДРОБНЕЕ

Документация 1С:Предприятие 8.3. Руководство разработчика. Глава 26. Разработка решений для мобильной платформы > Особенности использования > Специальные возможности > Средства геопозиционирования.

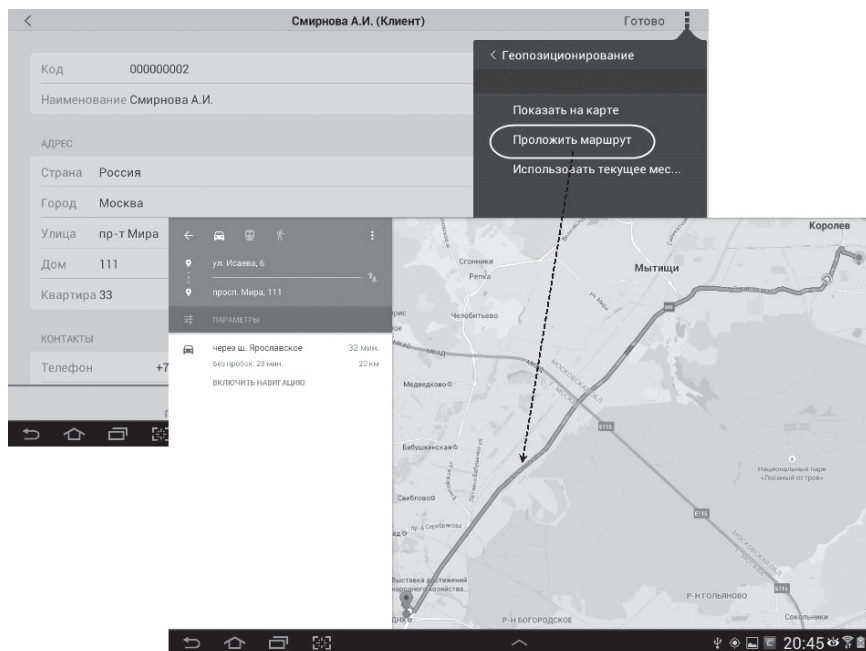


Рис. 2.77. Построение маршрута движения от курьера до клиента

Телефония

Связь с клиентом

Теперь реализуем команды для связи с клиентом. Для этого создадим клиентские обработчики для команд Позвонить и ОтправитьСМС и заполним их следующим образом (листинги 2.55, 2.56).

Листинг 2.55. Обработчик команды «Позвонить»

```
&НаКлиенте
Процедура Позвонить(Команда)
    Если ЗначениеЗаполнено(Объект.Телефон) Тогда
        СредстваТелефонии.НабратьНомер(Объект.Телефон, Ложь);
    Иначе
        Сообщение = Новый СообщениеПользователю();
        Сообщение.Текст = "Не указан телефон!";
        Сообщение.Поле = "Объект.Телефон";
        Сообщение.Сообщить();
    КонецЕсли
КонецПроцедуры
```


Листинг 2.56. Обработчик команды «ОтправитьСМС»

```
&НаКлиенте
Процедура ОтправитьСМС(Команда)

    Если ЗначениеЗаполнено(Объект.Телефон) Тогда
        Сообщение = Новый SMSCообщение();
        Сообщение.Получатели.Добавить(Объект.Телефон);
        СредстваТелефонии.ПослатьSMS(Сообщение, Истина);
    Иначе
        Сообщение = Новый СообщениеПользователю();
        Сообщение.Текст = "Не указан телефон!";
        Сообщение.Поле = "Объект.Телефон";
        Сообщение.Сообщить();
    КонецЕсли

КонецПроцедуры
```

В обработчиках обеих команд, в случае если телефон клиента заполнен, выполняется звонок или отправка сообщения клиенту с помощью объекта глобального контекста СредстваТелефонии, который предоставляет доступ к средствам телефонии на мобильном устройстве.

Для звонка клиенту в процедуре Позвонить() используется метод средств телефонии НабратьНомер, в который передается телефон клиента. Поскольку мы передаем во втором параметре метода Ложь, при выполнении метода будет открыто приложение работы со звонками, в котором уже установлен заданный номер. Курьеру нужно будет только нажать кнопку начала вызова.

Для отправки СМС клиенту в процедуре ОтправитьСМС() используется метод средств телефонии ПослатьSMS. Сначала создается объект SMSCообщение, затем в свойство Получатели этого объекта добавляется телефон клиента. После этого вызывается метод средств телефонии ПослатьSMS, в который первым параметром передается объект SMSCообщение, а вторым Истина. При этом будет запущено системное приложение для отправки SMS-сообщений. Курьеру остается ввести текст сообщения и нажать кнопку отправки.

Напоминание о звонке (локальное уведомление)

Теперь реализуем очень удобную для курьера возможность – создавать для себя напоминание о том, что надо позвонить клиенту в определенное время. Это напоминание появится в системе в указанные дату и время. При нажатии на это напоминание должна автоматически открыться форма звонка с переданными в нее номером телефона и текстом напоминания (см. рис. 2.80).

Для реализации описанной задачи мы должны сначала создать в системе так называемое локальное уведомление, информирующее пользователя о событии, произошедшем на конкретном мобильном устройстве. А затем соответствующим образом обработать это локальное уведомление в нашем мобильном приложении.

Для этого сначала создадим клиентский обработчик команды НапомнитьОЗвонке и заполним его следующим образом (листинг 2.57).

Листинг 2.57. Обработчик команды «НапомнитьОЗвонке»

```
&НаКлиенте
Процедура НапомнитьОЗвонке(Команда)

    Если ЗначениеЗаполнено(Объект.Телефон) Тогда
        ДатаНапоминания = ТекущаяДата();
        Оповещение = Новый ОписаниеОповещения("ПослеВводаДаты", ЭтотОбъект, Параметры);
        ПоказатьВводДаты(Оповещение, ДатаНапоминания, "Укажите дату и время напоминания");
    Иначе
        // Сообщим пользователю о том, что информация не консистентна
        Сообщение = Новый СообщениеПользователю();
        Сообщение.Текст = "Телефон не указан!";
        Сообщение.Поле = "Объект.Телефон";
        Сообщение.Сообщить();
    КонецЕсли

КонецПроцедуры
```

В этом обработчике, если телефон клиента заполнен, мы вызываем диалог для ввода пользователем даты и времени напоминания. Для этого мы используем немодальный метод глобального контекста ПоказатьВводДаты, который позволяет ввести дату, но не ожидает завершения ввода. Первым параметром в этот метод передается описание процедуры ПослеВводаДаты, которая будет выполнена, когда пользователь выберет дату (листинг 2.58).

Листинг 2.58. Процедура «ПослеВводаДаты»

```
&НаКлиенте
Процедура ПослеВводаДаты(Дата, Параметры) Экспорт

    Если НЕ Дата = Неопределено Тогда
        Уведомление = Новый ДоставляемоеУведомление();
        Уведомление.Текст = "Перезвоните клиенту: " + Объект.Наименование;
        Уведомление.Данные = Объект.Телефон;
        Уведомление.ДатаПоявленияУниверсальноеВремя = УниверсальноеВремя(Дата);
        ДоставляемыеУведомления.ДобавитьЛокальноеУведомление(Уведомление);
    КонецЕсли;

КонецПроцедуры
```

В эту процедуру передается выбранная пользователем дата или Неопределено, если он отказался от выбора.

Если дата выбрана, мы создаем объект `ДоставляемоеУведомление`, заполняем его свойство `Текст` как произвольный текст напоминания, `Данные` – как телефон клиента. В свойстве `ДатаПоявленияУниверсальноеВремя` устанавливаем дату, введенную пользователем, в формате универсального времени (UTC).

Затем с помощью свойства глобального контекста `ДоставляемыеУведомления` мы обращаемся к менеджеру доставляемых уведомлений и методом `ДобавитьЛокальноеУведомление` добавляем созданное нами локальное уведомление в систему.

Проверим, как это работает на планшете. Откроем форму клиента, из меню функций откроем группу команд `Телефония` и вызовем команду `Напомнить о звонке`. При выполнении данной команды появляется диалог для ввода даты и времени напоминания (рис. 2.78).

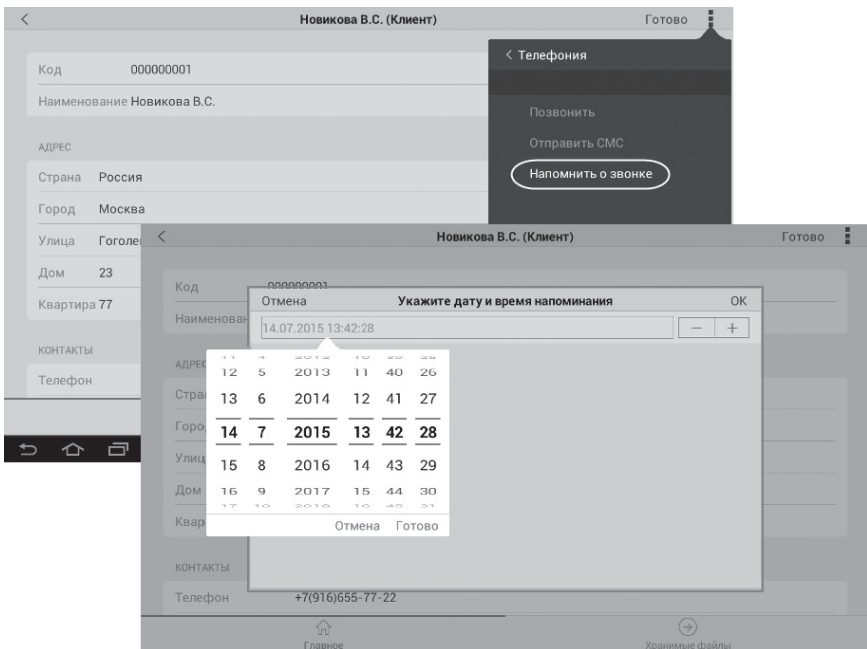


Рис. 2.78. Создание напоминания о звонке клиенту

После нажатия кнопки ОК в нашем мобильном приложении создается локальное уведомление, в него передается текст напоминания и телефон клиента, – в результате это уведомление появится на мобильном устройстве в указанное курьером время.

Если приложение запущено, но не активно, или закрыто, то в списке уведомлений в правом углу планшета (это справедливо для мобильных устройств, работающих под управлением ОС Android) появится значок уведомления. А также прозвучит стандартный сигнал, установленный в системе для уведомлений.

После того как пользователь мобильного приложения нажмет на уведомление, будет активизировано мобильное приложение, создавшее уведомление, и в этом приложении будет вызван обработчик уведомления. В рамках этого обработчика мы и должны реализовать необходимую обработку уведомления, а именно – появление формы для звонка клиенту.

Для этого нужно подключить обработчик уведомления в модуле управляемого приложения в обработчике события ПриНачалеРаботыСистемы (листинг 2.59).

Листинг 2.59. Процедура «ПриНачалеРаботыСистемы»

Процедура ПриНачалеРаботыСистемы()

```
// Подключение обработчика локальных и push-уведомлений  
ОписаниеОповещения = Новый ОписаниеОповещения("ОбработкаУведомлений", УведомленияКлиент);  
ДоставляемыеУведомления.ПодключитьОбработчикУведомлений(ОписаниеОповещения);
```

КонецПроцедуры

В этом обработчике методом ПодключитьОбработчикУведомлений менеджера доставляемых уведомлений в качестве обработчика уведомлений подключается процедура ОбработкаУведомлений, находящаяся в общем модуле УведомленияКлиент и описанная в объекте ОписаниеОповещения.

Создадим клиентский общий модуль УведомленияКлиент, поместим в нем процедуру ОбработкаУведомлений и заполним ее следующим образом (листинг 2.60).

Листинг 2.60. Процедура «ОбработкаУведомлений»

Процедура ОбработкаУведомлений(Уведомление, Локальное, Показано, Параметры) Экспорт

```
Если Локальное = Истина Тогда  
    ПараметрыФормы = Новый Структура("Текст, Данные", Уведомление.Текст, Уведомление.Данные);  
    ОткрытьФорму("ОбщаяФорма.Звонок", ПараметрыФормы);  
КонецЕсли;
```

КонецПроцедуры

В этом обработчике сначала проверяется, что полученное уведомление локальное (в параметре Локальное передано Истина). Это условие понадобилось нам потому, что в этом же обработчике будут обрабатываться также и push-уведомления, которые мы рассмотрим позже (см. раздел «Работа с push-уведомлениями» на стр. 328).

Если условие истинно, то создается структура параметров формы с полями Текст и Данные, которые заполняются соответственно текстом и данными полученного уведомления. Затем открывается общая форма Звонок и в нее передается структура параметров. Из этой формы и будет инициирован звонок клиенту.

Создадим в нашем мобильном приложении общую форму Звонок, добавим в нее команду Позвонить и строковые реквизиты Текст и Данные для хранения соответствующих параметров формы. В палитре свойств команды Позвонить в поле Картинка выберем картинку Позвонить из общих картинок нашего мобильного приложения и свойство команды Отображение установим как Картинка.

Реквизит Текст и команду Позвонить перетащим в окно элементов формы. В результате форма звонка в редакторе формы будет выглядеть следующим образом (рис. 2.79).

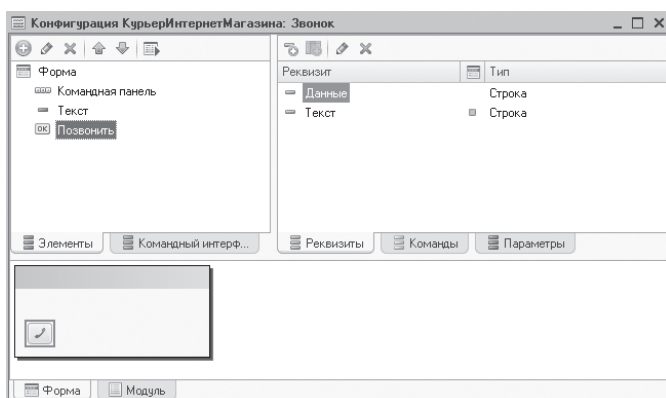


Рис. 2.79. Общая форма «Звонок» в редакторе формы

Чтобы при создании формы заполнить реквизиты формы соответствующими полями структуры параметров, переданной в форму, создадим обработчик события формы ПриСозданииНаСервере и заполним его следующим образом (листинг 2.61).

Листинг 2.61. Обработчик события формы «ПриСозданииНаСервере»

```
&НаСервере  
Процедура ПриСозданииНаСервере(Отказ, СтандартнаяОбработка)  
  
    Данные = Параметры.Данные;  
    Текст = Параметры.Текст;  
  
КонецПроцедуры
```

Таким образом, при создании формы в реквизитах Текст и Данные будут сохранены текст напоминания и телефон клиента, переданные в уведомлении.

Текст напоминания (содержимое реквизита Текст) мы просто отразим в форме. Для команды Позвонить создадим клиентский обработчик и заполним его следующим образом (листинг 2.62).

Листинг 2.62. Обработчик команды «Позвонить»

```
&НаКлиенте  
Процедура Позвонить(Команда)  
  
    СредстваТелефонии.НабратьНомер(Данные, Ложь);  
  
КонецПроцедуры
```

Для звонка клиенту в процедуре Позвонить() используется метод средств телефонии НабратьНомер, в который передается телефон клиента в реквизите формы Данные.

Таким образом, если приложение неактивно, – после того как курьер нажмет на значок уведомления, будет активизировано мобильное приложение, в котором будет вызван обработчик уведомлений. Если приложение активно и в этот момент наступает время напоминания, то обработчик уведомлений будет вызван сразу же, без значка уведомления и звукового сигнала. Поэтому, чтобы форма звонка не затерялась среди других форм (т.к. она немодальная), установим свойство формы РежимОткрытияОкна в значение Блокировать весь интерфейс.

В результате на экране поверх всех окон, блокируя весь остальной интерфейс, откроется форма звонка клиенту с установленным текстом напоминания. При нажатии на кнопку звонка будет открыто приложение работы со звонками, в котором уже установлен заданный номер. Курьеру останется только нажать кнопку начала вызова (рис. 2.80).

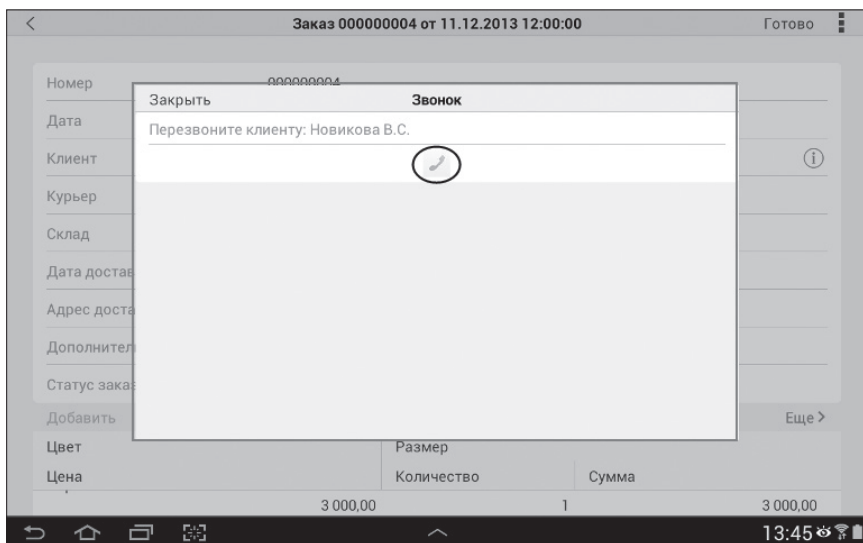


Рис. 2.80. Появление формы звонка клиенту

ПОДРОБНЕЕ

Документация 1С:Предприятие 8.3. Руководство разработчика. Глава 26. Разработка решений для мобильной платформы > Особенности использования > Специальные возможности > Работа с уведомлениями.

В заключение при открытии формы клиента установим видимость команд для работы со средствами мультимедиа только в случае, если мобильное устройство поддерживает работу с телефонией и отправку почты. Для этого создадим обработчик события формы ПриОткрытии и заполним его следующим образом (листинг 2.63).

Листинг 2.63. Обработчик события формы «ПриОткрытии»

```
&НаКлиенте
Процедура ПриОткрытии(Отказ)

    Элементы.ФормаПозвонить.Доступность = СредстваТелефонии.ПоддерживаетсяНаборНомера();
    Элементы.ФормаОтправитьСМС.Доступность =
        СредстваТелефонии.ПоддерживаетсяОтправкаSMS(Истина);
    Элементы.ФормаОтправитьПисьмо.Доступность = СредстваПочты.ПоддерживаетсяОтправка();

КонецПроцедуры
```

И аналогичную проверку на возможность звонка нужно добавить в форму Звонок в обработчик события формы ПриОткрытии (листинг 2.64).

Листинг 2.64. Обработчик события формы «ПриОткрытии» формы «Звонок»

```
&НаКлиенте
Процедура ПриОткрытии(Отказ)
```

```
    Элементы.ФормаПозвонить.Доступность = СредстваТелефонии.ПоддерживаетсяНаборНомера();
```

```
КонецПроцедуры
```

Электронная почта

Если телефон клиента по каким-то причинам недоступен, то курьер должен иметь возможность отправить клиенту письмо по электронной почте. Чтобы реализовать такую возможность, создадим клиентский обработчик для команды ОтправитьПисьмо и заполним его следующим образом (листинг 2.65).

Листинг 2.65. Обработчик команды «ОтправитьПисьмо»

```
&НаКлиенте
Процедура ОтправитьПисьмо(Команда)
```

```
    Если ЗначениеЗаполнено(Объект.ЭлектроннаяПочта) Тогда
        Сообщение = Новый ИнтернетПочтовоеСообщение;
        Сообщение.Тема = "Сообщение о заказе";
        Сообщение.Получатели.Добавить(СокрЛП(Объект.ЭлектроннаяПочта));
        Текст = Сообщение.Тексты.Добавить("Уважаемый клиент, " + СокрЛП(Объект.Наименование) + "!");
        Текст.ТипТекста = ТипТекстаПочтовогоСообщения.ПростойТекст;
        СредстваПочты.Послать(Сообщение);
```

```
Иначе
```

```
    // Сообщим пользователю о том, что информация не консистентна
    Сообщение = Новый СообщениеПользователю();
    Сообщение.Текст = "Электронный адрес не указан!";
    Сообщение.Поле = "Объект.ЭлектроннаяПочта";
    Сообщение.Сообщить();
```

```
КонецЕсли
```

```
КонецПроцедуры
```

В этом обработчике, если адрес электронной почты клиента заполнен, мы создаем объект ИнтернетПочтовоеСообщение, заполняем его свойства Тема и Тексты (текст не должен содержать элементы форматирования!). В свойство Получатели почтового сообщения добавляем адрес электронной почты клиента (Объект.ЭлектроннаяПочта).

Затем с помощью свойства глобального контекста СредстваПочты мы получаем доступ к менеджеру средств встроенной почты мобильной платформы.

Для отправки созданного нами почтового сообщения мы используем метод менеджера Послать и передаем в него сообщение в качестве параметра.

В результате при вызове команды Отправить письмо из меню функций в форме клиента запускается почтовое приложение для отправки электронной почты (E-mail, Gmail и т.п.), в котором уже заполнены адрес получателя, тема и частично текст (рис. 2.81).

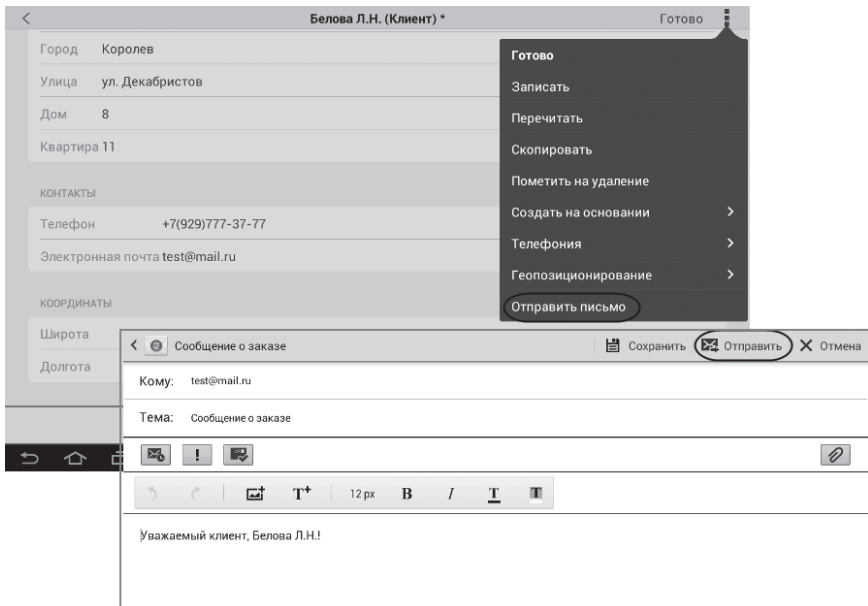


Рис. 2.81. Отправка электронной почты клиенту

Дальше курьер может работать с этим почтовым приложением как обычно (вложить файлы, дополнить текст сообщения и т.д.) и отправить письмо кнопкой Отправить.

Проверка заполнения

При разработке справочника Клиенты мы не устанавливали ни для каких реквизитов справочника свойство Проверка заполнения в значение Выдавать ошибку. Однако при записи клиента нам нужно программно проверить, чтобы все адресные реквизиты клиента Страна, Город, Улица, Дом были либо заполнены, либо пусты. Иначе отображение адреса на карте может быть неточным.

Такая проверка выполняется в обработчике `ОбработкаПроверкиЗаполнения()`, который описывается в модуле объекта. Этот обработчик вызывается автоматически при сохранении любой формы. Поместим в модуль справочника `Клиенты` следующую процедуру (листинг 2.66).

Листинг 2.66. Обработчик события справочника «`ОбработкаПроверкиЗаполнения`»

Процедура `ОбработкаПроверкиЗаполнения(Отказ, ПроверяемыеРеквизиты)`

```
// Если заполнено поле "Улица"
Если НЕ ПустаяСтрока(Улица) Тогда
    // Тогда должны быть заполнены и поля "Страна", "Город" и "Дом".
    ПроверяемыеРеквизиты.Добавить("Страна");
    ПроверяемыеРеквизиты.Добавить("Город");
    ПроверяемыеРеквизиты.Добавить("Дом");
КонецЕсли;

// Если заполнено поле "Дом"
Если НЕ ПустаяСтрока(Дом) Тогда
    // Тогда должны быть заполнены и поля "Страна", "Город" и "Улица".
    ПроверяемыеРеквизиты.Добавить("Страна");
    ПроверяемыеРеквизиты.Добавить("Город");
    ПроверяемыеРеквизиты.Добавить("Улица");
КонецЕсли;

// Если заполнено поле "Город"
Если НЕ ПустаяСтрока(Город) Тогда
    // Тогда должны быть заполнены и поля "Страна", "Улица" и "Дом".
    ПроверяемыеРеквизиты.Добавить("Страна");
    ПроверяемыеРеквизиты.Добавить("Улица");
    ПроверяемыеРеквизиты.Добавить("Дом");
КонецЕсли;

// Если заполнено поле "Страна"
Если НЕ ПустаяСтрока(Страна) Тогда
    // Тогда должны быть заполнены и поля "Улица", "Город" и "Дом".
    ПроверяемыеРеквизиты.Добавить("Улица");
    ПроверяемыеРеквизиты.Добавить("Город");
    ПроверяемыеРеквизиты.Добавить("Дом");
КонецЕсли;
```

КонецПроцедуры

В этом обработчике мы последовательно проверяем, заполнено ли значение одного из четырех реквизитов (Страна, Город, Улица, Дом), описывающих адрес клиента. Если да, то остальные три реквизита мы добавляем в массив `ПроверяемыеРеквизиты`. Этот массив передается в обработчик и содержит массив проверяемых реквизитов, у которых свойство `Проверка заполнения` установлено в значение `Выдавать ошибку` во время разработки мобильного приложения. На момент вызова обработчика `ОбработкаПроверкиЗаполнения()` в нем содержится только реквизит клиента `Наименование`.

Если одни адресные реквизиты клиента заполнены, а другие – нет, то при нажатии кнопки Записать и закрыть или Записать будет выдано сообщение об ошибке, а данные о клиенте не будут сохранены.

Список заказов

Ранее мы уже создавали форму списка заказов, но занимались только ее внешним видом (см. раздел «Документ "Заказ"» на стр. 77). Теперь сделаем этот список более удобным в работе и информативным для курьера за счет условного оформления и отбора.

Причем отбор будет выполняться программным образом, так как у пользователя в мобильном приложении нет возможности выполнять интерактивную настройку динамического списка, а условное оформление списка просто зададим в конфигураторе. Но и условное оформление, и отбор будут использовать виртуальное поле динамического списка, полученное с помощью произвольного запроса к таблице заказов.

Итак, приступим. Откроем форму списка документа Заказ и откроем палитру свойств основного реквизита формы динамического списка Список. Установим флажок у свойства списка ПроизвольныйЗапрос. Затем в строке Настройка списка нажмем гиперссылку Открыть (рис. 2.82).

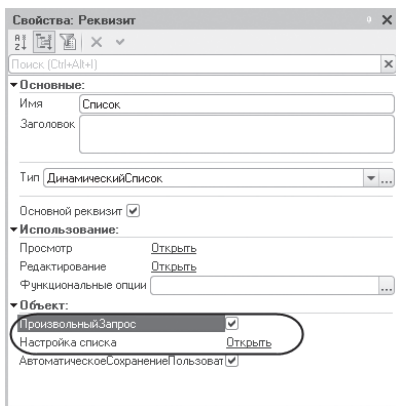


Рис. 2.82. Установка произвольного запроса для динамического списка

В открывшемся окне настройки динамического списка на закладке Запрос мы видим текст запроса, сформированный платформой, в котором выбраны все поля из таблицы заказов. Нам нужно немного изменить этот текст запроса.

Дело в том, что при проектировании структуры документа Заказ мы не запланировали поле, в котором бы содержался признак важности заказа. Например, этим признаком можно пометать заказы со срочной доставкой, со специфическими требованиями клиентов и т.д. Поэтому в интернет-магазине договорились, что важные заказы будут помечаться символом «!» в поле Дополнительная информация.

В нашем мобильном приложении нам этот признак нужен, чтобы выделить условным оформлением важные заказы, – тогда внимание курьера будет сразу привлечено к ним. Или чтобы курьер мог быстро отобрать только важные заказы из всего списка заказов.

Исходя из этого, изменим текст запроса для динамического списка заказов следующим образом (листинг 2.67).

Листинг 2.67. Текст запроса для динамического списка заказов

```

ВЫБРАТЬ
    ДокументЗаказ.Ссылка,
    ДокументЗаказ.ПометкаУдаления,
    ДокументЗаказ.Номер,
    ДокументЗаказ.Дата,
    ДокументЗаказ.Проведен,
    ДокументЗаказ.Клиент,
    ДокументЗаказ.Курьер,
    ДокументЗаказ.Склад,
    ДокументЗаказ.ДатаДоставки,
    ДокументЗаказ.АдресДоставки,
    ДокументЗаказ.ДополнительнаяИнформация,
    ДокументЗаказ.СтатусЗаказа,
    ДокументЗаказ.Товары,
    ДокументЗаказ.МоментВремени,
    ВЫБОР
        КОГДА ПОДСТРОКА(ДокументЗаказ.ДополнительнаяИнформация, 1, 1) = "!" ТОГДА 1
        ИНАЧЕ 0
    КОНЕЦ КАК Важность
ИЗ
    Документ.Заказ КАК ДокументЗаказ

```

В этом тексте мы добавляем в выборку запроса поле Важность. Для формирования этого поля используется операция выбора (ВЫБОР (КОГДА ... ТОГДА) ИНАЧЕ ... КОНЕЦ). С помощью этой операции выбираются все заказы, у которых поле ДополнительнаяИнформация начинается с символа «!». У этих заказов поле Важность = 1, у остальных заказов поле Важность = 0.

Чтобы обеспечить стандартное поведение списка документов, под окном с текстом запроса выберем в поле Основная таблица Документ.Заказ и нажмем ОК (рис. 2.83).

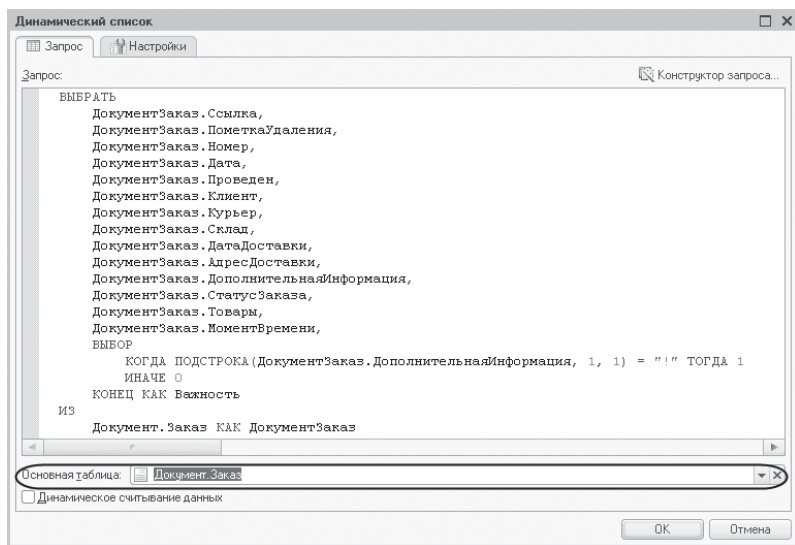


Рис. 2.83. Произвольный текст запроса для динамического списка заказов

Теперь зададим условное оформление динамического списка, чтобы важные заказы выделялись бы цветом в списке заказов. Для этого опять откроем окно настройки динамического списка и перейдем на закладку **Настройки** > **Условное оформление**.

Добавим новый элемент условного оформления. В колонке **Оформление** выберем особый цвет текста. В колонке **Условие** нажмем кнопку выбора и добавим группу условий отбора, связанных логическим союзом «И».

Первым условием будет отбор важных заказов. В качестве левого значения выберем поле **Важность**, которое появилось у динамического списка после установки произвольного текста запроса. Правое значение установим равным единице.

Второе условие будет отбирать только заказы, с которыми курьер в данный момент работает (поле **СтатусЗаказа** равно ссылке на значение **ВРработе** перечисления **СтатусыЗаказов**). Это условие нужно потому, что важность уже выполненных и закрытых заказов для курьера не имеет значения.

Колонку **Оформляемые поля** оставим пустой – в результате условное оформление будет применено сразу ко всей строке списка (рис. 2.84).

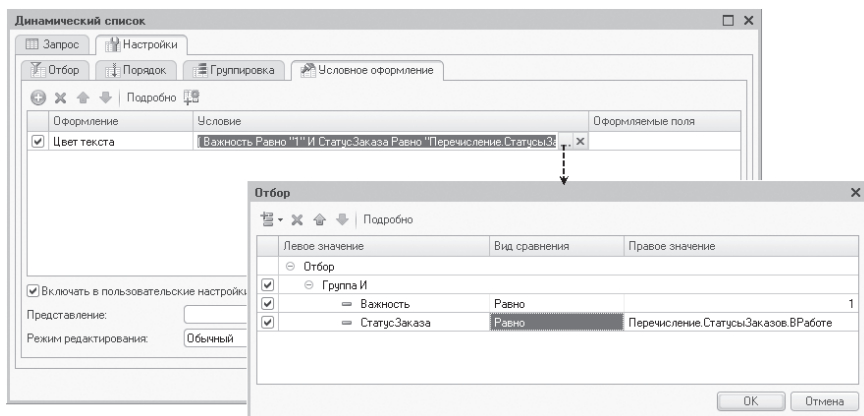


Рис. 2.84. Условное оформление динамического списка заказов

Теперь поместим в форму флажок, при установке которого выполнялся бы программный отбор важных заказов. Для этого создадим реквизит **Важные** числового типа и перетащим его в окно элементов формы над таблицей списка. У элемента формы **Важные** установим свойства Вид – Поле флажка и Заголовок – Показывать только важные. В результате форма списка заказов в редакторе формы будет выглядеть следующим образом (рис. 2.85).

Затем для поля флажка **Важные** создадим обработчик события **ПриИзменении** и заполним его следующим образом (листинг 2.68).

Листинг 2.68. Обработчик события «ПриИзменении» для поля формы «Важные»

```
&НаКлиенте
Процедура ВажныеПриИзменении(Элемент)

    Если Важные = 1 Тогда
        УстановитьФильтрПоВажнымЗаказам();
    Иначе
        УдалитьФильтрПоВажнымЗаказам();
    КонецЕсли;

КонецПроцедуры
```

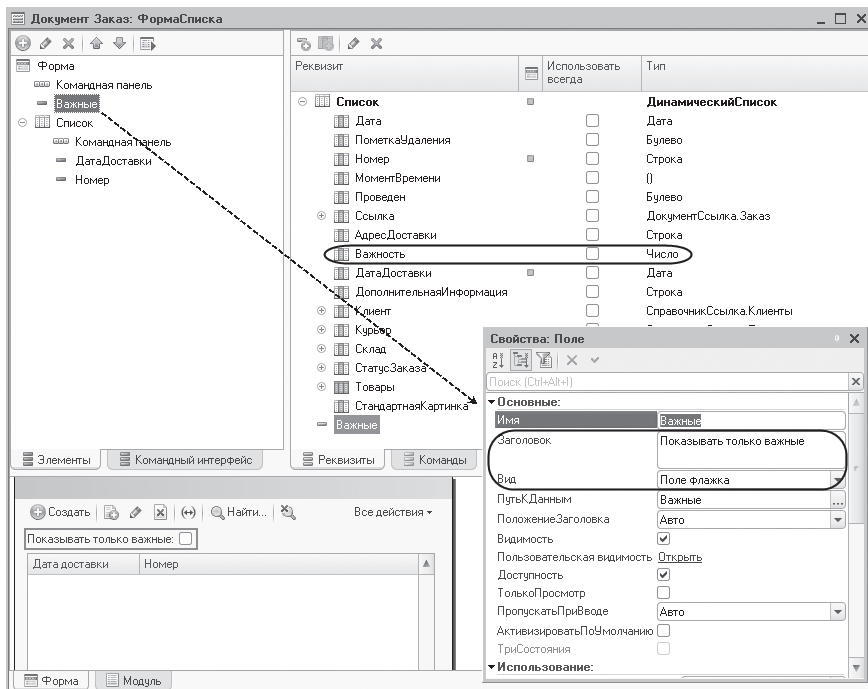


Рис. 2.85. Форма списка заказов в редакторе формы

В этом обработчике мы при изменении состояния флажка вызываем процедуры УстановитьФильтрПоВажнымЗаказам (листинг 2.69) или УдалитьФильтрПоВажнымЗаказам (листинг 2.70).

Листинг 2.69. Процедура «УстановитьФильтрПоВажнымЗаказам»

&НаКлиенте

Процедура УстановитьФильтрПоВажнымЗаказам()

Настройки = Список.КомпоновщикНастроек.Настройки;

ЭлементОтбора = Настройки.Отбор.Элементы.Добавить(Тип("ЭлементОтбораКомпоновкиДанных"));

ЭлементОтбора.ЛевоеЗначение = Новый ПолеКомпоновкиДанных("Важность");

ЭлементОтбора.ВидСравнения = ВидСравненияКомпоновкиДанных.Равно;

ЭлементОтбора.ПравоеЗначение = 1;

Список.КомпоновщикНастроек.ЗагрузитьНастройки(Настройки);

КонецПроцедуры

В процедуре установки фильтра мы сначала получаем настройки динамического списка с помощью его свойства `КомпоновщикНастроек`. Затем в коллекцию настроек добавляем элемент отбора по полю `Важность` (левое значение) и сравниваем ее с единицей (правое значение). И загружаем новые настройки в компоновщик настроек динамического списка методом `ЗагрузитьНастройки`.

Листинг 2.70. Процедура «УдалитьФильтрПоВажнымЗаказам»

```
&НаКлиенте
Процедура УдалитьФильтрПоВажнымЗаказам()

    Настройки = Список.КомпоновщикНастроек.Настройки;
    Для Каждого ЭлементНастроек Из Настройки.Отбор.Элементы Цикл
        Если ЭлементНастроек.ЛевоеЗначение = Новый ПолеКомпоновкиДанных("Важность") Тогда
            Настройки.Отбор.Элементы.Удалить(ЭлементНастроек);
        КонецЕсли;
    КонецЦикла;
    Список.КомпоновщикНастроек.ЗагрузитьНастройки(Настройки);

КонецПроцедуры
```

В процедуре удаления фильтра мы сначала получаем настройки динамического списка с помощью его свойства `КомпоновщикНастроек`. Затем обходим в цикле коллекцию элементов отбора этих настроек. Если в коллекции присутствует элемент отбора, в котором `ЛевоеЗначение` – поле компоновки данных `Важность`, то этот элемент удаляется. Затем измененные настройки загружаются в компоновщик настроек.

Таким образом, при открытии списка заказов важные заказы будут выделены цветом, и внимание курьера сразу будет привлечено к ним. Кроме того, установив флажок `Показывать только важные`, курьер может отобразить только важные заказы из общего списка заказов. Открыв такой заказ, в поле `Дополнительная информация` (оно начинается с восклицательного знака) курьер может прочитать важную для себя информацию (рис. 2.86).

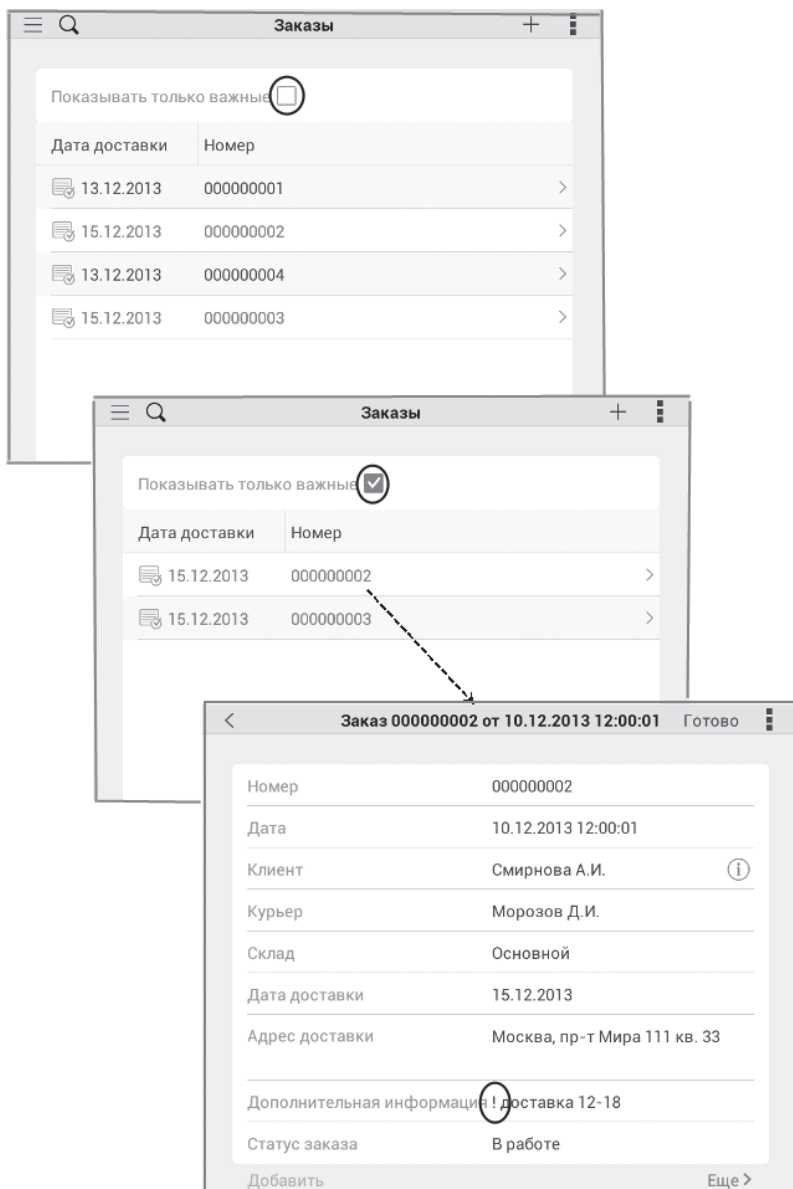


Рис. 2.86. Условное оформление и отбор в списке заказов

Обслуживание заказов

Ранее мы уже создавали форму списка документа, но занимались только ее внешним видом (см. раздел «Документ "Обслуживание заказов"» на стр. 76).

По условию нашей задачи курьер не может редактировать документ ОбслуживаниеЗаказов, поскольку этот документ формируется в самом интернет-магазине и определяет порядок обслуживания заказов.

Поскольку правила ограничения доступа к данным для роли Основная не позволяют редактировать документ, то форма документа открывается только в режиме просмотра. Поэтому из табличной части документа Обслуживание заказов нельзя открыть заказ.

Однако для удобства работы курьера надо предусмотреть такую возможность. Например, вероятно следующая последовательность действий (рис. 2.87):

- курьер открывает список обслуживания заказов;
- выбирает из списка конкретный документ и открывает его;
- в табличной части документа Обслуживание заказов курьер вызывает контекстное меню конкретного заказа, выполняет команду Открыть заказ и открывает форму выбранного заказа, в которой может с ним работать.

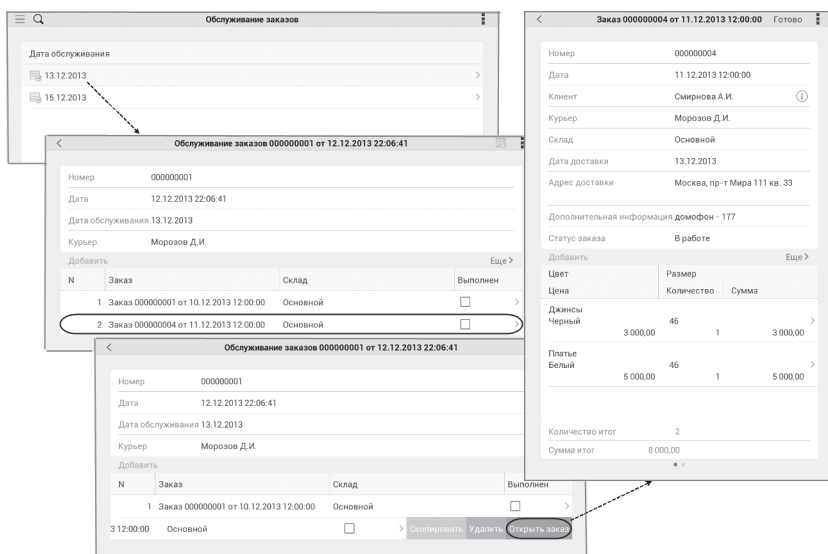


Рис. 2.87. Открытие заказа из документа «Обслуживание заказов»

Чтобы предусмотреть такую возможность, создадим форму документа ОбслуживаниеЗаказов. Затем на закладке Команды добавим команду ОткрытьЗаказ и перетащим ее в командную панель формы. В палитре свойств этой команды свойство Использование текущей строки установим в значение Использует и в свойстве Используемая таблица выберем таблицу с данными табличной части документа Заказы (рис. 2.88).

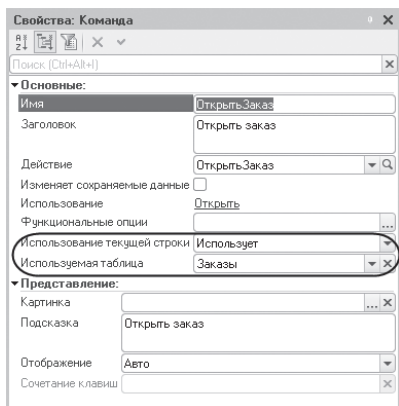


Рис. 2.88. Свойства команды «ОткрытьЗаказ»

С помощью этих свойств мы показываем, что команда будет использовать данные текущей строки таблицы Заказы. Эти данные нам нужны для получения ссылки на заказ, который мы хотим открыть.

Затем создадим клиентский обработчик этой команды и заполним его следующим образом (листинг 2.71).

Листинг 2.71. Обработчик команды «ОткрытьЗаказ»

```
&НаКлиенте
Процедура ОткрытьЗаказ(Команда)

    ПоказатьЗначение(, Элементы.Заказы.ТекущиеДанные.Заказ);

КонецПроцедуры
```

В этом обработчике мы вызываем метод глобального контекста ПоказатьЗначение() и передаем в него ссылку на заказ, форму которого нужно открыть. Чтобы получить эту ссылку, мы обращаемся к таблице формы Заказы, которая содержит данные табличной части документа. Используя свойство ТекущиеДанные этой таблицы, мы получаем объект, содержащий данные, находящиеся в текущей строке таблицы. Обращаясь через точку

к полю этого объекта Заказ, мы получаем ссылку на конкретный заказ, содержащуюся в данном поле в текущей строке таблицы.

Поскольку описанный выше сценарий действий кажется нам наиболее удобным и вероятным, можно поместить форму списка документа ОбслуживаниеЗаказов на начальную страницу нашего мобильного приложения.

Для этого выделим корень конфигурации КурьерИнтернетМагазина и выполним команду контекстного меню Открыть рабочую область начальной страницы. Добавим строку в левую колонку начальной страницы и выберем форму списка документа ОбслуживаниеЗаказов. В поле Отображение командного интерфейса основного раздела укажем значение Снизу (рис. 2.89).

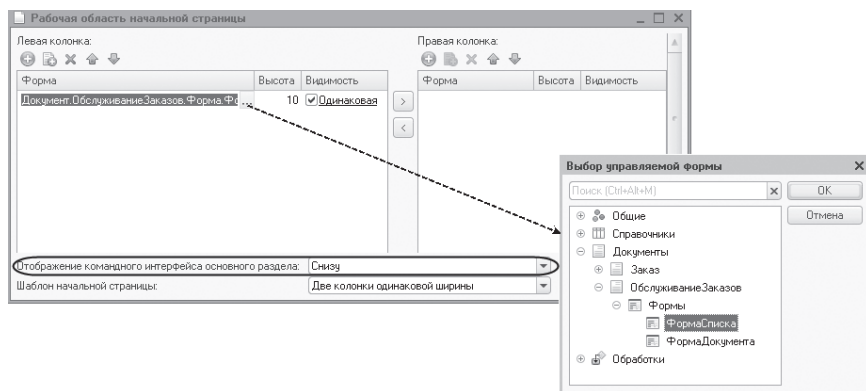


Рис. 2.89. Рабочая область начальной страницы

В результате при запуске мобильного приложения на планшете курьер увидит список заказов, отсортированный по дате обслуживания, и сможет работать с заказами, «не делая лишних движений». При этом в нижней половине экрана размещаются команды интерфейса основного раздела, и он может при необходимости выполнять другие команды, не вызывая главное меню приложения (рис. 2.90).

Заметьте, что если все команды уже не помещаются на один экран (например, при горизонтальной ориентации планшета), то они размещаются на нескольких страницах в нижней половине рабочего стола. Перейти к нужной странице можно с помощью стандартного жеста пролистывания.

Если вам мешают команды интерфейса основного раздела, то при настройке начальной страницы приложения нужно установить в поле Отображение командного интерфейса основного раздела значение Не использовать.

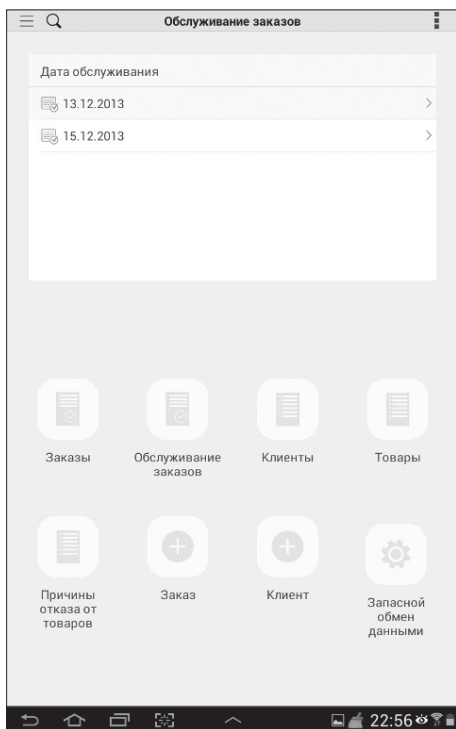


Рис. 2.90. Начальная страница приложения

Отчет по данным мобильного приложения

В следующей главе с помощью системы компоновки данных мы создадим в основном приложении несколько отчетов, характеризующих работу интернет-магазина по продаже товаров, и затем покажем, как с помощью веб-сервиса курьер может удаленно формировать нужный ему отчет в основном приложении и получать готовый отчет на свое мобильное устройство.

Но пока мы предоставим курьеру возможность проанализировать некоторые данные, которые уже есть на его мобильном устройстве, основанные на обслуженных этим курьером заказах.

Поскольку система компоновки данных в мобильном приложении недоступна, получим данные с помощью запроса к таблице заказов и выведем их в табличный документ и диаграмму.

Итак, создадим в нашем мобильном приложении обработку АнализЗаказов. В дерево элементов формы добавим группу Группа1 вида Страницы и установим свойство этой группы ОтображениеСтраниц в значение Прокли- стывание (сенсорный экран). Вложим в нее две группы ЗаказыКлиентов и ЗаказыТоваров вида Страница.

Затем добавим реквизиты формы Результат типа ТабличныйДокумент и ДиаграммаЗаказов типа Диаграмма и перетащим их поочередно в каждую из страниц. Установим у соответствующих элементов формы свойство ПоложениеЗаголовка в значение Нет.

Затем добавим команды формы ВыводВТабличныйДокумент и ВыводВДиаграмму и также перетащим их в каждую из страниц так, чтобы они располагались сверху. Зададим у соответствующих им кнопок заголовки ЗаказыКлиентов и ЗаказыТоваров. Ничего, что они совпадают с заголовками страниц. Дело в том, что на планшете мы не увидим привычных страниц: они будут отображаться в виде двух точек внизу экрана с возможностью пролистывания.

В результате в конфигураторе форма обработки должна принять следующий вид (рис. 2.91).

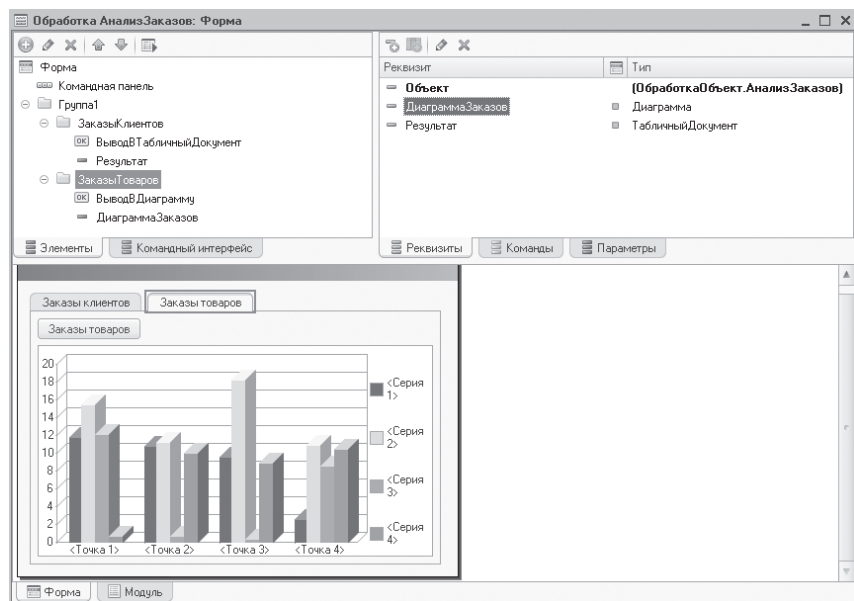


Рис. 2.91. Форма обработки «АнализЗаказов» в конфигураторе

Заказы клиентов (табличный документ)

Предположим, при нажатии на кнопку Заказы клиентов в форме обработки курьер хочет видеть список обслуженных им заказов, сгруппированных по клиентам и товарам, которые клиент купил, с выводом итоговых сумм по каждому товару и клиенту. Для этого сформируем запрос к таблице заказов и выведем результат запроса в табличный документ.

Создадим обработчик команды ВыводВТабличныйДокумент и заполним его следующим образом (листинг 2.72).

Листинг 2.72. Обработчик команды «ВыводВТабличныйДокумент»

```
&НаКлиенте  
Процедура ВыводВТабличныйДокумент(Команда)  
  
    ЗаполнитьТД(Результат);  
  
КонiecПроцедуры
```

В обработчике ВыводВТабличныйДокумент() мы вызываем серверную внеконтекстную процедуру ЗаполнитьТД() и передаем туда реквизит формы Результат для заполнения табличного документа данными запроса.

Для заполнения табличного документа мы для простоты и удобства будем использовать конструктор запроса с обработкой результата. Создадим шаблон процедуры ЗаполнитьТД() в модуле формы обработки, установим курсор в тело процедуры и вызовем из контекстного меню конструктор запроса с обработкой результата. Подтвердим, что мы хотим создать новый запрос.

На первой закладке конструктора Обработка результатов в группе Тип обработки выберем опцию Вывод в табличный документ, все остальные опции оставим без изменения (рис. 2.92).

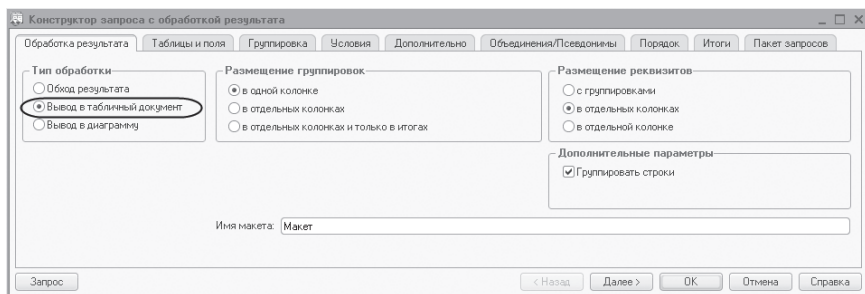


Рис. 2.92. Вывод данных запроса в табличный документ с помощью конструктора

Затем определим исходные данные для запроса. На закладке Таблицы и поля перенесем табличную часть Товары документа Заказ в список источников запроса и выберем из этой таблицы поля: Товар, Количество и Сумма. Для того чтобы выбрать поля основной таблицы документа Клиент и ДатаДоставки, раскроем поле Ссылка табличной части Товары, выберем эти поля и получим обращение «через точку» от ссылки – ЗаказТовары.Ссылка.Клиент и ЗаказТовары.Ссылка.ДатаДоставки.

Обратите внимание, что при выборе из таблицы ЗаказТовары ссылочных полей Клиент и Товар конструктор автоматически добавил в список выборки текстовое представление этих полей, полученное с помощью функции Представление(). Это позволит быстрее выполнить и вывести результат запроса (рис. 2.93).

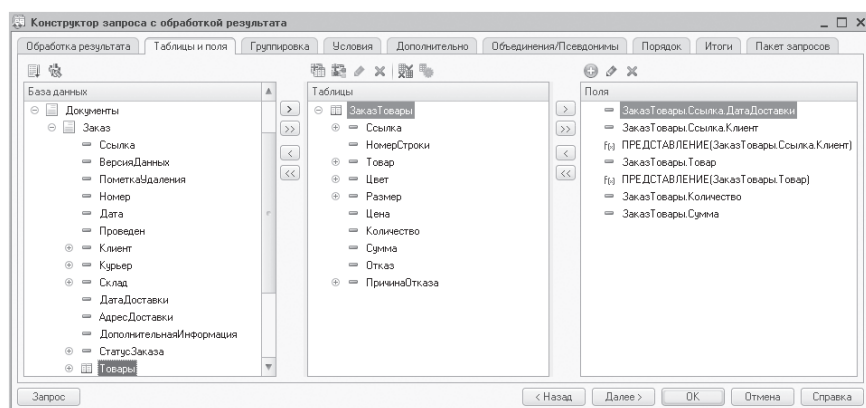


Рис. 2.93. Вывод данных запроса в табличный документ с помощью конструктора

На закладке Условия зададим условия отбора записей из таблицы заказов. Перенесем в список условий поля запроса ЗаказТовары.Ссылка.СтатусЗаказа и ЗаказТовары.Отказ. Укажем, что условия будут произвольными, и изменим их следующим образом (рис. 2.94, листинги 2.73, 2.74).

Листинг 2.73. Произвольное условие отбора записей

```
ЗаказТовары.Ссылка.СтатусЗаказа = ЗНАЧЕНИЕ(Перечисление.СтатусыЗаказов.Выполнен)
ИЛИ ЗаказТовары.Ссылка.СтатусЗаказа = ЗНАЧЕНИЕ(Перечисление.СтатусыЗаказов.Закрыт)
```

Листинг 2.74. Произвольное условие отбора записей

```
ЗаказТовары.Отказ = ЛОЖЬ
```

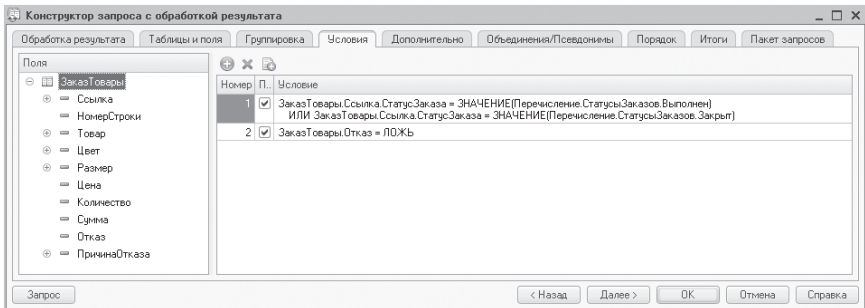



Рис. 2.94. Вывод данных запроса в табличный документ с помощью конструктора

Тем самым мы задали, что в результат запроса будут попадать только заказы, уже обслуженные курьером (со статусом Выполнен или Закрыт), и только те товары из списка заказанных товаров, которые клиент купил.

На закладке Порядок укажем, что записи в результате запроса будут сортироваться по возрастанию поля ДатаДоставки.

Затем перейдем на закладку Итоги и укажем, какие итоги нужно рассчитать для результата запроса. В окно Группировочное поле перенесем поля выборки запроса Клиент и Товар, а в окно Итоговое поле перенесем поля Количество и Сумма. Тем самым мы указали, что для каждого клиента по каждому товару из документа Заказ нужно подсчитать суммарное количество полей Количество и Сумма (рис. 2.95).

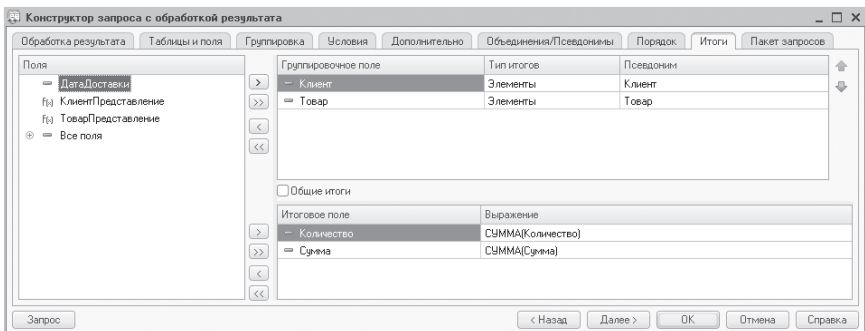


Рис. 2.95. Вывод данных запроса в табличный документ с помощью конструктора

Нажмем ОК. Конструктор запроса с обработкой результата создаст подчиненный объект обработки АнализЗаказов – макет табличного документа с полями, перечисленными в выборке запроса. Исходя из текста запроса, в макете присутствуют области итоговых группировок Клиент, Товар и область для вывода детальных записей, а также области для вывода заголовка и подвала табличного документа, шапки и подвала таблицы.

В процедуре ЗаполнитьТД() конструктор сформирует следующий код (листинг 2.75).

Листинг 2.75. Процедура «ЗаполнитьТД»

```
&НаСервереБезКонтекста
Процедура ЗаполнитьТД(Табдок)
```

```
// {{КОНСТРУКТОР_ЗАПРОСА_С_ОБРАБОТКОЙ_РЕЗУЛЬТАТА
// Данный фрагмент построен конструктором.
// При повторном использовании конструктора внесенные вручную изменения будут утеряны!!!

Макет = Обработки.АнализЗаказов.ПолучитьМакет("Макет");
Запрос = Новый Запрос;
Запрос.Текст =
    "ВЫБРАТЬ
    |      ЗаказТовары.Ссылка.ДатаДоставки КАК ДатаДоставки,
    |      ЗаказТовары.Ссылка.Клиент КАК Клиент,
    |      ПРЕДСТАВЛЕНИЕ(ЗаказТовары.Ссылка.Клиент),
    |      ЗаказТовары.Товар КАК Товар,
    |      ПРЕДСТАВЛЕНИЕ(ЗаказТовары.Товар),
    |      ЗаказТовары.Количество КАК Количество,
    |      ЗаказТовары.Сумма КАК Сумма
    |
    | ИЗ
    |
    | Документ.Заказ.Товары КАК ЗаказТовары
    |
    | ГДЕ
    |
    |      (ЗаказТовары.Ссылка.СтатусЗаказа = ЗНАЧЕНИЕ(Перечисление.СтатусыЗаказов.Выполнен)
    |      ИЛИ ЗаказТовары.Ссылка.СтатусЗаказа =
    |          ЗНАЧЕНИЕ(Перечисление.СтатусыЗаказов.Закрыт))
    |
    |      И ЗаказТовары.Отказ = ЛОЖЬ
    |
    | УПОРЯДОЧИТЬ ПО
    |      ДатаДоставки
    |
    | ИТОГИ
    |
    |      СУММА(Количество),
    |      СУММА(Сумма)
    |
    | ПО
    |
    |      Клиент,
    |      Товар";

РезультатЗапроса = Запрос.Выполнить();

ОбластьЗаголовок = Макет.ПолучитьОбласть("Заголовок");
ОбластьПодвал = Макет.ПолучитьОбласть("Подвал");
ОбластьШапкаТаблицы = Макет.ПолучитьОбласть("ШапкаТаблицы");
ОбластьПодвалТаблицы = Макет.ПолучитьОбласть("ПодвалТаблицы");
```

```
ОбластьКлиент = Макет.ПолучитьОбласть("Клиент");
ОбластьТовар = Макет.ПолучитьОбласть("Товар");
ОбластьДетальныхЗаписей = Макет.ПолучитьОбласть("Детали");

ТабДок.Очистить();
ТабДок.Вывести(ОбластьЗаголовков);
ТабДок.Вывести(ОбластьШапкаТаблицы);
ТабДок.НачатьАвтогруппировкуСтрок();

ВыборкаКлиент = РезультатЗапроса.Выбрать(ОбходРезультатаЗапроса.ПоГруппировкам);

Пока ВыборкаКлиент.Следующий() Цикл
    ОбластьКлиент.Параметры.Заполнить(ВыборкаКлиент);
    ТабДок.Вывести(ОбластьКлиент, ВыборкаКлиент.Уровень());

    ВыборкаТовар = ВыборкаКлиент.Выбрать(ОбходРезультатаЗапроса.ПоГруппировкам);

    Пока ВыборкаТовар.Следующий() Цикл
        ОбластьТовар.Параметры.Заполнить(ВыборкаТовар);
        ТабДок.Вывести(ОбластьТовар, ВыборкаТовар.Уровень());

        ВыборкаДетальныеЗаписи = ВыборкаТовар.Выбрать();

        Пока ВыборкаДетальныеЗаписи.Следующий() Цикл
            ОбластьДетальныхЗаписей.Параметры.Заполнить(ВыборкаДетальныеЗаписи);
            ТабДок.Вывести(ОбластьДетальныхЗаписей, ВыборкаДетальныеЗаписи.Уровень());
        КонецЦикла;
    КонецЦикла;
КонецЦикла;

ТабДок.ЗакончитьАвтогруппировкуСтрок();
ТабДок.Вывести(ОбластьПодвалТаблицы);
ТабДок.Вывести(ОбластьПодвал);

// }}КОНСТРУКТОР_ЗАПРОСА_С_ОБРАБОТКОЙ_РЕЗУЛЬТАТА

КонецПроцедуры
```

В процедуре ЗаполнитьТД() сначала методом ПолучитьМакет() получается макет табличного документа с именем, указанным в конструкторе. Затем создается и выполняется запрос, определенный в конструкторе. Затем методом ПолучитьОбласть() получают области табличного документа, заданные в макете. После этого табличный документ ТабДок, переданный в процедуру, очищается, и в него методом Вывести() выводятся области заголовка и шапки таблицы, полученные из макета. Затем начинается автоматическая группировка строк табличного документа методом НачатьАвтогруппировкуСтрок(), так как в конструкторе запроса с обработкой результата была по умолчанию включена опция Группировать строки.

Затем начинается обход трех вложенных друг в друга выборок – итоговых группировок Клиент, Товар и детальных записей. В цикле обхода каждой

выборки параметры соответствующих областей табличного документа, определенные в макете, заполняются данными выборок, и затем области с указанием уровня выборки выводятся в табличный документ методом Вывести(). Параметр Уровень этого метода используется для автоматической группировки строк результирующего табличного документа.

В заключение заканчивается автоматическая группировка строк табличного документа, и в него выводятся области подвала документа и подвала таблицы, полученные из макета.

Заказы товаров (диаграмма)

Теперь предположим, что при нажатии на кнопку Заказы товаров в форме обработки курьер хочет видеть диаграмму (в виде гистограммы), построенную на основе данных о купленных товарах в разрезе их цветов. Таким образом можно, например, легко проанализировать, какой товар и какого цвета пользуется наибольшим спросом. Для этого сформируем запрос к таблице заказов и выведем результат запроса в диаграмму.

Создадим обработчик команды ВыводВДиаграмму и заполним его следующим образом (листинг 2.76).

Листинг 2.76. Обработчик команды «ВыводВДиаграмму»

```
&НаКлиенте  
Процедура ВыводВДиаграмму(Команда)  
  
    ЗаполнитьДиаграмму(ДиаграммаЗаказов);  
  
КонецПроцедуры
```

В обработчике ВыводВДиаграмму() мы вызываем серверную внеконтекстную процедуру ЗаполнитьДиаграмму() и передаем туда реквизит формы ДиаграммаЗаказов для заполнения диаграммы данными запроса.

Для заполнения диаграммы мы для простоты и удобства будем использовать конструктор запроса с обработкой результата. Создадим шаблон процедуры ЗаполнитьДиаграмму() в модуле формы обработки, установим курсор в тело процедуры и вызовем из контекстного меню конструктор запроса с обработкой результата. Подтвердим, что мы хотим создать новый запрос.

На первой закладке конструктора Обработка результатов в группе Тип обработки выберем опцию Вывод в диаграмму.

Затем определим исходные данные для запроса. На закладке Таблицы и поля перенесем табличную часть Товары документа Заказ в список источников запроса и выберем из этой таблицы поля: Товар, Цвет, Размер и Количество (рис. 2.96).

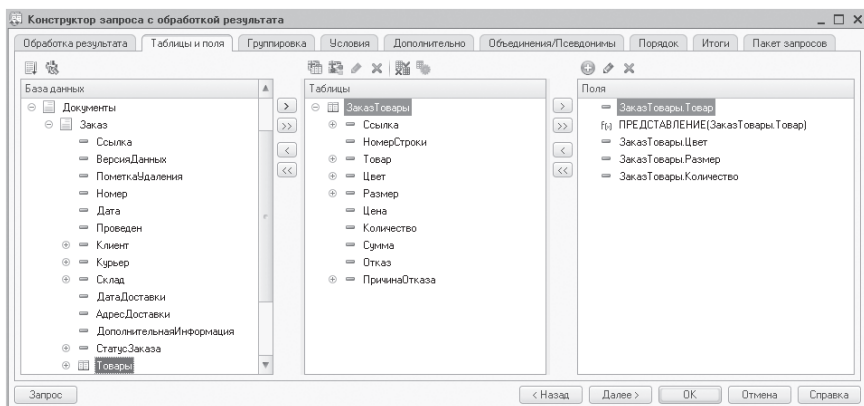


Рис. 2.96. Вывод данных запроса в диаграмму с помощью конструктора

На закладке **Условия** зададим условия отбора записей из таблицы заказов. Как и в предыдущем случае, в результат запроса должны попадать только заказы, уже обслуженные курьером (со статусом **Выполнен** или **Закрыт**), и только те товары из списка заказанных товаров, которые клиент купил (см. листинги 2.73, 2.74).

На закладке **Итоги** укажем, какие итоги нужно рассчитать для результата запроса. В окно **Группировочное поле** перенесем поля выборки запроса **Цвет** и **Товар**, а в окно **Итоговое поле** перенесем поле **Количество**. Тем самым мы указали, что для каждого цвета по каждому товару из документа **Заказ** нужно подсчитать суммарное количество заказанных товаров (рис. 2.97).

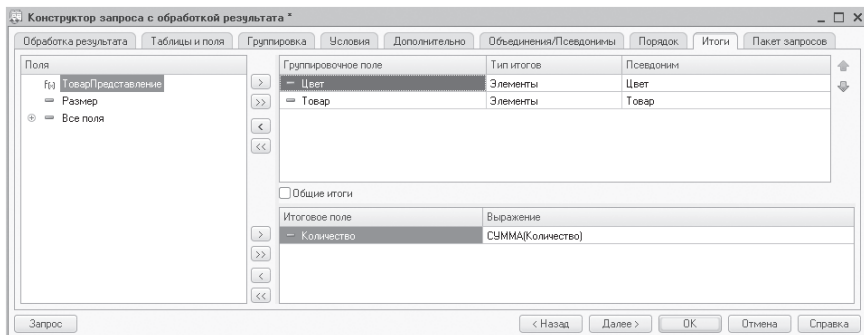


Рис. 2.97. Вывод данных запроса в диаграмму с помощью конструктора

Нажмем ОК. В процедуре ЗаполнитьДиаграмму() конструктор запроса с обработкой результата сформирует код, в который мы добавим фрагмент, выделенный жирным шрифтом (листинг 2.77).

Листинг 2.77. Процедура «ЗаполнитьДиаграмму»

&НаСервереБезКонтекста

Процедура ЗаполнитьДиаграмму(Диаграмма)

```
// {{КОНСТРУКТОР_ЗАПРОСА_С_ОБРАБОТКОЙ_РЕЗУЛЬТАТА
// Данный фрагмент построен конструктором.
// При повторном использовании конструктора, внесенные вручную изменения будут утеряны!!!

Запрос = Новый Запрос;
Запрос.Текст =
    "ВЫБРАТЬ
    |     ЗаказТовары.Товар КАК Товар,
    |     ПРЕДСТАВЛЕНИЕ(ЗаказТовары.Товар),
    |     ЗаказТовары.Цвет КАК Цвет,
    |     ЗаказТовары.Размер,
    |     ЗаказТовары.Количество КАК Количество
    | ИЗ
    |     Документ.Заказ.Товары КАК ЗаказТовары
    | ГДЕ
    |     (ЗаказТовары.Ссылка.СтатусЗаказа = ЗНАЧЕНИЕ(Перечисление.СтатусыЗаказов.Выполнен)
    |     ИЛИ ЗаказТовары.Ссылка.СтатусЗаказа =
    |         ЗНАЧЕНИЕ(Перечисление.СтатусыЗаказов.Закрыт))
    | И ЗаказТовары.Отказ = ЛОЖЬ
    | ИТОГИ
    |     СУММА(Количество)
    | ПО
    |     Товар,
    |     Цвет";

РезультатЗапроса = Запрос.Выполнить();

Диаграмма.Обновление = Ложь;
Диаграмма.Очистить();
Диаграмма.АвтоТранспонирование = Ложь;

ВыборкаЦвет = РезультатЗапроса.Выбрать(ОбходРезультатаЗапроса.ПоГруппировкам);

Пока ВыборкаЦвет.Следующий() Цикл
    Серия = Диаграмма.УстановитьСерию(ВыборкаЦвет.Цвет);
    Серия.Текст = ВыборкаЦвет.Цвет;
    Серия.Расшифровка = ВыборкаЦвет.Цвет;

    Если ВыборкаЦвет.Цвет.Наименование = "Красный" Тогда
        Серия.Цвет = WEBЦвета.Красный;
    ИначеЕсли ВыборкаЦвет.Цвет.Наименование = "Черный" Тогда
        Серия.Цвет = WEBЦвета.Черный;
    ИначеЕсли ВыборкаЦвет.Цвет.Наименование = "Фиолетовый" Тогда
        Серия.Цвет = WEBЦвета.Фиолетовый;
    КонецЕсли;
```

```
ВыборкаТовар = ВыборкаЦвет.Выбрать(ОбходРезультатаЗапроса.ПоГруппировкам);

Пока ВыборкаТовар.Следующий() Цикл
    Точка = Диаграмма.УстановитьТочку(ВыборкаТовар.Товар);
    Точка.Текст = ВыборкаТовар.ТоварПредставление;
    Точка.Расшифровка = ВыборкаТовар.Товар;
    Диаграмма.УстановитьЗначение(Точка, Серия, ВыборкаТовар.Количество);

    ВыборкаДетальныеЗаписи = ВыборкаТовар.Выбрать();

    Пока ВыборкаДетальныеЗаписи.Следующий() Цикл
        // Вставить обработку выборки ВыборкаДетальныеЗаписи
        КонецЦикла;
    КонецЦикла;

    Диаграмма.АвтоТранспонирование = Истина;
    Диаграмма.Обновление = Истина;

// }}КОНСТРУКТОР_ЗАПРОСА_С_ОБРАБОТКОЙ_РЕЗУЛЬТАТА

КонецПроцедуры
```

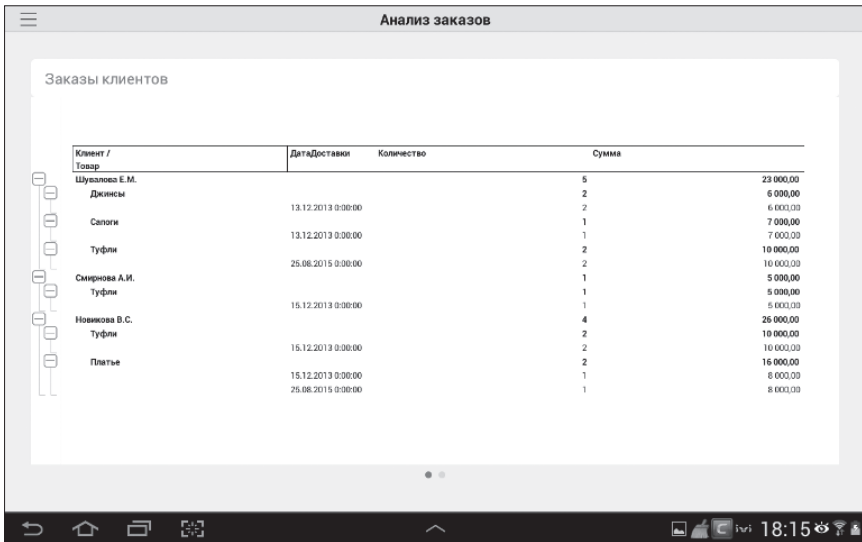
В процедуре ЗаполнитьДиаграмму() сначала создается и выполняется запрос, определенный в конструкторе. Затем диаграмма Диаграмма, переданная в процедуру, очищается, и на время ее заполнения данными отключаются обновление и автотранспонирование диаграммы. Тип диаграммы здесь не определяется, но по умолчанию он устанавливается как Гистограмма объемная.

Далее начинается обход двух вложенных друг в друга выборок – итоговых группировок Цвет и Товар. В цикле обхода первой выборки ВыборкаЦвет методом УстановитьСерию() определяются серии диаграммы. В цикле обхода вложенной в нее выборки ВыборкаТовар методом УстановитьТочку() определяются точки диаграммы и методом УстановитьЗначение() устанавливается значение серии диаграммы в точке.

В текст, сформированный конструктором, мы добавили фрагмент для определения цветов серий. Поскольку сериями диаграммы являются цвета товаров, то для наглядности мы задали в явном виде, чтобы цвета серий соответствовали названиям цветов. Это сделано «в лоб» для упрощения. На самом же деле цвета следовало бы хранить в справочнике цветов, в специальном реквизите типа хранилища значения, и выбирать их в диалоге выбора цвета. И затем устанавливать цвет серии диаграммы из справочника. В данном случае все это не является нашей прямой задачей, поэтому мы просто упростили наш отчет.

В заключение после заполнения данными диаграммы включаются ее обновление и автотранспонирование. Это позволяет избежать многократной перерисовки диаграммы во время ее формирования.

Проверим, как работает наш отчет на планшете. Из главного меню приложения в группе Сервис выполним команду для открытия обработки Анализ заказов. Нажмем на кнопку Заказы клиентов и выведем список заказов в табличный документ (рис. 2.98).



Клиент / Товар	Дата доставки	Количество	Сумма
Шувалова Е.М. Джинсы	13.12.2013 0:00:00	5	23 000,00
		2	6 000,00
		2	6 000,00
		1	7 000,00
Салопи	13.12.2013 0:00:00	1	7 000,00
		2	10 000,00
		2	10 000,00
		1	5 000,00
Смирнова А.И. Туфли	25.06.2015 0:00:00	1	5 000,00
		1	5 000,00
		1	5 000,00
		1	5 000,00
Новикова В.С. Туфли	15.12.2013 0:00:00	4	20 000,00
		2	10 000,00
		2	10 000,00
		2	16 000,00
Платье	15.12.2013 0:00:00	1	8 000,00
		1	8 000,00

Рис. 2.98. Отчет «Заказы клиентов»

Мы видим, что в табличном документе записи сгруппированы по клиентам и товарам, по ним рассчитаны итоги, и мы можем свернуть или развернуть эти уровни благодаря группировке строк табличного документа.

Кроме того, мобильная платформа стандартно поддерживает возможность расшифровки в табличном документе – можно выделить ячейку документа и нажать кнопку Расшифровать в командной панели, появившейся вверху формы обработки. В результате будет открыта, например, форма товара или клиента, название которого отображено в выбранной ячейке отчета.

Теперь с помощью пролистывания вправо перейдем на другую страницу обработки и нажмем кнопку Заказы товаров. Мы увидим диаграмму (в виде гистограммы), построенную на основе данных о купленных товарах в разрезе их цветов (рис. 2.99).

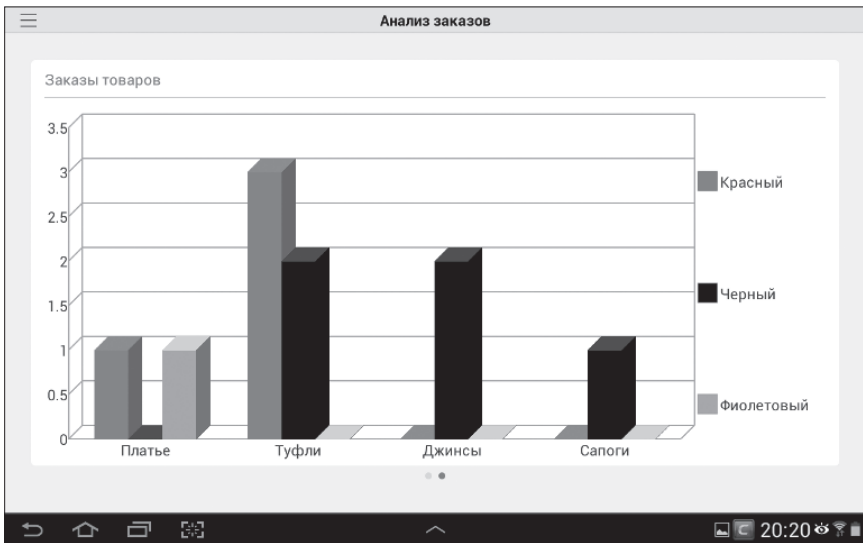


Рис. 2.99. Отчет «Заказы Товаров»

При касании названия товара или цвета в поле диаграммы сразу же открывается форма соответствующего элемента справочника.

ПРИМЕЧАНИЕ

Отчеты Заказы клиентов и Заказы товаров построены на данных, получившихся в мобильном приложении после выполнения обмена данными с основным приложением (см. раздел «Тестирование обмена данными в обе стороны» на стр. 276). Иначе данные отчетов были бы неинформативными.

Глава 3.

Разработка основного приложения

Скажем сразу, что при разработке основного приложения Интернет-магазин мы не ставим задачу создать конфигурацию, полностью охватывающую все реальные потребности и нюансы, возникающие в работе интернет-магазина. Мы реализуем в основном приложении только те функции, которые необходимы для совместной работы курьера интернет-магазина и офиса.

В первую очередь это ведение справочной информации и документов о поступлении товаров. Затем в офисе создаются заказы товаров и отправляются курьеру на планшет.

После выполнения заказов курьером и получения их обратно в офис на основании заказов создаются документы о продажах.

Затем документы проводятся и производят движения в регистрах накопления, на основе которых формируются отчеты об остатках товаров, о причинах отказа от товаров, о продажах клиентам и т. п.

Итак, в основном приложении мы реализуем следующие функции:

- работа с заказами;
- работа с хранимыми файлами;
- выбор картинки товара и ее сохранение в справочнике хранимых файлов;
- работа с клиентами;
- учет поступлений товаров;
- учет продаж товаров;

- проведение заказов, документов о поступлении и продаже товаров и формирование движений в регистрах накопления;
- формирование отчетов на основе данных регистров накопления.

Все описанные ниже действия по разработке основного приложения содержатся в выгрузке информационной базы «Интернет-магазин.dt».

Создание и редактирование информации для отправки курьеру

Прежде всего займемся формами объектов конфигурации, которые поставляют информацию для работы курьера на планшете, и наполним их нужной функциональностью.

В плане разработки интерфейса наша задача облегчается (по сравнению с мобильным приложением), так как формы большинства объектов, по умолчанию генерируемые платформой, позволяют редактировать информацию в них, что и требуется в основном приложении.

Кроме того, здесь мы не слишком озабочены экономией места и компактностью интерфейса, поэтому большинство автоматически генерируемых форм устраивают нас своим внешним видом. Например, формы справочников ПричиныОтказа, Пользователи, Склады, документа ОбслуживаниеЗаказов, регистра сведений ЦеныТоваров мы вообще создавать не будем, поскольку нам вполне достаточно того, что предлагает платформа.

Заказы

С точки зрения функциональности мы должны реализовать работу с заказами в основном приложении по аналогии с тем, что мы делали в мобильном приложении, за исключением функций, доступных только на мобильных устройствах.

Создадим форму документа Заказ. Сначала немного усовершенствуем внешний вид формы заказа.

Прежде всего, реквизиты шапки документа для большей читабельности расположим в две колонки. Для этого добавим в дерево элементов формы обычную группу без отображения Шапка с горизонтальной группировкой. Вложим в эту группу две обычные группы без отображения – ЛеваяКолонка и ПраваяКолонка – с вертикальной группировкой. В первую группу перетащим поля Номер, Клиент, Курьер, Склад, СтатусЗаказа. Во вторую

группу перенесем поля Дата, ДатаДоставки, АдресДоставки, ДополнительнаяИнформация.

Кроме того, чтобы отделить информацию о заказанных товарах, их ценах, количестве и т.п. от информации об отказе от товаров, используем группы страниц, содержащие таблицы с различной информацией о товарах.

Для этого добавим над таблицей Товары группу ГруппаТовары вида Страницы. Затем вложим в нее две группы – СписокТоваров и ОтказОтТоваров – вида Страница. В группу СписокТоваров перетащим таблицу Товары и удалим из нее поля ТоварыОтказ и ТоварыПричинаОтказа.

Затем раскроем основной реквизит формы Объект и перетащим в группу ОтказОтТоваров табличную часть документа Товары. Согласимся с созданием в форме колонок таблицы (по умолчанию она будет названа Товары1). Затем удалим из этой таблицы поля Товары1Цвет, Товары1Размер, Товары1Цена, Товары1Количество, Товары1Сумма.

После этого дерево элементов формы документа в редакторе формы будет выглядеть следующим образом (рис. 3.1).

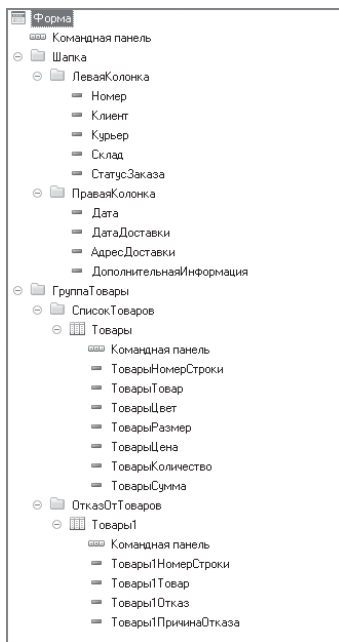


Рис. 3.1. Дерево элементов формы заказа в редакторе формы в конфигураторе

В заключение придадим полю СтатусЗаказа, которое имеет тип перечисления СтатусыЗаказов, вид переключателя в виде тумблера. Для этого в палитре свойств поля СтатусЗаказа установим свойство Вид в значение Поле переключателя, Вид переключателя в значение Тумблер, а для свойства Список выбора зададим список значений, состоящий из элементов перечисления СтатусыЗаказов (рис. 3.2).

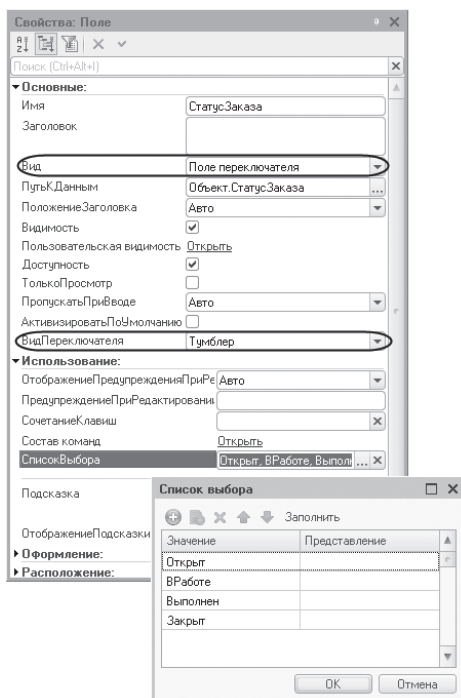


Рис. 3.2. Палитра свойств поля «СтатусЗаказа»

А также в палитре свойств поля ДополнительнаяИнформация установим свойство Заголовок как Дополнительно.

В окне предварительного просмотра редактора формы документа мы можем оценить внешний вид получившейся формы заказа (рис. 3.3).

Теперь наполним форму функциональностью, описанной во второй главе в разделе «Заказы» на стр. 85. Здесь мы не будем еще раз подробно объяснять все детали, а лишь кратко перечислим сделанные изменения и их назначение.

Провести и закрыть Записать Провести Создать на основании Еще

Номер: Дата: Клиент: Дата доставки: Курьер: Адрес доставки: Склад: Дополнительно: Статус заказа: Открыт ВРаботе Выполнен Закрыт Список товаров Отказ от товаров Добавить Еще

N	Товар	Цвет	Размер	Цена	Количес...	Сумма
---	-------	------	--------	------	------------	-------

Рис. 3.3. Форма документа «Заказ» в редакторе формы в конфигураторе

Расчет суммы по товару

Прежде всего реализуем алгоритм расчета суммы по товару как произведения его цены на количество.

Цена товара на определенную дату задается менеджером интернет-магазина и хранится в периодическом регистре сведений Цены товаров.

При изменении или добавлении товаров в заказ цена товара из регистра сведений должна умножаться на его количество, введенное вручную, и записываться в колонку Сумма в измененной строке таблицы заказанных товаров. Для этого в обработчиках события ПриИзменении этих полей таблицы мы должны задать необходимый алгоритм расчета.

Для этого создадим клиентские обработчики события ПриИзменении для полей таблицы ТоварыЦена и ТоварыКоличество и заполним их следующим образом (листинг 3.1).

Листинг 3.1. Обработчик события «ПриИзменении» для поля «ТоварыЦена»

```
&НаКлиенте
Процедура ТоварыЦенаПриИзменении(Элемент)
    СтрокаТабличнойЧасти = Элементы.Товары.ТекущиеДанные;
    СтрокаТабличнойЧасти.Сумма = СтрокаТабличнойЧасти.Количество * СтрокаТабличнойЧасти.Цена;
КонецПроцедуры
```

В переменной СтрокаТабличнойЧасти мы получаем доступ к данным колонок, содержащимся в текущей строке табличной части, и рассчитываем сумму как произведение цены на количество.

А для поля ТоварыСумма в палитре свойств установим свойство ТолькоПрочтено в значение Истина, так как это поле будет расчетным и не должно изменяться вручную.

Проверим изменения. Откроем любой заказ и изменим количество товара или его цену. При этом соответственно будет пересчитана сумма – как произведение цены товара на его количество.

Получение актуальной цены товара

Однако до сих пор мы просто указывали произвольную цену товара. На самом деле, как уже говорилось, данные о ценах товаров на определенную дату заполняются менеджером интернет-магазина и хранятся в периодическом регистре сведений Цены товаров.

При создании или редактировании заказа менеджером нужно, чтобы при выборе товара в строке таблицы автоматически подставлялась бы его цена из регистра сведений. Цена товара должна быть актуальной на дату заказа. И после этого должна пересчитаться сумма по товару с учетом новой цены.

Для этого сначала создадим в модуле формы серверную внеконтекстную функцию, которая будет возвращать нам актуальную цену товара на определенную дату. Эту дату и ссылку на товар мы будем передавать в функцию в качестве параметров (листинг 3.2).

Листинг 3.2. Функция для получения актуальной цены товара

```
&НаСервереБезКонтекста
Функция ПолучитьЦенуТовара(Дата, Товар)

    Отбор = Новый Структура("Товар", Товар);
    ЦенаТовара = РегистрыСведений.ЦеныТоваров.ПолучитьПоследнее(Дата, Отбор);
    Возврат ЦенаТовара.Цена;

КонецФункции
```

В этой функции мы выполняем метод менеджера регистра сведений ЦеныТоваров ПолучитьПоследнее(), который возвращает нам значения ресурсов самой поздней записи регистра (соответствующей переданной в функцию дате), в которой измерение регистра Товар равно переданной в функцию ссылке на элемент справочника товаров.

Теперь эту функцию нужно вызвать при выборе товара в табличной части заказа.

Для этого создадим клиентский обработчик события ПриИзменении для поля табличной части ТоварыТовар и заполним его следующим образом (листинг 3.3).

Листинг 3.3. Обработчик события «ПриИзменении» для поля «ТоварыТовар»

```
&НаКлиенте
Процедура ТоварыТоварПриИзменении(Элемент)
```



```
СтрокаТабличнойЧасти = Элементы.Товары.ТекущиеДанные;
```

```
СтрокаТабличнойЧасти.Цена = ПолучитьЦенуТовара(Объект.Дата, СтрокаТабличнойЧасти.Товар);  
СтрокаТабличнойЧасти.Количество = ?(СтрокаТабличнойЧасти.Количество = 0, 1,  
СтрокаТабличнойЧасти.Количество);
```

```
СтрокаТабличнойЧасти.Сумма = СтрокаТабличнойЧасти.Количество * СтрокаТабличнойЧасти.Цена;
```

КонецПроцедуры

Используя переменную `СтрокаТабличнойЧасти`, мы передаем значение, содержащееся в колонке `Товар` (ссылку на элемент справочника `Товары`), вторым параметром в функцию `ПолучитьЦенуТовара()`, определенную нами выше (см. листинг 3.2).

Первым параметром мы передаем в функцию дату документа, на которую нужно получить цену товара. Дату документа мы получаем из основного реквизита формы (`Объект.Дата`).

Возвращенное функцией последнее значение цены товара мы присваиваем полю `Цена` в текущей строке табличной части документа. Если строка создается заново, то полю `Количество` присваивается единица.

Затем так же, как и в процедурах `ТоварыКоличествоПриИзменении` и `ТоварыЦенаПриИзменении`, сумма в текущей строке табличной части документа пересчитывается как произведение цены товара на его количество.

Итоговые суммы заказанных товаров

Теперь поместим внизу формы заказа итоговые суммы заказанных товаров без учета товаров, от которых клиент отказался.

Для этого создадим реквизиты формы `КоличествоИтог` (типа `Число 10`) и `СуммаИтог` (типа `Число 10,2`) и перетащим их в самый низ дерева элементов формы. Установим для соответствующих элементов формы свойство `Вид` в значение `Поле надписи` (рис. 3.4).

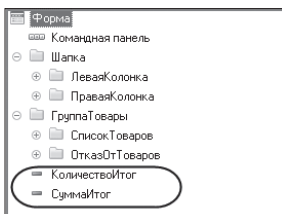


Рис. 3.4. Добавление итоговых сумм в форму

Для расчета итоговых сумм по табличной части документа мы создадим следующую серверную процедуру (листинг 3.4).

Листинг 3.4. Процедура «ПересчитатьИтоговуюСумму()»

```
&НаСервере
Процедура ПересчитатьИтоговуюСумму()

    СуммаИтог = 0;
    КоличествоИтог = 0;
    Для Каждого ТекущаяСтрока Из Объект.Товары Цикл
        Если ТекущаяСтрока.Отказ = Ложь Тогда
            СуммаИтог = СуммаИтог + ТекущаяСтрока.Сумма;
            КоличествоИтог = КоличествоИтог + ТекущаяСтрока.Количество;
        КонецЕсли;
    КонецЦикла;

КонецПроцедуры
```

В данной процедуре мы обходим в цикле табличную часть документа (Объект.Товары) и накапливаем итоговые значения в реквизитах формы КоличествоИтог и СуммаИтог в том случае, если у товара нет отметки об отказе.

Процедуру для пересчета итоговых сумм табличной части будем вызывать каждый раз при изменении цены и количества товара, а также при перевыборе товара.

Соответственно в процедуры ТоварыКоличествоПриИзменении(), ТоварыЦенаПриИзменении(), ТоварыТоварПриИзменении() последней строкой нужно добавить процедуру ПересчитатьИтоговуюСумму().

Кроме того, создадим обработчик события формы ПриСозданииНаСервере и также поместим туда процедуру для пересчета итогов (листинг 3.5).

Листинг 3.5. Обработчик события «ПриСозданииНаСервере»

```
&НаСервере
Процедура ПриСозданииНаСервере(Отказ, СтандартнаяОбработка)

    ПересчитатьИтоговуюСумму();

КонецПроцедуры
```

В результате при открытии заказа внизу формы будут отображаться итоговое количество и сумма заказанных товаров без учета товаров, от которых клиент отказался (рис. 3.5). И при любых изменениях в табличной части эти итоги будут пересчитываться.

← → ☆ Заказ 000000004 от 11.12.2013 12:00:00

Провести и закрыть

Записать

Провести

Еще ▾

Номер: 000000004

Дата: 11.12.2013 12:00:00

Клиент: Смирнова А.И.

Дата доставки: 13.12.2013

Курьер: Морозов Д.И.

Адрес доставки: Москва, пр-т Мира 111 кв. 33

Склад: Основной

Дополнительно: домофон - 177

Статус заказа: Открыт

В работе

Выполнен

Закрыт

Список товаров

Отказ от товаров

Добавить

Еще ▾

N	Товар	Цвет	Размер	Цена	Количе...	Сумма
1	Джинсы	Черный	46	3 000,00	3	9 000,00
2	Платье	Белый	46	5 000,00	1	5 000,00

Количество итог: 4

Сумма итог: 14 000,00

Рис. 3.5. Форма заказа с итоговыми суммами

При подсчете итоговых сумм в заказе для упрощения сделано одно допущение. Если клиент отказывается от товара, то считается, что он не берет сразу все количество товара, которое заказано.

Отказ от товаров

Теперь сделаем так, чтобы информация об отказе от товаров была недоступна для изменения, так как по логике нашей задачи эту информацию вводит курьер при доставке заказа клиенту.

Для этого в палитре свойств таблицы формы Товары1 установим свойство ПоложениеКоманднойПанели в значение Нет и свойство ТолькоПросмотр в значение Истина.

Добавление нового клиента

Платформа «1С:Предприятия» предоставляет удобную возможность добавления нового клиента, для которого создается заказ, непосредственно из формы заказа.

Как только пользователь начинает вводить наименование клиента в поле заказа Клиент, под ним открывается выпадающий список. Если клиента с подходящим наименованием в справочнике нет, то можно нажать кнопку Создать (+) в правом нижнем углу выпадающего списка (рис. 3.6). При этом откроется форма для добавления нового клиента, и затем ссылка на этого клиента попадет в заказ.

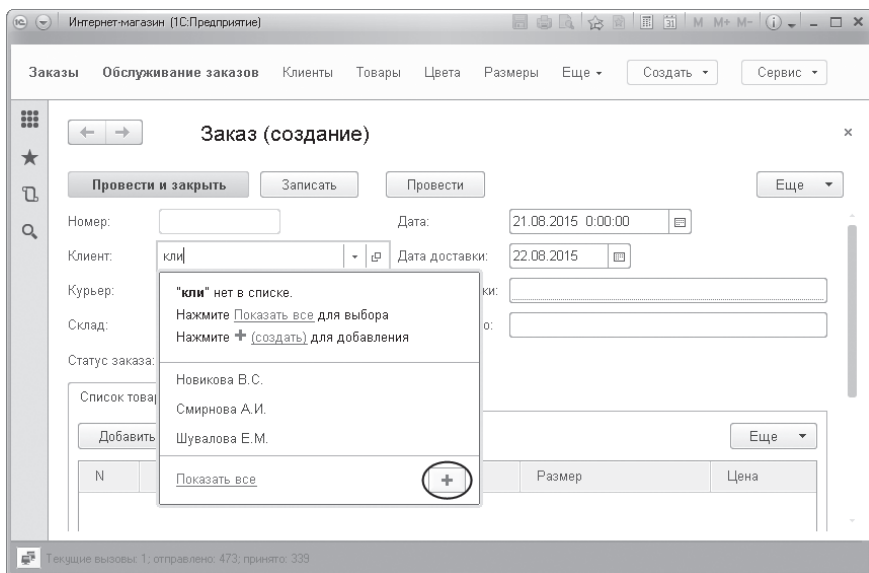


Рис. 3.6. Добавление нового клиента в заказ

При добавлении нового клиента в заказ (или изменении уже существующего клиента) было бы удобно сразу же заполнить его адрес доставки (реквизит АдресДоставки) исходя из адресных реквизитов клиента. Чтобы предоставить такую возможность, создадим обработчик события ПриИзменении для поля формы Клиент и поместим туда вызов функции для получения адреса доставки клиента (листинг 3.6).

Листинг 3.6. Обработчик события «ПриИзменении» для поля «Клиент»

```
&НаКлиенте
Процедура КлиентПриИзменении(Элемент)

    Объект.АдресДоставки = ПолучитьАдресДоставки(Объект.Клиент);

КонецПроцедуры
```

В этом обработчике мы вызываем серверную внеконтекстную функцию ПолучитьАдресДоставки(), которая возвращает адресные реквизиты клиента (Город, Улица, Дом и т.д.), перечисленные через запятую в одной строке (листинг 3.7).

Листинг 3.7. Функция для получения адреса доставки для клиента

&НаСервереБезКонтекста

Функция ПолучитьАдресДоставки(Клиент)

```
АдресДоставки = СокрЛП(Клиент.Страна) + "; " + СокрЛП(Клиент.Город) + "; " +  
                СокрЛП(Клиент.Улица) + "; " + СокрЛП(Клиент.Дом) + "; " + СокрЛП(Клиент.Квартира);  
Возврат АдресДоставки;
```

КонецФункции

Ввод на основании

Теперь добавим в основное приложение еще одну удобную возможность – ввода новых заказов на основании отдельных товаров и клиентов.

Для этого в окне редактирования объекта конфигурации Документ Заказ на закладке Ввод на основании добавим справочники Товары и Клиенты в список Вводится на основании (рис. 3.7).

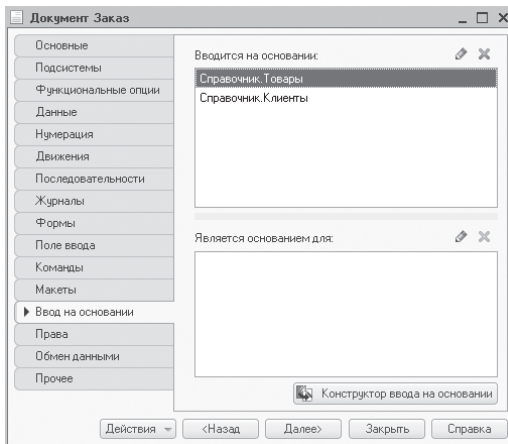


Рис. 3.7. Список объектов конфигурации, на основании которых вводится заказ

Это значит, что новый заказ можно будет ввести на основании элементов справочников товаров или клиентов. После этого в форме списка и в форме элемента этих справочников станет доступна команда Создать на основании > Заказ. При выполнении этой команды нам нужно обеспечить следующее заполнение заказа:

- Если заказ вводится на основании клиента, ссылка на этого клиента должна попасть в поле Клиент нового заказа, а поле Адрес доставки должно быть заполнено исходя из адресных реквизитов клиента.

- Если заказ вводится на основании товара, то в табличную часть нового документа должна быть добавлена строка, содержащая ссылку на этот товар. Причем у добавленного товара должна быть получена актуальная цена из регистра сведений и подставлена в строку табличной части (вместе с одинарным количеством заказанного товара). С учетом цены и количества товара должна быть посчитана сумма по товару, на основании которого вводился заказ.

Данный алгоритм мы должны прописать в обработчике события ОбработкаЗаполнения, которое описывается в модуле документа. Откроем модуль документа Заказ и поместим в него следующую процедуру (листинг 3.8).

Листинг 3.8. Обработчик события документа «ОбработкаЗаполнения»

Процедура ОбработкаЗаполнения(ДанныеЗаполнения, ТекстЗаполнения, СтандартнаяОбработка)

```
Если ТипЗнч(ДанныеЗаполнения) = Тип("СправочникСсылка.Клиенты") Тогда
    // Заполнение шапки
    Клиент = ДанныеЗаполнения.Ссылка;
    АдресДоставки = СокрЛП(Клиент.Страна) + ", " + СокрЛП(Клиент.Город) + ", " +
        СокрЛП(Клиент.Улица) + ", " + СокрЛП(Клиент.Дом) + ", " + СокрЛП(Клиент.Квартира);

ИначеЕсли ТипЗнч(ДанныеЗаполнения) = Тип("СправочникСсылка.Товары") Тогда
    // Заполнение табличной части
    НоваяСтрока = Товары.Добавить();
    НоваяСтрока.Товар = ДанныеЗаполнения.Ссылка;
    НоваяСтрока.Количество = 1;

    ЦенаТовара = РегистрыСведений.ЦеныТоваров.ПолучитьПоследнее(Дата,
        Новый Структура("Товар", ДанныеЗаполнения.Ссылка));
    НоваяСтрока.Цена = ЦенаТовара.Цена;
    НоваяСтрока.Сумма = НоваяСтрока.Цена;
КонецЕсли;
```

КонецПроцедуры

В зависимости от типа объекта-основания, содержащегося в параметре ДанныеЗаполнения данного обработчика, мы реализуем нужный алгоритм действий. Для получения актуальной цены товара и адреса доставки клиента здесь используются фрагменты функций ПолучитьЦенуТовара() и ПолучитьАдресДоставки(), которые рассматривались ранее.

Также при создании нового заказа автоматически установим дату доставки на следующий день после даты документа. Для этого создадим обработчик события формы ПриОткрытии и заполним его следующим образом (листинг 3.9).

Листинг 3.9. Обработчик события формы «ПриОткрытии»

```
&НаКлиенте  
Процедура ПриОткрытии(Отказ)  
  
    Если Параметры.Ключ.Пустая() Тогда  
        Объект.ДатаДоставки = Объект.Дата + 24*60*60;  
    КонечЕсли;  
  
КонечПроцедуры
```

При открытии формы в параметре формы Ключ мы определяем, что создается новый документ, если ссылка на создаваемый объект пустая. В этом случае мы устанавливаем дату доставки следующей датой после даты документа.

Редактирование уже существующего заказа

В заключение ограничим возможность изменения информации в заказе в зависимости от статуса заказа.

Если заказ вернулся от курьера со статусом Выполнен, то менеджер интернет-магазина может только закрыть этот заказ. Если статус заказа Открыт или В работе, то заказ можно редактировать. А если заказ уже закрыт, такой заказ можно только просматривать.

Для этого создадим обработчик события формы ПриЧтенииНаСервере, который выполняется только для уже существующих в информационной базе объектов, и заполним его следующим образом (листинг 3.10).

Листинг 3.10. Обработчик события формы документа «ПриЧтенииНаСервере»

```
&НаСервере  
Процедура ПриЧтенииНаСервере(ТекущийОбъект)  
  
    Если ТекущийОбъект.СтатусЗаказа = Перечисления.СтатусыЗаказов.Закрыт Тогда  
        ЭтаФорма.ТолькоПросмотр = Истина;  
    ИначеЕсли ТекущийОбъект.СтатусЗаказа = Перечисления.СтатусыЗаказов.Выполнен Тогда  
        Элементы.Номер.ТолькоПросмотр = Истина;  
        Элементы.Дата.ТолькоПросмотр = Истина;  
        Элементы.ДатаДоставки.ТолькоПросмотр = Истина;  
        Элементы.АдресДоставки.ТолькоПросмотр = Истина;  
        Элементы.Клиент.ТолькоПросмотр = Истина;  
        Элементы.Курьер.ТолькоПросмотр = Истина;  
        Элементы.Склад.ТолькоПросмотр = Истина;  
        Элементы.ДополнительнаяИнформация.ТолькоПросмотр = Истина;  
        Элементы.Товары.ТолькоПросмотр = Истина;  
    КонечЕсли;  
  
КонечПроцедуры
```

При создании новых заказов данный обработчик вызываться не будет, и новые заказы менеджер в офисе сможет редактировать, кроме информации об отказе от товаров. Эта информация недоступна в интернет-магазине как для новых, так и для уже существующих заказов.

Список заказов

Теперь немного улучшим внешний вид списка заказов. Сейчас в этом списке, автоматически сгенерированном платформой, отображаются все реквизиты документа Заказ. Этих реквизитов довольно много (Клиент, Курьер, Склад и т.д.), поэтому пользователю трудно выделить среди них основную информацию о заказе. Для этого ограничим состав полей списка заказов только самыми важными реквизитами.

Создадим форму списка документа Заказ. Конструктор форм стандартно добавляет в форму списка два поля: Код и Наименование. Раскроем основной реквизит формы Список и перетащим в дерево элементов формы поля динамического списка: Клиент, Курьер, ДатаДоставки и СтатусЗаказа.

Затем настроим условное оформление списка заказов, чтобы менеджеру интернет-магазина было удобнее ориентироваться в этом списке в зависимости от статуса заказов. Выделим различным цветом фона строки списка с открытыми, выполненными и закрытыми заказами. Кроме того, заказы со статусом Открыт выделим жирным шрифтом.

Для этого откроем палитру свойств основного реквизита формы Список и нажмем ссылку Открыть. На закладке Условное оформление добавим три строки с соответствующим оформлением строк списка в зависимости от статуса заказов (рис. 3.8).

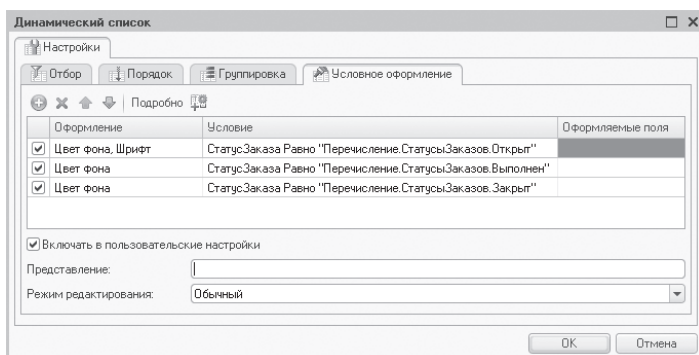


Рис. 3.8. Условное оформление динамического списка

Таким образом, список заказов, когда он заполнится документами, в режиме 1С:Предприятие будет выглядеть более информативно (рис. 3.9).

Дата	↓	Номер	Клиент	Курьер	Дата доставки	Статус заказа
10.12.2013 12:00:00		000000001	Шувалова Е.М.	Морозов Д.И.	13.12.2013	Закрыт
10.12.2013 12:00:01		000000002	Смирнова А.И.	Морозов Д.И.	15.12.2013	Закрыт
11.12.2013 12:00:00		000000004	Смирнова А.И.	Морозов Д.И.	13.12.2013	Закрыт
12.12.2013 22:00:14		000000003	Новикова В.С.	Морозов Д.И.	15.12.2013	Закрыт
20.08.2015 12:00:00		000000006	Шувалова Е.М.	Морозов Д.И.	25.08.2015	Выполнен
23.08.2015 12:00:00		000000005	Новикова В.С.	Морозов Д.И.	25.08.2015	Выполнен
31.08.2015 12:00:00		ЦО-000002	Шувалова Е.М.	Алексеев А.Н.	03.09.2015	Выполнен
01.09.2015 12:00:00		ЦО-000001	Смирнова А.И.	Алексеев А.Н.	03.09.2015	В работе
01.09.2015 12:00:01		ЦО-000003	Новикова В.С.	Морозов Д.И.	05.09.2015	В работе

Рис. 3.9. Список заказов в «1С:Предприятии»

ПРИМЕЧАНИЕ

В списке заказов отображены документы, получившиеся в основном приложении после выполнения обмена данными с мобильным приложением (см. раздел «Тестирование обмена данными в обе стороны» на стр. 276). Иначе список заказов был бы неинформативным.

Клиенты

Прежде всего немного улучшим внешний вид списка клиентов. Сейчас в этом списке, автоматически сгенерированном платформой, отображаются все реквизиты справочника Клиенты. Этих реквизитов довольно много (Страна, Город, Улица и т.д.), поэтому пользователю трудно выделить среди них основную информацию о клиенте. Для этого ограничим состав полей списка клиентов только самыми важными реквизитами.

Создадим форму списка справочника Клиенты. Конструктор форм стандартно добавляет в форму списка два поля: Код и Наименование. Раскроем основной реквизит формы Список и перетащим в дерево элементов формы поля динамического списка: Телефон, Город и ЭлектроннаяПочта.

Можно было, конечно, этого и не делать, но, на наш взгляд, в таком виде список клиентов выглядит более информативно и аккуратно.

Теперь создадим форму элемента справочника Клиенты. Поскольку элементов в форме клиента много, хорошо бы их зрительно выделить, объединив в группы.

В дереве формы создадим группы (вида Обычная группа) Адрес, Контакты, Координаты и установим для них свойство Поведение в значение Свертываемая и свойство ОтображениеУправления в значение Картинка (рис. 3.10).

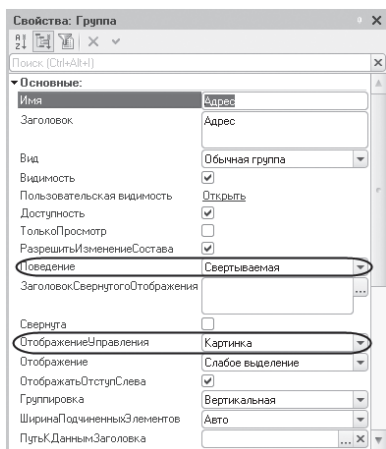


Рис. 3.10. Свойства групп формы элемента справочника «Клиенты»

Затем перетащим в эти группы элементы формы, отображающие соответствующую информацию о клиенте. В результате в режиме 1С:Предприятие форма клиента будет выглядеть следующим образом (рис. 3.11).

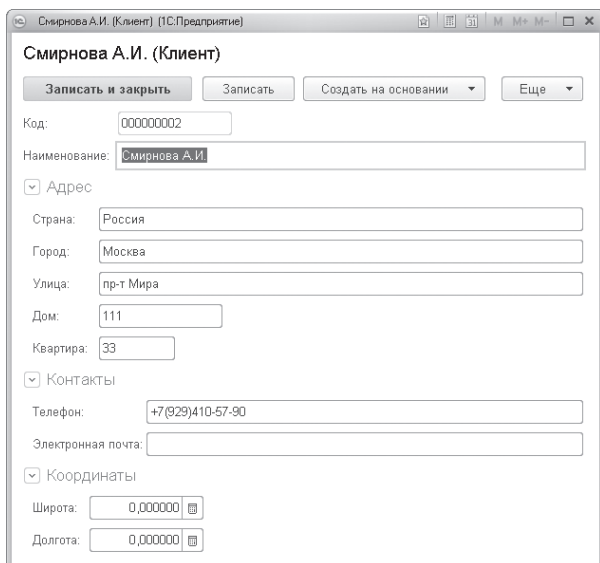


Рис. 3.11. Форма клиента в режиме «1С:Предприятие»

Таким образом, информация о клиенте более структурирована и легче воспринимается, к тому же группы с данными можно свернуть/развернуть, нажав на галочку слева от названия группы.

В заключение при записи клиента выполним программную проверку, чтобы все адресные реквизиты клиента Страна, Город, Улица, Дом были либо заполнены, либо пусты. Иначе отображение адреса на карте может быть неточным.

Такая проверка выполняется в обработчике ОбработкаПроверкиЗаполнения(), который описывается в модуле объекта. Поместим в модуль справочника Клиенты следующую процедуру (листинг 3.11).

Листинг 3.11. Обработчик события справочника «ОбработкаПроверкиЗаполнения»

```
Процедура ОбработкаПроверкиЗаполнения(Отказ, ПроверяемыеРеквизиты)
// Если заполнено поле "Улица"
Если НЕ ПустаяСтрока(Улица) Тогда
    // Тогда должны быть заполнены и поля "Страна", "Город" и "Дом".
    ПроверяемыеРеквизиты.Добавить("Страна");
    ПроверяемыеРеквизиты.Добавить("Город");
    ПроверяемыеРеквизиты.Добавить("Дом");
КонецЕсли;

// Если заполнено поле "Дом"
Если НЕ ПустаяСтрока(Дом) Тогда
    // Тогда должны быть заполнены и поля "Страна", "Город" и "Улица".
    ПроверяемыеРеквизиты.Добавить("Страна");
    ПроверяемыеРеквизиты.Добавить("Город");
    ПроверяемыеРеквизиты.Добавить("Улица");
КонецЕсли;

// Если заполнено поле "Город"
Если НЕ ПустаяСтрока(Город) Тогда
    // Тогда должны быть заполнены и поля "Страна", "Улица" и "Дом".
    ПроверяемыеРеквизиты.Добавить("Страна");
    ПроверяемыеРеквизиты.Добавить("Улица");
    ПроверяемыеРеквизиты.Добавить("Дом");
КонецЕсли;

// Если заполнено поле "Страна"
Если НЕ ПустаяСтрока(Страна) Тогда
    // Тогда должны быть заполнены и поля "Улица", "Город" и "Дом".
    ПроверяемыеРеквизиты.Добавить("Улица");
    ПроверяемыеРеквизиты.Добавить("Город");
    ПроверяемыеРеквизиты.Добавить("Дом");
КонецЕсли;

КонецПроцедуры
```

В этом обработчике мы последовательно проверяем, заполнено ли значение одного из четырех реквизитов (Страна, Город, Улица, Дом), описывающих адрес клиента. Если да, то остальные три реквизита мы добавляем в массив ПроверяемыеРеквизиты. Этот массив передается в обработчик и содержит массив проверяемых реквизитов, у которых свойство Проверка заполнения

установлено в значение **Выдавать ошибку** во время разработки мобильного приложения.

Если одни адресные реквизиты клиента заполнены, а другие – нет, то при нажатии кнопки **Записать** и **закрыть** или **Записать** будет выдано сообщение об ошибке, а данные о клиенте не будут сохранены.

Хранимые файлы

Теперь пришла пора заняться справочником **ХранимыеФайлы**. Мы уже добавили этот справочник в мобильное приложение и использовали его для хранения файлов мультимедиа, сделанных курьером при доставке заказов клиенту. В основном приложении в этом справочнике будут храниться, прежде всего, картинки товаров. Вообще же хранимые файлы могут быть использованы для хранения любых типов данных, которые можно поместить в хранилище значения.

Справочник хранимых файлов также будет участвовать в обмене данными между основным и мобильным приложением. Таким образом, после обмена данными этого справочника на планшете в форме товара появится его изображение, а в основном приложении появятся фотоснимки, видео- и аудиозаписи, сделанные курьером, которые можно будет открыть и сохранить на стационарном компьютере в виде отдельных файлов.

Итак, создадим в нашей мобильной конфигурации справочник с именем **ХранимыеФайлы**, представлением объекта **Файл** и представлением списка **Файлы**. Для хранения имени файла и собственно содержимого создадим у этого справочника два реквизита: **ИмяФайла** (типа **Строка** неограниченной длины) и **ДанныеФайла** (типа **ХранилищеЗначения**). А также увеличим длину основного реквизита справочника **Наименование** до 50 символов.

Обратите внимание на то, что последовательность реквизитов справочника (как и всех других объектов конфигурации, участвующих в обмене) должна быть такой же, как и в мобильном приложении. Иначе при обмене данными стандартным способом будет получена ошибка.

Дело в том, что в процедурах обмена данными мы (для упрощения) выполняем стандартные методы записи и чтения данных XML, которые подразумевают одинаковую структуру объектов в узлах обмена. Вообще же мы используем универсальный обмен данными, при котором структура объектов в узлах обмена может быть разной. Однако при этом не получится выполнять стандартные методы записи и чтения данных XML. Позднее (см. раздел «Обмен данными с разной структурой» на стр. 255) мы доработаем обмен данными так, чтобы в нем могли участвовать данные с разной структурой, но конкретно для справочника **ХранимыеФайлы** этого делать не планируется.

Укажем, что у справочника ХранимыеФайлы могут быть два владельца: справочник Товары (для просмотра картинок, относящихся к конкретному товару) и справочник Клиенты (для просмотра файлов, сделанных курьером у конкретного клиента), рис. 3.12.

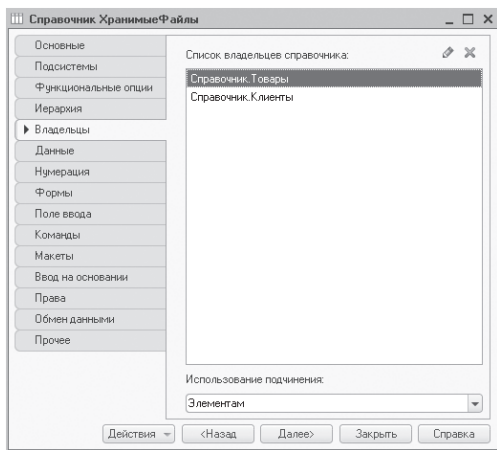


Рис. 3.12. Список владельцев справочника «ХранимыеФайлы»

Чтобы выполнять обмен данными справочника между планшетом и основным приложением, на закладке Обмен данными окна редактирования справочника ХранимыеФайлы включим его в план обмена Мобильные (рис. 3.13).

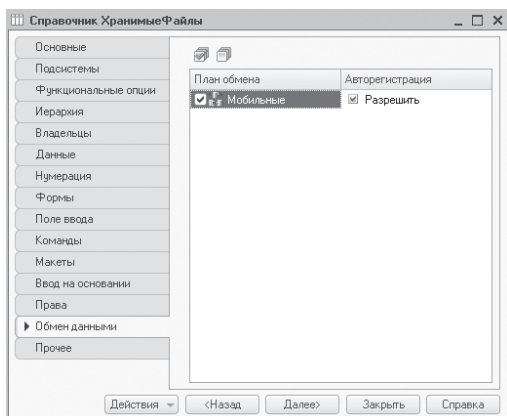


Рис. 3.13. Принадлежность справочника к плану обмена «Мобильные»

Создание и открытие хранимых файлов

Затем создадим форму элемента справочника `ХранимыеФайлы` и предоставим пользователю две возможности. Нужно, чтобы в этой форме можно было выбрать файл на стационарном компьютере и сохранить его содержимое в справочнике, а также открыть содержимое элемента справочника соответствующим приложением и сохранить его в локальную файловую систему пользователя.

Для этого на закладке `Команды` добавим команды `ВыбратьФайлСДискаИЗаписать`, `ПрочитатьФайлИСохранитьНаДиск` и перетащим их в форму.

В результате форма элемента справочника в редакторе формы примет следующий вид (рис. 3.14).

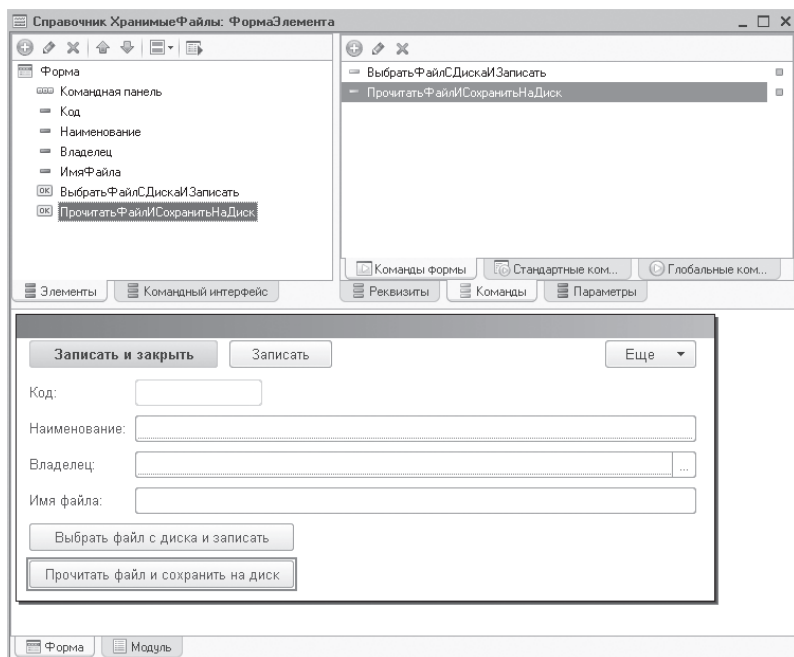


Рис. 3.14. Форма хранимого файла в редакторе формы

При создании формы элемента справочника хранимых файлов нужно очистить имя файла в том случае, если новый элемент справочника создается путем копирования существующего элемента – чтобы не возникало иллюзии, что содержимое файла тоже копируется. Для этого создадим

обработчик события формы ПриСозданииНаСервере и заполним его следующим образом (листинг 3.12).

Листинг 3.12. Обработчик события формы «ПриСозданииНаСервере»

```
&НаСервере
Процедура ПриСозданииНаСервере(Отказ, СтандартнаяОбработка)

    Если Параметры.Ключ.Пустая() И НЕ Параметры.ЗначениеКопирования.Пустая() Тогда
        // при копировании очищаем имя файла, чтобы не возникало иллюзии, что содержимое
        // файла тоже копируется.
        Объект.ИмяФайла = "";
    КонецЕсли;

КонецПроцедуры
```

Перед записью данных из формы убедимся, что имя файла заполнено. Иначе запись данных не должна выполняться. Для этого создадим обработчик события формы ПередЗаписью и заполним его следующим образом (листинг 3.13).

Листинг 3.13. Обработчик события формы «ПередЗаписью»

```
&НаКлиенте
Процедура ПередЗаписью(Отказ, ПараметрыЗаписи)

    Если Объект.ИмяФайла = "" Тогда
        ПоказатьПредупреждение(, "Не выбран файл!");
        Отказ = Истина;
    КонецЕсли;

КонецПроцедуры
```

После этого создадим клиентский обработчик для команды ВыбратьФайлСДискаИЗаписать и заполним его следующим образом (листинг 3.14).

Листинг 3.14. Обработчик команды «ВыбратьФайлСДискаИЗаписать»

```
&НаКлиенте
Процедура ВыбратьФайлСДискаИЗаписать(Команда)

    НовыйОбъект = Объект.Ссылка.Пустая();
    Оповещение = Новый ОписаниеОповещения("ВыбратьФайлСДискаИЗаписатьЗавершение",
        ЭтотОбъект, НовыйОбъект);

    НачатьПомещениеФайла(Оповещение, , "", Истина);

КонецПроцедуры
```

При обработке этой команды во встроенном языке реализуется методика отказа от модальности – то есть вывод пользователю блокирующих окон

вместо модальных, для того чтобы модальные окна не останавливали работу программного кода.

Сначала в переменной НовыйОбъект мы получаем признак того, создается ли новый хранимый файл.

Затем создается объект ОписаниеОповещения, в конструкторе которого первым параметром указывается имя процедуры-обработчика оповещения, которая будет выполнена после получения ответа пользователя. Во втором параметре указывается, в каком модуле расположена эта процедура (в нашем случае – в модуле формы). А в третьем в качестве дополнительного параметра передается признак создания нового объекта (Истина или Ложь).

После этого с помощью немодального метода НачатьПомещениеФайла() файл, выбранный пользователем из локальной файловой системы, помещается во временное хранилище. После того как пользователь сделает выбор в диалоге выбора файла, файл с именем ВыбранноеИмя будет помещен во временное хранилище по адресу АдресВременногоХранилища.

Эти значения в качестве параметров передаются в экспортную процедуру модуля формы ВыбратьФайлСДискаИЗаписатьЗавершение() (листинг 3.15), которая описывается в объекте ОписаниеОповещения, переданном в первом параметре в процедуру НачатьПомещениеФайла().

Листинг 3.15. Обработчик оповещения «ВыбратьФайлСДискаИЗаписатьЗавершение»

```
&НаКлиенте
Процедура ВыбратьФайлСДискаИЗаписатьЗавершение(Результат, АдресВременногоХранилища,
                                                    ВыбранноеИмя, НовыйОбъект) Экспорт

    Если Результат Тогда
        Объект.ИмяФайла = ВыбранноеИмя;

        Если Не ЗначениеЗаполнено(Объект.Наименование) Тогда
            ОписаниеФайла = Новый Файл(ВыбранноеИмя);
            Объект.Наименование = ОписаниеФайла.Имя;
        КонецЕсли;

        ПоместитьФайлОбъекта(АдресВременногоХранилища);

        Если НовыйОбъект Тогда
            ОтобразитьИзменениеДанных(Объект.Ссылка, ВидИзмененияДанных.Добавление);
        Иначе
            ОтобразитьИзменениеДанных(Объект.Ссылка, ВидИзмененияДанных.Изменение);
        КонецЕсли;
    КонецЕсли;

КонецПроцедуры
```


В параметре Результат процедуры-обработчика оповещения содержится результат ответа пользователя. В случае положительного результата (пользователь выбрал файл) мы присваиваем имя выбранного пользователем файла (которое сохраняется в параметре ВыбранноеИмя) реквизиту объекта ИмяФайла.

Если наименование хранимого файла не заполнено, то создаем объект Файл на основе имени выбранного файла и присваиваем наименованию хранимого файла имя созданного нами объекта Файл.

Затем мы вызываем серверную процедуру ПоместитьФайлОбъекта() для сохранения содержимого выбранного файла в справочнике ХранимыеФайлы и передаем в нее адрес временного хранилища, в которое было помещено это содержимое (листинг 3.16).

Листинг 3.16. Процедура для сохранения содержимого выбранного файла в справочнике «ХранимыеФайлы»

&НаСервере

Процедура ПоместитьФайлОбъекта(АдресВременногоХранилища)

```
ЭлементСправочника = РеквизитФормыВЗначение("Объект");  
ДвоичныеДанные = ПолучитьИзВременногоХранилища(АдресВременногоХранилища);  
ЭлементСправочника.ДанныеФайла = Новый ХранилищеЗначения(ДвоичныеДанные,  
Новый СжатиеДанных());
```

```
Файл = Новый Файл(ЭлементСправочника.ИмяФайла);  
ЭлементСправочника.ИмяФайла = Файл.Имя;  
ЭлементСправочника.Записать();
```

```
Модифицированность = Ложь;  
УдалитьИзВременногоХранилища(АдресВременногоХранилища);  
ЗначениеВРеквизитФормы(ЭлементСправочника, "Объект");
```

КонецПроцедуры

В данной процедуре мы преобразуем основной реквизит формы в объект – элемент справочника ХранимыеФайлы.

Затем получаем из временного хранилища двоичные данные по адресу временного хранилища, переданному в процедуру. На основе этих данных мы создаем объект ХранилищеЗначения и помещаем его в реквизит элемента справочника ДанныеФайла.

Затем создаем объект Файл на основе значения реквизита элемента справочника ИмяФайла. Этот объект мы создаем для того, чтобы простым способом получить короткое имя файла из его полного имени. Это короткое имя мы записываем в соответствующий реквизит элемента справочника ИмяФайла. После этого сохраняем созданный элемент справочника ХранимыеФайлы.

В заключение удаляем данные из временного хранилища и преобразуем данные объекта обратно в основной реквизит формы Объект. А также свойству Модифицированность формы присваиваем значение Ложь, так как данные элемента справочника уже записаны.

После этого в процедуре ВыбратьФайлСДискаИЗаписатьЗавершение() (см. листинг 3.15) вызываем метод ОтобразитьИзменениеДанных() для вывода пользователю сообщения об изменении или создания нового объекта и оповещения об этом всех динамических списков об изменениях.

Затем создадим клиентский обработчик для команды ПрочитатьФайлИСохранитьНаДиск и заполним его следующим образом (листинг 3.17).

Листинг 3.17. Обработчик команды «ПрочитатьФайлИСохранитьНаДиск»

```
&НаКлиенте
Процедура ПрочитатьФайлИСохранитьНаДиск(Команда)

    Если Объект.Ссылка.Пустая() Тогда
        ПоказатьПредупреждение(, "Данные не записаны!");
        Возврат;
    КонецЕсли;

    Если ПустаяСтрока(Объект.ИмяФайла) Тогда
        ПоказатьПредупреждение(, "Имя не задано!");
        Возврат;
    КонецЕсли;

    Адрес = ПолучитьНавигационнуюСсылку(Объект.Ссылка, "ДанныеФайла");
    ПолучитьФайл(Адрес, Объект.ИмяФайла, Истина);

КонецПроцедуры
```

В данном обработчике мы выполняем открытие файла мультимедиа только в том случае, если данные объекта записаны и имя файла не пустое.

Если это так, то с помощью метода ПолучитьНавигационнуюСсылку() переменной Адрес мы присваиваем навигационную ссылку на реквизит объекта ДанныеФайла. В первом параметре метода ПолучитьНавигационнуюСсылку() передается ссылка на объект – Объект.Ссылка, а во втором – имя реквизита.

Далее с помощью метода ПолучитьФайл() на основе навигационной ссылки, сохраненной в переменной Адрес, мы получаем файл с именем, записанным в реквизите ИмяФайла, и сохраняем его в локальную файловую систему пользователя. Третьим параметром в метод передается Истина – признак запуска в интерактивном режиме. В этом случае пользователю будет выдано диалоговое окно, позволяющее указать действие с полученным файлом: запустить его или сохранить по указанному пользователем расположению.

Проверим, как это работает. Запишем в справочник хранимых файлов несколько картинок, которые мы чуть позже установим в качестве картинок товаров.

Откроем список хранимых файлов, выполнив команду **Файлы** из панели команд «1С:Предприятия». Пока список файлов пуст. Нажмем кнопку **Создать**. В появившейся форме нового хранимого файла укажем его наименование (в нашем случае – **Туфли**) и владельца (в нашем случае – **товар Туфли**), см. рис. 3.15.

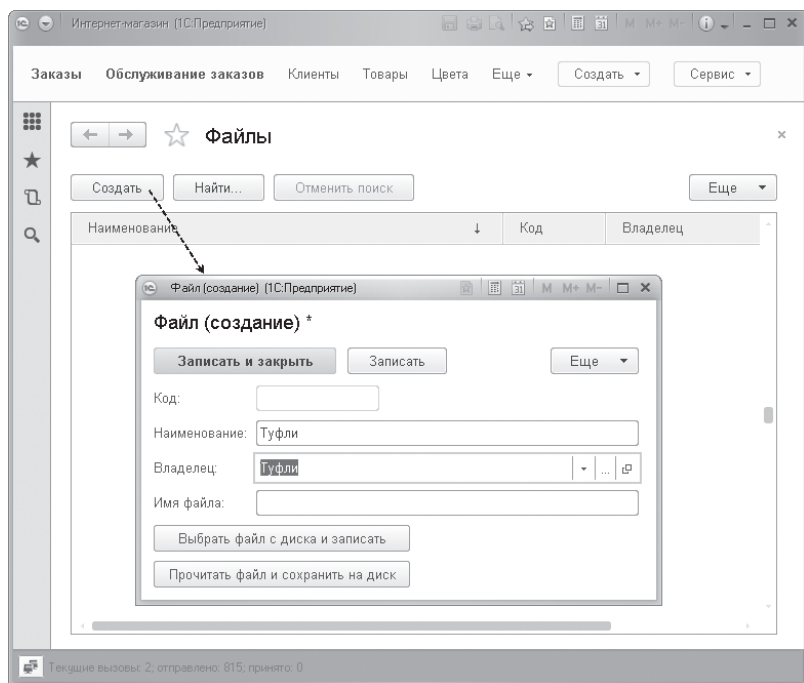


Рис. 3.15. Создание хранимого файла

Затем нажмем кнопку **Выбрать файл с диска и записать** и выберем нужный файл картинки (рис. 3.16).

Нажмем **Записать и закрыть** и вернемся в список хранимых файлов, в котором появится файл с наименованием **Туфли**. Таким же образом добавим в справочник хранимых файлов еще несколько файлов-картинок.

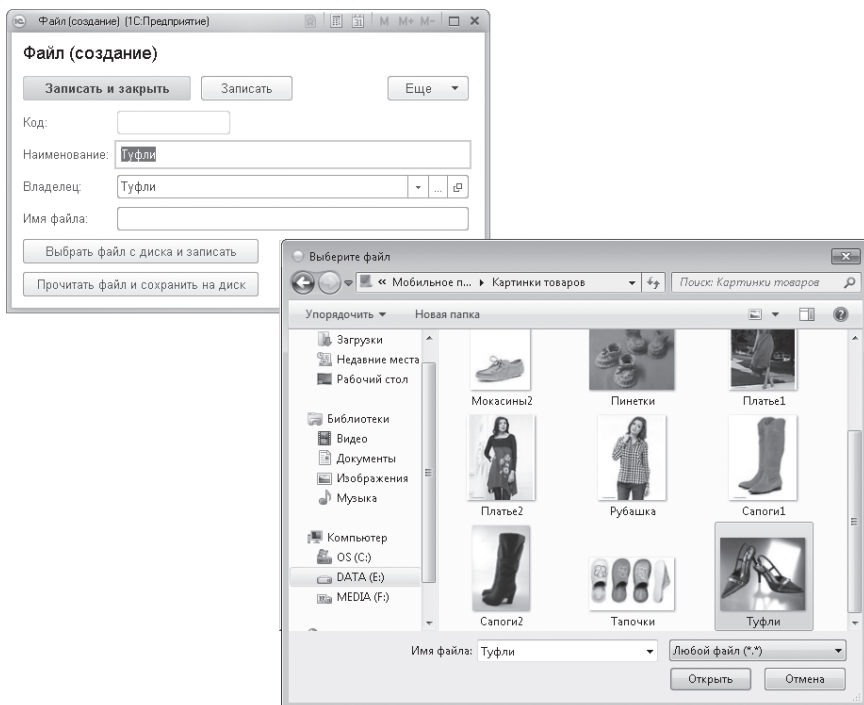


Рис. 3.16. Создание хранимого файла

Теперь откроем форму уже существующего хранимого файла и нажмем кнопку **Прочитать файл и сохранить на диск**. На вопрос «Открыть или сохранить файл?» ответим **Открыть** (рис. 3.17), так как сохранение картинки вряд ли имеет смысл, поскольку мы ее только что выбрали с диска того же компьютера. А вот файлы мультимедиа, полученные с планшета, мы попробуем сохранить на компьютер.

В результате файл картинки будет открыт программой просмотра изображений, которая установлена как программа по умолчанию для работы с файлами типа jpg (рис. 3.18).

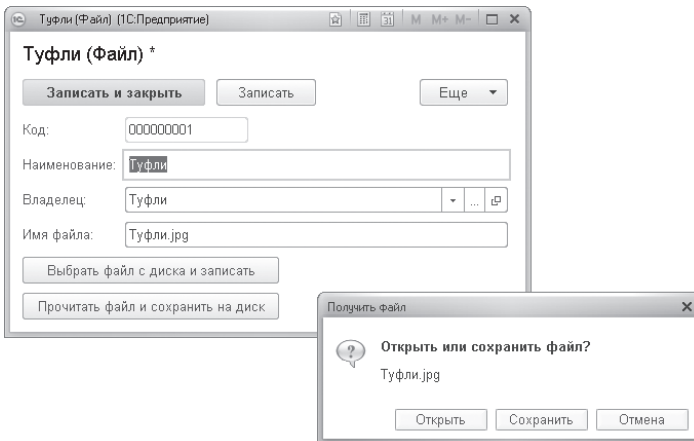


Рис. 3.17. Открытие и сохранение хранимого файла

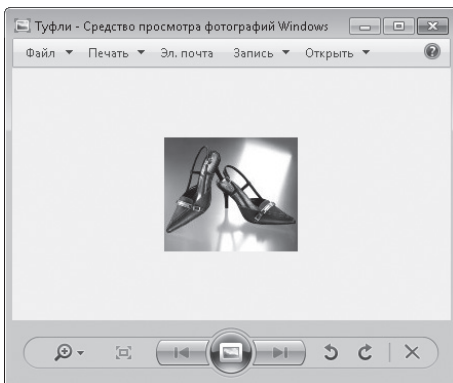


Рис. 3.18. Открытие хранимого файла

Открытие текущего хранимого файла

Теперь создадим форму списка справочника ХранимыеФайлы и предоставим пользователю возможность открыть содержимое текущего файла ассоциированным с ним приложением.

Для этого на закладке Команды добавим команду ОткрытьФайл и перетащим ее в командную панель формы. А также добавим реквизит Владелец в состав полей списка хранимых файлов. В результате форма списка справочника в редакторе формы примет следующий вид (рис. 3.19).

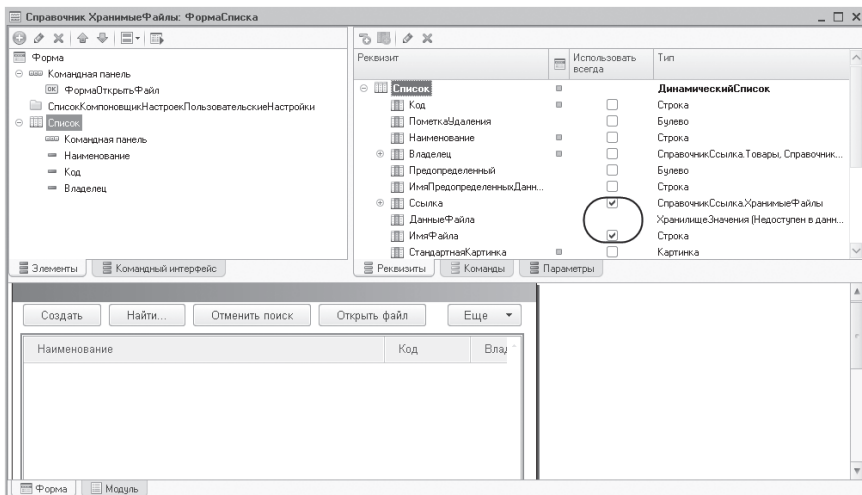


Рис. 3.19. Форма списка хранимых файлов в редакторе формы

Затем создадим клиентский обработчик для команды **ОткрытьФайл** и заполним его следующим образом (листинг 3.18).

Листинг 3.18. Обработчик команды «ОткрытьФайл»

&НаКлиенте

Процедура ОткрытьФайл(Команда)

```
ХранимыйФайл = Элементы.Список.ТекущиеДанные;
Файл = Новый Файл(ХранимыйФайл.ИмяФайла);
ИмяФайла = ПолучитьИмяВременногоФайла(Файл.Расширение);
Адрес = ПолучитьНавигационнуюСсылку(ХранимыйФайл.Ссылка, "ДанныеФайла");
ПолучитьФайл(Адрес, ИмяФайла, Ложь);
ЗапуститьПриложение(ИмяФайла);
```

КонецПроцедуры

В этом обработчике мы используем таблицу формы **Список**, которая отображает данные динамического списка, полученного на основе справочника **ХранимыеФайлы**. Используя свойство **ТекущиеДанные** этой таблицы, в переменной **ХранимыйФайл** мы получаем объект, содержащий данные, находящиеся в текущей строке таблицы.

Далее сначала мы создаем объект **Файл** на основе значения реквизита **ИмяФайла** текущего элемента справочника хранимых файлов.

Затем, для того чтобы записать содержимое файла во временный файл, методом глобального контекста `ПолучитьИмяВременногоФайла()` мы получаем уникальное имя временного файла с желаемым расширением, полученным из созданного нами объекта `Файл`.

Затем с помощью метода `ПолучитьНавигационнуюСсылку()` переменной `Адрес` мы присваиваем навигационную ссылку на реквизит `ДанныеФайла` текущего элемента справочника хранимых файлов. В первом параметре метода `ПолучитьНавигационнуюСсылку()` передается ссылка на объект – `ХранимыйФайл.Ссылка`, а во втором – имя реквизита.

Далее с помощью метода `ПолучитьФайл()` на основе навигационной ссылки, сохраненной в переменной `Адрес`, мы получаем временный файл с именем, записанным в переменной `ИмяФайла`.

И после этого с помощью метода глобального контекста `ЗапуститьПриложение()` выполняется открытие этого временного файла с помощью ассоциированного с ним приложения (например, при открытии видео запускается видеоплеер и т.п.).

Для того чтобы в процедуре `ОткрытьФайл()` нам были доступны реквизиты `ИмяФайла` и `Ссылка` справочника `ХранимыеФайлы`, нужно установить флажок `Использовать всегда` для этих полей в свойствах основного реквизита формы `Список` (см. рис. 3.19). Дело в том, что по умолчанию в форме на клиенте доступны данные только тех полей динамического списка, которые отображаются в форме. Но при установке флажка `Использовать всегда` данные этих полей будут передаваться на клиента даже несмотря на то, что в форме они нигде не отображаются.

Откроем список хранимых файлов в режиме `1С:Предприятие`, выделим нужный элемент списка и нажмем кнопку `Открыть файл`. Содержимое выбранного файла картинки будет помещено во временный файл и открыто соответствующим приложением (рис. 3.20).

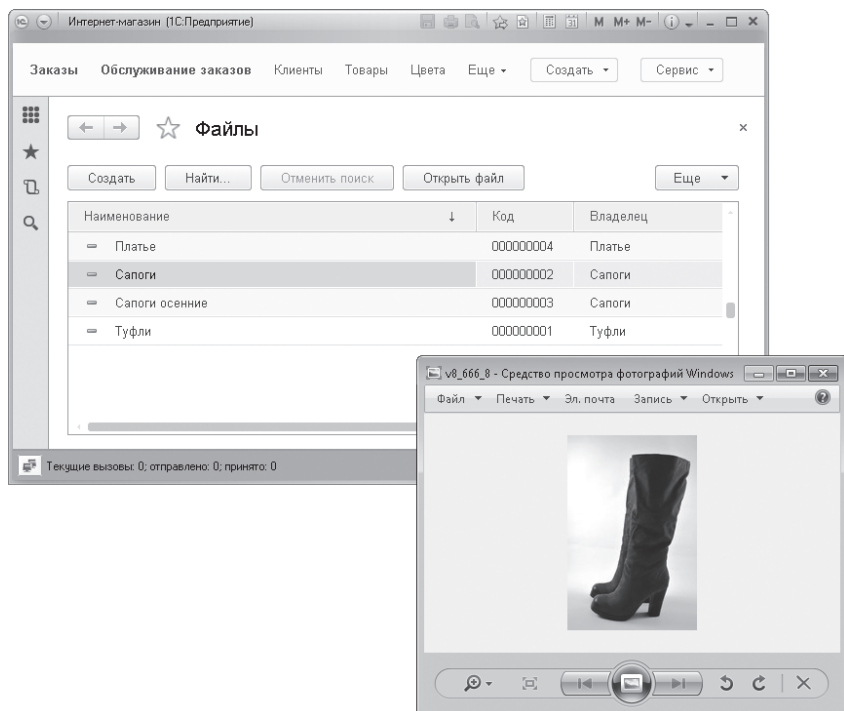


Рис. 3.20. Открытие текущего хранимого файла из списка файлов

Товары

Вначале сделаем список товаров более информативным. Для этого создадим форму списка справочника Товары и ограничим состав полей списка товаров реквизитами Наименование, Код, Артикул и Описание.

Отображение и выбор картинки товара

Теперь, используя справочник хранимых файлов, мы можем добавить ссылку на файл картинки в список реквизитов товара и отобразить картинку в форме товара, а также предоставить возможность перевыбора этой картинки.

Для этого в окне редактирования справочника Товары на закладке Данные добавим реквизит ФайлКартинки типа СправочникСсылка.ХранимыеФайлы и установим у него в свойстве Связи параметров выбора отбор по владельцу – Отбор.Владелец(Ссылка), а также свойство Синоним зададим как Картинка (рис. 3.21).

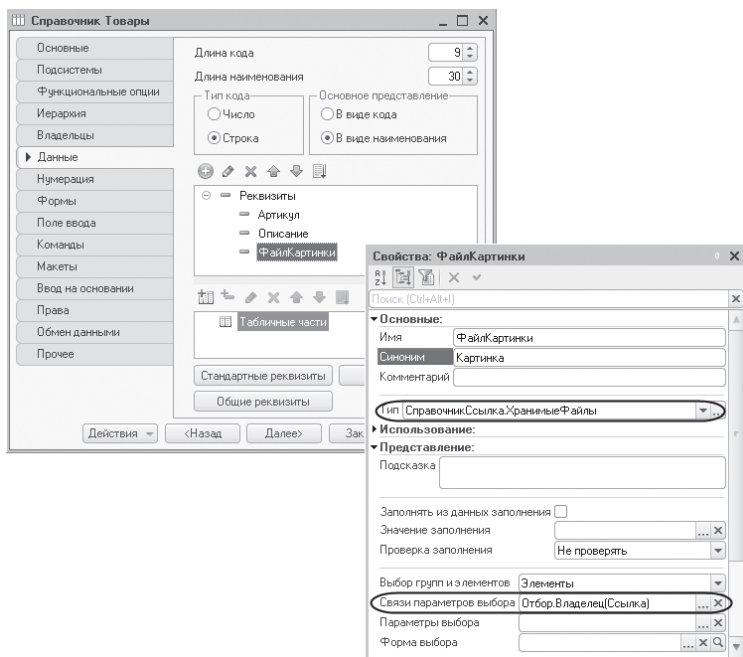


Рис. 3.21. Добавление реквизита «ФайлКартинки» в справочник «Товары»

То есть в справочнике Товары будет храниться не сама картинка товара, а ссылка на нее. Такой подход, когда все данные типа ХранилищеЗначения хранятся отдельно от других данных объекта, является методически правильным. Дело в том, что при обращении к данным объектов они целиком считываются из базы данных. Поэтому манипулирование этими объектами может замедляться из-за необходимости считывания из базы данных больших объемов информации. Особенно это чувствительно на мобильном устройстве.

Отбор по владельцу в свойствах реквизита нужен для того, чтобы при выборе хранимых файлов их список отбирался бы по владельцу. В нашем случае при выборе картинки товара должны предлагаться только файлы картинок, относящиеся к конкретному товару.

Также сделаем небольшое улучшение в интерфейсе – нажмем кнопку Стандартные реквизиты и установим для поля Родитель синоним Группа.

Теперь создадим форму элемента справочника Товары и отобразим в ней поле для выбора картинки, а также поместим само изображение товара.

Для этого в палитре свойств элемента формы **ФайлКартинки** установим свойства: **КнопкаВыпадающегоСписка** – **Нет**, **КнопкаОчистки** – **Да**, **ОтображениеКнопкиВыбора** – **В поле ввода** (рис. 3.22).

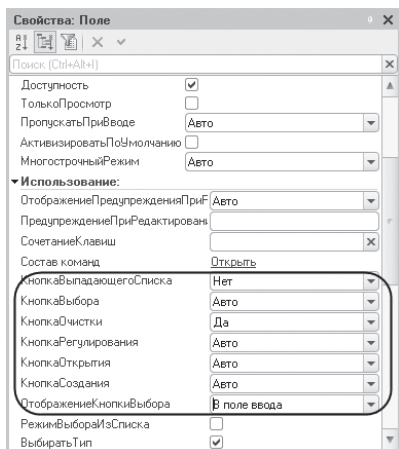


Рис. 3.22. Палитра свойств поля формы товара «ФайлКартинки»

Для отображения картинки в форме добавим строковый реквизит **АдресКартинки**, в котором будет храниться навигационная ссылка на данные картинки в справочнике хранимых файлов. Перетащим реквизит **АдресКартинки** в форму. У соответствующего поля формы установим свойства: **Вид** – **Поле картинки**, **ПоложениеЗаголовка** – **Нет** (рис. 3.23).

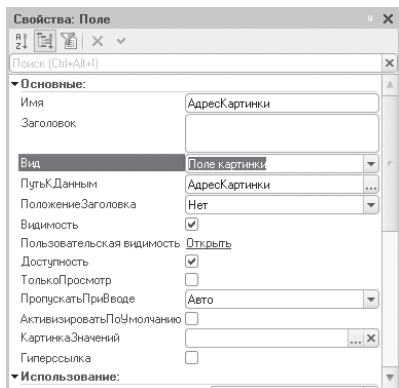


Рис. 3.23. Палитра свойств поля формы товара «АдресКартинки»

В результате форма элемента справочника в редакторе формы примет следующий вид (рис. 3.24).

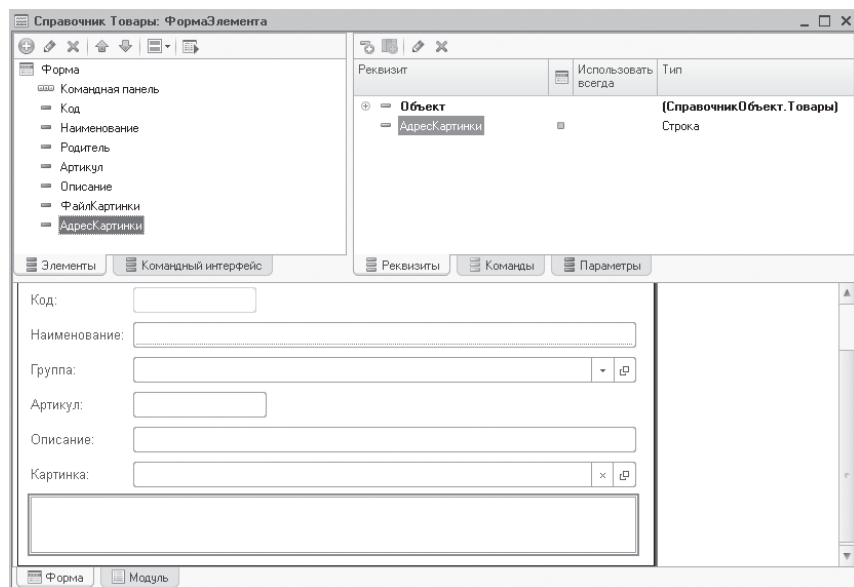


Рис. 3.24. Форма товара в редакторе формы

Затем установим значение реквизита АдресКартинки при создании формы товара. Для этого создадим обработчик события формы ПриСозданииНаСервере и заполним его следующим образом (листинг 3.19).

Листинг 3.19. Обработчик события формы «ПриСозданииНаСервере»

```
&НаСервере
Процедура ПриСозданииНаСервере(Отказ, СтандартнаяОбработка)
    ФайлКартинки = Объект.ФайлКартинки;
    Если НЕ ФайлКартинки.Пустая() Тогда
        АдресКартинки = ПолучитьНавигационнуюСсылку(ФайлКартинки, "ДанныеФайла")
    Конечесли;
КонечПроцедуры
```

В этом обработчике мы присваиваем реквизиту АдресКартинки навигационную ссылку на реквизит ДанныеФайла хранимого файла, на который указывает реквизит товара ФайлКартинки. Таким образом, при создании формы товара, если у товара установлена картинка, она автоматически отображается в форме.

При перевыборе картинки товара в поле **ФайлКартинки** мы должны отследить изменение картинки и обновить навигационную ссылку в реквизите **АдресКартинки**. Для этого создадим обработчик события **ПриИзменении** для поля формы **ФайлКартинки** и заполним его следующим образом (листинг 3.20).

Листинг 3.20. Обработчик события формы «**ПриИзменении**»
поля формы «**ФайлКартинки**»

```
&НаКлиенте
Процедура ФайлКартинкиПриИзменении(Элемент)

    ФайлКартинки = Объект.ФайлКартинки;
    Если НЕ ФайлКартинки.Пустая() Тогда
        АдресКартинки = ПолучитьНавигационнуюСсылку(ФайлКартинки, "ДанныеФайла")
    Иначе
        АдресКартинки = "";
    КонецЕсли;

КонецПроцедуры
```

Также при начале выбора картинки товара нужно проверить, чтобы данные товара были записаны. Для этого создадим обработчик события **НачалоВыбора** для поля формы **ФайлКартинки** и заполним его следующим образом (листинг 3.21).

Листинг 3.21. Обработчик события формы «**НачалоВыбора**»
поля формы «**ФайлКартинки**»

```
&НаКлиенте
Процедура ФайлКартинкиНачалоВыбора(Элемент, ДанныеВыбора, СтандартнаяОбработка)

    Если Объект.Ссылка.Пустая() Тогда
        ПоказатьПредупреждение("Данные не записаны!");
        СтандартнаяОбработка = Ложь;
    Возврат;
    КонецЕсли;

КонецПроцедуры
```

Теперь проверим, как это работает.

Откроем форму товара **Туфли** и нажмем кнопку выбора в поле **Картинка**. Поскольку раньше мы добавили картинку **Туфли** в справочник **ХранимыеФайлы** и указали этот товар в качестве владельца файла (см. рис. 3.16, 3.17), то нам будет показана именно эта запись из списка хранимых файлов (рис. 3.25).

Нажмем кнопку **Выбрать**, и выбранная картинка будет установлена как картинка товара и сразу же показана в форме (рис. 3.26).

Однако если у товара еще нет картинки, можно прямо здесь же создать ее, записать в справочник хранимых файлов и установить ее в качестве картинки товара.

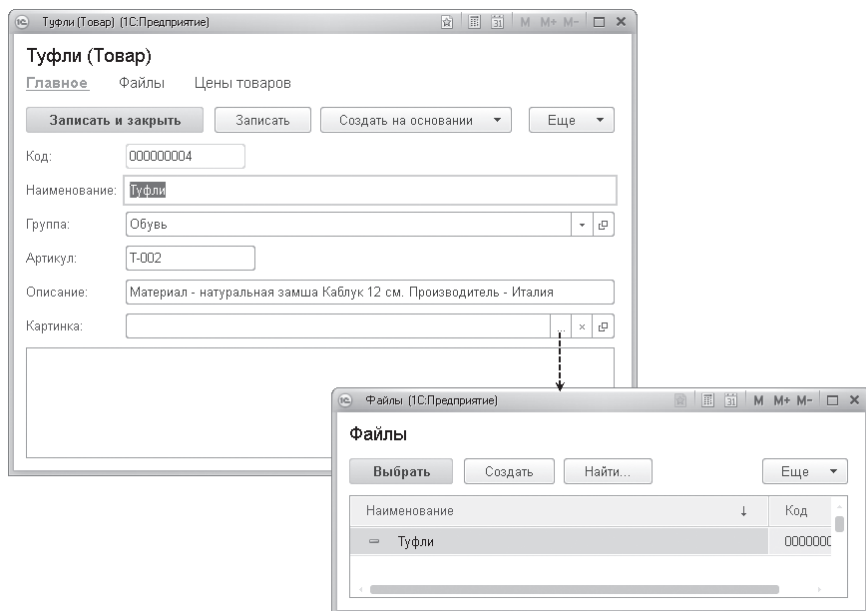


Рис. 3.25. Выбор картинки товара в форме товара

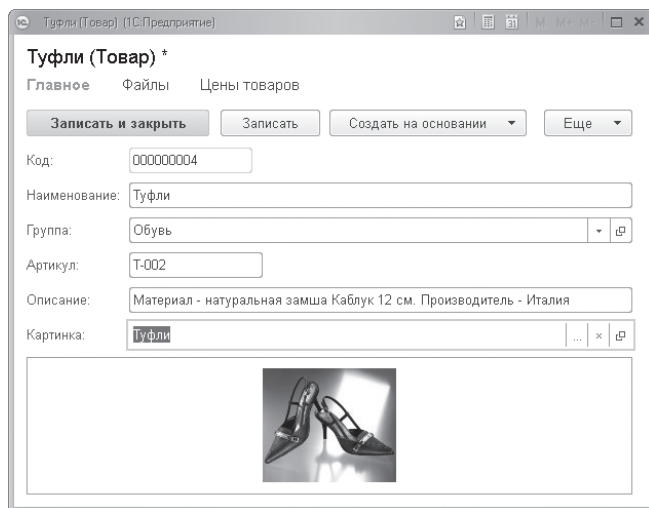


Рис. 3.26. Отображение картинки товара

Например, откроем товар Джинсы и нажмем кнопку выбора в поле Картинка. Для выбора откроется пустой список хранимых файлов, отобранный по товару-владельцу. Нажмем в форме выбора кнопку Создать. Откроется форма создания хранимого файла, в которой в поле Владелец уже будет указан товар Джинсы. Затем нажмем кнопку Выбрать файл с диска и запишем выбранный файл в справочник хранимых файлов. После этого нажмем кнопку Выбрать, и выбранная картинка будет установлена как картинка товара и сразу же показана в форме (рис. 3.27, 3.28).

Если в форме товара нажать на ссылку Файлы, то можно увидеть список всех подчиненных этому товару файлов. Здесь можно, например, создать несколько картинок для одного товара. Поскольку ранее в форму списка хранимых файлов мы добавили команду Открыть файл, то можно открыть выделенный в списке файл ассоциированным с ним приложением (рис. 3.29).

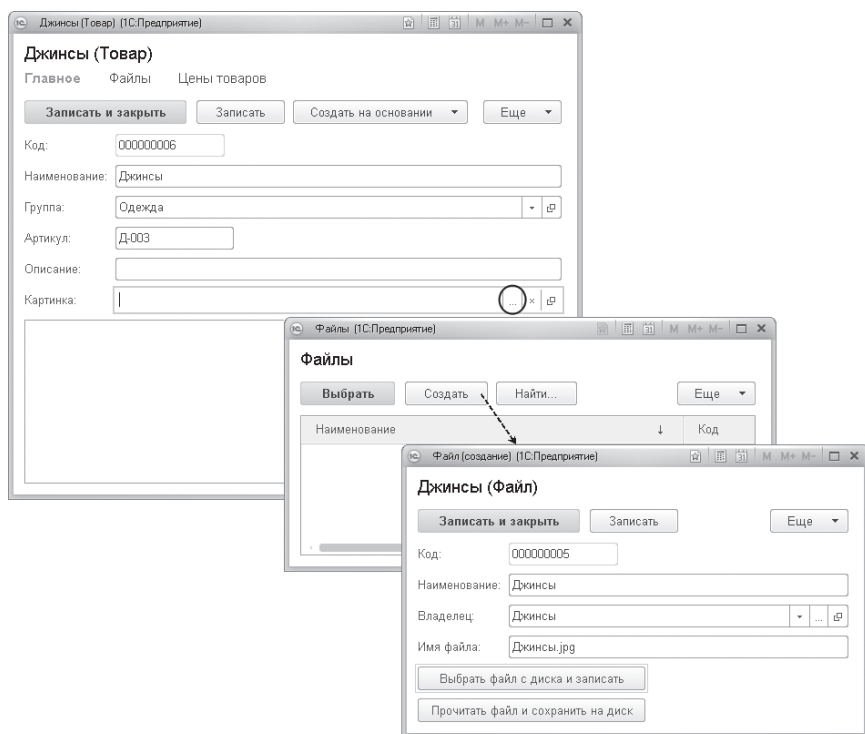


Рис. 3.27. Создание картинки товара в выбор ее в форме товара

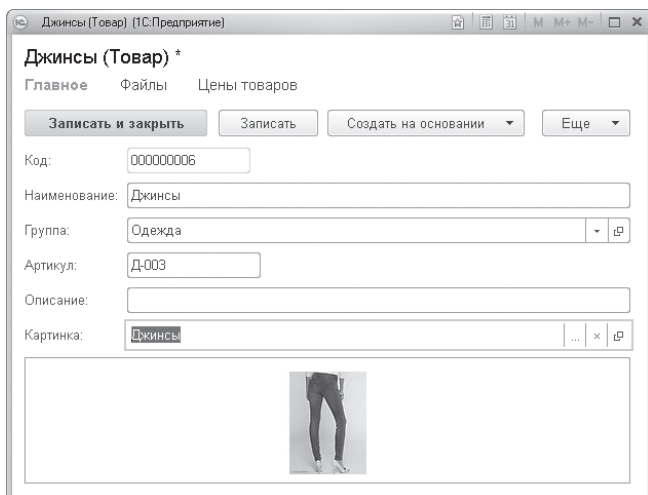


Рис. 3.28. Отображение картинки товара

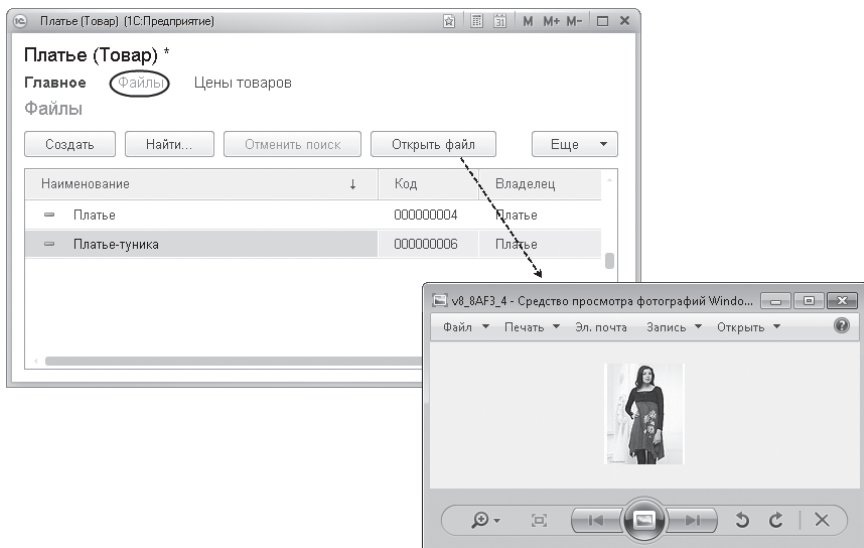


Рис. 3.29. Открытие файла, подчиненного товару

Отображение картинки товара в мобильном приложении

В мобильном приложении мы должны обеспечить отображение картинки в форме товара, но курьер не должен иметь возможности ее пере выбора.

По условию нашей задачи файлы картинок выбираются и записываются в справочник `ХранимыеФайлы` в основном приложении, затем ссылки на них устанавливаются в качестве картинок товара, как мы только что показали. Эти данные передаются на планшет в процессе обмена данными. В мобильном приложении нам осталось только отобразить картинку в форме товара.

Итак, откроем мобильное приложение `КурьерИнтернетМагазина`. В окне редактирования справочника `Товары` на закладке `Данные` добавим реквизит `ФайлКартинки` типа `СправочникСсылка.ХранимыеФайлы` и установим у него в свойстве `Связи параметров выбора` отбор по владельцу – `Отбор.Владелец(Ссылка)`, а также свойство `Синоним` зададим как `Картинка`.

Затем создадим форму справочника `Товары` и добавим в нее строковый реквизит `АдресКартинки`, в котором будет храниться навигационная ссылка на данные картинки в справочнике хранимых файлов. Перетащим реквизит `АдресКартинки` в форму. У соответствующего поля формы установим свойства: Вид – `Поле картинки`, `ПоложениеЗаголовка` – `Нет`.

Затем установим значение реквизита `АдресКартинки` при открытии формы уже существующего товара. Для этого создадим обработчик события формы `ПриЧтенииНаСервере` и заполним его следующим образом (листинг 3.22).

Листинг 3.22. Обработчик события формы «ПриЧтенииНаСервере»

```
&НаСервере  
Процедура ПриЧтенииНаСервере(ТекущийОбъект)  
  
    ФайлКартинки = Объект.ФайлКартинки;  
    Если НЕ ФайлКартинки.Пустая() Тогда  
        АдресКартинки = ПолучитьНавигационнуюСсылку(ФайлКартинки, "ДанныеФайла")  
    Конечесли;  
  
КонецПроцедуры
```

В этом обработчике мы присваиваем реквизиту `АдресКартинки` навигационную ссылку на реквизит `ДанныеФайла` хранимого файла, на который указывает реквизит товара `ФайлКартинки`.

Таким образом, при открытии формы уже существующего товара, если у товара установлена картинка, она автоматически отображается в форме (рис. 3.30).

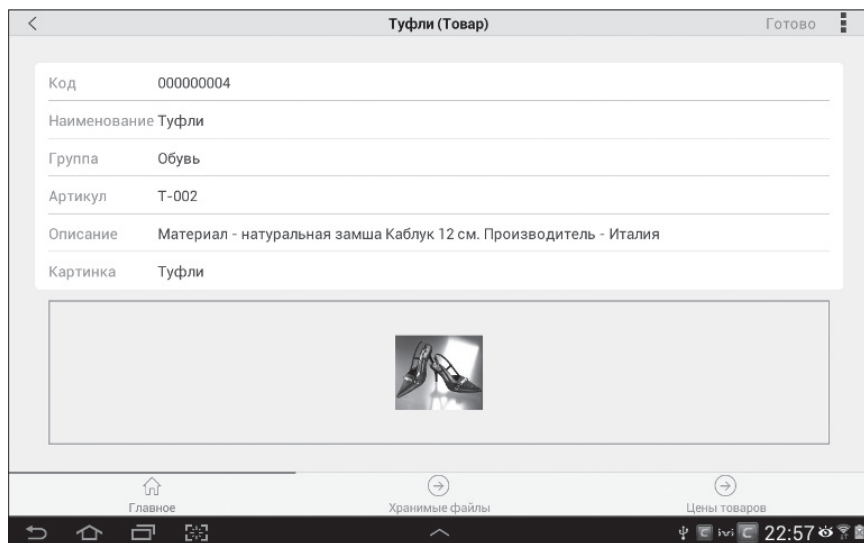


Рис. 3.30. Форма товара в мобильном приложении

ПРИМЕЧАНИЕ

Чтобы увидеть картинки товаров в мобильном приложении, нужно еще раз выполнить обмен данными — выгрузить данные из основного приложения и загрузить в мобильное (см. раздел «Перенос данных на планшет» на стр. 53).

Отчеты

В реальной жизни, конечно же, прежде чем принимать заказы от клиентов, интернет-магазин должен сначала получить эти товары на склад. После выполнения заказов курьером и получения их обратно в офис на основании заказов создаются документы о продажах. Затем документы проводятся и производят движения в регистрах накопления, которые служат источниками данных для различных отчетов.

Приходовать и расходовать товары мы будем в разрезе цветов и размеров. Данная функциональность будет весьма схематичной и упрощенной, так как нам она нужна не сама по себе, а только как источник данных для отчетов о наличии товаров на складах.

Приход товаров

Для учета поступлений товаров создадим документ Приходная Накладная (представление списка – Поступления).

Добавим реквизит документа Склад (тип СправочникСсылка.Склады, Проверка заполнения – Выдавать ошибку).

Добавим табличную часть документа Товары с реквизитами:

- Товар (тип СправочникСсылка.Товары, Проверка заполнения – Выдавать ошибку);
- Цвет (тип СправочникСсылка.Цвета, Проверка заполнения – Выдавать ошибку);
- Размер (тип СправочникСсылка.Размеры, Проверка заполнения – Выдавать ошибку);
- Цена (тип Число, длина 10, 2, Неотрицательное, Проверка заполнения – Выдавать ошибку);
- Количество (тип Число, длина 5, 0);
- Сумма (тип Число, длина 10, 2).

Затем создадим основную форму этого документа. Внешним видом формы мы заниматься не будем, только добавим под таблицей товаров стандартные итоговые поля ИтогКоличество и ИтогСумма из табличной части документа (рис. 3.31).

А также добавим в модуль формы процедуры для автоматического пересчета суммы по товару при изменении его цены и количества (листинги 3.23, 3.24).

Листинг 3.23. Обработчик события «ПриИзменении» для поля «ТоварыЦена»

```
&НаКлиенте
Процедура ТоварыЦенаПриИзменении(Элемент)

    СтрокаТабличнойЧасти = Элементы.Товары.ТекущиеДанные;
    СтрокаТабличнойЧасти.Сумма = СтрокаТабличнойЧасти.Количество * СтрокаТабличнойЧасти.Цена;

КонецПроцедуры
```

Листинг 3.24. Обработчик события «ПриИзменении» для поля «ТоварыКоличество»

```
&НаКлиенте
Процедура ТоварыКоличествоПриИзменении(Элемент)

    СтрокаТабличнойЧасти = Элементы.Товары.ТекущиеДанные;
    СтрокаТабличнойЧасти.Сумма = СтрокаТабличнойЧасти.Количество * СтрокаТабличнойЧасти.Цена;

КонецПроцедуры
```

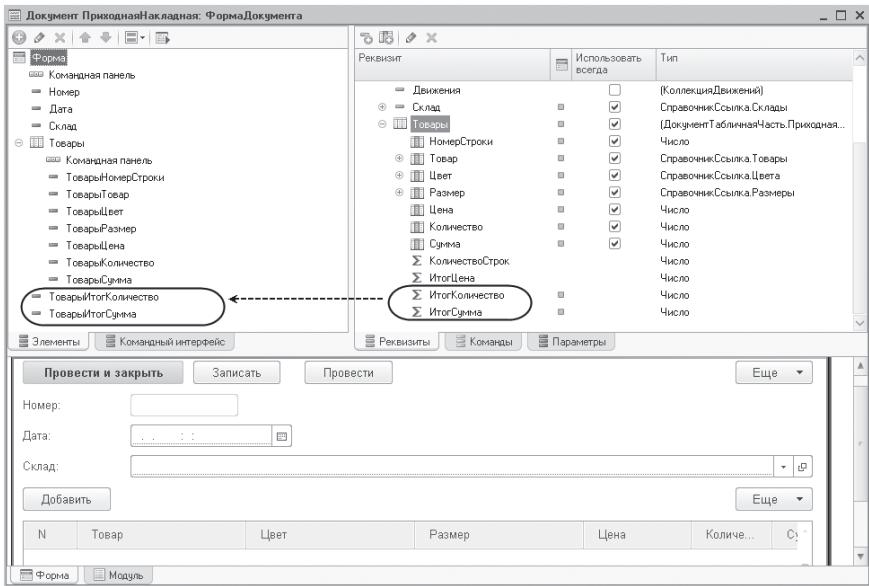


Рис. 3.31. Форма приходной накладной в редакторе формы

Назначение этих процедур уже неоднократно разбиралось ранее (см. раздел «Расчет суммы по товару» на стр. 85), поэтому мы не будем еще раз на этом останавливаться.

Затем создадим клиентские обработчики события ПриИзменении для полей таблицы формы ТоварыЦена и ТоварыКоличество и свяжем их с соответствующими процедурами в модуле формы.

А для поля ТоварыСумма в палитре свойств установим свойство ТолькоПрочитать в значение Истина, так как это поле будет расчетным и не должно изменяться вручную.

Расход товаров

После того как курьер доставил заказ покупателю, поставил отметку о выполнении заказа (статус заказа Выполнен) и в процессе обмена данными передал заказ обратно в офис, на основании этого заказа формируется документ о продаже товаров клиенту.

Для учета продаж товаров создадим документ РасходнаяНакладная (представление списка – Продажи).

Добавим реквизиты документа:

- Клиент (тип СправочникСсылка.Клиенты, Проверка заполнения – Выдавать ошибку);
- Курьер (тип СправочникСсылка.Пользователи, Проверка заполнения – Выдавать ошибку);
- Склад (тип СправочникСсылка.Склады, Проверка заполнения – Выдавать ошибку);
- Основание (тип ДокументСсылка.Заказ).

Добавим табличную часть документа Товары с реквизитами:

- Товар (тип СправочникСсылка.Товары, Проверка заполнения – Выдавать ошибку);
- Цвет (тип СправочникСсылка.Цвета, Проверка заполнения – Выдавать ошибку);
- Размер (тип СправочникСсылка.Размеры, Проверка заполнения – Выдавать ошибку);
- Цена (тип Число, длина 10, 2, Неотрицательное, Проверка заполнения – Выдавать ошибку);
- Количество (тип Число, длина 5, 0);
- Сумма (тип Число, длина 10, 2).

На закладке Ввод на основании окна редактирования документа укажем, что документ вводится на основании документа Заказ (рис. 3.32).

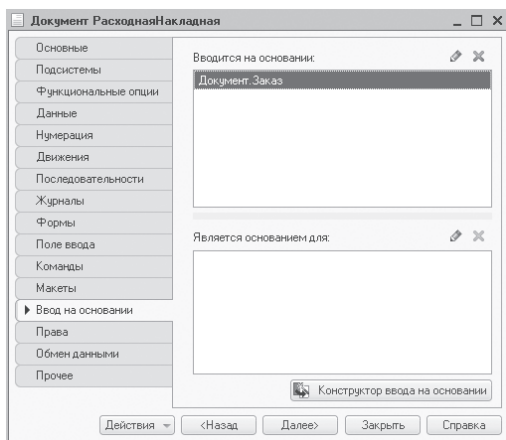


Рис. 3.32. Список объектов, на основании которых вводится документ «Расходная накладная»

Затем создадим основную форму этого документа. Внешним видом формы мы заниматься не будем, только добавим под таблицей товаров стандартные итоговые поля ИтогКоличество и ИтогСумма из табличной части документа (см. рис. 3.31). Кроме того, установим у поля Основание свойство ТолькоПросмотр, чтобы пользователь не мог изменять ссылку на заказ, на основании которого создана расходная накладная.

А также добавим в модуль формы процедуры для автоматического пересчета суммы по товару при изменении его цены и количества (листинги 3.25, 3.26).

Листинг 3.25. Обработчик события «ПриИзменении» для поля «ТоварыЦена»

```
&НаКлиенте
Процедура ТоварыЦенаПриИзменении(Элемент)

    СтрокаТабличнойЧасти = Элементы.Товары.ТекущиеДанные;
    СтрокаТабличнойЧасти.Сумма = СтрокаТабличнойЧасти.Количество * СтрокаТабличнойЧасти.Цена;

КонецПроцедуры
```

Листинг 3.26. Обработчик события «ПриИзменении» для поля «ТоварыКоличество»

```
&НаКлиенте
Процедура ТоварыКоличествоПриИзменении(Элемент)

    СтрокаТабличнойЧасти = Элементы.Товары.ТекущиеДанные;
    СтрокаТабличнойЧасти.Сумма = СтрокаТабличнойЧасти.Количество * СтрокаТабличнойЧасти.Цена;

КонецПроцедуры
```

Назначение этих процедур уже неоднократно разбиралось ранее (см. раздел «Расчет суммы по товару» на стр. 85), поэтому мы не будем еще раз на этом останавливаться.

Затем создадим клиентские обработчики события ПриИзменении для полей таблицы формы ТоварыЦена и ТоварыКоличество и свяжем их с соответствующими процедурами в модуле формы.

А для поля ТоварыСумма в палитре свойств установим свойство ТолькоПросмотр в значение Истина, так как это поле будет расчетным и не должно изменяться вручную.

Кроме того, добавим в модуль формы процедуры для автоматической подстановки актуальной цены товара из регистра сведений при выборе товара (листинги 3.27, 3.28).

Листинг 3.27. Обработчик события «ПриИзменении» для поля «ТоварыТовар»

```

&НаКлиенте
Процедура ТоварыТоварПриИзменении(Элемент)

    СтрокаТабличнойЧасти = Элементы.Товары.ТекущиеДанные;

    СтрокаТабличнойЧасти.Цена = ПолучитьЦенуТовара(Объект.Дата, СтрокаТабличнойЧасти.Товар);
    СтрокаТабличнойЧасти.Количество = ?(СтрокаТабличнойЧасти.Количество = 0, 1,
                                         СтрокаТабличнойЧасти.Количество);

    СтрокаТабличнойЧасти.Сумма = СтрокаТабличнойЧасти.Количество * СтрокаТабличнойЧасти.Цена;
КонецПроцедуры

```

Листинг 3.28. Функция для получения актуальной цены товара

```

&НаСервереБезКонтекста
Функция ПолучитьЦенуТовара(Дата, Товар)

    Отбор = Новый Структура("Товар", Товар);
    ЦенаТовара = РегистрыСведений.ЦеныТоваров.ПолучитьПоследнее(Дата, Отбор);
    Возврат ЦенаТовара.Цена;

КонецФункции

```

Назначение этих процедур уже разбиралось ранее (см. раздел «Получение актуальной цены товара» на стр. 86), поэтому мы не будем еще раз на этом останавливаться.

Затем создадим клиентский обработчик события ПриИзменении для поля таблицы ТоварыТовар и свяжем его с соответствующей процедурой в модуле формы.

В заключение, поскольку расходная накладная будет создаваться на основании заказа, нам нужно обеспечить заполнение расходной накладной соответствующими значениями реквизитов заказа.

Для этого мы должны создать обработчик события ОбработкаЗаполнения и поместить его в модуль документа. Этот обработчик будет вызываться как при программном заполнении документа при выполнении метода Заполнить(), так и при интерактивном вводе нового документа на основании.

Итак, откроем модуль документа РасходнаяНакладная и поместим в него процедуру ОбработкаЗаполнения(). В параметре ДанныеЗаполнения этой процедуры нам будет доступен объект-основание (листинг 3.29).

Листинг 3.29 Обработчик события документа «ОбработкаЗаполнения»

Процедура ОбработкаЗаполнения(ДанныеЗаполнения, ТекстЗаполнения, СтандартнаяОбработка)

```
Если ТипЗнч(ДанныеЗаполнения) = Тип("ДокументСсылка.Заказ") Тогда
    // Заполнение шапки
    Основание = ДанныеЗаполнения.Ссылка;
    Клиент = ДанныеЗаполнения.Клиент;
    Курьер = ДанныеЗаполнения.Курьер;
    Склад = ДанныеЗаполнения.Склад;

    // Заполнение табличной части
    Для Каждого ТекСтрокаТовары Из ДанныеЗаполнения.Товары Цикл
        Если ТекСтрокаТовары.Отказ = Ложь Тогда
            НоваяСтрока = Товары.Добавить();
            НоваяСтрока.Товар = ТекСтрокаТовары.Товар;
            НоваяСтрока.Цвет = ТекСтрокаТовары.Цвет;
            НоваяСтрока.Размер = ТекСтрокаТовары.Размер;
            НоваяСтрока.Цена = ТекСтрокаТовары.Цена;
            НоваяСтрока.Количество = ТекСтрокаТовары.Количество;
            НоваяСтрока.Сумма = ТекСтрокаТовары.Сумма;
        КонецЕсли;
    КонецЦикла;
КонецЕсли;
КонецПроцедуры
```

В этом обработчике мы заполняем реквизиты новой расходной накладной соответствующими значениями реквизитов объекта-основания. В реквизит документа Основание попадает ссылка на заказ-основание. Заметьте, что в табличную часть расходной накладной попадает информация только о тех товарах, по которым не было отказа.

Надо понимать, что расходная накладная будет заполняться данными заказа в основном программно – при записи заказа в основную базу после получения выполненного заказа от курьера. Но на всякий случай мы предусмотрели возможность интерактивного создания расходной накладной на основании заказа. В этом случае вся ответственность за достоверность данных лежит уже на менеджере интернет-магазина.

Главное, что в обоих случаях будет срабатывать обработчик события документа ОбработкаЗаполнения() и расходная накладная будет заполнена данными заказа по описанному нами алгоритму.

Регистры накопления

Теперь создадим структуры для накопления информации, содержащейся в созданных нами документах. Это регистры накопления, которые будут хранить движения проведенных документов и служить источниками данных для отчетов.

Прежде всего, это регистр накопления остатков `ОстаткиТоваров`, который будет накапливать информацию о приходе, заказе и расходе товаров на складах в разрезе цветов и размеров.

Регистраторами этого регистра будут документы `ПриходнаяНакладная` (тип движения `Приход`), `Заказ` (тип движения `Приход/Расход`) и `РасходнаяНакладная` (тип движения `Расход`).

Измерениями регистра будут:

- `Товар` (тип `СправочникСсылка.Товары`);
- `Цвет` (тип `СправочникСсылка.Цвета`);
- `Размер` (тип `СправочникСсылка.Размеры`);
- `Склад` (тип `СправочникСсылка.Склады`).

Ресурсами регистра будут:

- `Заказано` (тип `Число`, длина 10, 0);
- `Количество` (тип `Число`, длина 10, 0).

Затем нам понадобится регистр накопления оборотов `Продажи`, который будет накапливать информацию о продажах товаров в разрезе клиентов и курьеров.

Регистратором этого регистра будет документ `РасходнаяНакладная`.

Измерениями регистра будут:

- `Товар` (тип `СправочникСсылка.Товары`);
- `Клиент` (тип `СправочникСсылка.Клиенты`);
- `Курьер` (тип `СправочникСсылка.Пользователи`).

Ресурсами регистра будут:

- `Количество` (тип `Число`, длина 10, 0);
- `Сумма` (тип `Число`, длина 15, 2).

А также для анализа причин отказа от товаров нам понадобится регистр накопления оборотов `ПричиныОтказа`, который будет накапливать информацию об отказах от товаров в разрезе причин отказа.

Регистратором этого регистра будет документ `Заказ`.

Измерениями регистра будут:

- `Товар` (тип `СправочникСсылка.Товары`);
- `ПричинаОтказа` (тип `СправочникСсылка.ПричиныОтказа`).

Ресурсами регистра будут:

- `Отказ` (тип `Число`, длина 10, 0).

Движения документов при проведении

Теперь сформируем движения в этих регистрах накопления при проведении документов. Это делается в обработчике события `ОбработкаПроведения`, который помещается в модуль документов. Именно в эту процедуру разработчик должен поместить собственные алгоритмы по учету данных, выполняемые в момент проведения документа.

Приходная накладная

Откроем окно редактирования объекта конфигурации `Документ ПриходнаяНакладная`. Перейдем на закладку `Движения`, раскроем список `Регистры накопления` и отметим регистр накопления `ОстаткиТоваров` (рис. 3.33).

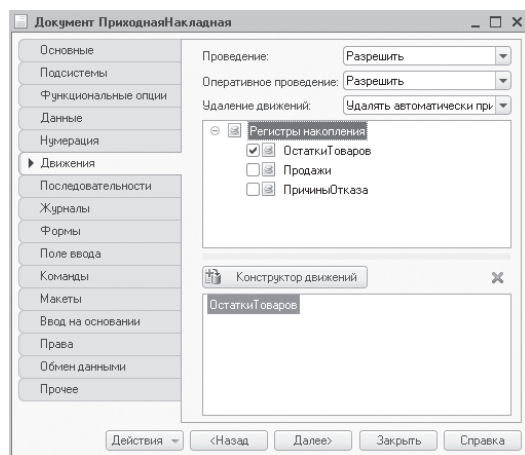


Рис. 3.33. Создание движений документа «ПриходнаяНакладная» в регистре «ОстаткиТоваров»

Здесь же можно нажать на кнопку `Конструктор движений` и сформировать движения документа в регистре с помощью конструктора. Но мы сделаем это вручную.

Откроем модуль документа `ПриходнаяНакладная` и поместим туда процедуру `ОбработкаПроведения()`, заполненную следующим образом (листинг 3.30).

Листинг 3.30. Обработчик события документа «ОбработкаПроведения»

Процедура ОбработкаПроведения(Отказ, Режим)

```
// регистр ОстаткиТоваров Приход
Движения.ОстаткиТоваров.Записывать = Истина;
Для Каждого ТекСтрокаТовары Из Товары Цикл
    Движение = Движения.ОстаткиТоваров.Добавить();
    Движение.ВидДвижения = ВидДвиженияНакопления.Приход;
    Движение.Период = Дата;
    Движение.Товар = ТекСтрокаТовары.Товар;
    Движение.Цвет = ТекСтрокаТовары.Цвет;
    Движение.Размер = ТекСтрокаТовары.Размер;
    Движение.Склад = Склад;
    Движение.Количество = ТекСтрокаТовары.Количество;
КонецЦикла;
КонецПроцедуры
```

В первой строке процедуры мы устанавливаем свойство `Записывать` набора записей регистра `ОстаткиТоваров` в значение `Истина`. То есть в явном виде указываем, что после завершения обработки проведения платформа должна будет записать этот набор записей в базу данных.

Затем обходим табличную часть документа в цикле. На каждом шаге цикла мы добавляем новую запись в коллекцию движений документа в регистре `ОстаткиТоваров`. Тем самым мы создаем объект `РегистрНакопленияЗапись` и сохраняем его в переменной `Движение`.

Используя этот объект, мы можем обратиться к полям этой записи, указав имя поля через точку от этой переменной (например, `Движение.Количество`).

Сначала мы задаем вид движения (`Движение.ВидДвижения`) как значение системного перечисления (`ВидДвиженияНакопления.Приход`), которое определяет вид движения регистра накопления как `Приход`.

Чтобы присвоить полям новой записи регистра соответствующие значения полей документа, мы обращаемся к полям табличной части, указав имя поля через точку от переменной `ТекСтрокаТовары` (например, `ТекСтрокаТовары.Цвет`).

Причем в цикле меняются только значения полей табличной части документа. Поля `Дата` и `Склад` не меняются, так как относятся к документу в целом и не зависят от текущей строки табличной части документа.

Таким образом мы присваиваем нужные значения всем полям новой записи. После завершения цикла в наборе записей (`Движения.ОстаткиТоваров`) будет содержаться столько записей, сколько строк в табличной части проводимого документа.

Расходная накладная

Затем откроем в конфигураторе окно редактирования объекта конфигурации Документ РасходнаяНакладная и на закладке Движения укажем, что этот документ будет создавать движения в двух регистрах: ОстаткиТоваров и Продажи.

Откроем модуль документа РасходнаяНакладная и поместим туда процедуру ОбработкаПроведения(), заполненную следующим образом (листинг 3.31).

Листинг 3.31. Обработчик события документа «ОбработкаПроведения»

Процедура ОбработкаПроведения(Отказ, Режим)

```
Движения.ОстаткиТоваров.Записывать = Истина;  
Движения.Продажи.Записывать = Истина;
```

Для Каждого ТекСтрокаТовары Из Товары Цикл

```
    // регистр ОстаткиТоваров Расход  
    Движение = Движения.ОстаткиТоваров.Добавить();  
    Движение.ВидДвижения = ВидДвиженияНакопления.Расход;  
    Движение.Период = Дата;  
    Движение.Товар = ТекСтрокаТовары.Товар;  
    Движение.Цвет = ТекСтрокаТовары.Цвет;  
    Движение.Размер = ТекСтрокаТовары.Размер;  
    Движение.Склад = Склад;  
    Движение.Количество = ТекСтрокаТовары.Количество;  
    Движение.Заказано = ТекСтрокаТовары.Количество;
```

```
    // регистр Продажи  
    Движение = Движения.Продажи.Добавить();  
    Движение.Период = Дата;  
    Движение.Товар = ТекСтрокаТовары.Товар;  
    Движение.Клиент = Клиент;  
    Движение.Курьер = Курьер;  
    Движение.Количество = ТекСтрокаТовары.Количество;  
    Движение.Сумма = ТекСтрокаТовары.Сумма;
```

КонецЦикла;

КонецПроцедуры

Сначала мы указываем платформе на необходимость записи движений документа по регистрам ОстаткиТоваров и Продажи. Затем обходим табличную часть документа в цикле и на каждом шаге цикла формируем движения в обоих регистрах.

В регистре ОстаткиТоваров ситуация идентичная рассмотренной выше для приходной накладной, но движения формируются с обратным знаком – это определяет вид движения регистра накопления как Расход.

Кроме того, значение поля **Количество** в строке табличной части присваивается двум ресурсам регистра **Количество** и **Заказано**. То есть суммарное количество заказанного товара уменьшается на количество его продаж – иными словами, если товар продан, то он уже не может быть заказан.

Регистр **Продажи** – оборотный, поэтому при формировании движений по этому регистру отсутствует вид движения. В ресурсах этого регистра, помимо количества, учитывается также сумма продаж.

Обратите внимание, что в обработке проведения расходной накладной отсутствует контроль остатков товара. Во-первых, это сделано для упрощения, а во-вторых – такой контроль имеет смысл только при оперативном проведении документа. В нашем же случае считается, что расходная накладная будет программно создаваться, записываться и проводиться в неоперативном режиме при поступлении выполненных заказов от курьера в процессе обмена данными.

Заказ

При проведении заказа мы реализуем следующую бизнес-логику:

- Первоначально заказ создается в интернет-магазине (или иногда на планшете) со статусом **Открыт**.
- Затем у заказа устанавливается курьер и статус заказа **В работе**, и заказ отправляется курьеру на планшет. При проведении заказа с этим статусом формируются движения типа **Приход** в регистре **ОстаткиТоваров**.
- Курьер обслуживает заказ, отмечает товары, от которых клиент отказался, присваивает заказу статус **Выполнен** и отправляет его обратно в офис. При проведении заказа с этим статусом, в случае если клиент отказался от каких-то товаров, формируются движения в регистре **ПричиныОтказа**, а также формируются движения типа **Расход** в регистре **ОстаткиТоваров**.
- Менеджер интернет-магазина проверяет заказ и присваивает ему статус **Закрыт**. После этого на основании заказа создается расходная накладная со списком товаров, которые клиент купил.

Для реализации данного алгоритма при проведении заказа откроем модуль документа **Заказ** и поместим туда процедуру **ОбработкаПроведения()**, заполненную следующим образом (листинг 3.32).

Листинг 3.32. Обработчик события документа «ОбработкаПроведения»

Процедура ОбработкаПроведения(Отказ, Режим)

Если СтатусЗаказа = Перечисления.СтатусыЗаказов.ВРаботе Тогда

```
// регистр ОстаткиТоваров Приход
Движения.ОстаткиТоваров.Записывать = Истина;
Для Каждого ТекСтрокаТовары Из Товары Цикл
    Движение = Движения.ОстаткиТоваров.Добавить();
    Движение.ВидДвижения = ВидДвиженияНакопления.Приход;
    Движение.Период = Дата;
    Движение.Товар = ТекСтрокаТовары.Товар;
    Движение.Цвет = ТекСтрокаТовары.Цвет;
    Движение.Размер = ТекСтрокаТовары.Размер;
    Движение.Склад = Склад;
    Движение.Заказано = ТекСтрокаТовары.Количество;
КонецЦикла;
```

ИначеЕсли СтатусЗаказа = Перечисления.СтатусыЗаказов.Выполнен Тогда

```
Движения.ПричиныОтказа.Записывать = Истина;
Движения.ОстаткиТоваров.Записывать = Истина;
```

Для Каждого ТекСтрокаТовары Из Товары Цикл

```
// регистр ОстаткиТоваров Приход
Движение = Движения.ОстаткиТоваров.Добавить();
Движение.ВидДвижения = ВидДвиженияНакопления.Приход;
Движение.Период = Дата;
Движение.Товар = ТекСтрокаТовары.Товар;
Движение.Цвет = ТекСтрокаТовары.Цвет;
Движение.Размер = ТекСтрокаТовары.Размер;
Движение.Склад = Склад;
Движение.Заказано = ТекСтрокаТовары.Количество;
```

Если ТекСтрокаТовары.Отказ Тогда

```
// регистр ПричиныОтказа
Движение = Движения.ПричиныОтказа.Добавить();
Движение.Период = ДатаДоставки;
Движение.Товар = ТекСтрокаТовары.Товар;
Движение.ПричинаОтказа = ТекСтрокаТовары.ПричинаОтказа;
Движение.Отказ = 1;
```

```
// регистр ОстаткиТоваров Расход
Движение = Движения.ОстаткиТоваров.ДобавитьРасход();
Движение.Период = ДатаДоставки;
Движение.Товар = ТекСтрокаТовары.Товар;
Движение.Цвет = ТекСтрокаТовары.Цвет;
Движение.Размер = ТекСтрокаТовары.Размер;
Движение.Склад = Склад;
Движение.Заказано = ТекСтрокаТовары.Количество;
```

КонецЕсли;

КонецЦикла;

ИначеЕсли СтатусЗаказа = Перечисления.СтатусыЗаказов.Закрыт Тогда

```

Продажа = Ложь;
Для Каждого ТекСтрокаТовары Из Товары Цикл
    Если Не ТекСтрокаТовары.Отказ Тогда
        Продажа = Истина;
        Прервать;
    КонецЕсли;
КонецЦикла;

Если Продажа Тогда
    // проверяем, создавалась ли расходная накладная на основании этого заказа
    Запрос = Новый Запрос;
    Запрос.Текст = "ВЫБРАТЬ * ИЗ
        | Документ.РасходнаяНакладная КАК Продажи
        | ГДЕ
        |     Продажи.Основание = &Заказ";
    Запрос.УстановитьПараметр("Заказ", Ссылка);
    РезультатЗапроса = Запрос.Выполнить();

    Если РезультатЗапроса.Пустой() Тогда
        // создаем расходную накладную на основании заказа
        НовыйДокумент = Документы.РасходнаяНакладная.СоздатьДокумент();
        НовыйДокумент.Дата = ДатаДоставки + 60*60*24;
        НовыйДокумент.Заполнить(Ссылка);
        НовыйДокумент.Записать(РежимЗаписиДокумента.Проведение,
            РежимПроведенияДокумента.Неоперативный);
    КонецЕсли;
КонецЕсли;

КонецЕсли;

КонецПроцедуры

```

В обработчике реализован описанный выше алгоритм действий.

При поступлении заказа в работу количество заказанных товаров учитывается в ресурсе **Заказано** регистра **ОстаткиТоваров** с типом движения накопления **Приход**.

При выполнении заказа отказ от товаров учитывается в ресурсе **Отказ** регистра **ПричиныОтказа**. Дата движений устанавливается равной дате доставки заказа (реквизит **ДатаДоставки**).

А также для товаров, от которых клиент отказался, формируются движения типа **Расход** в регистре **ОстаткиТоваров** в поле **Заказано**. Дата движений устанавливается равной дате доставки заказа. Чтобы не потерять движения типа **Приход**, сформированные ранее этим же заказом при поступлении заказа в работу, мы просто дублируем их.

При закрытии заказа, в случае если клиент купил хотя бы один из заказанных товаров, с помощью запроса проверяется, создавалась ли ранее расходная накладная на основании этого заказа.

Если нет (результат запроса пустой), то на основе товаров, купленных клиентами, создается расходная накладная. Дата документа устанавливается следующим днем после даты доставки заказа.

Расходная накладная программно заполняется с помощью метода Заполнить() на основании ссылки на заказ. Затем документ записывается с проведением (РежимЗаписиДокумента.Проведение) в неоперативном режиме (РежимПроведенияДокумента.Неоперативный).

При проведении расходной накладной формируются движения типа Расход в регистре ОстаткиТоваров и в оборотном регистре Продажи (см. листинг 3.31).

Следует учитывать, что, как правило, выполненные заказы доставляются в основную базу в процессе обмена данными от мобильного приложения. Стандартно полученные при обмене заказы записываются без проведения, то есть процедура обработки проведения не вызывается.

Поэтому в модуле заказа, в процедуре ПередЗаписью(), в случае если запись выполненного заказа происходит в процессе обмена данными, продублируем соответствующую часть кода из процедуры ОбработкаПроведения(), рассмотренной выше (листинг 3.33).

Листинг 3.33. Обработчик события документа «ПередЗаписью»

Процедура ПередЗаписью(РежимЗаписи, РежимПроведения)

Если ОбменДанными.Загрузка И СтатусЗаказа = Перечисления.СтатусыЗаказов.Выполнен Тогда

 Движения.ПричиныОтказа.Записывать = Истина;

 Движения.ОстаткиТоваров.Записывать = Истина;

 Для Каждого ТекСтрокаТовары Из Товары Цикл

 // регистр ОстаткиТоваров Приход

 Движение = Движения.ОстаткиТоваров.Добавить();

 Движение.ВидДвижения = ВидДвиженияНакопления.Приход;

 Движение.Период = Дата;

 Движение.Товар = ТекСтрокаТовары.Товар;

 Движение.Цвет = ТекСтрокаТовары.Цвет;

 Движение.Размер = ТекСтрокаТовары.Размер;

 Движение.Склад = Склад;

 Движение.Заказано = ТекСтрокаТовары.Количество;

 Если ТекСтрокаТовары.Отказ Тогда

 // регистр ПричиныОтказа

 Движение = Движения.ПричиныОтказа.Добавить();

 Движение.Период = ДатаДоставки;

 Движение.Товар = ТекСтрокаТовары.Товар;

 Движение.ПричинаОтказа = ТекСтрокаТовары.ПричинаОтказа;

 Движение.Отказ = 1;

 // регистр ОстаткиТоваров Расход

```

Движение = Движения.ОстаткиТоваров.ДобавитьРасход();
Движение.Период = ДатаДоставки;
Движение.Товар = ТекСтрокаТовары.Товар;
Движение.Цвет = ТекСтрокаТовары.Цвет;
Движение.Размер = ТекСтрокаТовары.Размер;
Движение.Склад = Склад;
Движение.Заказано = ТекСтрокаТовары.Количество;
КонецЕсли;

```

```

КонецЦикла;

```

```

КонецЕсли;

```

```

КонецПроцедуры

```

Для определения того, что запись документа происходит в процессе обмена данными, используется значение параметра Загрузка свойства документа ОбменДанными. Это свойство появляется у всех объектов конфигурации, участвующих в обмене.

Команда перехода к записям регистра

В заключение отредактируем командный интерфейс форм документов Заказ, ПриходнаяНакладная и РасходнаяНакладная так, чтобы в панели навигации формы можно было бы переходить к списку записей регистров накопления, связанных с этими документами.

Для этого откроем форму документа Заказ. В левом верхнем окне перейдем на закладку Командный интерфейс. В разделе Панель навигации раскроем группу Перейти и установим свойство Видимость для команд открытия соответствующих регистров накопления (рис. 3.34).

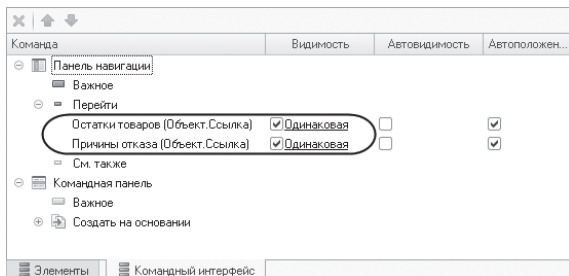


Рис. 3.34. Настройка командного интерфейса формы документа «Заказ»

Аналогичные изменения сделаем для форм документов ПриходнаяНакладная и РасходнаяНакладная.

Проведение документов в режиме «1С:Предприятие»

Теперь запустим «1С:Предприятие» и посмотрим, какие движения формируют документы в регистрах накопления.

Прежде всего создадим две-три приходные накладные, учитывающие поступление товара на склад, и проведем их. Откроем одну из накладных (рис. 3.35).

← → ☆ Приходная накладная 000000001 от 03.12.2013 12:00:00

Главное Остатки товаров

Провести и закрыть Записать Провести Еще ▾

Номер: 000000001

Дата: 03.12.2013 12:00:00

Склад: Основной

Добавить Еще ▾

N	Товар	Цвет	Размер	Цена	Количе...	Сумма
1	Сапоги	Черный	37	5 000,00	5	25 000,00
2	Сапоги	Фиолетовый	38	5 000,00	3	15 000,00
3	Туфли	Черный	37	3 000,00	5	15 000,00
4	Туфли	Красный	38	3 000,00	5	15 000,00
5	Сапоги	Белый	38	5 000,00	3	15 000,00
6	Туфли	Черный	38	3 000,00	5	15 000,00

Количество (итог): 26

Сумма (итог): 100 000,00

Рис. 3.35. «Приходная накладная № 000000001»

В табличной части документа мы видим шесть строк. Общее количество поступивших товаров – 26.

Нажмем на ссылку Остатки товаров в командной панели формы документа и перейдем к движениям регистра ОстаткиТоваров, которые сформировал этот документ (рис. 3.36).

Мы видим, что в наборе записей регистра Остатки товаров, связанном с документом, также шесть строк. Их состав отражает содержание табличной части документа. Количество поступивших товаров учитывается в поле Количество с видом движения накопления Приход.

Теперь введем еще пару заказов, установим у всех шести заказов статус В работе, проведем их и отправим курьеру на планшет. Для этого мы снова воспользуемся обработкой Запасной обмен данными. Откроем ее и выгрузим измененные данные для узла обмена Мобильное приложение – планшет в файл обмена MessageЦО_МПП. Скопируем этот файл в корень планшета и произведем загрузку этих данных в мобильное приложение из узла обмена Центральный офис (см. раздел «Перенос данных на планшет» на стр. 53).

Период	Регистратор	№	Номер стр.	Товар	Цвет	Размер	Склад	Заказано	Количество
+ 03.12.2013 12:00:00	Приходная наклад...	1		Сапоги	Черный	37	Основной		5
+ 03.12.2013 12:00:00	Приходная наклад...	2		Сапоги	Фиолетовый	38	Основной		3
+ 03.12.2013 12:00:00	Приходная наклад...	3		Туфли	Черный	37	Основной		5
+ 03.12.2013 12:00:00	Приходная наклад...	4		Туфли	Красный	38	Основной		5
+ 03.12.2013 12:00:00	Приходная наклад...	5		Сапоги	Белый	38	Основной		3
+ 03.12.2013 12:00:00	Приходная наклад...	6		Туфли	Черный	38	Основной		5

Рис. 3.36. Движения в регистре «Остатки товаров» документа «Приходная накладная № 000000001»

Затем симулируем работу курьера на планшете при обслуживании заказов у клиентов. Откажемся от некоторых позиций заказанных товаров, в одном из заказов откажемся вообще от всех товаров. У всех шести обслуженных заказов установим статус Выполнен. Затем отправим эти данные в основное приложение.

Для этого откроем на планшете обработку Запасной обмен данными и выгрузим измененные данные для узла обмена Центральный офис в файл обмена MessageМПП_ЦО. Скопируем этот файл на компьютер и произведем загрузку этих данных в основное приложение из узла обмена Мобильное приложение – планшет.

Затем в основном приложении откроем список заказов. Установим у четырех первых заказов статус Закрыт и проведем их (рис. 3.37).

Дата	№	Номер	Клиент	Курьер	Дата доставки	Статус заказа
10.12.2013 12:00:00	000000001		Шувалова Е.М.	Морозов Д.И.	13.12.2013	Закрыт
10.12.2013 12:00:01	000000002		Смирнова А.И.	Морозов Д.И.	15.12.2013	Закрыт
11.12.2013 12:00:00	000000004		Смирнова А.И.	Морозов Д.И.	13.12.2013	Закрыт
12.12.2013 22:00:14	000000003		Новикова В.С.	Морозов Д.И.	15.12.2013	Закрыт
20.08.2015 12:00:00	000000006		Шувалова Е.М.	Морозов Д.И.	25.08.2015	Выполнен
23.08.2015 12:00:00	000000005		Новикова В.С.	Морозов Д.И.	25.08.2015	Выполнен

Рис. 3.37. Список заказов

Откроем заказ № 000000002. В табличной части документа – две позиции, от одного из заказанных товаров клиент отказался (закладка Отказ от товаров), см. рис. 3.38.

← → ☆ Заказ 000000002 от 10.12.2013 12:00:01

Главное Остатки товаров Причины отказа

Провести и закрыть Записать Провести Создать на основании Еще

Номер: 000000002 Дата: 10.12.2013 12:00:01

Клиент: Смирнова А.И. Дата доставки: 15.12.2013

Курьер: Морозов Д.И. Адрес доставки: Москва, пр-т Мира 111 кв. 33

Склад: Основной Дополнительно: доставка 12-18

Статус заказа: Открыт В работе Выполнен Закрыт

Список товаров Отказ от товаров

Добавить Еще

N	Товар	Цвет	Размер	Цена	Количество	Сумма
1	Туфли	Красный	38	5 000,00	1	5 000,00
2	Салоги	Белый	38	10 000,00	1	10 000,00

Количество итог: 1

Сумма итог: 5 000,00

Рис. 3.38. «Заказ № 000000002»

Нажмем на ссылку Остатки товаров в командной панели формы документа и перейдем к движениям регистра ОстаткиТоваров, которые сформировал этот документ (рис. 3.39).

← → ☆ Заказ 000000002 от 10.12.2013 12:00:01

Главное **Остатки товаров** Причины отказа

Остатки товаров

Найти... Отменить поиск Еще

Период	Регистратор	N...	Товар	Цвет	Размер	Склад	Заказано	Количество
+ 10.12.2013 12:00:01	Заказ 000000002 от ...	1	Туфли	Красный	38	Основной	1	
+ 10.12.2013 12:00:01	Заказ 000000002 от ...	2	Салоги	Белый	38	Основной	1	
- 15.12.2013 0:00:00	Заказ 000000002 от ...	3	Салоги	Белый	38	Основной	1	

Рис. 3.39. Движения в регистре «Остатки товаров» документа «Заказ № 000000002»

Мы видим, что в наборе записей регистра Остатки товаров, связанном с документом, – три строки. Первые две из них отражают содержание табличной части документа, размещенной на странице Список товаров. Количество заказанных товаров учитывается в поле Заказано с видом движения накопления Приход. Третья строка с видом движения накопления Расход появилась при загрузке данных от планшета, когда заказ получил статус Выполнен. Это строка табличной части документа с информацией о товаре, от которого клиент отказался (страница Отказ от товаров).

Нажмем на ссылку Причины отказа в командной панели формы документа и перейдем к движениям регистра ПричиныОтказа, которые сформировал этот документ (рис. 3.40).

Период	Регистратор	Номер стр...	Товар	Причина отказа	Отказ
15.12.2013 0:00:00	Заказ 000000002 от 10.12.201...	1	Сапоги	Не подошел цвет/размер	1

Рис. 3.40. Движения в регистре «Причины отказа» документа «Заказ № 000000002»

Мы видим, что в наборе записей регистра Причины отказа, связанном с документом, – одна строка. Это строка табличной части документа с информацией о товаре, от которого клиент отказался (страница Отказ от товаров). Факт отказа от товара учитывается в поле Отказ.

Теперь откроем список расходных накладных, созданных на основе закрытых заказов. Их только три, несмотря на то что закрытых заказов – четыре. Так получилось потому, что в одном из заказов клиент не купил ни одного из заказанных товаров. Поэтому расходные накладные в данном случае не создавались, но данный заказ, тем не менее, произвел движения в регистре Причины отказа для их последующего анализа руководством интернет-магазина.

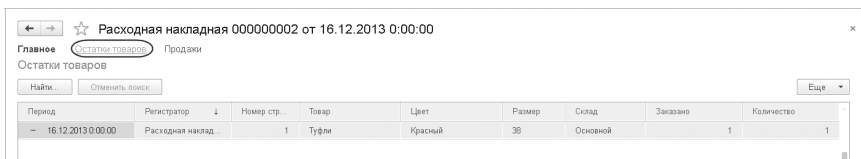
Откроем расходную накладную № 000000002, созданную на основании заказа № 000000002 (см. рис. 3.38). В табличной части документа содержится одна строка с информацией о товаре, который клиент купил (рис. 3.41).

N	Товар	Цена	Размер	Цена	Коллич...	Сумма
1	Туфли	Красный	38	5 000,00	1	5 000,00

Количество (итог): 1
Сумма (итог): 5 000,00

Рис. 3.41. «Расходная накладная № 000000002»

Нажмем на ссылку Остатки товаров в командной панели формы документа и перейдем к движениям регистра ОстаткиТоваров, которые сформировал этот документ (рис. 3.42).



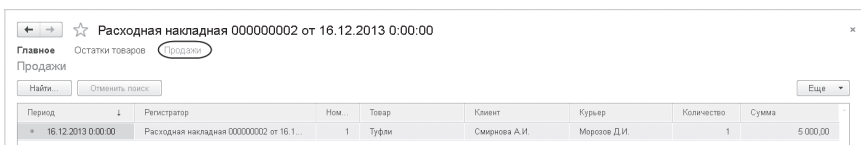
The screenshot shows a web application window titled 'Расходная накладная 000000002 от 16.12.2013 0:00:00'. Below the title bar is a navigation bar with 'Главное', 'Остатки товаров' (highlighted), and 'Продажи'. Below this is a search bar with 'Найти...' and 'Отменить поиск'. The main area contains a table with the following data:

Период	Регистратор	Номер стр...	Товар	Цвет	Размер	Склад	Заказано	Количество
16.12.2013 0:00:00	Расходная наклад...	1	Туфли	Красный	38	Основной	1	1

Рис. 3.42. Движения в регистре «Остатки товаров» документа «Расходная накладная № 000000002»

Мы видим, что в наборе записей регистра Остатки товаров, связанном с документом, также находится одна строка с информацией о проданном клиенту товаре. Количество проданных товаров учитывается в поле Количество с видом движения накопления Расход. Поле Заказано также имеет отрицательный прирост, равный количеству проданных товаров.

Нажмем на ссылку Продажи в командной панели формы документа и перейдем к движениям регистра Продажи, которые сформировал этот документ (рис. 3.43).



The screenshot shows the same web application window, but the 'Продажи' register is selected in the navigation bar. The table below shows the following data:

Период	Регистратор	Ном...	Товар	Клиент	Курьер	Количество	Сумма
16.12.2013 0:00:00	Расходная накладная 000000002 от 16.1...	1	Туфли	Смирнова А.И.	Морозов Д.И.	1	5 000,00

Рис. 3.43. Движения в регистре «Продажи» документа «Расходная накладная № 000000002»

Мы видим, что в наборе записей регистра Продажи, связанном с документом, также находится одна строка с информацией о проданном клиенту товаре. Количество и сумма проданных товаров учитываются в поле Количество и Сумма с видом движения накопления Расход.

Последние два заказа со статусом Выполнен (они показываются фиолетовым цветом в списке заказов) по каким-то причинам еще не закрыты в интернет-магазине, поэтому на их основании пока не созданы расходные накладные с данными о продажах. При этом товары, которые клиент купил, учитываются как заказанные в регистре Остатки товаров, а товары, от которых клиент отказался, учитываются в регистре Причины отказа и становятся свободными в регистре Остатки товаров.

Таким образом, мы убедились, что все движения в регистрах накопления формируются согласно заданному нами алгоритму, и, следовательно, вся информация для построения отчетов готова.

Формировать отчеты мы будем с помощью системы компоновки данных. Так как отчеты достаточно простые и поскольку система компоновки данных подробно рассматривалась в предыдущих книгах (например, в книге «Разработка сложных отчетов в «1С:Предприятии 8.2». Система компоновки данных»), то мы не будем детально объяснять каждый шаг построения отчетов.

Остатки товаров на складах

Итак, сначала мы разработаем отчет, который будет показывать приход, расход и остатки товаров на складах в разрезе их цветов и размеров. А также в отдельных колонках будет показываться количество заказанных товаров и товаров, которые находятся в свободном остатке, без учета заказанных.

Этот отчет потребуется как в самом интернет-магазине (чтобы видеть, какие товары проданы, какие – в наличии и т.д.), так и курьеру на планшете (чтобы в случае новых заказов иметь оперативную информацию из офиса о наличии товаров).

Поэтому этот отчет будет формироваться в интернет-магазине наряду с другими отчетами, на основе информации из регистров накопления. Но позднее (см. раздел «Получение отчета» на стр. 309) в мобильном приложении мы предоставим курьеру возможность получать данные этого отчета через веб-сервис и просматривать их на планшете.

Добавим новый объект конфигурации Отчет. Назовем его **ОстаткиТоваровНаСкладах**. Откроем конструктор схемы компоновки данных и добавим новый набор данных – запрос. Откроем конструктор запроса.

Запрос для набора данных

В качестве источника данных для запроса выберем виртуальную таблицу регистра накопления **ОстаткиТоваров.ОстаткиИОбороты**. В список полей перенесем поля из этой таблицы (рис. 3.44):

- Товар,
- Цвет,
- Размер,
- Склад,
- ЗаказаноКонечныйОстаток,

- КоличествоНачальныйОстаток,
- КоличествоКонечныйОстаток,
- КоличествоПриход,
- КоличествоРасход.

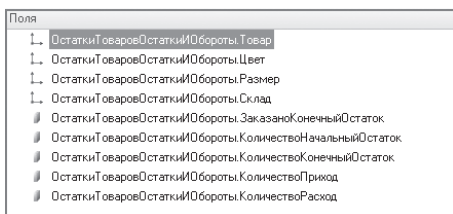


Рис. 3.44. Список полей отчета в конструкторе запроса

После этого на закладке Объединения/Псевдонимы зададим псевдонимы числовых полей без слова Количество (рис. 3.45).

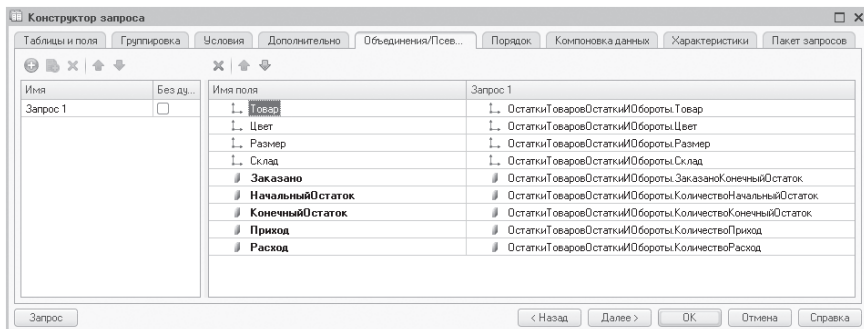


Рис. 3.45. «Объединения/Псевдонимы»

На этом создание запроса закончено. Нажмем ОК, вернемся в конструктор схемы компоновки данных и приступим к редактированию схемы компоновки данных.

Вычисляемые поля

Кроме количества поступивших, заказанных и проданных товаров, в отчете мы хотим видеть количество товаров в свободном остатке, то есть разницу между конечным остатком товаров (поле регистра КонечныйОстаток) и количеством товаров, остающихся в заказе (поле регистра Заказано). Значит, для

этого нам понадобится создать вычисляемое поле. Перейдем на закладку Вычисляемые поля и добавим поле с именем ВНаличи. В колонку Выражение для этого поля введем следующий текст (листинг 3.34, рис. 3.46).

Листинг 3.34. Выражение для расчета вычисляемого поля «ВНаличи»

КонечныйОстаток - Заказано

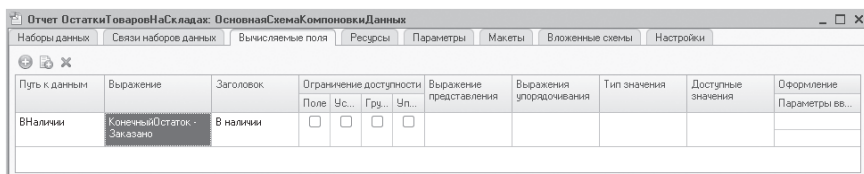


Рис. 3.46. Создание вычисляемого поля

Ресурсы

Затем на закладке Ресурсы выберем все доступные ресурсы (рис. 3.47).

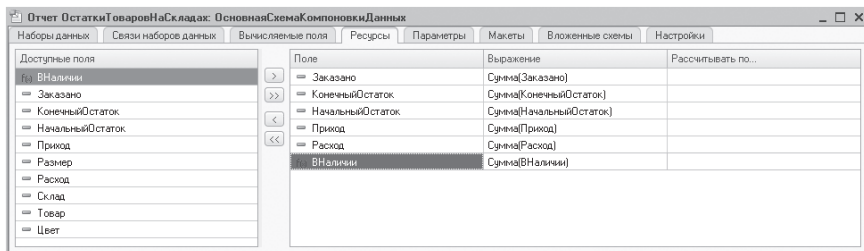


Рис. 3.47. Описание ресурсов

Обратите внимание, что созданное нами вычисляемое поле ВНаличи также присутствует в списке ресурсов. Значит, по нему можно рассчитывать итоги.

Настройки

Перейдем на закладку Настройки. Создадим структуру отчета – добавим в отчет группировку по полям Склад, Товар, Размер и последовательно вложим их друг в друга. В группировку Размер добавим группировку без указания группировочного поля – Детальные записи.

Затем на закладке Выбранные поля выберем те поля, которые будут выводиться в отчет: Цвет, Заказано, НачальныйОстаток, Приход, Расход, КонечныйОстаток и ВНаличи (рис. 3.48).

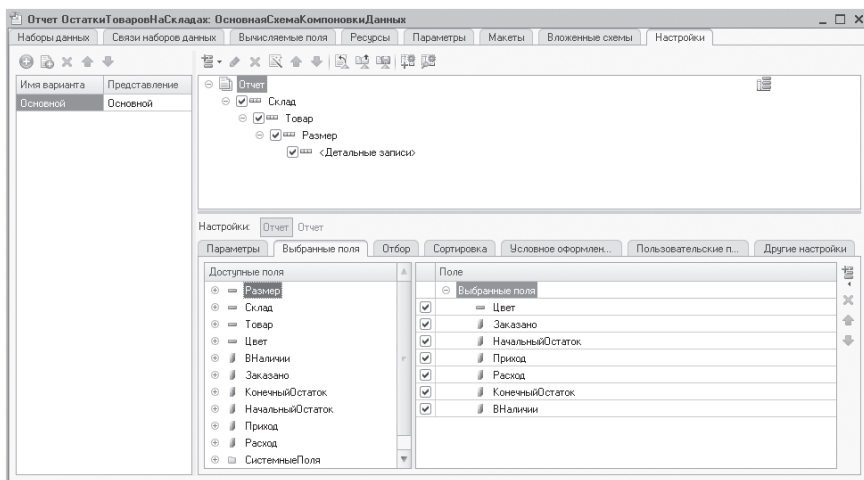


Рис. 3.48. Группировки и поля отчета

Названия полей группировок Склад, Товар, Размер также будут выводиться в отчет, но указывать их не обязательно.

В заключение предоставим пользователю возможность задавать отчетный период перед формированием отчета непосредственно в отчетной форме. Для этого на закладке Параметры настроек схемы компоновки данных выделим поочередно параметры Начало периода и Конец периода. Для каждого из них нажмем кнопку Свойства элемента пользовательских настроек и установим флажок Включать в пользовательские настройки (рис. 3.49).

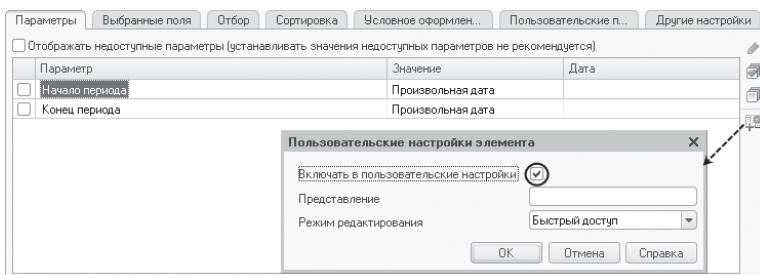


Рис. 3.49. Пользовательские настройки

В результате в режиме 1С:Предприятие сформированный отчет примет следующий вид (рис. 3.50).

Остатки товаров на складах							
Сформировать		Выбрать вариант...		Настройки...		Еще ▾	
Начало периода:				Конец периода:			
Склад	Заказано	Начальный остаток	Приход	Расход	Конечный остаток	В наличии	
Товар							
Размер							
Цвет							
Основной	3		49	7	42	39	
Джинсы			15	2	13	13	
44			10	2	8	8	
Белый			5		5	5	
Черный			5	2	3	3	
46			5		5	5	
Черный			5		5	5	
Платье	1		8	1	7	6	
44	1		2		2	1	
Красный	1		2		2	1	
46			6	1	5	5	
Белый			3		3	3	
Фиолетовый			3	1	2	2	
Сапоги			11	1	10	10	
37			5	1	4	4	
Черный			5	1	4	4	
38			6		6	6	
Белый			3		3	3	
Черный							
Фиолетовый			3		3	3	
Туфли	2		15	3	12	10	
37			5		5	5	
Черный			5		5	5	
38	2		10	3	7	5	
Черный			5	2	3	3	
Красный	2		5	1	4	2	
Итого	3		49	7	42	39	

Рис. 3.50. Результат отчета

Таким образом, с помощью этого отчета можно проследить заказ, приход и расход товаров по складам в разрезе цветов и размеров товаров.

Причины отказа от товаров

Для анализа причин отказа от товаров руководству интернет-магазина понадобится отчет, показывающий, от каких товаров и по каким причинам наиболее часто отказывались клиенты. Нагляднее всего такой отчет будет выглядеть в виде диаграммы.

Добавим новый объект конфигурации Отчет. Назовем его ПричиныОтказаОтТоваров. Откроем конструктор схемы компоновки данных и добавим новый набор данных – запрос. Откроем конструктор запроса.

Запрос для набора данных

В качестве источника данных для запроса выберем виртуальную таблицу регистра накопления ПричиныОтказа.Обороты. В список полей перенесем поля из этой таблицы (рис. 3.51):

- Товар,
- ПричинаОтказа,
- ОтказОборот.

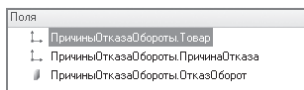


Рис. 3.51. Список полей отчета в конструкторе запроса

После этого на закладке Объединения/Псевдонимы зададим псевдоним поля Отказ без слова Оборот (рис. 3.52).

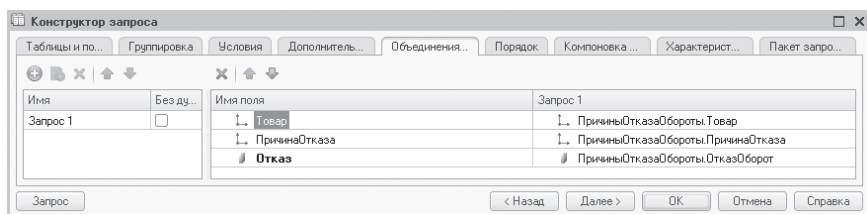


Рис. 3.52. «Объединения/Псевдонимы»

Нажмем ОК, вернемся в конструктор схемы компоновки данных и приступим к редактированию схемы компоновки данных.

Ресурсы

Прежде всего на закладке Ресурсы выберем все доступные ресурсы (рис. 3.53).

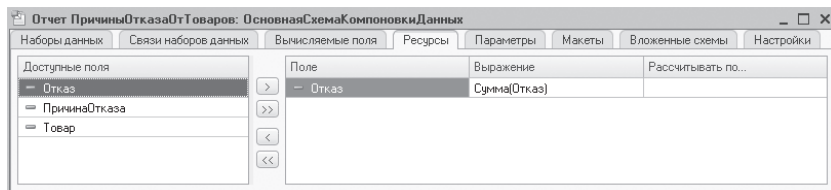


Рис. 3.53. Описание ресурсов

Настройки

Перейдем на закладку Настройки. Добавим в структуру отчета диаграмму. В точки диаграммы добавим группировку по полю ПричинаОтказа, а серии диаграммы оставим пустыми. Затем на закладке Выбранные поля выберем ресурс Отказ, который будет отображаться в диаграмме (рис. 3.54).

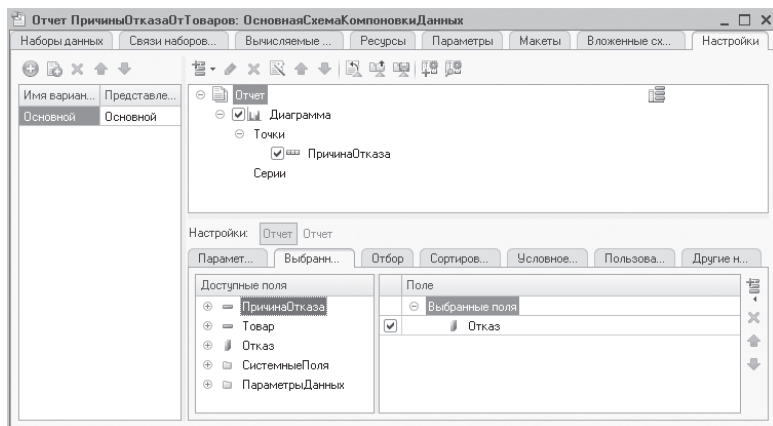


Рис. 3.54. Группировки и поля отчета

На закладке Другие настройки настроек схемы компоновки данных установим тип диаграммы – Круговая объемная (рис. 3.55).

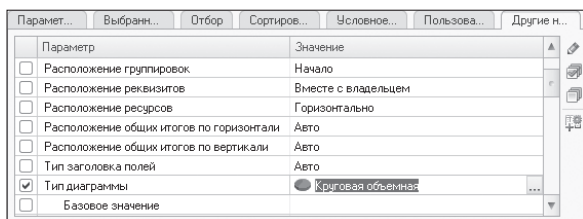


Рис. 3.55. Настройка типа диаграммы

В результате в режиме 1С:Предприятие сформированный отчет примет вид (рис. 3.56).

Таким образом, с помощью этого отчета руководитель может наглядно увидеть долю каждой причины отказа в общем количестве отказов от товаров.

Немного усовершенствуем этот отчет – сделаем его более универсальным.

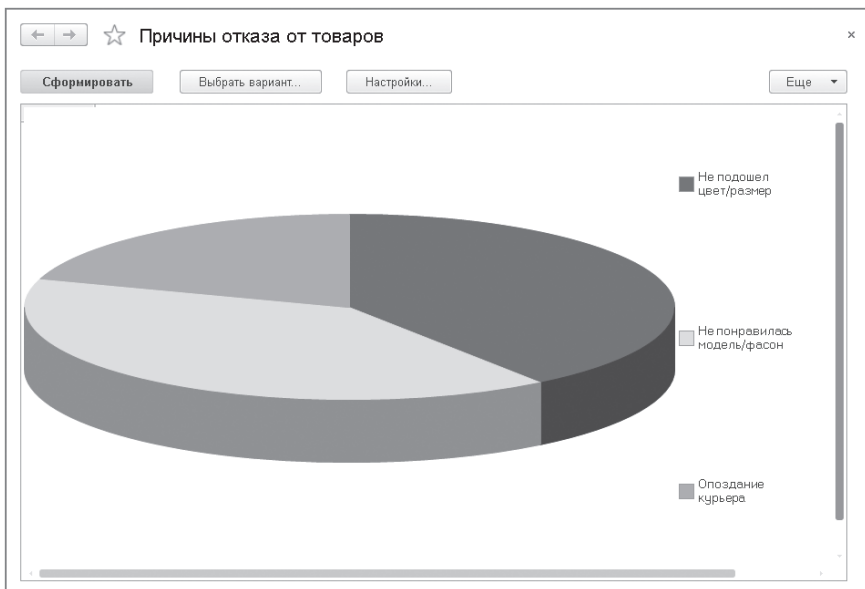


Рис. 3.56. Результат отчета

Пользовательские настройки

В конструкторе схемы компоновки данных на закладке **Настройки** удалим из точек диаграммы группировку по полю **ПричинаОтказа**. Мы не будем задавать точки и серии диаграммы, так как хотим предоставить полную свободу пользователю в этих действиях.

Для этого выделим в структуре элементов отчета элемент **Диаграмма** и нажмем кнопку **Свойства элемента** пользовательских настроек, расположенную сверху в командной панели окна настроек. Установим признак использования для настроек **Группировки точек** и **Группировки серий** (рис. 3.57).

На закладке **Другие настройки** включим настройку **Тип диаграммы** в состав быстрых пользовательских настроек (рис. 3.58).

Запустим «1С:Предприятие» и вызовем отчет **Причины отказа от товаров**. Нажмем кнопку выбора в поле **Серии** и добавим в серии диаграммы группировку по полю **Товар**. Нажмем **Сформировать**. Отчет примет следующий вид (рис. 3.59).

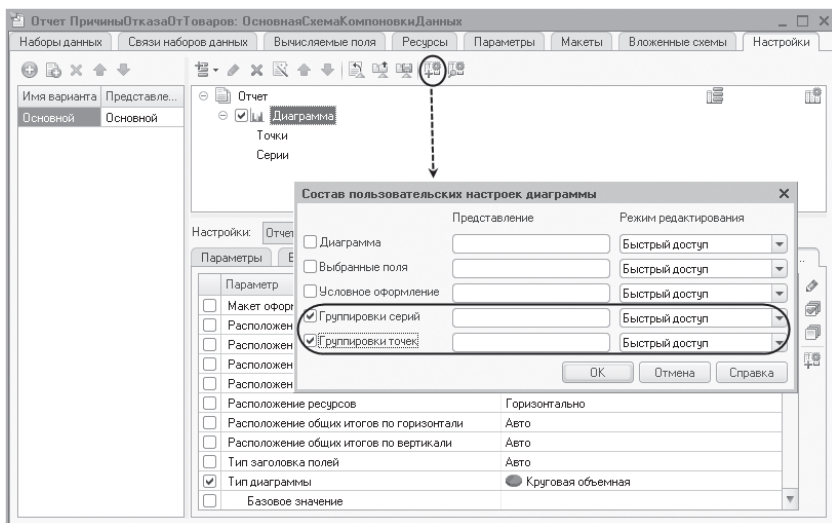


Рис. 3.57. Состав пользовательских настроек диаграммы

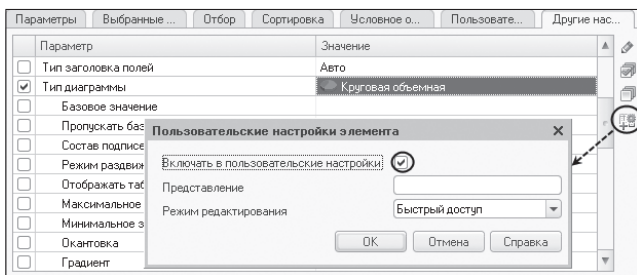


Рис. 3.58. Пользовательские настройки

Теперь нажмем кнопку выбора в поле Точки и добавим в точки диаграммы группировку по полю ПричинаОтказа. В поле Тип диаграммы выберем Гистограмма объемная. Нажмем Сформировать. Отчет примет следующий вид (рис. 3.60).

Таким образом, меняя настройки диаграммы, пользователь может увидеть и проанализировать данные об отказе от товаров в разных ракурсах.

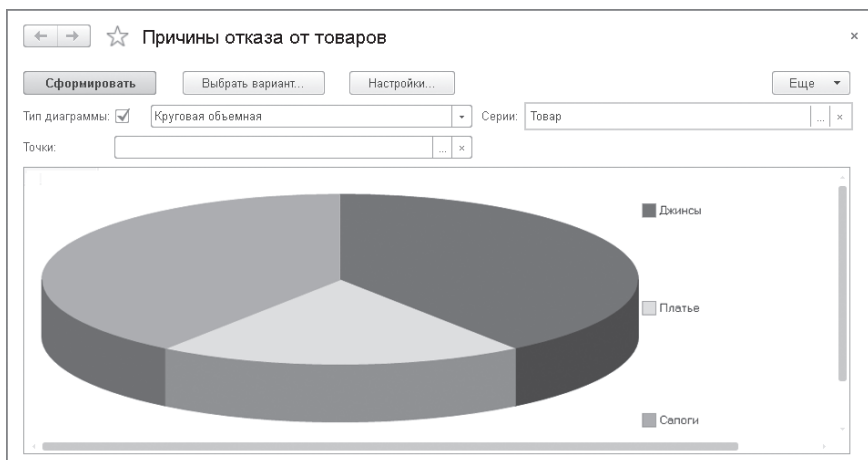


Рис. 3.59. Результат отчета

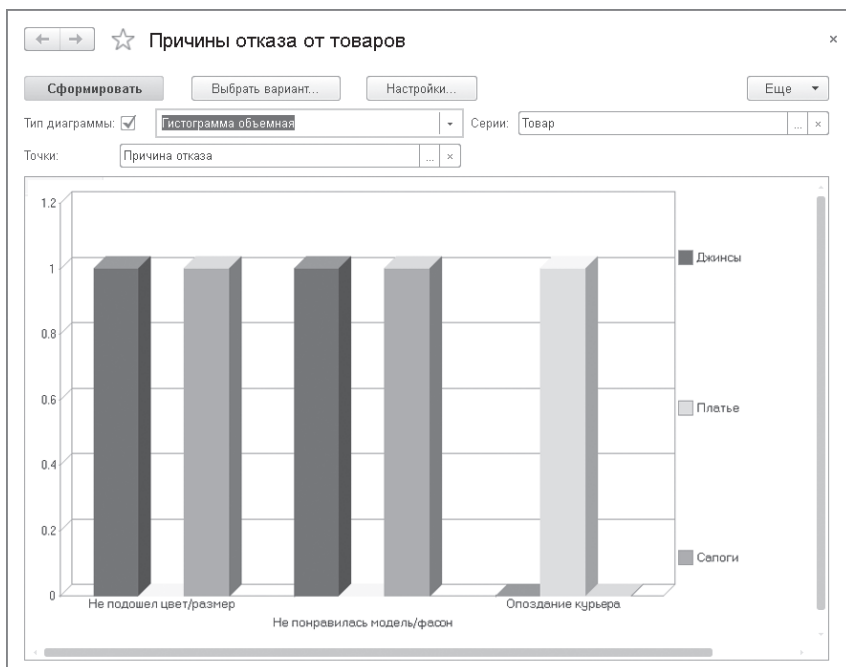


Рис. 3.60. Результат отчета

Продажи клиентам

Для анализа объемов продаж товаров клиентам в интернет-магазине понадобится отчет, показывающий, сколько, на какую сумму и каких товаров куплено каждым клиентом. Такую информацию удобнее всего представить в виде таблицы.

Добавим новый объект конфигурации Отчет. Назовем его ПродажиКлиентам. Откроем конструктор схемы компоновки данных и добавим новый набор данных – запрос. Откроем конструктор запроса.

Запрос для набора данных

В качестве источника данных для запроса выберем виртуальную таблицу регистра накопления Продажи.Обороты. В список полей перенесем все поля из этой таблицы (рис. 3.61).

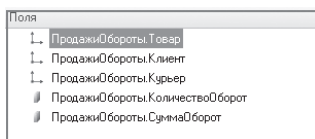


Рис. 3.61. Список полей отчета в конструкторе запроса

После этого на закладке Объединения/Псевдонимы зададим псевдонимы числовых полей без слова Оборот (рис. 3.62).

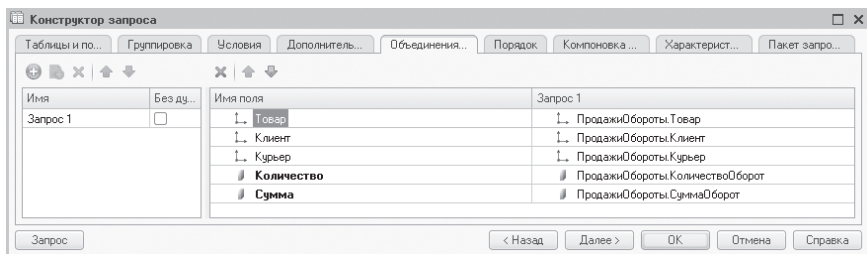


Рис. 3.62. «Объединения/Псевдонимы»

Нажмем ОК, вернемся в конструктор схемы компоновки данных и приступим к редактированию схемы компоновки данных.

Ресурсы

На закладке Ресурсы выберем все доступные ресурсы (рис. 3.63).

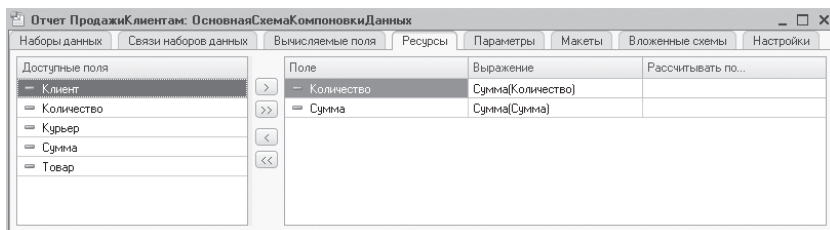


Рис. 3.63. Описание ресурсов

Настройки

Перейдем на закладку Настройки. Добавим в структуру отчета таблицу. В строки таблицы добавим группировку по полю Товар, а в колонки – группировку по полю Клиент. Затем на закладке Выбранные поля выберем ресурсы Количество и Сумма, которые будут отображаться в ячейках таблицы (рис. 3.64).

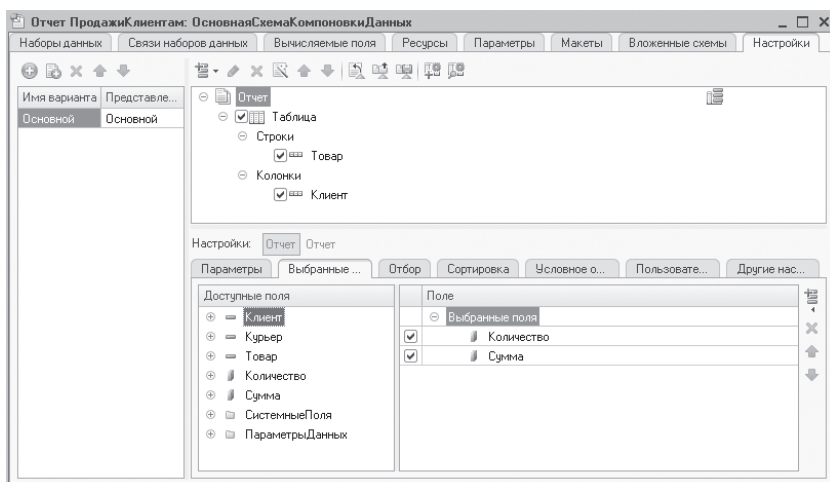
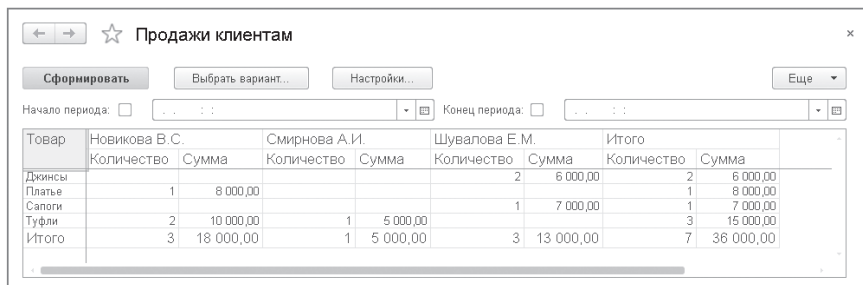


Рис. 3.64. Группировки и поля отчета

В заключение предоставим пользователю возможность задавать отчетный период перед формированием отчета непосредственно в отчетной форме.

В результате в режиме 1С:Предприятие сформированный отчет примет следующий вид (рис. 3.66).



Теперь хотелось бы представить эти же данные в виде измерительной диаграммы, в которой наглядно отражалась бы доля покупок каждого клиента в общем объеме продаж.

Для этого в конструкторе схемы компоновки данных на закладке Настройки создадим новый вариант отчета с именем **Диаграмма** и представлением **Диаграмма покупок**.

Добавим в структуру отчета диаграмму. В точки диаграммы добавим группировку по полю Клиент, а серии диаграммы оставим пустыми. Затем

на закладке Выбранные поля выберем ресурс Сумма, который будет отображаться в диаграмме (рис. 3.67).

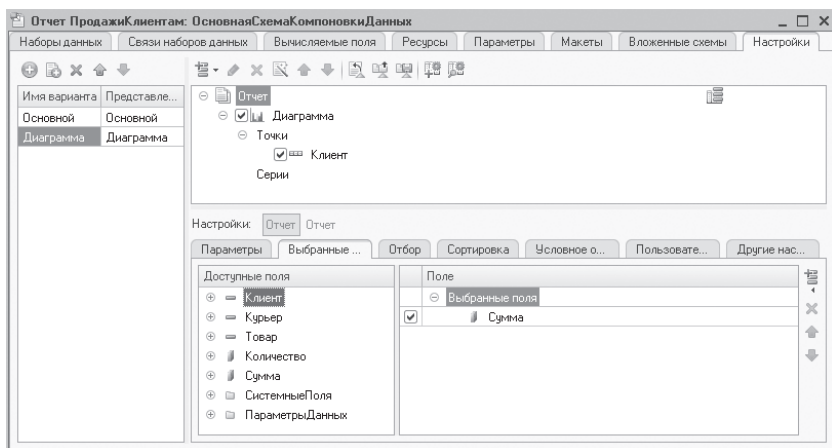


Рис. 3.67. Группировки и поля отчета

На закладке Другие настройки настроек схемы компоновки данных установим тип диаграммы – Измерительная, а также зададим значения полос диаграммы (рис. 3.68).

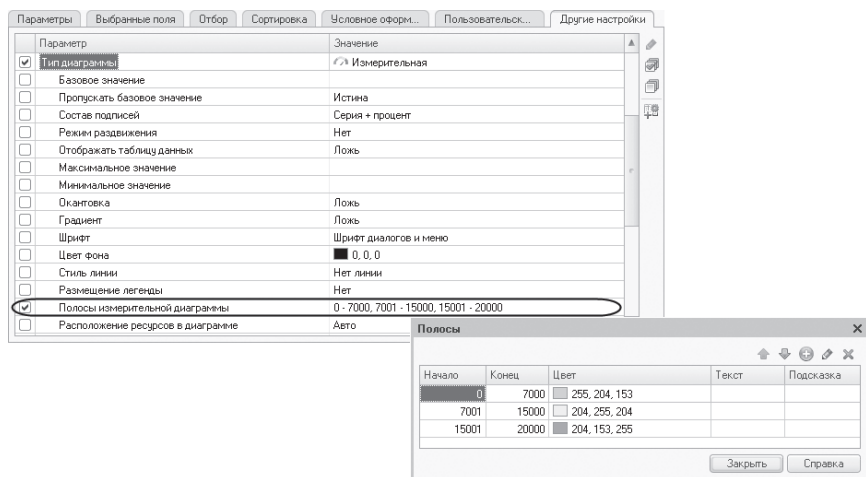


Рис. 3.68. Настройка типа диаграммы

На закладке **Параметры** включим параметры для установки отчетного периода в состав быстрых пользовательских настроек (рис. 3.69).

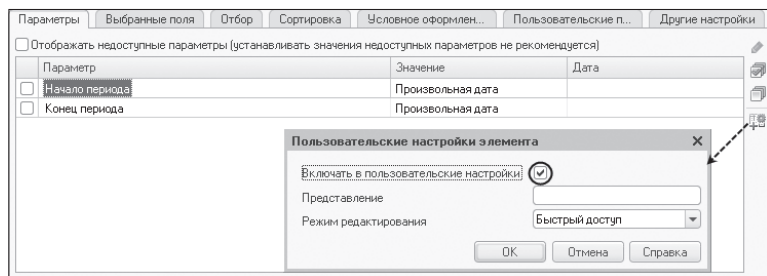


Рис. 3.69. Пользовательские настройки

Мы уже делали это для основного варианта отчета, но следует иметь в виду, что состав пользовательских настроек у каждого варианта отчета свой.

В режиме 1С:Предприятие запустим отчет **Продажи клиентам**. Нажмем кнопку **Выбрать вариант**, выберем вариант **Диаграмма покупок** и сформируем отчет (рис. 3.70).

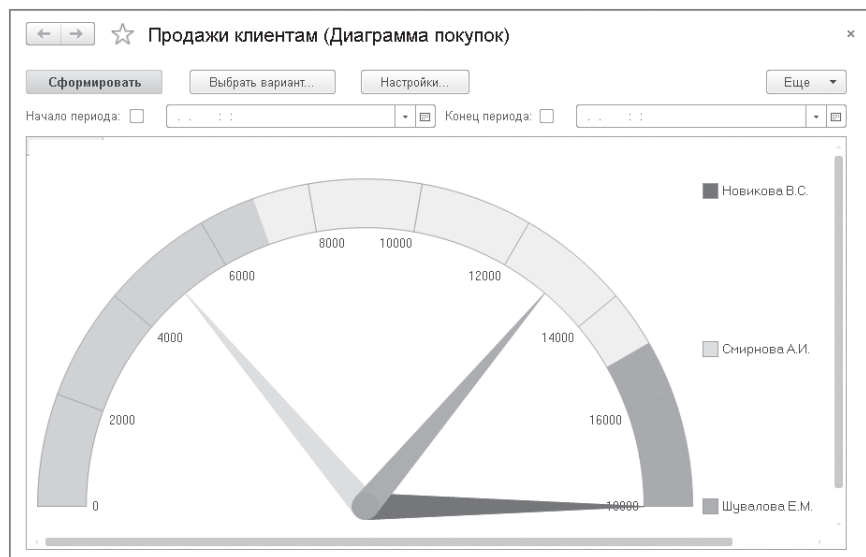


Рис. 3.70. Результат отчета

Таким образом, руководство интернет-магазина может представить данные о продажах товаров клиентам в виде таблицы или диаграммы в зависимости от надобности.

В этой же диаграмме можно сгруппировать данные по курьерам и увидеть долю каждого курьера в общем объеме продаж. Причем можно предоставить пользователю возможность самому выбирать поля группировок диаграммы перед формированием отчета, как мы показали выше.

А также можно создать еще несколько полезных отчетов, основываясь на движениях регистров накопления, которые производят документы при проведении.

Но мы на этом закончим разработку основного приложения и в следующей главе приступим к реализации обмена данными с мобильным приложением через веб-сервис.

Глава 4.

Обмен данными

В этой главе мы расскажем, как реализовать обмен данными между основным и мобильным приложением. Здесь мы займемся обменом данными более углубленно: рассмотрим выгрузку данных в разные узлы обмена, покажем, как разрешается коллизия при обмене данными, а также рассмотрим обмен данными различной структуры.

Основным способом обмена будет обмен данными через веб-сервис. Этот вариант обмена предполагается использовать всегда, когда возможно интернет-соединение. Кроме того, мы доработаем первоначальный обмен данными так, чтобы его можно было использовать как аварийный способ обмена в том случае, если отсутствует доступ в Интернет. Этот способ мы рассмотрим вначале, так как проще на нем все отладить и затем перенести в основной обмен данными через веб-сервис.

Запасной способ обмена данными с основным приложением

Запасным способом обмена данными с интернет-магазином будет обмен через USB-кабель, при котором данные обмена выгружаются и загружаются из файла обмена. Мы уже пользовались этим способом при первоначальном обмене данными от основного приложения к мобильному, для того чтобы перенести тестовые данные на планшет (см. раздел «Перенос данных на планшет» на стр. 53).

Теперь доработаем этот обмен так, чтобы он стал запасным способом обмена с основным приложением в случае отсутствия интернет-соединения.

Напомним, что в состав плана обмена Мобильные в нашем мобильном приложении входят следующие объекты конфигурации:

- Справочники:
 - ☐ Товары,
 - ☐ Клиенты,
 - ☐ Склады,
 - ☐ Цвета,
 - ☐ Размеры,
 - ☐ ПричиныОтказа,
 - ☐ Пользователи,
 - ☐ ХранимыеФайлы.
- Документы:
 - ☐ Заказ,
 - ☐ ОбслуживаниеЗаказов.
- Регистры сведений:
 - ☐ ЦеныТоваров.

Но по условию нашей задачи курьер на планшете может изменять лишь некоторые объекты, а именно:

- Справочники:
 - ☐ Клиенты,
 - ☐ ПричиныОтказа,
 - ☐ ХранимыеФайлы.
- Документы:
 - ☐ Заказ.

Обмен новыми данными

Прежде всего мы должны сделать так, чтобы при создании новых объектов, которые могут добавляться и в мобильном, и в основном приложении, номер документа или код справочника начинался бы с префикса – кода узла обмена.

Этот префикс необходим для обеспечения уникальности номеров и кодов в каждой из наших баз. То есть, например, заказы, созданные на планшете,

будут иметь номер «МПП-00001», «МПП-00002», «МПП-00003» и т.д., в то время как заказы, созданные в интернет-магазине, будут иметь номера «ЦО-000001», «ЦО-000002», «ЦО-000003» и т.д.

Функцию формирования префикса номера мы вынесем в общий модуль, поскольку не исключена возможность того, что в будущем алгоритм формирования префикса может быть изменен.

Мобильное приложение

В мобильном приложении КурьерИнтернетМагазина в дереве объектов конфигурации раскроем ветвь Общие и добавим общий модуль Обмен. В палитре его свойств установим флажок Вызов сервера, чтобы можно было использовать процедуры и функции этого модуля на клиенте, т.е. на мобильном устройстве (рис. 4.1).

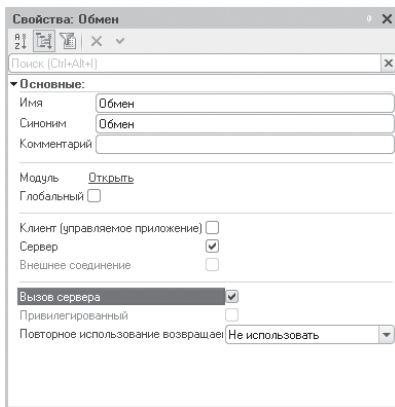


Рис. 4.1. Свойства общего модуля «Обмен»

В модуль поместим следующую функцию (листинг 4.1).

Листинг 4.1. Функция формирования префикса номера

```
Функция ПолучитьПрефиксНомера() Экспорт
```

```
    Возврат ПланыОбмена.Мобильные.ЭтотУзел().Код + "-";
```

```
КонецФункции
```

Эта функция возвращает строковый код predetermined узла информационной базы мобильного приложения.

Теперь откроем модуль объекта справочника Клиенты и добавим в него обработчик события ПриУстановкеНовогоКода (листинг 4.2).

Листинг 4.2. Обработчик события «ПриУстановкеНовогоКода»

```
Процедура ПриУстановкеНовогоКода(СтандартнаяОбработка, Префикс)
```

```
    Префикс = Обмен.ПолучитьПрефиксНомера();
```

```
КонецПроцедуры
```

Событие ПриУстановкеНовогоКода возникает в момент, когда выполняется установка нового кода элемента справочника. Обратите внимание, что мы пишем этот код не в модуле формы, а в модуле объекта, поскольку это событие возникает не для формы, а для объекта в целом.

Вторым параметром вызова обработчика передается префикс, который будет заполнен в данной процедуре и использован системой для генерации кода.

В обработчике события мы вызываем функцию общего модуля. Поскольку модуль неглобальный, то обращаемся к ней по имени модуля и имени функции (Обмен.ПолучитьПрефиксНомера).

Такие же обработчики нужно добавить в модули справочников ПричиныОтказа и ХранимыеФайлы.

В модуль документа Заказ также добавим обработчик события ПриУстановкеНовогоНомера (листинг 4.3).

Листинг 4.3. Обработчик события «ПриУстановкеНовогоНомера»

```
Процедура ПриУстановкеНовогоНомера(СтандартнаяОбработка, Префикс)
```

```
    Префикс = Обмен.ПолучитьПрефиксНомера();
```

```
КонецПроцедуры
```

Основное приложение

Аналогичные изменения необходимо выполнить и в основном приложении (конфигурация ИнтернетМагазин): создать процедуру ПолучитьПрефиксНомера() общего модуля Обмен и добавить процедуры установки префикса кодов/номеров в модули объектов, которые могут добавляться как в основном, так и в мобильном приложении. Это документ Заказ и справочники Клиенты, ПричиныОтказа, ХранимыеФайлы.

Обмен данными с разной структурой

На практике у одного и того же справочника или документа в основном и в мобильном приложении обычно бывает разный состав реквизитов. Так как, исходя из логики работы, не все реквизиты, которые есть, например, у справочника в основном приложении, нужны ему в мобильном приложении. Поэтому мы будем передавать на планшет курьеру не все данные, которые есть в офисе, а только ограниченный набор реквизитов некоторых справочников.

Для примера реализуем ситуацию с различной структурой данных одного иерархического и одного простого справочника. Структура остальных справочников и документов совпадает, поэтому для них мы будем использовать стандартные процедуры обмена.

Например, коды элементов справочников Товары и Склады совершенно не нужны на планшете. Поэтому при дальнейшей доработке мобильного приложения мы установим у этих справочников длину стандартного реквизита Код равной нулю. А также удалим реквизит Артикул из списка реквизитов справочника Товары. А в основном приложении мы, наоборот, добавим реквизит Расположение у справочника Склады. Информация о расположении складов также не нужна на планшете.

Таким образом, у справочника Склады в обоих приложениях «общими» являются только стандартные реквизиты Ссылка и Наименование, а у справочника Товары – стандартные реквизиты Ссылка, Родитель, ЭтоГруппа, Наименование и реквизиты Описание и ФайлКартинки. Поэтому для этих справочников мы создадим и будем использовать собственные процедуры обмена данными.

Основное приложение

Итак, в основном приложении для справочника Склады добавим реквизит Расположение, в котором будет храниться информация о расположении складов.

Затем откроем модуль узла плана обмена Мобильные и внесем следующие изменения в процедуру ЗаписатьСообщениеСИзменениями(), см. листинг 4.4.

Листинг 4.4. Процедура для записи данных обмена

Процедура ЗаписатьСообщениеСИзменениями(Каталог) Экспорт

Сообщение = Новый СообщенияПользователю;

Сообщение.Текст = "----- Выгрузка в узел " + Строка(ЭтотОбъект) + " -----";

Сообщение.Сообщить();

```
// Сформировать имя временного файла.
ИмяФайла = Каталог + ?(Прав(Каталог, 1) = "\", "", "\") + "Message" +
    СокрЛП(ПланыОбмена.Мобильные.ЭтотУзел().Код) + "_" + СокрЛП(Ссылка.Код) + ".xml";

// Создать объект записи XML
// *** ЗаписьXML-документов.
ЗаписьXML = Новый ЗаписьXML;
ЗаписьXML.ОткрытьФайл(ИмяФайла);
ЗаписьXML.ЗаписатьОбъявлениеXML();

// *** Инфраструктура сообщений.
ЗаписьСообщения = ПланыОбмена.СоздатьЗаписьСообщения();
ЗаписьСообщения.НачатьЗапись(ЗаписьXML, Ссылка);
Сообщение = Новый СообщениеПользователю;
Сообщение.Текст = " Номер сообщения: " + ЗаписьСообщения.НомерСообщения;
Сообщение.Сообщить();

// Получить выборку измененных данных
// *** Механизм регистрации изменений.
ВыборкаИзменений = ПланыОбмена.ВыбратьИзменения(
    ЗаписьСообщения.Получатель, ЗаписьСообщения.НомерСообщения);
Пока ВыборкаИзменений.Следующий() Цикл
    Данные = ВыборкаИзменений.Получить();

    // Записать данные в сообщение *** XML-сериализация.
    ЗаписатьДанные(ЗаписьXML, Данные);
КонецЦикла;

ЗаписьСообщения.ЗакончитьЗапись();
ЗаписьXML.Закрыть();

Сообщение = Новый СообщениеПользователю;
Сообщение.Текст = "----- Конец выгрузки -----";
Сообщение.Сообщить();
```

КонецПроцедуры

Измененный фрагмент выделен жирным шрифтом, но мы специально привели здесь текст процедуры полностью, чтобы заодно прокомментировать его, так как не делали этого раньше.

Итак, сначала мы формируем имя файла, содержащего данные обмена, с указанием каталога его расположения. Этот каталог выбирается пользователем на стационарном компьютере и передается в процедуру обмена в параметре Каталог из обработки ЗапаснойОбменДанными. Имена файлов обмена стандартизованы и имеют вид MessageКодУзлаОтправителя_КодУзлаПолучателя.xml.

После этого мы обращаемся к механизмам записи/чтения XML-документов и создаем новый объект – ЗаписьXML. С помощью него открываем новый XML-файл для записи и записываем в него объявление XML.

Затем мы обращаемся к механизмам инфраструктуры сообщений и создаем новый объект `ЗаписьСообщенияОбмена`, метод которого `НачатьЗапись()` позволяет, кроме всего прочего, создать очередной номер сообщения и записать заголовок сообщения в XML.

В качестве ссылки на текущий узел обмена мы используем значение стандартного реквизита `Ссылка плана обмена Мобильные`, в модуле которого мы находимся.

После этого, чтобы получить данные, которые необходимо сохранить в этом файле, мы обращаемся к механизму регистрации изменений и получаем выборку из записей регистрации изменений, предназначенных данному узлу. При формировании выборки методом менеджера планов обмена `ВыбратьИзменения()` мы передаем вторым параметром номер сообщения.

Затем мы перебираем выборку записей в цикле. Выбранные данные сериализуются в открытый XML-файл с помощью процедуры `ЗаписатьДанные()`. В параметре `Данные` в функцию передается полученный объект обмена (листинг 4.5).

Листинг 4.5. Процедура «ЗаписатьДанные»

```
Процедура ЗаписатьДанные(ЗаписьXML, Данные) Экспорт
    Если ТипЗнч(Данные) = Тип("СправочникОбъект.Товары") Тогда
        // Записываем элемент справочника вручную.
        ЗаписатьXMLТовары(ЗаписьXML, Данные);
    ИначеЕсли ТипЗнч(Данные) = Тип("СправочникОбъект.Склады") Тогда
        // Записываем элемент справочника вручную.
        ЗаписатьXMLСклады(ЗаписьXML, Данные);
    Иначе
        // Записываем данные с помощью стандартного метода.
        ЗаписатьXML(ЗаписьXML, Данные);
    КонецЕсли
КонецПроцедуры
```

В процедуре `ЗаписатьДанные()` проверяется: если записываемый объект является элементом справочника `Товары` или `Склады`, то вызываются процедуры `ЗаписатьXMLТовары()` (см. листинг 4.6) или `ЗаписатьXMLСклады()` (см. листинг 4.7) для записи элемента справочника вручную, в противном случае запись производится с помощью стандартного метода `ЗаписатьXML()`.

Листинг 4.6. Процедура «ЗаписатьXMLТовары»

Процедура ЗаписатьXMLТовары(ЗаписьXML, Товар)

```
// Записываем начало элемента XML
ЗаписьXML.ЗаписатьНачалоЭлемента("CatalogObject.Товары.Вручную");

// Ссылка
ЗаписатьXML(ЗаписьXML, Товар.Ссылка, "Ref", НазначениеТипаXML.Явное);
// ЭтоГруппа
ЗаписатьXML(ЗаписьXML, Товар.ЭтоГруппа, "IsFolder", НазначениеТипаXML.Явное);
// Родитель
ЗаписатьXML(ЗаписьXML, Товар.Родитель, "Parent", НазначениеТипаXML.Явное);
// Наименование
ЗаписатьXML(ЗаписьXML, Товар.Наименование, "Description", НазначениеТипаXML.Явное);

// Реквизиты, выгружаемые только для элемента справочника
Если Не Товар.ЭтоГруппа Тогда
    // Описание
    ЗаписатьXML(ЗаписьXML, Товар.Описание, "Описание", НазначениеТипаXML.Явное);
    // ФайлКартинки
    ЗаписатьXML(ЗаписьXML, Товар.ФайлКартинки, "ФайлКартинки", НазначениеТипаXML.Явное);
КонецЕсли;

// Записываем конец элемента
ЗаписьXML.ЗаписатьКонецЭлемента();
```

КонецПроцедуры

Листинг 4.7. Процедура «ЗаписатьXMLСклады»

Процедура ЗаписатьXMLСклады(ЗаписьXML, Склад)

```
// Записываем начало элемента XML
ЗаписьXML.ЗаписатьНачалоЭлемента("CatalogObject.Склады.Вручную");

// Ссылка
ЗаписатьXML(ЗаписьXML, Склад.Ссылка, "Ref", НазначениеТипаXML.Явное);
// Наименование
ЗаписатьXML(ЗаписьXML, Склад.Наименование, "Description", НазначениеТипаXML.Явное);

// Записываем конец элемента
ЗаписьXML.ЗаписатьКонецЭлемента();
```

КонецПроцедуры

В процедурах ЗаписатьXMLТовары()/ЗаписатьXMLСклады() производится «ручное» формирование элемента CatalogObject.Товары.Вручную/CatalogObject.Склады.Вручную.

Сначала в открытый XML-файл мы записываем начало элемента с заголовком, указывающим на ручное формирование элемента. Затем при

помощи метода `ЗаписатьXML()` записываем значения тех реквизитов, которые хотим передать в мобильное приложение. Третьим параметром передаем строковое имя записываемого реквизита и в качестве четвертого параметра указываем значение перечисления `НазначениеТипаXML`, определяющее необходимость явного указания типа данных XML. После этого мы завершаем запись элемента и записываем в XML-файл конец элемента.

После выхода из цикла в процедуре `ЗаписатьСообщениеСИзмениями()` (см. листинг 4.4) мы заканчиваем запись методом `ЗакончитьЗапись()` объекта `ЗаписьСообщенияОбмена`. Затем закрываем файл с сообщением обмена методом `Закреть()` объекта `ЗаписьXML` и выводим сообщение об окончании выгрузки данных.

Теперь изменим процедуру `ПрочитатьСообщениеСИзмениями()` в модуле узла плана обмена `Мобильные` так, чтобы для элементов справочников `Товары` и `Склады` читать только те реквизиты, которые есть у этих справочников и в основном, и в мобильном приложении (листинг 4.8).

Листинг 4.8. Процедура для чтения данных обмена

Процедура `ПрочитатьСообщениеСИзмениями(Каталог) Экспорт`

```
// Сформировать имя файла.  
ИмяФайла = Каталог + "?"(Прав(Каталог, 1) = "\", "" , "\" ) + "Message" + СокрЛП(Ссылка.Код) + "-" +  
СокрЛП(ПланыОбмена.Мобильные.ЭтотУзел().Код) + ".xml";  
  
Файл = Новый Файл(ИмяФайла);  
Если Не Файл.Существует() Тогда  
    Возврат;  
КонецЕсли;  
  
// *** Чтение документов XML  
// Попытаться открыть файл.  
ЧтениеXML = Новый ЧтениеXML;  
Попытка  
    ЧтениеXML.ОткрытьФайл(ИмяФайла);  
  
Исключение  
    Сообщение = Новый СообщениеПользователю;  
    Сообщение.Текст = "Невозможно открыть файл обмена данными.";  
    Сообщение.Сообщить();  
    Возврат;  
КонецПопытки;  
  
Сообщение = Новый СообщениеПользователю;  
Сообщение.Текст = "----- Загрузка из " + Строка(ЭтотОбъект) + " -----";  
Сообщение.Сообщить();  
Сообщение = Новый СообщениеПользователю;  
Сообщение.Текст = " - Считывается файл " + ИмяФайла;  
Сообщение.Сообщить();  
  
// Загрузить из найденного файла
```

```

// *** Инфраструктура сообщений.
ЧтениеСообщения = ПланыОбмена.СоздатьЧтениеСообщения();

// Читать заголовок сообщения обмена данными – файла XML.
ЧтениеСообщения.НачатьЧтение(ЧтениеXML);

// Сообщение предназначено не для этого узла.
Если ЧтениеСообщения.Отправитель <> Ссылка Тогда
    ВызватьИсключение "Неверный узел";
КонецЕсли;

// Удаляем регистрацию изменений для узла отправителя сообщения.
// *** Служба регистрации изменений.
ПланыОбмена.УдалитьРегистрациюИзменений(
    ЧтениеСообщения.Отправитель, ЧтениеСообщения.НомерПринятого);

// Читаем данные из сообщения *** XML-сериализация.
Пока ВозможностьЧтенияДанных(ЧтениеXML) Цикл

    // Читаем очередное значение.
    Данные = ПрочитатьДанные(ЧтениеXML);

    // Записать полученные данные.
    Данные.ОбменДанными.Отправитель = ЧтениеСообщения.Отправитель;
    Данные.ОбменДанными.Загрузка = Истина;
    Данные.Записать();

КонецЦикла;

ЧтениеСообщения.ЗакончитьЧтение();
ЧтениеXML.Закрыть();
УдалитьФайлы(ИмяФайла);

Сообщение = Новый СообщениеПользователю;
Сообщение.Текст = "----- Конец загрузки -----";
Сообщение.Сообщить();

```

КонецПроцедуры

Измененный фрагмент выделен жирным шрифтом, но мы специально привели здесь текст процедуры полностью, чтобы заодно прокомментировать его, так как не делали этого раньше.

Итак, сначала мы формируем имя файла, содержащего данные обмена, с указанием каталога его расположения. Этот каталог выбирается пользователем на стационарном компьютере и передается в процедуру обмена в параметре Каталог из обработки ЗапаснойОбменДанными. Имена файлов обмена стандартизованы и имеют вид `MessageКодУзлаОтправителя_КодУзлаПолучателя.xml`.

На основе имени файла создаем объект Файл и проверяем, существует ли он. Если файл обмена найден, то мы пытаемся открыть его для чтения с помощью объекта ЧтениеXML. В случае успеха выводим сообщение о начале загрузки данных из файла.

Затем мы обращаемся к механизмам инфраструктуры сообщений планов обмена и создаем объект `ЧтениеСообщенияОбмена`. Используя метод этого объекта `НачатьЧтение()`, мы считываем заголовок XML-сообщения, в котором содержится, в том числе, информация об отправителе сообщения.

После этого проверяем, является ли отправитель сообщения тем узлом плана обмена, для которого мы в данном вызове этой процедуры производим обмен данными. В качестве ссылки на текущий узел обмена мы используем значение стандартного реквизита `Ссылка плана обмена Мобильные`, в модуле которого мы находимся.

Если все в порядке, то, перед тем как начать чтение данных, следует удалить все записи регистрации изменений, которые были сделаны для этого узла и соответствовали номерам сообщений, меньшим или равным указанному в обрабатываемом нами сообщении как номер принятого. Это делается затем, чтобы исключить дублирование данных, которые уже были ранее посланы этому узлу и им обработаны. Для этого мы обращаемся к службе регистрации изменений менеджера планов обмена и используем метод `УдалитьРегистрациюИзменений()`.

Затем в цикле мы выполняем чтение данных, содержащихся в сообщении обмена. При этом для каждого элемента в функции `ВозможностьЧтенияДанных()` (см. листинг 4.9) проверяется возможность чтения данных. Причем, если имя типа `CatalogObject.Товары.Вручную` или `CatalogObject.Склады.Вручную`, возвращается значение `Истина`. В противном случае вызывается стандартный метод `ВозможностьЧтенияXML()`, с помощью которого из объекта `ЧтениеXML` получается очередной тип данных XML и определяется, имеется ли соответствующий тип «1С:Предприятия».

Листинг 4.9. Функция «ВозможностьЧтенияДанных»

Функция `ВозможностьЧтенияДанных(ЧтениеXML)`

```
// Получаем тип данных XML, который может быть считан в данный момент
ТипXML = ПолучитьXMLТип(ЧтениеXML);
Если ТипXML = Неопределено Тогда
    Возврат Ложь;
КонецЕсли;

Если (ТипXML.ИмяТипа = "CatalogObject.Товары.Вручную" ИЛИ
    ТипXML.ИмяТипа = "CatalogObject.Склады.Вручную")
    И ТипXML.URIПространстваИмен = "" Тогда
    Возврат Истина;
КонецЕсли;

Возврат ВозможностьЧтенияXML(ЧтениеXML);
```

КонецФункции

В цикле чтения данных вызывается функция ПрочитатьДанные() (см. листинг 4.10), в которой производится чтение данных из элемента. При этом если читается элемент CatalogObject.Товары.Вручную/CatalogObject.Склады.Вручную, то вызывается функция ПрочитатьXMLТовары() (см. листинг 4.12) ПрочитатьXMLСклады() (см. листинг 4.11) для «ручного» чтения данных, в противном случае чтение осуществляется с помощью стандартного метода ПрочитатьXML.

Листинг 4.10. Функция «ПрочитатьДанные»

Функция ПрочитатьДанные(ЧтениеXML)

```

ТипXML = ПолучитьXMLТип(ЧтениеXML);
Если ТипXML = Неопределено Тогда
    Возврат Неопределено;
КонецЕсли;

Если ТипXML.ИмяТипа = "CatalogObject.Склады.Вручную"
    И ТипXML.URIПространстваИмен = "" Тогда
    // Пытаемся прочесть значение справочника "Склады".
    Возврат ПрочитатьXMLСклады(ЧтениеXML);
КонецЕсли;

Если ТипXML.ИмяТипа = "CatalogObject.Товары.Вручную"
    И ТипXML.URIПространстваИмен = "" Тогда
    // Пытаемся прочесть значение справочника Товары
    Возврат ПрочитатьXMLТовары(ЧтениеXML);
КонецЕсли;

// Пытаемся прочесть значение из объекта ЧтениеXML стандартным образом.
Возврат ПрочитатьXML(ЧтениеXML);

КонецФункции // ПрочитатьДанные(ЧтениеXML)

```

Листинг 4.11. Функция «ПрочитатьXMLСклады»

Функция ПрочитатьXMLСклады(ЧтениеXML)

```

Если ЧтениеXML.ТипУзла <> ТипУзлаXML.НачалоЭлемента Тогда
    ВызватьИсключение "Ошибка чтения XML";
КонецЕсли;

// Чтение следующего узла.
ЧтениеXML.Прочитать();

// Чтение ссылки на элемент справочника.
СкладСсылка = ПрочитатьXML(ЧтениеXML);
Если ТипЗнч(СкладСсылка) <> Тип("СправочникСсылка.Склады") Тогда
    ВызватьИсключение "Ошибка чтения XML";
КонецЕсли;

// Пытаемся создать объект по полученной ссылке.

```

```
Склад = СкладСсылка.ПолучитьОбъект();

Если Склад = Неопределено Тогда
    // Создаем элемент справочника.
    Склад = Справочники.Склады.СоздатьЭлемент();

    // Устанавливаем значение ссылки для нового объекта.
    Склад.УстановитьСсылкуНового(СкладСсылка);
КонецЕсли;

// Наименование.
Склад.Наименование = ПрочитатьXML(ЧтениеXML);

// Проверяем, что текущим узлом является КонецЭлемента.
Если ЧтениеXML.ТипУзла <> ТипУзлаXML.КонецЭлемента Тогда
    ВызватьИсключение "Ошибка чтения XML";
КонецЕсли;

// Чтение следующего узла для завершения чтения элемента.
ЧтениеXML.Прочитать();
Возврат Склад;

КонецФункции
```

В функции `ПрочитатьXMLСклады()` производится последовательное считывание каждого узла элемента справочника `Склады`, записанного вручную в сообщение обмена.

Сначала мы проверяем, что находимся в XML-файле на начале элемента, затем двигаемся на следующий узел методом `Прочитать` объекта `ЧтениеXML`. Затем считываем ссылку на элемент справочника и пытаемся создать объект по полученной ссылке. Если это сделать не удалось, значит, такого объекта еще не существует, и мы создаем новый склад. Затем считываем наименование переданного объекта и устанавливаем его для нового или уже существующего склада. В завершение проверяем: если считана вся информация о складе (достигнут конец элемента), то двигаемся на следующий элемент данных в сообщении обмена.

В результате функция возвращает объект `Склад`, содержащий данные обмена, в процедуру `ПрочитатьСообщениеСИИзменениями()`, в переменную `Данные` (см. листинг 4.8).

Листинг 4.12. Функция «ПрочитатьXMLТовары»

```
Функция ПрочитатьXMLТовары(ЧтениеXML)

    Если ЧтениеXML.ТипУзла <> ТипУзлаXML.НачалоЭлемента Тогда
        ВызватьИсключение "Ошибка чтения XML";
    КонецЕсли;

    // Чтение следующего узла.
```

```
ЧтениеXML.Прочитать();

// Чтение ссылки на элемент справочника.
ТоварСсылка = ПрочитатьXML(ЧтениеXML);
Если ТипЗнч(ТоварСсылка) <> Тип("СправочникСсылка.Товары") Тогда
    ВызватьИсключение "Ошибка чтения XML";
КонецЕсли;

// Пытаемся создать объект по полученной ссылке.
Товар = ТоварСсылка.ПолучитьОбъект();

// Чтение признака группы.
ЭтоГруппа = ПрочитатьXML(ЧтениеXML);
Если Товар <> Неопределено Тогда
    Если Товар.ЭтоГруппа <> ЭтоГруппа Тогда
        ВызватьИсключение "Некорректные данные";
    КонецЕсли;

Иначе
    // Создаем элемент справочника.
    Если ЭтоГруппа = Истина Тогда
        Товар = Справочники.Товары.СоздатьГруппу();
    Иначе
        Товар = Справочники.Товары.СоздатьЭлемент();
    КонецЕсли;

    // Устанавливаем значение ссылки для нового объекта.
    Товар.УстановитьСсылкуНового(ТоварСсылка);
КонецЕсли;

// Родитель.
Товар.Родитель = ПрочитатьXML(ЧтениеXML);
// Наименование.
Товар.Наименование = ПрочитатьXML(ЧтениеXML);

// Реквизиты, загружаемые только для элемента справочника.
Если Не Товар.ЭтоГруппа Тогда
    // Описание.
    Товар.Описание = ПрочитатьXML(ЧтениеXML);
    // ФайлКартинки.
    Товар.ФайлКартинки = ПрочитатьXML(ЧтениеXML);
КонецЕсли;

// Проверяем, что текущим узлом является "КонецЭлемента".
Если ЧтениеXML.ТипУзла <> ТипУзлаXML.КонецЭлемента Тогда
    ВызватьИсключение "Ошибка чтения XML";
КонецЕсли;

// Чтение следующего узла для завершения чтения элемента.
ЧтениеXML.Прочитать();
Возврат Товар;
```

Функция ПрочитатьXMLТовары() работает аналогично описанной выше функции для считывания информации о складах. С той лишь разницей, что справочник Товары иерархический. Поэтому в случае несуществующего объекта мы создаем или новую группу, или новый элемент, в зависимости от значения реквизита ЭтоГруппа, считанного из сообщения обмена. Затем устанавливаем значения реквизитов Родитель и Наименование для нового или уже существующего товара. Для товаров, не являющихся группой, устанавливаем также значения реквизитов Описание и ФайлКартинки. Все эти реквизиты есть у товаров и в основном, и в мобильном приложении.

В результате функция возвращает объект Товар, содержащий данные обмена, в процедуру ПрочитатьСообщениеСИзменениями(), в переменную Данные (см. листинг 4.8).

В случае стандартного чтения данных методом ПрочитатьXML переменная Данные будет содержать любой другой объект «1С:Предприятия», полученный из сообщения обмена.

После этого в процедуре ПрочитатьСообщениеСИзменениями() (см. листинг 4.8) мы записываем полученные данные методом Записать(). Перед записью полученного объекта мы устанавливаем у него в параметрах обмена данными узел отправителя, для того чтобы система при записи этого объекта в нашей базе данных не формировала записи регистрации изменений этого объекта для того узла, от которого мы его только что получили.

Кроме этого, в параметрах обмена данными мы устанавливаем свойство Загрузка, информирующее систему о том, что запись объекта будет происходить в режиме обновления данных, полученных в результате обмена. Такое указание позволяет системе упростить процедуру записи объекта, отказавшись от ряда стандартных проверок и исключив изменения связанных данных, которые выполняются при обычной записи.

После того как все сообщение обмена будет нами обработано, мы заканчиваем чтение методом ЗакончитьЧтение() объекта ЧтениеСообщенияОбмена. Затем закрываем файл с сообщением обмена методом Закреть() объекта ЧтениеXML и удаляем этот файл с планшета. В заключение выводим сообщение об окончании загрузки данных из файла.

Мобильное приложение

В мобильном приложении мы хотим хранить только необходимые данные. Поэтому для справочников Склады и Товары установим длину стандартного реквизита Код равной нулю. В результате этого поля не будет в таблице справочника. А также удалим реквизит Артикул из списка реквизитов справочника Товары (рис. 4.2).

Чтобы не возникало ошибки при сохранении структуры базы данных, удалим поле Код из списка свойства Ввод по строке на закладке Поле ввода в окне редактирования объекта конфигурации справочников Товары и Склады (рис. 4.2).

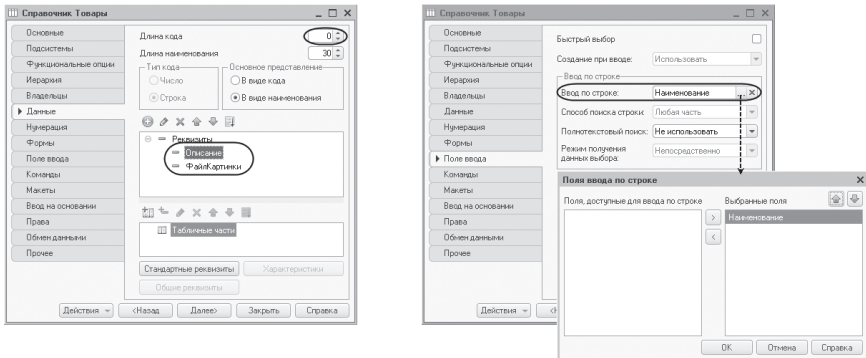


Рис. 4.2. Свойства справочника «Товары»

Поскольку этих данных у товара больше не будет, удалим поля Код и Артикул из дерева формы элемента справочника Товары.

Затем откроем модуль узла плана обмена Мобильные и внесем следующие изменения в процедуру ЗаписатьСообщениеСИзмениями(), см. листинг 4.13.

Листинг 4.13. Процедура для записи данных обмена

Процедура ЗаписатьСообщениеСИзмениями(Каталог) Экспорт

```
Сообщение = Новый СообщениеПользователю;
Сообщение.Текст = "----- Выгрузка в узел " + Строка(ЭтотОбъект) + " -----";

// Сформировать имя временного файла.
ИмяФайла = Каталог + "Message" + СокрЛП(ПланыОбмена.Мобильные.ЭтотУзел().Код) + " " +
СокрЛП(Ссылка.Код) + ".xml";

// Создать объект записи XML
// *** Запись XML-документов.
ЗаписьXML = Новый ЗаписьXML;
ЗаписьXML.ОткрытьФайл(ИмяФайла);
ЗаписьXML.ЗаписатьОбъявлениеXML();

// *** Инфраструктура сообщений.
ЗаписьСообщения = ПланыОбмена.СоздатьЗаписьСообщения();
ЗаписьСообщения.НачатьЗапись(ЗаписьXML, Ссылка);
Сообщение.Текст = Сообщение.Текст + Символы.ПС + " Номер сообщения: " +
ЗаписьСообщения.НомерСообщения;
```

```
// Получить выборку измененных данных.  
// *** Механизм регистрации изменений.  
ВыборкаИзменений = ПланыОбмена.ВыбратьИзменения(ЗаписьСообщения.Получатель,  
                                                    ЗаписьСообщения.НомерСообщения);  
  
Пока ВыборкаИзменений.Следующий() Цикл  
    Данные = ВыборкаИзменений.Получить();  
  
    // Записать данные в сообщение *** XML-сериализация.  
    ЗаписатьДанные(ЗаписьXML, Данные);  
КонецЦикла;  
  
ЗаписьСообщения.ЗакончитьЗапись();  
ЗаписьXML.Закрыть();  
  
Сообщение.Текст = Сообщение.Текст + Символы.ПС + "----- Конец выгрузки -----";  
Сообщение.Сообщить();  
  
КонецПроцедуры
```

Эта процедура практически аналогична процедуре для записи данных в сообщение обмена в основном приложении (см. листинг 4.4) – с той лишь разницей, что каталог расположения файла обмена, который передается в процедуру обмена в параметре Каталог из обработки ЗапаснойОбменДанными, не выбирается пользователем интерактивно, а задается явно. Это корень планшета – `/storage/sdcard0/`.

Кроме того, все сообщения пользователю о ходе выгрузки данных собираются в одно многострочное сообщение, которое показывается один раз, по окончании выгрузки.

Текст и назначение процедур `ЗаписатьДанные()` (см. листинг 4.5), `ЗаписатьXMLТовары()` (см. листинг 4.6) и `ЗаписатьXMLСклады()` (см. листинг 4.7) уже рассматривались нами в предыдущем разделе. Поэтому мы их не будем здесь еще раз повторять, но в модуль узла плана обмена их надо внести обязательно.

В процедуру `ПрочитатьСообщениеСИзменениями()` в модуле узла плана обмена Мобильные внесем следующие изменения (листинг 4.14).

Листинг 4.14. Процедура для чтения данных обмена

Процедура ПрочитатьСообщениеСИзменениями(Каталог) Экспорт

```
// Сформировать имя файла.  
ИмяФайла = Каталог + "Message" + СокрЛП(Ссылка.Код) + "_" +  
                                                    СокрЛП(ПланыОбмена.Мобильные.ЭтотУзел().Код) + ".xml";  
  
Файл = Новый Файл(ИмяФайла);  
Если Не Файл.Существует() Тогда  
    Возврат;  
КонецЕсли;
```

```

// *** Чтение документов XML.
// Попытаться открыть файл.
ЧтениеXML = Новый ЧтениеXML;
Попытка
    ЧтениеXML.ОткрытьФайл(ИмяФайла);

Исключение
    Сообщение = Новый СообщениеПользователю;
    Сообщение.Текст = "Невозможно открыть файл обмена данными.";
    Сообщение.Сообщить();
    Возврат;
КонецПопытки;

Сообщение = Новый СообщениеПользователю;
Сообщение.Текст = "----- Загрузка из " + Строка(ЭтотОбъект) + " -----";
Сообщение.Текст = Сообщение.Текст + Символы.ПС + " – Считывается файл " + ИмяФайла;

// Загрузить из найденного файла.
// *** Инфраструктура сообщений.
ЧтениеСообщения = ПланыОбмена.СоздатьЧтениеСообщения();

// Читать заголовок сообщения обмена данными – файла XML.
ЧтениеСообщения.НачатьЧтение(ЧтениеXML);

// Сообщение предназначено не для этого узла.
Если ЧтениеСообщения.Отправитель <> Ссылка Тогда
    ВызватьИсключение "Неверный узел";
КонецЕсли;

// Удаляем регистрацию изменений для узла отправителя сообщения.
// *** Служба регистрации изменений.
ПланыОбмена.УдалитьРегистрациюИзменений(ЧтениеСообщения.Отправитель,
ЧтениеСообщения.НомерПринятого);

// Читаем данные из сообщения *** XML-сериализация.
Пока ВозможностьЧтенияДанных(ЧтениеXML) Цикл

    // Читаем очередное значение.
    Данные = ПрочитатьДанные(ЧтениеXML);

    // Записать полученные данные.
    Данные.ОбменДанными.Отправитель = ЧтениеСообщения.Отправитель;
    Данные.ОбменДанными.Загрузка = Истина;
    Данные.Записать();

КонецЦикла;

ЧтениеСообщения.ЗакончитьЧтение();
ЧтениеXML.Закрыть();
УдалитьФайлы(ИмяФайла);

Сообщение.Текст = Сообщение.Текст + Символы.ПС + "----- Конец загрузки -----";
Сообщение.Сообщить();

КонецПроцедуры

```


Эта процедура практически аналогична процедуре для чтения данных из сообщения обмена в основном приложении (см. листинг 4.8) – с той лишь разницей, что каталог расположения файла обмена, который передается в процедуру обмена в параметре Каталог из обработки ЗапаснойОбменДанными, не выбирается пользователем интерактивно, а задается явно. Это корень планшета – `"/storage/sdcard0/"`.

Кроме того, все сообщения пользователю о ходе загрузки данных собираются в одно многострочное сообщение, которое показывается один раз, по окончании загрузки.

Текст и назначение функций `ВозможностьЧтенияДанных()` (см. листинг 4.9), `ПрочитатьДанные()` (см. листинг 4.10), `ПрочитатьXMLТовары()` (см. листинг 4.12), `ПрочитатьXMLСклады()` (см. листинг 4.11) уже рассматривались нами в предыдущем разделе. Поэтому мы их не будем здесь еще раз повторять, но в модуль узла плана обмена их надо внести обязательно.

Разрешение коллизий при обмене существующими данными

Надо понимать, что при вводе новых объектов на планшете и в основном приложении одновременно ссылки на объекты базы данных будут разными, а все новые объекты «будут доставлены по назначению».

Но при обмене уже существующими объектами базы данных (с одинаковыми ссылками), которые были изменены как в основном, так и в мобильном приложении после последнего сеанса обмена, может возникнуть коллизия, когда требуется определить, какие изменения принять, а какие – отклонить. Чтобы разрешить эту коллизию, нужно исходить из логики работы нашего примера.

Например, заказ создается в интернет-магазине и в процессе обмена данными посылается курьеру на планшет. После этого уже курьер будет работать с этим заказом и результат своей работы возвращать в офис. Значит, дальнейшее изменение этого заказа в основном приложении бессмысленно, так как при чтении данных обмена от планшета к интернет-магазину изменения курьера будут иметь больший приоритет. Соответственно, на планшете должны быть проигнорированы изменения этого заказа, сделанные в офисе уже после того, как заказ был передан на планшет.

Что касается справочников Клиенты и Причины отказа, то здесь ситуация обратная, так как нормативно-справочная информация ведется в офисе, т.е. изменения интернет-магазина должны иметь больший приоритет. Однако это не мешает курьеру вводить новых клиентов и новые причины отказа,

как и вводить новые заказы. Все эти данные будут приняты при обмене данными от планшета к интернет-магазину.

В случае обмена данными справочника Хранимые файлы коллизии возникнуть не должно, так как работа с файлами картинок товаров возможна только на стационарном компьютере, а работа с файлами мультимедиа возможна только на планшете.

Исходя из вышеизложенного, внесем изменения в процедуры чтения сообщений обмена (и в мобильном, и в основном приложении), определенных в модуле узла плана обмена Мобильные.

Мобильное приложение

В мобильном приложении откроем модуль узла плана обмена Мобильные и внесем следующие изменения в процедуру ПрочитатьСообщениеСИзмениями(), листинг 4.15.

Листинг 4.15. Процедура для чтения данных обмена

Процедура ПрочитатьСообщениеСИзмениями(Каталог) Экспорт

.....

// Читаем данные из сообщения *** XML-сериализация.

Пока ВозможностьЧтенияДанных(ЧтениеXML) Цикл

 // Читаем очередное значение.

 Данные = ПрочитатьДанные(ЧтениеXML);

 // Не переносим изменение, полученное из офиса,

 // если есть регистрация изменения на планшете.

 Если Не ПринятьИзменения(ЧтениеСообщения.Отправитель, Данные) Тогда

 Сообщение.Текст = Сообщение.Текст + Символы.ПС +

 Строка(Данные.Метаданные()) + "": " + Строка(Данные) +

 " – Изменения отклонены";

 Продолжить;

 КонецЕсли;

 // Записать полученные данные.

.....

КонецЦикла;

.....

КонецПроцедуры

Добавленный фрагмент выделен жирным шрифтом.

В теле цикла чтения данных следует разрешить возможную коллизию. Для этого мы вызываем функцию ПринятьИзменения() и передаем туда полу-

ченный объект обмена (Данные) и ссылку на узел обмена, для которого отправлено сообщение (СообщениеОбмена.Отправитель), см. листинг 4.16.

Листинг 4.16. Функция для разрешения коллизии при обмене данными

Функция ПринятьИзменения(Отправитель, Данные)

```
Прием = Истина;  
Если ПланыОбмена.ИзменениеЗарегистрировано(Отправитель, Данные) Тогда  
    Если ТипЗнч(Данные) = Тип("ДокументОбъект.Заказ") Тогда  
        Прием = Ложь;  
    КонецЕсли;  
  
КонецЕсли;  
Возврат Прием;  
  
КонецФункции
```

С помощью метода менеджера планов обмена `ИзменениеЗарегистрировано()` мы проверяем, зарегистрировано ли изменение объекта в узле обмена (на планшете), для которого отправлены данные. Если да и если тип объекта `ДокументОбъект.Заказ`, то функция возвращает `Ложь`, во всех остальных случаях возвращается `Истина`.

Таким образом, в процедуре `ПрочитатьСообщениеСИзменениями()` мы отклоняем изменения объекта `Документ Заказ`, полученное из офиса, если этот объект уже изменялся на планшете. Во всех остальных случаях мы принимаем изменения полученного объекта.

Основное приложение

Теперь внесем аналогичные изменения в процедуру `ПрочитатьСообщениеСИзменениями()`, описанную в модуле узла плана обмена `Мобильные` в основном приложении (конфигурация `ИнтернетМагазин`), см. листинг 4.17.

Листинг 4.17. Процедура для чтения данных обмена

Процедура ПрочитатьСообщениеСИзменениями(Каталог) Экспорт

...

```
// Читаем данные из сообщения *** XML-сериализация.  
Пока ВозможностьЧтенияДанных(ЧтениеXML) Цикл
```

```
    // Читаем очередное значение.  
    Данные = ПрочитатьДанные(ЧтениеXML);
```

```
    // Не переносим изменение, полученное от планшета,  
    // если есть регистрация изменения в офисе.  
    Если Не ПринятьИзменения(ЧтениеСообщения.Отправитель, Данные) Тогда  
        Сообщение = Новый СообщениеПользователю;
```

```

Сообщение.Текст = Строка(Данные.Метаданные()) + ": " + Строка(Данные) +
" – Изменения отклонены";
Сообщение.Сообщить();

Продолжить;
КонецЕсли;

// Записать полученные данные.
.....
КонецЦикла;

.....
КонецПроцедуры

```

Добавленный фрагмент выделен жирным шрифтом.

В теле цикла чтения данных коллизия, возникающая при обмене данными, разрешается в функции `ПринятьИзменения()` по-другому, поскольку это уже не мобильное, а основное приложение (листинг 4.18).

Листинг 4.18. Функция для разрешения коллизии при обмене данными

```

Функция ПринятьИзменения(Отправитель, Данные)

    Прием = Истина;
    Если ПланыОбмена.ИзменениеЗарегистрировано(Отправитель, Данные) Тогда

        Если ТипЗнч(Данные) = Тип("СправочникОбъект.Клиенты") ИЛИ
            ТипЗнч(Данные) = Тип("СправочникОбъект.ПричиныОтказа") Тогда
            Прием = Ложь;
        КонецЕсли;

    КонецЕсли;
    Возврат Прием;

КонецФункции

```

В этой функции проверяется, зарегистрировано ли изменение объекта в узле обмена (в офисе), для которого отправлены данные. Если да и если тип объекта `СправочникОбъект.Клиенты` или `СправочникОбъект.ПричиныОтказа`, то функция возвращает `Ложь`, во всех остальных случаях возвращается `Истина`.

Таким образом, в процедуре `ПрочитатьСообщениеСИзменениями()` мы отклоняем изменения объектов `Справочник Клиенты` и `Справочник ПричиныОтказа`, полученные от планшета, если эти объекты уже изменялись в офисе. Во всех остальных случаях мы принимаем изменения полученного объекта.

Передача данных из основного приложения

Теперь в процедуре записи сообщения обмена в основном приложении нужно учесть еще один важный момент.

Дело в том, что при передаче данных от интернет-магазина к планшету нужно передавать не все данные, а только те, которые относятся к конкретному курьеру, работающему на планшете. То есть в документах **Заказ** и **Обслуживание заказов** должен быть указан тот же курьер, что и в реквизите узла плана обмена **Мобильные**, описывающего мобильное устройство (рис. 4.3).

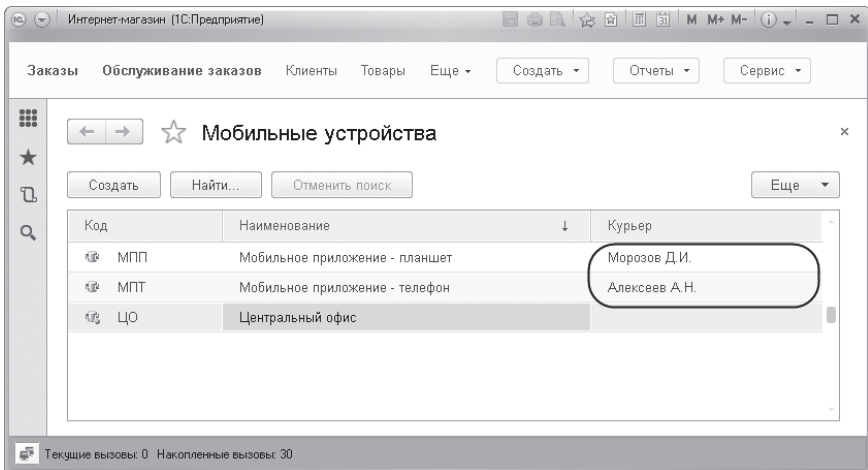


Рис. 4.3. Список узлов плана обмена «Мобильные устройства»

Заказы для других курьеров должны отправиться на другие мобильные устройства, которых в общем случае может быть несколько. Это касается только основного приложения, так как от планшета мы будем принимать все данные.

А также при выгрузке данных из основного приложения на планшет должен передаваться только тот элемент справочника Пользователи, ссылкой на который является реквизит Курьер узла-планшета.

Итак, в основном приложении откроем модуль узла плана обмена **Мобильные** и внесем следующие изменения в процедуру **ЗаписатьСообщениеСИИзменениями()**, см. листинг 4.19.

Листинг 4.19. Процедура для записи данных обмена

Процедура ЗаписатьСообщениеСИзмениями(Каталог) Экспорт

.....

Пока ВыборкаИзменений.Следующий() Цикл
 Данные = ВыборкаИзменений.Получить();

// Если перенос данных не нужен, то, возможно, необходимо записать удаление данных.
Если Не НуженПереносДанных(Данные) Тогда

// Получаем значение с возможным удалением данных.
УдалениеДанных(Данные);
КонецЕсли;

// Записать данные в сообщение * XML-сериализация.**
ЗаписатьДанные(ЗаписьXML, Данные);
КонецЦикла;

.....

КонецПроцедуры

Добавленный фрагмент выделен жирным шрифтом.

В цикле перебора измененных данных, перед записью в XML-файл, данные с помощью вызова функции НуженПереносДанных() (листинг 4.20) проверяются на необходимость выгрузки. В случае если выгрузка не должна производиться (функция возвращает Ложь), вызывается процедура УдалениеДанных() (листинг 4.21).

Листинг 4.20. Функция для реализации стратегии переноса данных

Функция НуженПереносДанных(Данные)

Перенос = Истина;
 Если ТипЗнч(Данные) = Тип("ДокументОбъект.Заказ") ИЛИ
 ТипЗнч(Данные) = Тип("ДокументОбъект.ОбслуживаниеЗаказов") Тогда

// Проверяем, что у документов курьер совпадает с реквизитом узла обмена.
Если Не Курьер.Пустая() И Данные.Курьер <> Курьер Тогда
Перенос = Ложь;
КонецЕсли;

ИначеЕсли ТипЗнч(Данные) = Тип("СправочникОбъект.Пользователи") Тогда

Если Не Курьер.Пустая() И Данные.Ссылка <> Курьер Тогда
Перенос = Ложь;
КонецЕсли;

КонецЕсли;
Возврат Перенос;

КонецФункции

Функция `НуженПереносДанных()` реализует стратегию распространения данных, описанную выше. В параметре `Данные` в функцию передается полученный объект обмена.

Если тип объекта `ДокументОбъект.Заказ` или `ДокументОбъект.ОбслуживаниеЗаказов`, то проверяется следующее условие переноса данных. Если реквизит `Курьер` для узла-получателя заполнен и не равен значению одноименного реквизита документов `Заказ` и `ОбслуживаниеЗаказов`, то функция возвращает `Ложь`.

Если тип объекта `СправочникОбъект.Пользователи`, то проверяется следующее условие переноса данных. Если реквизит `Курьер` для узла-получателя заполнен и не равен значению ссылки на элемент справочника `Пользователи`, то функция также возвращает `Ложь`. Во всех остальных случаях возвращается `Истина`.

Таким образом, в процедуре `ЗаписатьСообщениеСИИзменениями()` мы выгружаем только те данные, которые относятся к курьеру, указанному в плане обмена `Мобильные` при описании узла обмена, в который мы хотим передать данные (см. рис. 4.3).

В случае если данные (полученные при выборке изменений) не подлежат выгрузке в текущий узел плана обмена, производится вызов процедуры `УдалениеДанных()`. В этой процедуре для данных, имеющих объектную природу, создается объект `УдалениеОбъекта`. Этим достигается удаление в узле-приемнике «ранее неправильно отосланных данных». Наборы записей регистров очищаются (листинг 4.21).

Листинг 4.21. Функция для удаления данных при обмене

Процедура `УдалениеДанных(Данные)`

```
// Получаем объект описания метаданного, соответствующий данным.  
ОбъектМетаданных = ?(ТипЗнч(Данные) = Тип("УдалениеОбъекта"),  
                        Данные.Ссылка.Метаданные(), Данные.Метаданные());  
Если Метаданные.Справочники.Содержит(ОбъектМетаданных)  
ИЛИ Метаданные.Документы.Содержит(ОбъектМетаданных) Тогда  
  
    // Перенос удаления объекта для объектных.  
    Данные = Новый УдалениеОбъекта(Данные.Ссылка);  
  
ИначеЕсли Метаданные.РегистрыСведений.Содержит(ОбъектМетаданных) Тогда  
  
    // Очищаем данные.  
    Данные.Очистить();  
  
КонецЕсли;  
КонецПроцедуры
```

Тестирование обмена данными в обе стороны

Итак, нам нужно симитировать ситуацию, когда интернет-магазин обменивается данными с несколькими курьерами, работающими на разных мобильных устройствах. При этом он отсылает каждому курьеру «свои» данные, принимает от них информацию и разрешает возможные коллизии, возникающие при обмене данными. А также мы протестируем ситуацию, когда некоторые данные (в нашем случае справочники складов и товаров) имеют разную структуру данных на планшете и в офисе. При этом обмен этими данными выполняется «точно так же» (с точки зрения пользователя) – все переданные данные доставляются по назначению.

Чтобы все это проверить, мы создадим в основном приложении в списке плана обмена еще два узла обмена (помимо предопределенного узла), описывающие мобильные устройства и относящиеся к разным курьерам.

Затем на планшете мы создадим две информационные базы, подключенные к нашему мобильному приложению. То есть программная начинка у этих баз будет одной и той же, а данные будут предназначены для различных курьеров.

Самое простое – начать все с чистого листа. Поэтому откроем основное приложение Интернет-магазин в режиме 1С:Предприятие и в плане обмена Мобильные устройства удалим все узлы обмена, кроме предопределенного узла Центральный офис (с кодом ЦО). Это делается для того, чтобы регистрация уже измененных объектов и номера отправленных/принятых сообщений обмена для прежнего узла с кодом МПП «не портили общую картину».

Затем добавим два новых узла, описывающих мобильные устройства, с которыми будут работать различные курьеры. Заполним код первого узла – МПП, наименование – Мобильное приложение – планшет и укажем курьера – Морозов Д.И.; код второго узла – МПТ, наименование – Мобильное приложение – телефон, курьер – Алексеев А.Н. (рис. 4.4).

ПРИМЕЧАНИЕ

Коды узлов обмена МПП (мобильное приложение – планшет) и МПТ (мобильное приложение – телефон) могут быть другими. Главное, что им должны соответствовать такие же коды в мобильном приложении.

После этого добавим два новых заказа для курьера Алексеев А.Н. Обратите внимание, что номера новых заказов будут иметь префикс ЦО: ЦО-000001, ЦО-000002. Затем создадим документ Обслуживание заказов № 000000004 для этого же курьера и внесем новые заказы в его табличную часть.

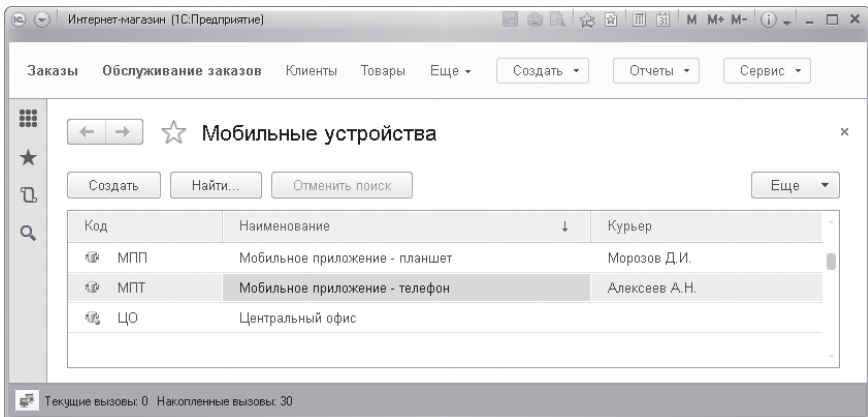


Рис. 4.4. Список узлов плана обмена «Мобильные устройства»

Также добавим еще один заказ (с номером ЦО-000003) для курьера Морозов Д.И. Затем создадим документ Обслуживание заказов № 000000005 для этого курьера и внесем новый заказ в его табличную часть.

Все описанные выше изменения содержатся в выгрузке информационной базы «Запасной способ обмена.dt».

Перенос данных из основного приложения в разные узлы обмена. Обмен данными разной структуры

Теперь создадим в основном приложении новый товар (например, Рубашка), заполним все его данные и выберем для него картинку.

Изменим наименование склада и заполним данные о его расположении (например, Офис).

А также изменим статус созданных заказов ЦО-000001, ЦО-000002, ЦО-000003 на значение В работе и проведем их.

Затем выгрузим данные в каждый из узлов обмена.

Откроем обработку Запасной обмен данными, выберем в поле Узел обмена узел, в который мы хотим передать данные (Мобильное приложение – планшет), и нажмем кнопку Зарегистрировать изменения (рис. 4.5).

Таким образом, мы выполнили начальную синхронизацию нового узла обмена всеми данными обмена. Затем нажмем кнопку Выгрузить данные и выберем каталог для записи XML-файла с сообщением обмена.

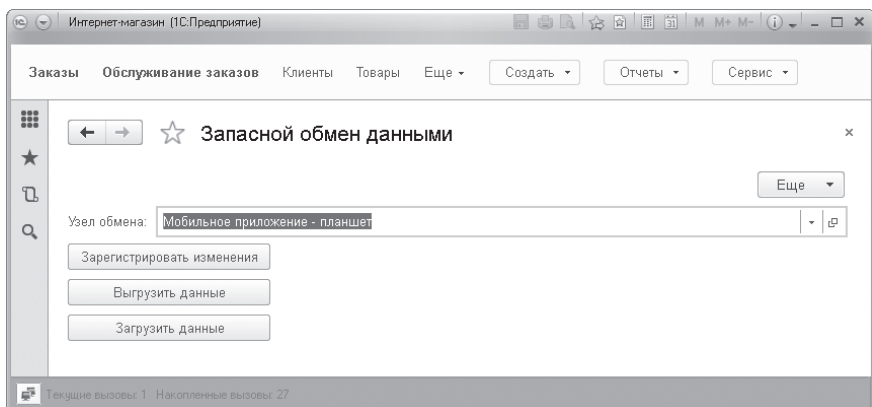


Рис. 4.5. Обработка «Обмен данными»

После этого выберем в поле Узел обмена другой узел – Мобильное приложение – телефон и нажмем кнопку Зарегистрировать изменения. Далее нажмем кнопку Выгрузить данные и выберем каталог для записи XML-файла с сообщением обмена.

Затем подсоединим планшет через USB-кабель к компьютеру и скопируем в корень планшета файлы обмена с именами MessageЦО_МПП, MessageЦО_МПТ из выбранного каталога.

Теперь создадим на планшете две информационные базы, в которые мы будем загружать данные из основного приложения.

Для чистоты эксперимента желательно удалить уже имеющуюся информационную базу Курьер Интернет-магазина. Дело в том, что в процессе изучения данной главы на ней могли выполняться различные примеры и в результате там могли оказаться «ненужные» данные.

Чтобы удалить мобильное приложение, нужно выполнить долгое нажатие на этом приложении в списке приложений мобильной платформы разработчика. В открывшемся контекстном меню нужно выбрать пункт Удалить. Будьте внимательны: эта операция необратима!

Запустим на планшете мобильную платформу разработчика «1С:Предприятия», выполним долгое нажатие на приложении Курьер Интернет-магазина и в появившемся контекстном меню нажмем кнопку Удалить (рис. 4.6).

Затем снова добавим это приложение в список приложений мобильной платформы, но в нем уже не будет никаких данных.

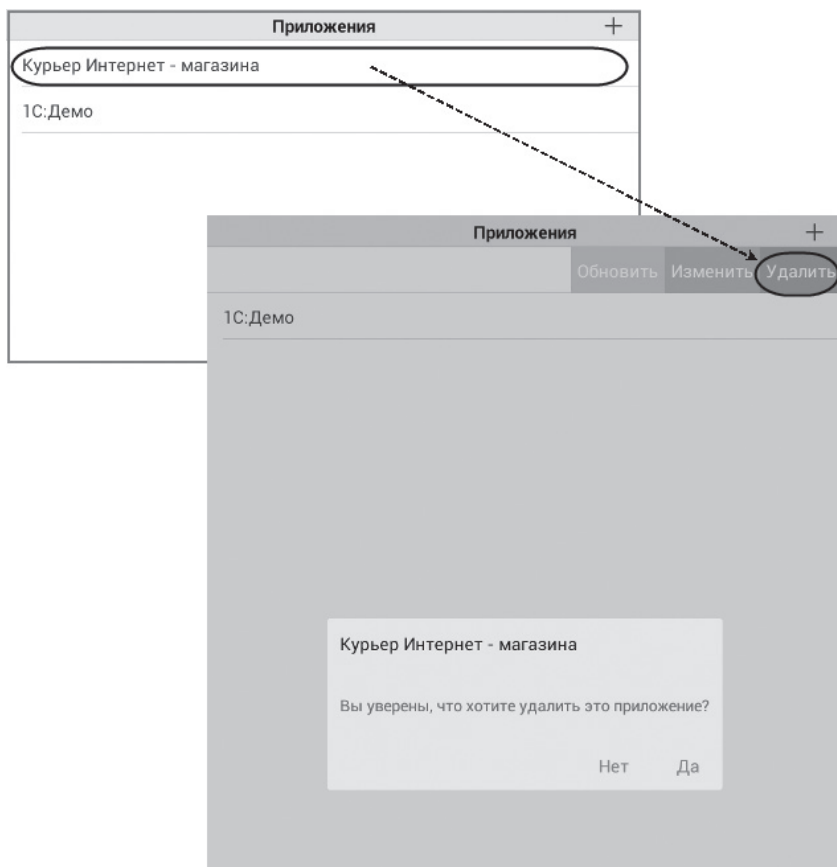


Рис. 4.6. Удаление мобильного приложения

Для этого нажмем кнопку со значком «+» в правом верхнем углу экрана в строке Приложения. В появившемся окне в поле Адрес укажем URL веб-сервера, на котором опубликовано наше мобильное приложение, и нажмем кнопку Загрузить в правом верхнем углу экрана. Далее укажем наименование нового мобильного приложения – Курьер Интернет-магазина (Морозов), включим флажок Перезапуск из конфигулятора и нажмем кнопку ОК в правом верхнем углу экрана (рис. 4.7).

Аналогичным образом добавим другую информационную базу для курьера Алексеев (рис. 4.8).

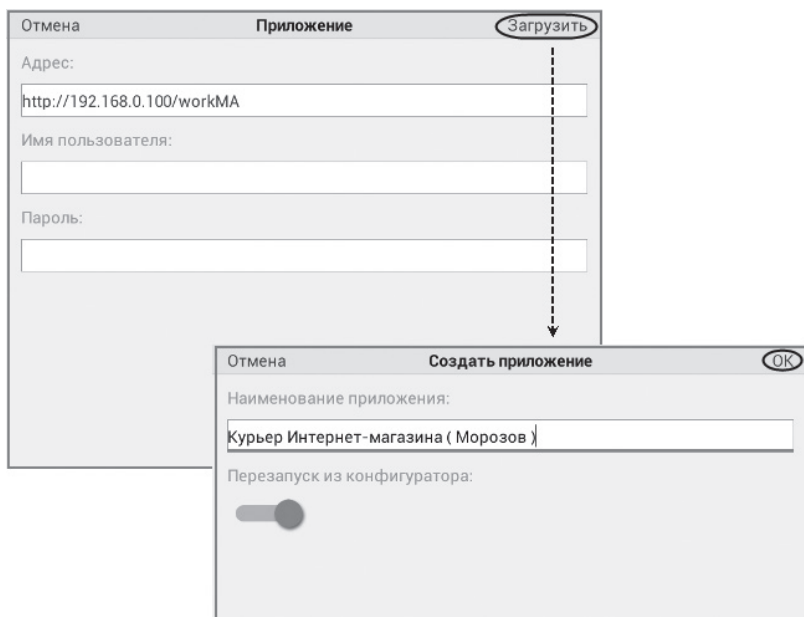


Рис. 4.7. Добавление нового мобильного приложения



Рис. 4.8. Добавление нового мобильного приложения

Теперь запустим на планшете первое мобильное приложение Курьер Интернет-магазина (Морозов) и из главного меню приложения выполним команду для открытия списка узлов плана обмена Мобильные устройства.

В списке узлов плана обмена уже присутствует одна запись с пиктограммой – предопределенный узел. Установим для нее значения полей: Код – МПП, Наименование – Планшет – Samsung. Затем добавим еще один узел плана обмена со значениями полей: Код – ЦО, Наименование – Центральный офис. Поле Курьер можно не задавать, так как от планшета в офис в любом случае отправляются все данные (см. рис. 4.9).

Затем откроем обработку Запасной обмен данными. Выберем в поле Узел обмена узел Центральный офис (узел, от которого мы принимаем данные) и нажмем кнопку Загрузить данные (рис. 4.9).

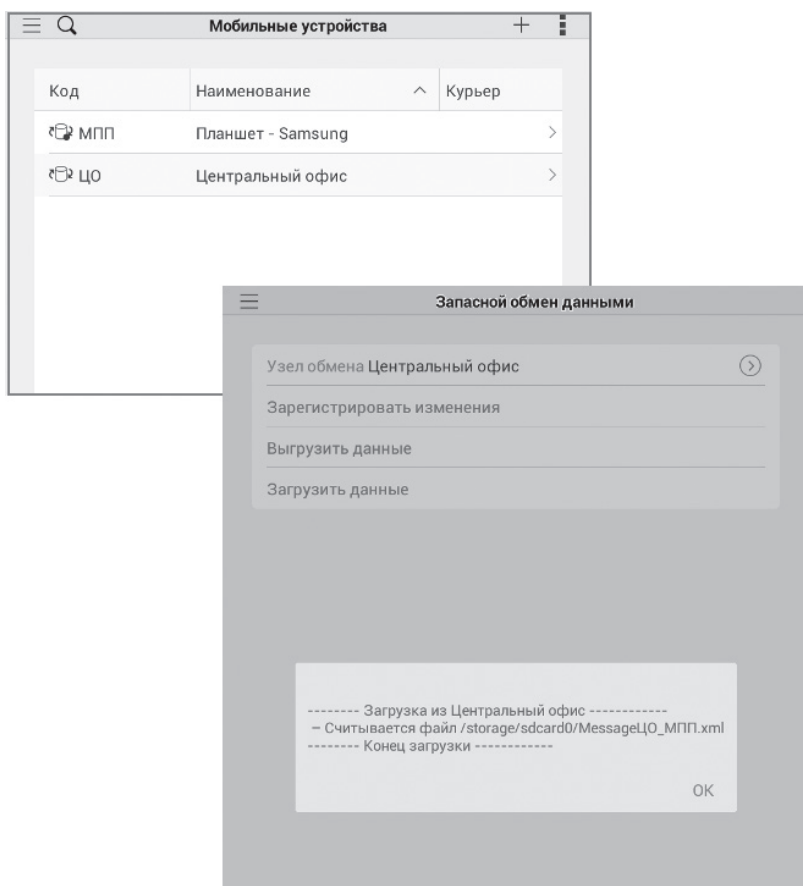


Рис. 4.9. Загрузка данных для мобильного приложения

Затем откроем списки документов Заказ и Обслуживание заказов и убедимся, что из основного приложения перенесены только документы для курьера Морозов Д.И.

А также посмотрим, как произошла передача данных с различной структурой из основного приложения на планшет. Откроем форму нового товара Рубашка. Мы видим, что данных о коде и артикуле товара в мобильном приложении нет, но все остальные данные доставлены по назначению (рис. 4.10).

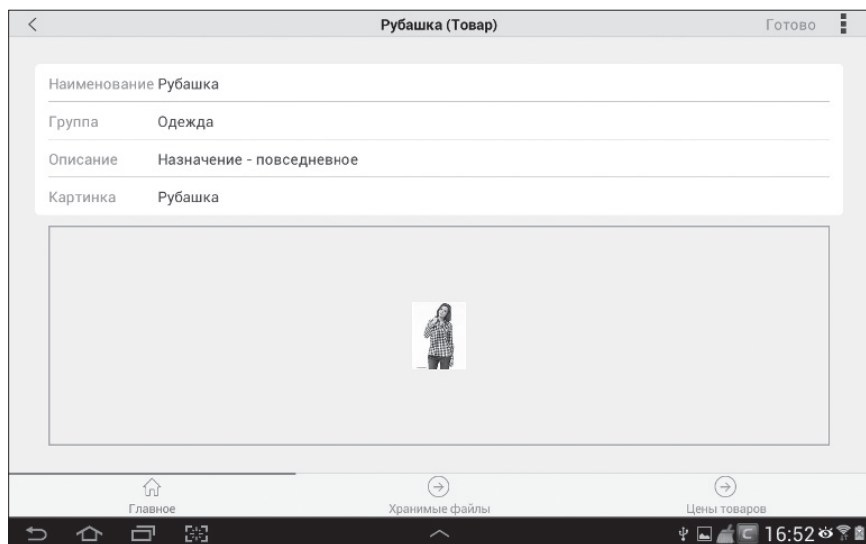


Рис. 4.10. Форма нового товара, переданного из офиса на планшет

В справочнике хранимых файлов также появился новый элемент, соответствующий картинке товара.

Откроем форму нового заказа № ЦО-000003, переданного из офиса на планшет, и убедимся, что наименование склада изменилось на новое (Большой), установленное в офисе (рис. 4.11).

Теперь запустим на планшете второе мобильное приложение Курьер Интернет-магазина (Алексеев) и из главного меню приложения выполним команду для открытия списка узлов плана обмена Мобильные устройства.

Заказ ЦО-000003 от 01.09.2015 12:00:01 Готово

Номер	ЦО-000003
Дата	01.09.2015 12:00:01
Клиент	Новикова В.С. (i)
Курьер	Морозов Д.И.
Склад	Большой
Дата доставки	05.09.2015
Адрес доставки	Россия, Москва, Гоголевский б...
Дополнительная информация	
Статус заказа	В работе

Добавить Еще >

Цвет	Размер	Количество	Сумма
Сапоги			
Черный	37	1	8 000,00

Количество итог 1

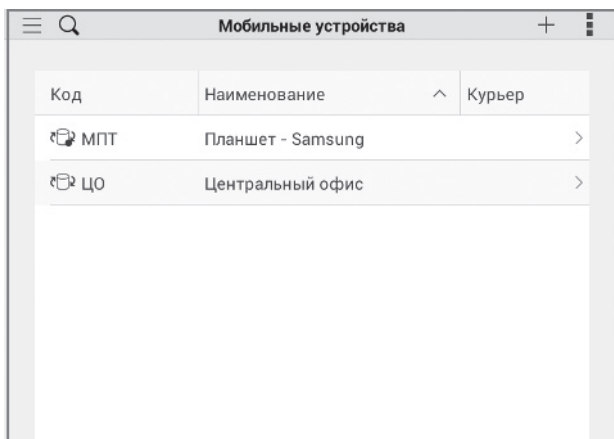
Сумма итог 8 000,00

Рис. 4.11. Форма нового заказа, переданного из офиса на планшет

В списке узлов плана обмена уже присутствует одна запись с пиктограммой – предопределенный узел. Установим для нее значения полей: Код – МПТ, Наименование – Планшет – Samsung. Затем добавим еще один узел плана обмена со значениями полей: Код – ЦО, Наименование – Центральный офис (рис. 4.12).

Поле Курьер можно не задавать, так как от планшета в офис в любом случае отправляются все данные.

После этого откроем обработку Запасной обмен данными. Выберем в поле Узел обмена узел Центральный офис (узел, от которого мы принимаем данные) и нажмем кнопку Загрузить данные.



Код	Наименование	Курьер
МПТ	Планшет - Samsung	>
ЦО	Центральный офис	>

Рис. 4.12. Список узлов обмена мобильного приложения

Затем откроем списки документов **Заказ** и **Обслуживание заказов** и убедимся, что из основного приложения перенесены только документы для курьера **Алексеев А.Н.**

Данные справочников и регистра сведений перенесены полностью в каждую из баз, за исключением списка пользователей. Этот список включает единственный элемент и соответствует курьеру, указанному в основном приложении при описании мобильных узлов обмена.

Итак, мы протестировали отдельную выгрузку данных из основного приложения. Все работает правильно! Таким образом, даже если на одном планшете работает несколько курьеров, можно создать информационную базу для каждого курьера и работать, как мы только что показали.

Получение данных от планшета. Разрешение коллизии

Теперь нам нужно ввести новые данные на планшете, посмотреть, как они передаются в интернет-магазин и как разрешается коллизия при обмене данными.

Для тестирования этих ситуаций введем сначала данные на планшете.

Во второй информационной базе **Курьер Интернет-магазина (Алексеев)** добавим новую причину отказа. Обратите внимание, что префикс кода причины отказа будет **МПТ**.

Затем откроем форму клиента **Смирнова А.И.** и изменим его адрес (рис. 4.13).

Смирнова А.И. (Клиент) * Готово

Код 000000002

Наименование Смирнова А.И.

АДРЕС

Страна Россия

Город Москва

Улица пр-т Мира

Дом 55

Квартира 127

КОНТАКТЫ

Телефон +7(929)410-57-90

Главное Хранимые файлы

Рис. 4.13. Изменение клиента 000000002 на планшете

А также откроем заказ № ЦО-000002, для одной из позиций в списке заказанных товаров установим признак Отказ и укажем новую причину отказа. А также установим статус заказа в значение Выполнен (рис. 4.14).

Теперь выгрузим измененные данные из этой базы в файл обмена. Для этого откроем обработку Запасной обмен данными. Выберем в поле Узел обмена узел Центральный офис и нажмем кнопку Выгрузить данные. В результате в корне планшета появится файл MessageМПТ_ЦО.xml.

Заказ ЦО-000002 от 31.08.2015 12:00:00 * Готово

Номер	ЦО-000002
Дата	31.08.2015 12:00:00
Клиент	Шувалова Е.М. ⓘ
Курьер	Алексеев А.Н.
Склад	Большой
Дата доставки	03.09.2015
Адрес доставки	Россия, Москва, ул. Грибоедов...

Дополнительная информация

Статус заказа: **Выполнен**

Отказ: Причина отказа

Джинсы ☐

Платье ☒ Не соответствует заказу

Рис. 4.14. Изменение заказа ЦО-000002 на планшете

После этого изменим те же данные в основной информационной базе Интернет-магазина. В основном приложении откроем заказ № ЦО-000002 и изменим количество и сумму заказанного товара (рис. 4.15).

Затем откроем информацию о клиенте Смирнова А.И. и изменим его адрес (рис. 4.16).

Теперь выгрузим измененные данные из этой базы в файл обмена. Для этого откроем обработку Запасной обмен данными. Выберем в поле Узел обмена узел Мобильное приложение – телефон и нажмем кнопку Выгрузить

данные. В результате в каталоге стационарного компьютера, выбранного для обмена, появится файл MessageЦО_МПТ.xml.

← → ☆ Заказ ЦО-000002 от 31.08.2015 12:00:00 *

Главное Остатки товаров Причины отказа

Провести и закрыть Записать Провести Создать на основании

Еще

Добавить

N	Товар	Цвет	Размер	Цена	Количе...	Сумма
1	Джинсы	Черный	46	4 000,00	2	8 000,00
2	Платье	Белый	46	5 000,00	3	15 000,00

Количество итог: 5

Сумма итог: 23 000,00

Рис. 4.15. Изменение заказа ЦО-000002 в офисе

Смирнова А.И. (Клиент) (1С:Предприятие)

Смирнова А.И. (Клиент)

Главное Файлы

Записать и закрыть Записать Создать на основании

Еще

Код: 000000002

Наименование: Смирнова А.И.

Адрес

Страна: Россия

Город: Москва

Улица: Кутузовский пр-т

Дом: 111

Квартира: 33

Контакты

Телефон: +7(929)410-57-90

Электронная почта:

Координаты

Широта: 0,000000

Долгота: 0,000000

Рис. 4.16. Изменение клиента 000000002 в офисе

Затем подсоединим планшет через USB-кабель к компьютеру и скопируем в корень планшета файл обмена с именем MessageЦО_МПТ из выбранного каталога. А в этот каталог положим файл MessageМПТ_ЦО, который находится в корне планшета.

Теперь загрузим в основное приложение данные обмена, полученные от второго мобильного приложения. Откроем это приложение, в обработке Запасной обмен данными выберем в поле узел обмена Мобильное приложение – телефон и нажмем кнопку Загрузить данные.

При загрузке данных произойдет конфликт для клиента Смирнова А.И., так как его адрес изменялся одновременно и в основном приложении, и на планшете. В соответствии с нашей стратегией разрешения коллизии при обмене данными изменения в справочниках, сделанные на планшете, отклоняются (рис. 4.17).

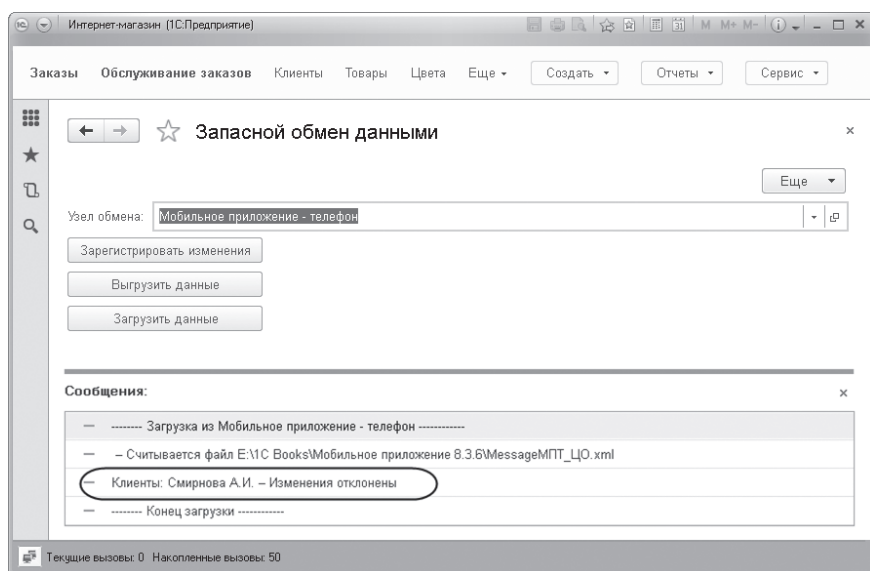


Рис. 4.17. Загрузка данных в основное приложение

В результате адрес клиента останется таким, каким он был задан в офисе (см. рис. 4.16).

А вот при загрузке новой причины отказа, добавленной на планшете, проблем не возникнет. В результате в основной базе появится новая причина отказа с префиксом МПТ (рис. 4.18).

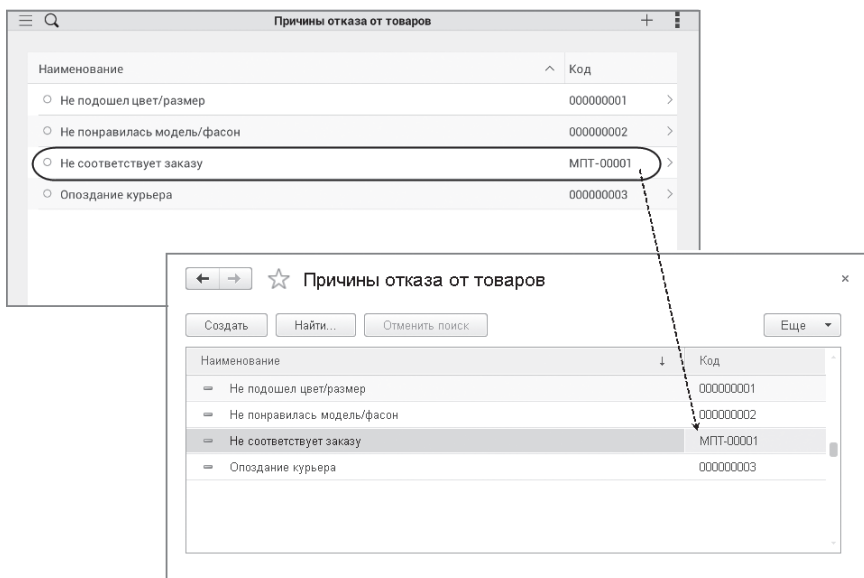


Рис. 4.18. Загрузка новой причины отказа в основное приложение

Теперь загрузим во второе мобильное приложение данные обмена, полученные от основного приложения. Откроем это приложение на планшете, в обработке Запасной обмен данными выберем в поле узел обмена Центральный офис и нажмем кнопку Загрузить данные.

При загрузке данных произойдет конфликт для заказа № ЦО-000002, так как он изменялся одновременно и в основном приложении, и на планшете. В соответствии с нашей стратегией разрешения коллизии при обмене данными изменения заказов, сделанные в офисе, отклоняются (рис. 4.19).

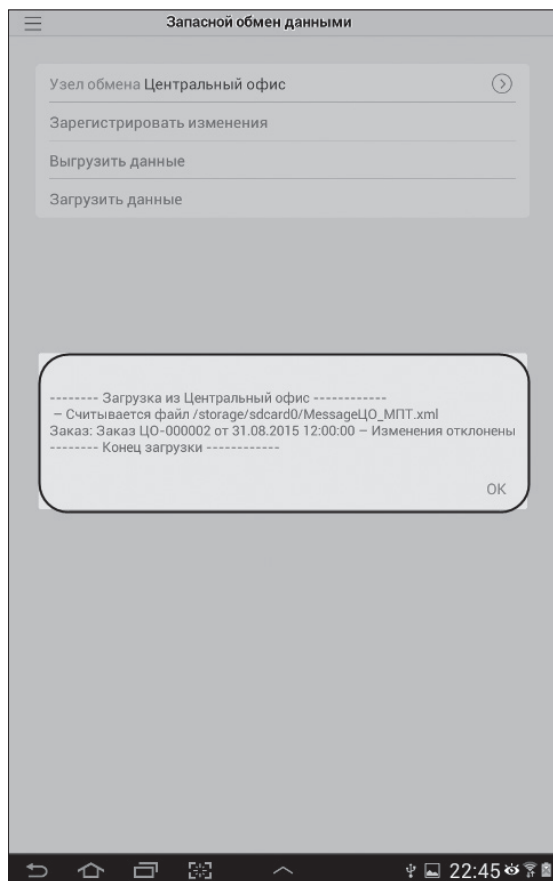


Рис. 4.19. Загрузка данных в мобильное приложение

В результате обмена данные заказа в основном приложении будут такими, какими они были заданы на планшете (см. рис. 4.14, 4.20).

А адрес клиента на планшете будет таким, каким его задали в офисе (см. рис. 4.16, 4.21).

← → ☆ Заказ ЦО-000002 от 31.08.2015 12:00:00

Главное Остатки товаров Причины отказа

Провести и закрыть Записать Провести Создать на основании Ещё ▾

Номер: ЦО-000002 Дата: 31.08.2015 12:00:00

Клиент: Шувалова Е.М. ☒ Дата доставки: 03.09.2015

Курьер: Алексеев А.Н. ☒ Адрес доставки: Россия, Москва, ул. Грибоедова, 51, 140

Склад: Большой ☒ Дополнительно:

Статус заказа: Открыт В работе Выполнен Закрыт

Список товаров Отказ от товаров

N	Товар	Отказ	Причина отказа
1	Джинсы		
2	Платье	✓	Не соответствует заказу

Количество итог: 2

Сумма итог: 8 000,00

Рис. 4.20. Заказ № ЦО-000002 в основном приложении

< Смирнова А.И. (Клиент) Готово ≡

Код 000000002

Наименование Смирнова А.И.

АДРЕС

Страна Россия

Город Москва

Улица Кутузовский пр-т

Дом 111

Квартира 33

КОНТАКТЫ

Телефон +7(929)410-57-90

Электронная почта

КООРДИНАТЫ

Широта 0,000000

Долгота 0,000000

Главное Хранимые файлы

22:46

Рис. 4.21. Информация о клиенте 000000002 в мобильном приложении

Все описанные выше изменения содержатся в выгрузке информационной базы «Тестирование обмена.dt».

После того как мы протестировали обмен данными для разных информационных баз, удалим с планшета вторую базу Курьер Интернет-магазина (Алексеев), так как она больше нам не нужна.

Основной обмен данными через Web-сервис

По условию нашей задачи в начале каждого рабочего дня курьер получает все необходимые ему данные из интернет-магазина на свой планшет. Затем, в процессе обслуживания заказов, происходит автоматическая синхронизация данных с основным приложением, благодаря чему в обеих базах появляется оперативная информация о деятельности интернет-магазина.

На основе заказов, полученных от курьера, в основном приложении можно построить отчеты: о наличии товаров на складах, о причинах отказа от товаров и т.п. Хотя у курьера на планшете нет нужной информации для построения отчетов, он может выполнить нужный ему отчет удаленно в основном приложении и получить его через веб-сервис себе на планшет.

Для реализации описанной выше функциональности мы используем возможности сервис-ориентированной архитектуры, с помощью которой система «1С:Предприятие» может экспортировать свою функциональность через Web-сервисы.

Команды для выполнения обмена данными и получения отчета будут инициированы из мобильного приложения. При этом будет произведено обращение к Web-сервису, предоставляемому основным приложением, и вызваны его соответствующие операции, которые возвращают данные обмена или данные отчета.

Кроме того, мы рассмотрим ситуацию, при которой обмен данными может быть инициирован из основного приложения путем рассылки push-уведомлений, сообщающих мобильному приложению о необходимости синхронизации данных с основным приложением. Получив такое уведомление, мобильное приложение выполняет обмен данными через веб-сервис основного приложения.

Создание Web-сервиса в основном приложении

Прежде всего, нам понадобится создать объект конфигурации WebСервис, описать его свойства и выполняемые им операции и опубликовать Web-сервис на веб-сервере. После этого функции, выполняемые Web-сер-

висом, станут доступны внешним информационным системам, в том числе мобильному приложению.

Для описания типов параметров и возвращаемых значений Web-сервиса в дереве объектов конфигурации в ветке Общие создадим ПакетXDTO ОбменДанными с пространством имен `http://localhost/wsExchange`. В свойстве URI пространства имен содержится `http://localhost` – адрес веб-сервера, установленного на локальном компьютере, `/wsExchange` – каталог, в который будет опубликован Web-сервис (рис. 4.22).

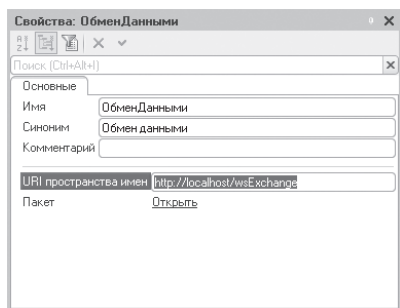


Рис. 4.22. Свойства пакета XDTO «ОбменДанными»

Теперь добавим в конфигурацию WebСервис MAExchange со следующими свойствами:

- URI пространства имен – `http://localhost/wsExchange`. Каждый Web-сервис может быть однозначно идентифицирован по своему имени и URI пространства имен, которому он принадлежит.
- Пакеты XDTO. Содержит перечень пакетов XDTO, типы которых могут использоваться для описания типов параметров и возвращаемых значений Web-сервиса:
 - `http://localhost/wsExchange`,
 - `http://v8.1c.ru/8.1/data-composition-system/details`,
 - `http://v8.1c.ru/8.1/data/core`,
 - `http://v8.1c.ru/8.1/data/enterprise/current-config`,
 - `http://v8.1c.ru/8.2/data/spreadsheet`.
- Имя файла публикации – `wsExchange.1cws`. Содержит имя файла описания Web-сервиса, который расположен на веб-сервере.

Для выбора пакетов XDTO, используемых Web-сервисом, нажмем кнопку выбора в соответствующем поле и в появившемся списке отметим только

что созданный нами пакет, а также пакеты XDTO, стандартно предоставляемые платформой (рис. 4.23).

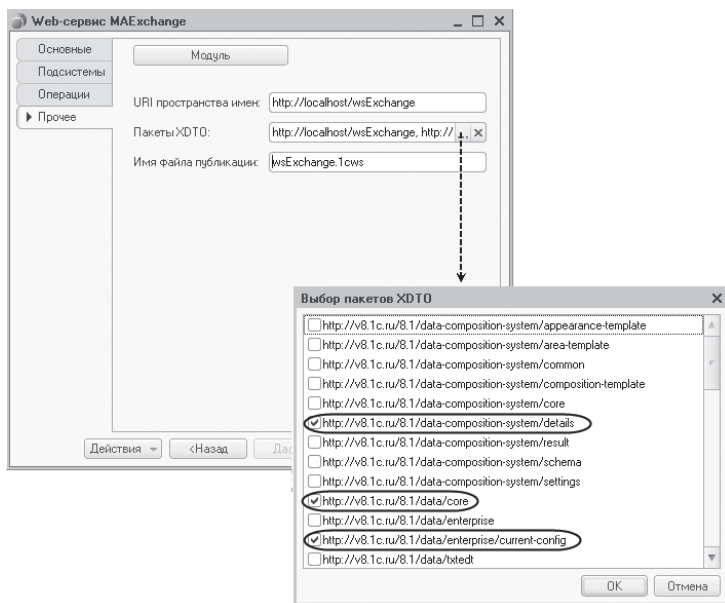


Рис. 4.23. Свойства Web-сервиса «MAExchange»

У созданного Web-сервиса определим четыре операции (рис. 4.24):

1. НачатьОбмен (для инициализации обмена) со свойствами:
 - тип возвращаемого значения – string из пространства имен <http://www.w3.org/2001/XMLSchema> (значение по умолчанию);
 - Возможно пустое значение – установлен;
 - Имя процедуры – НачатьОбмен.
2. ПолучитьДанные (для формирования пакета изменений, предназначенных для узла обмена) со свойствами:
 - тип возвращаемого значения – ValueStorage из пространства имен <http://v8.1c.ru/8.1/data/core>;
 - Возможно пустое значение – не установлен;
 - Имя процедуры – ПолучитьДанные.

3. **ЗаписатьДанные** (для записи изменений, принятых от узла обмена) со свойствами:
 - тип возвращаемого значения – string из пространства имен <http://www.w3.org/2001/XMLSchema> (значение по умолчанию);
 - Возможно пустое значение – установлен;
 - Имя процедуры – ЗаписатьДанные.
4. **ПолучитьОтчет** (для удаленного формирования и получения отчета) со свойствами:
 - тип возвращаемого значения – SpreadsheetDocument из пространства имен <http://v8.1c.ru/8.2/data/spreadsheet>;
 - Возможно пустое значение – не установлен;
 - Имя процедуры – ПолучитьОтчет.

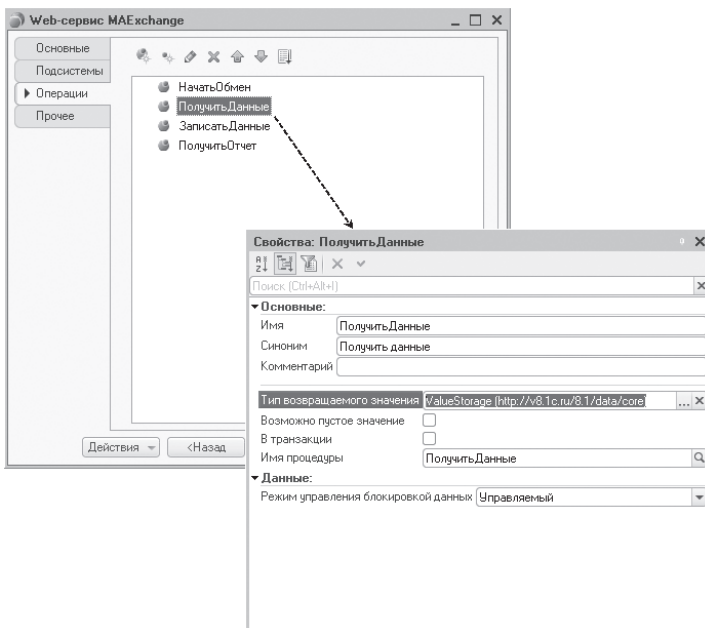


Рис. 4.24. Операции Web-сервиса «MAExchange»

Затем для каждой операции Web-сервиса определим параметры, которые будут передаваться в процедуру, описанную в модуле Web-сервиса и указанную в свойстве Имя процедуры (рис. 4.25):

1. Параметры операции НачатьОбмен:

- КодУзла со свойствами:
 - Тип значения — string из пространства имен <http://www.w3.org/2001/XMLSchema>;
 - Возможно пустое значение — не установлен;
 - Направление передачи — Входной.
- НомерОтправленного со свойствами:
 - Тип значения — decimal из пространства имен <http://www.w3.org/2001/XMLSchema>;
 - Возможно пустое значение — установлен;
 - Направление передачи — Входной.
- НомерПринятого со свойствами:
 - Тип значения — decimal из пространства имен <http://www.w3.org/2001/XMLSchema>;
 - Возможно пустое значение — установлен;
 - Направление передачи — Входной.

2. Параметры операции ПолучитьДанные:

- КодУзла со свойствами:
 - Тип значения — string из пространства имен <http://www.w3.org/2001/XMLSchema>;
 - Возможно пустое значение — не установлен;
 - Направление передачи — Входной.

3. Параметры операции ЗаписатьДанные:

- КодУзла со свойствами:
 - Тип значения — string из пространства имен <http://www.w3.org/2001/XMLSchema>;
 - Возможно пустое значение — не установлен;
 - Направление передачи — Входной.
- ДанныеМобильногоПриложения со свойствами:
 - Тип значения — ValueStorage из пространства имен <http://v8.1c.ru/8.1/data/core>;
 - Возможно пустое значение — не установлен;
 - Направление передачи — Входной.

4. Параметры операции ПолучитьОтчет:

- ДатаНачала со свойствами:
 - Тип значения — date из пространства имен <http://www.w3.org/2001/XMLSchema>;

- Возможно пустое значение — установлен;
- Направление передачи — Входной.
- **ДатаОкончания со свойствами:**
 - Тип значения — date из пространства имен `http://www.w3.org/2001/XMLSchema`;
 - Возможно пустое значение — установлен;
 - Направление передачи — Входной.
- **ИнформацияРасшифровки со свойствами:**
 - Тип значения — string из пространства имен `http://www.w3.org/2001/XMLSchema`;
 - Возможно пустое значение — установлен;
 - Направление передачи — Выходной.

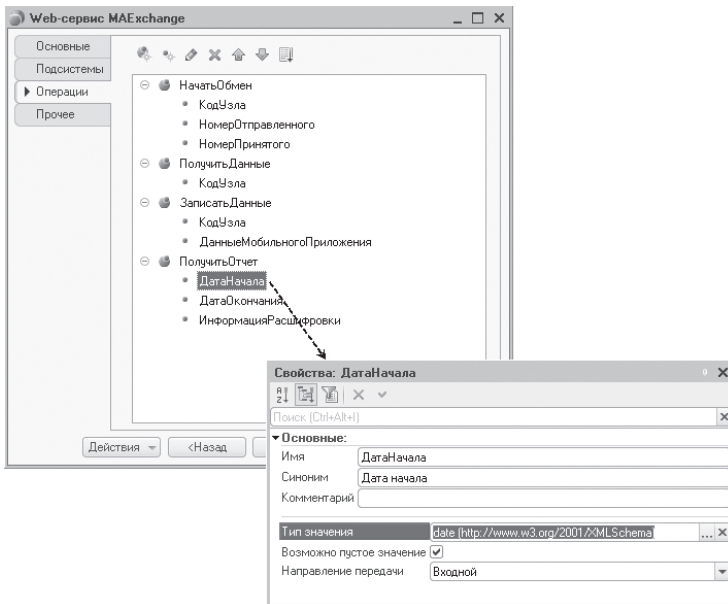


Рис. 4.25. Свойства параметров операций Web-сервиса

Теперь откроем модуль Web-сервиса и поместим в нем процедуры, которые будут вызываться при выполнении описанных выше операций.

Обмен данными

Начнем с функции НачатьОбмен(), которая инициализирует обмен (листинг 4.22).

Листинг 4.22. Функция «НачатьОбмен()»

Функция НачатьОбмен(КодУзла, НомерОтправленного, НомерПринятого)

```

    УзелОбмена = ПланыОбмена.Мобильные.НайтиПоКоду(КодУзла);
    Если УзелОбмена.Пустая() Тогда
        Возврат "";
    КонецЕсли;

    Если УзелОбмена.НомерОтправленного <> НомерОтправленного ИЛИ
        УзелОбмена.НомерПринятого <> НомерПринятого Тогда
        Узел = УзелОбмена.ПолучитьОбъект();
        Узел.НомерОтправленного = НомерОтправленного;
        Узел.НомерПринятого = НомерПринятого;
        Узел.Записать();
        Обмен.ЗарегистрироватьИзмененияДанных(УзелОбмена);
    КонецЕсли;

    Возврат УзелОбмена.Код;

```

КонецФункции

В эту функцию передается код узла обмена (базы на планшете) и номера принятых и отправленных этим узлом сообщений обмена. Узел обмена находится по коду в плане обмена Мобильные.

В случае несоответствия номеров принятых или отправленных сообщений для узла обмена и номеров сообщений, переданных в функцию, от ссылки на найденный узел обмена получается объект, для которого устанавливаются переданные значения номеров сообщений. Затем объект записывается и для него регистрируются изменения данных.

Процедуру для регистрации всех измененных объектов, входящих в состав плана обмена, для узла поместим в общий неглобальный модуль Обмен (листинг 4.23).

Листинг 4.23. Процедура «ЗарегистрироватьИзмененияДанных()»

Процедура ЗарегистрироватьИзмененияДанных(УзелОбмена) Экспорт

```

    СоставПланаОбмена = УзелОбмена.Метаданные().Состав;
    Для Каждого ЭлементСоставаПланаОбмена Из СоставПланаОбмена Цикл
        ПланыОбмена.ЗарегистрироватьИзменения(
            УзелОбмена,ЭлементСоставаПланаОбмена.Метаданные);
    КонецЦикла;

```

КонецПроцедуры

В данной процедуре используется метод менеджера планов обмена `ЗарегистрироватьИзменения()`, с помощью которого в цикле для каждого объекта из состава плана обмена регистрируются изменения данных для узла обмена, переданного в процедуру.

Затем в модуле Web-сервиса поместим функцию `ПолучитьДанные()` для формирования пакета изменений, предназначенных для узла обмена, код которого передается в функцию (листинг 4.24).

Листинг 4.24. Функция «ПолучитьДанные()»

Функция `ПолучитьДанные(КодУзла)`

```
УзелОбмена = ПланыОбмена.Мобильные.НайтиПоКоду(КодУзла);
Если УзелОбмена.Пустая() Тогда
    ВызватьИсключение("Неизвестное устройство – " + КодУзла);
КонецЕсли;

Возврат Обмен.СформироватьПакетОбмена(УзелОбмена);
```

КонецФункции

Узел обмена находится по коду в плане обмена `Мобильные`. Если узел не найден, то вызывается исключение. Затем вызывается функция `СформироватьПакетОбмена()` для формирования пакета изменений, предназначенных для узла, которую мы поместим в общий модуль `Обмен` (листинг 4.25).

Листинг 4.25. Функция «СформироватьПакетОбмена()»

Функция `СформироватьПакетОбмена(УзелОбмена) Экспорт`

```
ЗаписьXML = Новый ЗаписьXML;
ЗаписьXML.УстановитьСтроку("UTF-8");
ЗаписьXML.ЗаписатьОбъявлениеXML();

ЗаписьСообщения = ПланыОбмена.СоздатьЗаписьСообщения();
ЗаписьСообщения.НачатьЗаписи(ЗаписьXML, УзелОбмена);

ЗаписьXML.ЗаписатьСоответствиеПространстваИмен("xsi", "http://www.w3.org/2001/XMLSchema-instance");
ЗаписьXML.ЗаписатьСоответствиеПространстваИмен("v8", "http://v8.1c.ru/data");

ВыборкаИзменений = ПланыОбмена.ВыбратьИзменения(УзелОбмена,
                                                    ЗаписьСообщения.НомерСообщения);
Пока ВыборкаИзменений.Следующий() Цикл
    Данные = ВыборкаИзменений.Получить();

    // Если перенос данных не нужен, то, возможно, необходимо записать удаление данных.
    Если Не НуженПереносДанных(Данные, УзелОбмена) Тогда
        // Получаем значение с возможным удалением данных.
        УдалениеДанных(Данные);
    КонецЕсли;
```

```

// Записываем данные в сообщение.
ЗаписатьДанные(ЗаписьXML, Данные);
КонецЦикла;

ЗаписьСообщения.ЗакончитьЗапись();

Возврат Новый ХранилищеЗначения(ЗаписьXML.Закреть(), Новый СжатиеДанных(9));

КонецФункции

```

В функцию передается узел обмена (база на планшете), для которого формируется пакет изменений.

В целом код процедуры идентичен процедуре для записи сообщения обмена, которую мы рассматривали в разделе «Запасной способ обмена данными с основным приложением» (стр. 251). Разница состоит в том, что изменения записываются не в файл обмена, а в XML-строку при помощи метода УстановитьСтроку() объекта ЗаписьXML.

Кроме того, для сокращения размера XML-строки определяется соответствие используемым пространствам имен. Затем создается новое сообщение обмена, производится выборка изменений и т. д.

Выбранные данные с помощью вызова функции НуженПереносДанных() (листинг 4.26) проверяются на необходимость выгрузки. В случае если выгрузка не должна производиться (функция возвращает Ложь), вызывается процедура УдалениеДанных() (листинг 4.27).

Листинг 4.26. Функция для реализации стратегии переноса данных

Функция НуженПереносДанных(Данные, УзелОбмена) Экспорт

```

Перенос = Истина;
Если ТипЗнч(Данные) = Тип("ДокументОбъект.Заказ") ИЛИ
    ТипЗнч(Данные) = Тип("ДокументОбъект.ОбслуживаниеЗаказов") Тогда

    // Проверяем, что у документов курьер совпадает с реквизитом узла обмена.
    Если Не УзелОбмена.Курьер.Пустая() И Данные.Курьер <> УзелОбмена.Курьер Тогда
        Перенос = Ложь;
    КонецЕсли;

    ИначеЕсли ТипЗнч(Данные) = Тип("СправочникОбъект.Пользователи") Тогда

        Если Не УзелОбмена.Курьер.Пустая() И Данные.Ссылка <> УзелОбмена.Курьер Тогда
            Перенос = Ложь;
        КонецЕсли;

    КонецЕсли;
    Возврат Перенос;

```

КонецФункции

Функция `НуженПереносДанных()` реализует стратегию распространения данных из основного приложения на мобильные устройства. По этой стратегии для планшета выгружаются только те данные, которые относятся к курьеру, указанному в плане обмена Мобильные при описании узла обмена – мобильного устройства.

В случае если данные (полученные при выборке изменений) не подлежат выгрузке в узел обмена, производится вызов процедуры `УдалениеДанных()`. В этой процедуре для данных, имеющих объектную природу, создается объект `УдалениеОбъекта`. Этим достигается удаление в узле приемнике «ранее неправильно отосланных данных». Наборы записей регистров очищаются (листинг 4.27).

Листинг 4.27. Процедура для удаления данных при обмене

Процедура `УдалениеДанных(Данные)` Экспорт

```
// Получаем объект описания метаданного, соответствующий данным.  
ОбъектМетаданных = ?(ТипЗнч(Данные) = Тип("УдалениеОбъекта"),  
                        Данные.Ссылка.Метаданные(), Данные.Метаданные());  
Если Метаданные.Справочники.Содержит(ОбъектМетаданных)  
ИЛИ Метаданные.Документы.Содержит(ОбъектМетаданных) Тогда  
  
    // Перенос удаления объекта для объектных.  
    Данные = Новый УдалениеОбъекта(Данные.Ссылка);  
  
ИначеЕсли Метаданные.РегистрыСведений.Содержит(ОбъектМетаданных) Тогда  
  
    // Очищаем данные.  
    Данные.Очистить();  
  
КонецЕсли;
```

КонецПроцедуры

Затем в процедуре `СформироватьПакетОбмена()` вызывается процедура общего модуля `ЗаписатьДанные()`. В этой процедуре проверяется: если записываемый объект является элементом справочника `Товары` или `Склады`, то вызываются процедуры `ЗаписатьXMLТовары()` (см. листинг 4.29) или `ЗаписатьXMLСклады()` (см. листинг 4.30) для записи элемента справочника вручную, в противном случае запись производится с помощью стандартного метода `ЗаписатьXML` (листинг 4.28).

Листинг 4.28. Процедура для записи данных обмена в сообщение

Процедура `ЗаписатьДанные(ЗаписьXML, Данные)` Экспорт

```
Если ТипЗнч(Данные) = Тип("СправочникОбъект.Товары") Тогда  
  
    // Записываем элемент справочника вручную.  
    ЗаписатьXMLТовары(ЗаписьXML, Данные);
```

```
ИначеЕсли ТипЗнч(Данные) = Тип("СправочникОбъект.Склады") Тогда
```

```
    // Записываем элемент справочника вручную.  
    ЗаписатьXMLСклады(ЗаписьXML, Данные);
```

```
Иначе
```

```
    // Записываем данные с помощью стандартного метода.  
    ЗаписатьXML(ЗаписьXML, Данные);
```

```
КонецЕсли
```

```
КонецПроцедуры
```

Листинг 4.29. Процедура «ЗаписатьXMLТовары»

Процедура ЗаписатьXMLТовары(ЗаписьXML, Товар) Экспорт

```
    // Записываем начало элемента XML.  
    ЗаписьXML.ЗаписатьНачалоЭлемента("CatalogObject.Товары.Вручную");  
  
    // Ссылка.  
    ЗаписатьXML(ЗаписьXML, Товар.Ссылка, "Ref", НазначениеТипаXML.Явное);  
    // ЭтоГруппа.  
    ЗаписатьXML(ЗаписьXML, Товар.ЭтоГруппа, "IsFolder", НазначениеТипаXML.Явное);  
    // Родитель.  
    ЗаписатьXML(ЗаписьXML, Товар.Родитель, "Parent", НазначениеТипаXML.Явное);  
    // Наименование.  
    ЗаписатьXML(ЗаписьXML, Товар.Наименование, "Description", НазначениеТипаXML.Явное);  
  
    // Реквизиты, выгружаемые только для элемента справочника.  
    Если Не Товар.ЭтоГруппа Тогда  
        // Описание.  
        ЗаписатьXML(ЗаписьXML, Товар.Описание, "Описание", НазначениеТипаXML.Явное);  
        // ФайлКартинки.  
        ЗаписатьXML(ЗаписьXML, Товар.ФайлКартинки, "ФайлКартинки", НазначениеТипаXML.Явное);  
    КонецЕсли;  
  
    // Записываем конец элемента.  
    ЗаписьXML.ЗаписатьКонецЭлемента();
```

```
КонецПроцедуры
```

Листинг 4.30. Процедура «ЗаписатьXMLСклады»

Процедура ЗаписатьXMLСклады(ЗаписьXML, Склад) Экспорт

```
    // Записываем начало элемента XML.  
    ЗаписьXML.ЗаписатьНачалоЭлемента("CatalogObject.Склады.Вручную");  
  
    // Ссылка.  
    ЗаписатьXML(ЗаписьXML, Склад.Ссылка, "Ref", НазначениеТипаXML.Явное);  
    // Наименование.  
    ЗаписатьXML(ЗаписьXML, Склад.Наименование, "Description", НазначениеТипаXML.Явное);  
  
    // Записываем конец элемента.  
    ЗаписьXML.ЗаписатьКонецЭлемента();
```

```
КонецПроцедуры
```

Эти процедуры подробно рассматривались нами в разделе «Обмен данными с разной структурой» на стр. 255.

После выхода из цикла мы завершаем запись текста XML методом `Закрыть()` объекта `ЗаписьXML`. При этом, поскольку запись производилась в XML-строку, результирующая строка будет получена в качестве возвращаемого значения метода.

На основе этой строки мы создаем объект `ХранилищеЗначения` и возвращаем его в функцию `ПолучитьДанные()` нашего Web-сервиса. Эту строку, помещенную в хранилище значения, и вернет Web-сервис при вызове его соответствующей операции из мобильного приложения.

Затем в модуле Web-сервиса поместим функцию `ЗаписатьДанные()` для записи пакета изменений, принятых от узла, код которого передается в функцию (листинг 4.31).

Листинг 4.31. Функция «`ЗаписатьДанные()`»

Функция `ЗаписатьДанные(КодУзла, ДанныеМобильногоПриложения)`

```
УзелОбмена = ПланыОбмена.Мобильные.НайтиПоКоду(КодУзла);  
Если УзелОбмена.Пустая() Тогда  
    ВызватьИсключение("Неизвестное устройство - " + КодУзла);  
КонецЕсли;  
  
Обмен.ПринятьПакетОбмена(УзелОбмена, ДанныеМобильногоПриложения);
```

КонецФункции

Узел обмена находится по коду в плане обмена `Мобильные`. Если узел не найден, то вызывается исключение. Затем вызывается процедура `ПринятьПакетОбмена()` для записи пакета изменений, принятых от узла, которую поместим в общий модуль `Обмен` (листинг 4.32).

Листинг 4.32. Процедура «`ПринятьПакетОбмена()`»

Процедура `ПринятьПакетОбмена(УзелОбмена, ДанныеОбмена)` Экспорт

```
ЧтениеXML = Новый ЧтениеXML;  
ЧтениеXML.УстановитьСтроку(ДанныеОбмена.Получить());  
ЧтениеСообщения = ПланыОбмена.СоздатьЧтениеСообщения();  
ЧтениеСообщения.НачатьЧтение(ЧтениеXML);  
  
ПланыОбмена.УдалитьРегистрациюИзменений(ЧтениеСообщения.Отправитель,  
    ЧтениеСообщения.НомерПринятого);  
  
НачатьТранзакцию();  
Пока ВозможностьЧтенияДанных(ЧтениеXML) Цикл  
  
    Данные = ПрочитатьДанные(ЧтениеXML);
```

```

Если Не Данные = Неопределено Тогда
// Не переносим изменение, полученное от планшета, если есть регистрация
// изменения в офисе.
    Если Не ПринятьИзменения(ЧтениеСообщения.Отправитель, Данные) Тогда
        Продолжить;
    КонецЕсли;

    Данные.ОбменДанными.Отправитель = ЧтениеСообщения.Отправитель;
    Данные.ОбменДанными.Загрузка = Истина;

    Данные.Записать();
КонецЕсли;

КонецЦикла;
ЗафиксироватьТранзакцию();

ЧтениеСообщения.ЗакончитьЧтение();
ЧтениеXML.Закрыть();

КонецПроцедуры

```

В процедуру передается узел обмена (база на планшете), от которого принимаются изменения и пакет обмена (ДанныеОбмена), полученный от этого узла и помещенный в ХранилищеЗначения.

В целом код процедуры идентичен процедуре для чтения сообщения обмена, которую мы рассматривали в разделе «Запасной способ обмена данными с основным приложением» (стр. стр. 251). Разница состоит в том, что данные обмена читаются не из файла, а из XML-строки. Эта строка получается из хранилища значения и с помощью метода УстановитьСтроку() устанавливается для чтения объектом ЧтениеXML.

Чтение данных обмена происходит в транзакции, в цикле, пока метод ВозможностьЧтенияДанных() возвращает истину. Причем, если имя типа CatalogObject.Товары.Вручную или CatalogObject.Склады.Вручную, возвращается значение Истина. В противном случае вызывается стандартный метод ВозможностьЧтенияXML(), с помощью которого из объекта ЧтениеXML получается очередной тип данных XML и определяется, имеется ли соответствующий тип «1С:Предприятия» (листинг 4.33).

Листинг 4.33. Функция «ВозможностьЧтенияДанных»

Функция ВозможностьЧтенияДанных(ЧтениеXML) Экспорт

```

// Получаем тип данных XML, который может быть считан в данный момент.
ТипXML = ПолучитьXMLТип(ЧтениеXML);
Если ТипXML = Неопределено Тогда
    Возврат Ложь;
КонецЕсли;

```

```
Если (ТипXML.ИмяТипа = "CatalogObject.Товары.Вручную" ИЛИ  
    ТипXML.ИмяТипа = "CatalogObject.Склады.Вручную")  
И ТипXML.URIПространстваИмен = " Тогда  
    Возврат Истина;  
КонецЕсли;
```

```
Возврат ВозможностьЧтенияXML(ЧтениеXML);
```

```
КонецФункции
```

В цикле чтения данных вызывается функция ПрочитатьДанные() (см. листинг 4.32), в которой производится чтение данных из элемента. При этом если читается элемент CatalogObject.Товары.Вручную/CatalogObject.Склады.Вручную, то вызывается функция ПрочитатьXMLТовары() (см. листинг 4.36), ПрочитатьXMLСклады() (см. листинг 4.35) для «ручного» чтения данных, в противном случае чтение осуществляется с помощью стандартного метода ПрочитатьXML (см. листинг 4.34).

Листинг 4.34. Функция «ПрочитатьДанные»

Функция ПрочитатьДанные(ЧтениеXML) Экспорт

```
ТипXML = ПолучитьXMLТип(ЧтениеXML);  
Если ТипXML = Неопределено Тогда  
    Возврат Неопределено;  
КонецЕсли;
```

```
Если ТипXML.ИмяТипа = "CatalogObject.Склады.Вручную"  
    И ТипXML.URIПространстваИмен = "" Тогда  
    // Пытаемся прочесть значение справочника "Склады".  
    Возврат ПрочитатьXMLСклады(ЧтениеXML);  
КонецЕсли;
```

```
Если ТипXML.ИмяТипа = "CatalogObject.Товары.Вручную"  
    И ТипXML.URIПространстваИмен = "" Тогда  
    // Пытаемся прочесть значение справочника "Товары".  
    Возврат ПрочитатьXMLТовары(ЧтениеXML);  
КонецЕсли;
```

```
// Пытаемся прочесть значение из объекта ЧтениеXML стандартным образом.  
Возврат ПрочитатьXML(ЧтениеXML);
```

```
КонецФункции // ПрочитатьДанные(ЧтениеXML)
```

Листинг 4.35. Функция «ПрочитатьXMLСклады»

Функция ПрочитатьXMLСклады(ЧтениеXML) Экспорт

```
Если ЧтениеXML.ТипУзла <> ТипУзлаXML.НачалоЭлемента Тогда  
    ВызватьИсключение "Ошибка чтения XML";  
КонецЕсли;
```

```
// Чтение следующего узла.  
ЧтениеXML.Прочитать();  
  
// Чтение ссылки на элемент справочника.  
СкладСсылка = ПрочитатьXML(ЧтениеXML);  
Если ТипЗнч(СкладСсылка) <> Тип("СправочникСсылка.Склады") Тогда  
    ВызватьИсключение "Ошибка чтения XML";  
КонецЕсли;  
  
// Пытаемся создать объект по полученной ссылке.  
Склад = СкладСсылка.ПолучитьОбъект();  
  
Если Склад = Неопределено Тогда  
    // Создаем элемент справочника.  
    Склад = Справочники.Склады.СоздатьЭлемент();  
  
    // Устанавливаем значение ссылки для нового объекта.  
    Склад.УстановитьСсылкуНового(СкладСсылка);  
КонецЕсли;  
  
// Наименование.  
Склад.Наименование = ПрочитатьXML(ЧтениеXML);  
  
// Проверяем, что текущим узлом является "КонецЭлемента".  
Если ЧтениеXML.ТипУзла <> ТипУзлаXML.КонецЭлемента Тогда  
    ВызватьИсключение "Ошибка чтения XML";  
КонецЕсли;  
  
// Чтение следующего узла для завершения чтения элемента.  
ЧтениеXML.Прочитать();  
Возврат Склад;
```

КонецФункции

Листинг 4.36. Функция «ПрочитатьXMLТовары»

Функция ПрочитатьXMLТовары(ЧтениеXML) Экспорт

```
Если ЧтениеXML.ТипУзла <> ТипУзлаXML.НачалоЭлемента Тогда  
    ВызватьИсключение "Ошибка чтения XML";  
КонецЕсли;  
  
// Чтение следующего узла  
ЧтениеXML.Прочитать();  
  
// Чтение ссылки на элемент справочника  
ТоварСсылка = ПрочитатьXML(ЧтениеXML);  
Если ТипЗнч(ТоварСсылка) <> Тип("СправочникСсылка.Товары") Тогда  
    ВызватьИсключение "Ошибка чтения XML";  
КонецЕсли;  
  
// Пытаемся создать объект по полученной ссылке.  
Товар = ТоварСсылка.ПолучитьОбъект();
```

```
// Чтение признака группы.
ЭтоГруппа = ПрочитатьXML(ЧтениеXML);
Если Товар <> Неопределено Тогда
    Если Товар.ЭтоГруппа <> ЭтоГруппа Тогда
        ВызватьИсключение "Некорректные данные";
    КонецЕсли;

Иначе
    // Создаем элемент справочника.
    Если ЭтоГруппа = Истина Тогда
        Товар = Справочники.Товары.СоздатьГруппу();
    Иначе
        Товар = Справочники.Товары.СоздатьЭлемент();
    КонецЕсли;

    // Устанавливаем значение ссылки для нового объекта.
    Товар.УстановитьСсылкуНового(ТоварСсылка);
КонецЕсли;

// Родитель.
Товар.Родитель = ПрочитатьXML(ЧтениеXML);
// Наименование.
Товар.Наименование = ПрочитатьXML(ЧтениеXML);

// Реквизиты, загружаемые только для элемента справочника.
Если Не Товар.ЭтоГруппа Тогда
    // Описание.
    Товар.Описание = ПрочитатьXML(ЧтениеXML);
    // ФайлКартинки.
    Товар.ФайлКартинки = ПрочитатьXML(ЧтениеXML);
КонецЕсли;

// Проверяем, что текущим узлом является "КонецЭлемента".
Если ЧтениеXML.ТипУзла <> ТипУзлаXML.КонецЭлемента Тогда
    ВызватьИсключение "Ошибка чтения XML";
КонецЕсли;

// Чтение следующего узла для завершения чтения элемента.
ЧтениеXML.Прочитать();
Возврат Товар;

КонецФункции
```

Эти процедуры подробно рассматривались нами в разделе «Обмен данными с разной структурой» на стр. 255.

В результате в переменной Данные, в процедуре ПрочитатьСообщениеСИзменениями(), будет содержаться объект, содержащий данные обмена.

Затем мы анализируем прочитанные данные, чтобы разрешить возможную коллизию. Для этого мы вызываем функцию ПринятьИзменения() и передаем туда полученный объект обмена (Данные) и ссылку на узел обмена,

для которого отправлено сообщение (СообщениеОбмена.Отправитель), см. листинг 4.37.

Листинг 4.37. Функция для разрешения коллизии при обмене данными

Функция ПринятьИзменения(Отправитель, Данные) Экспорт

Прием = Истина;

Если ПланыОбмена.ИзменениеЗарегистрировано(Отправитель, Данные) Тогда

Если ТипЗнч(Данные) = Тип("СправочникОбъект.Клиенты") ИЛИ

ТипЗнч(Данные) = Тип("СправочникОбъект.ПричиныОтказа") Тогда

Прием = Ложь;

КонецЕсли;

КонецЕсли;

Возврат Прием;

КонецФункции

В этой функции проверяется, зарегистрировано ли изменение объекта в узле обмена (в офисе), для которого отправлены данные. Если да и если тип объекта СправочникОбъект.Клиенты или СправочникОбъект.ПричиныОтказа, то функция возвращает Ложь, во всех остальных случаях возвращается Истина.

Таким образом, в процедуре ПринятьПакетОбмена() мы отклоняем изменения объектов Справочник Клиенты и Справочник ПричиныОтказа, полученные от планшета, если эти объекты уже изменялись в офисе. Во всех остальных случаях мы принимаем изменения полученного объекта.

После этого мы записываем полученные данные методом Записать().

После того как все сообщение обмена будет нами обработано, мы фиксируем транзакцию и заканчиваем чтение XML-данных из строки.

Как можно заметить, процедуры ПринятьИзменения (см. листинг 4.37), ПрочитатьXMLТовары (см. листинг 4.36), ПрочитатьXMLСклады (см. листинг 4.35), ПрочитатьДанные (см. листинг 4.34), ВозможностьЧтенияДанных (см. листинг 4.33), ЗаписатьXMLТовары (см. листинг 4.29), ЗаписатьXMLСклады (см. листинг 4.30), ЗаписатьДанные (см. листинг 4.28), УдалениеДанных (см. листинг 4.27) полностью идентичны одноименным процедурам, описанным нами в модуле плана обмена Мобильные при тестировании запасного способа обмена с мобильным приложением. Поэтому можно их оттуда убрать и вызывать через точку от имени общего модуля – например: Обмен.ПринятьИзменения(Отправитель, Данные). Обратите внимание, что эти процедуры должны быть объявлены, как экспортные.

Получение отчета

Еще раз напомним, что отчет формируется в основном приложении на основе данных интернет-магазина. Для получения отчета в мобильном приложении вызывается операция Web-сервиса, в которой отчет формируется с учетом переданных в нее параметров и результат отчета возвращается в виде значения XDTO.

Для этого в модуле Web-сервиса поместим функцию ПолучитьОтчет(), в которую передаются даты начала и окончания отчетного периода, а также объект СтрокаИнформацииРасшифровки для заполнения информации расшифровки отчета (листинг 4.38).

Листинг 4.38. Функция «ПолучитьОтчет()»

Функция ПолучитьОтчет(ДатаНачала, ДатаОкончания, СтрокаИнформацииРасшифровки)

```
Отчет = Отчеты.ОстаткиТоваровНаСкладах.Создать();

ПараметрыВывода = Отчет.КомпоновщикНастроек.Настройки.ПараметрыВывода;
ПараметрыВывода.УстановитьЗначениеПараметра("ВертикальноеРасположениеОбщихИтогов",
                                              РасположениеИтоговКомпоновкиДанных.Начало);
ПараметрыВывода.УстановитьЗначениеПараметра("ВыводитьЗаголовок",
                                              ТипВыводаТекстаКомпоновкиДанных.НеВыводить);
ПараметрыВывода.УстановитьЗначениеПараметра("ВыводитьПараметрыДанных",
                                              ТипВыводаТекстаКомпоновкиДанных.НеВыводить);

ПараметрДанныхНачалоПериода =
    Отчет.КомпоновщикНастроек.ПользовательскиеНастройки.Элементы[0];
ПараметрДанныхНачалоПериода.Значение = ДатаНачала;
ПараметрДанныхНачалоПериода.Использование = Истина;

ПараметрДанныхКонецПериода = Отчет.КомпоновщикНастроек.ПользовательскиеНастройки.Элементы[1];
ПараметрДанныхКонецПериода.Значение = ДатаОкончания;
ПараметрДанныхКонецПериода.Использование = Истина;

ТабличныйДокумент = Новый ТабличныйДокумент();
ДанныеРасшифровки = Неопределено;
Отчет.СкомпоноватьРезультат(ТабличныйДокумент, ДанныеРасшифровки);

ИнформацияРасшифровки = Новый Соответствие;
Для Каждого Элемент из ДанныеРасшифровки.Элементы Цикл
    Если ТипЗнч(Элемент) = Тип("ЭлементРасшифровкиКомпоновкиДанныхПоля") Тогда
        Поля = Элемент.ПолучитьПоля();
        Если Поля.Количество() > 0 Тогда
            ИнформацияРасшифровки.Вставить(Элемент.Идентификатор, Поля[0].Значение);
        КонецЕсли;
    КонецЕсли;
КонецЦикла;
СтрокаИнформацииРасшифровки = СериализаторXDTO.ЗаписатьXDTO(ИнформацияРасшифровки);

Возврат СериализаторXDTO.ЗаписатьXDTO(ТабличныйДокумент);
```

КонецФункции

В этой функции сначала создается отчет `ОстаткиТоваровНаСкладах`. Затем отменяется вывод заголовка и параметров отчета, а итоги отчета выводятся в начале. Это сделано для того, чтобы отчет был максимально виден на мобильном устройстве.

Затем параметры отчетного периода, которые мы включили в состав пользовательских настроек (см. раздел «Настройки» на стр. 236) при разработке отчета находятся по индексу (0, 1) в коллекции пользовательских настроек отчета. Это сделано для упрощения, так как мы знаем, что это – первая и вторая пользовательские настройки. Этим настройкам присваиваются соответственно значения параметров `ДатаНачала` и `ДатаОкончания`, переданных в операцию `Web-сервиса`.

После этого отчет формируется методом `СкомпоноватьРезультат()`. При этом результат отчета сохраняется в табличном документе `ТабличныйДокумент`, а данные расшифровки отчета записываются в объект `ДанныеРасшифровки`.

Далее создается соответствие `ИнформацияРасшифровки` и организуется цикл для обхода коллекции элементов данных расшифровки отчета. В этом цикле получают значения полей компоновки данных, содержащихся в элементе расшифровки, и добавляются в соответствие.

После выхода из цикла соответствие `ИнформацияРасшифровки` с помощью метода `ЗаписатьXDTO()` объекта `СериализаторXDTO` глобальной фабрики `XDTO` сериализуется в XML-строку и сохраняется в выходном параметре `СтрокаИнформацииРасшифровки` операции `Web-сервиса` `ПолучитьОтчет`.

Как результат выполнения этой операции в мобильное приложение возвращается сериализованный в `XDTO` табличный документ, содержащий данные отчета.

Публикация Web-сервиса на веб-сервере

Теперь нам осталось только сохранить конфигурацию базы данных и опубликовать наш `Web-сервис` на веб-сервере. Для этого выполним команду конфигуриатора `Администрирование > Публикация на веб-сервере ...` (рис. 4.26).

В появившемся диалоге в поле `Имя` зададим имя виртуального каталога на веб-сервере, в который будет выполнена публикация `Web-сервиса` (это имя будет использовать мобильное приложение для обращения к `Web-сервису`).

Поле `Веб-сервер` автоматически заполнилось единственным установленным на компьютере веб-сервером `Apache 2.2`.

В поле Каталог укажем физический каталог компьютера, в котором будет находиться файл публикации Web-сервиса.

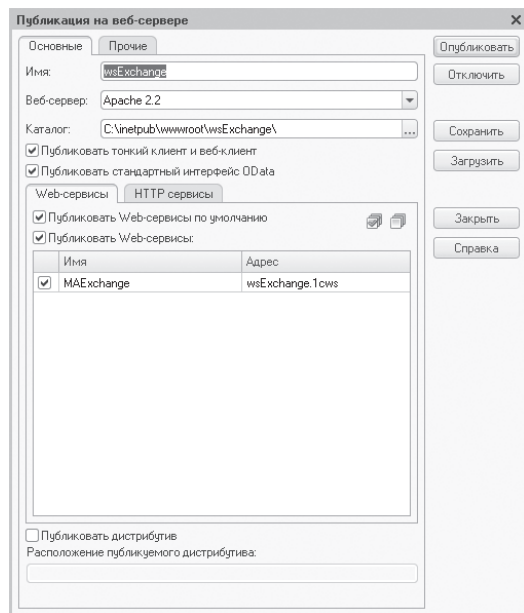


Рис. 4.26. Диалог публикации Web-сервиса

В списке сервисов появится созданный нами в конфигурации Web-сервис MAExchange. В поле Адрес укажем для него имя файла публикации – wsExchange.1cws.

Затем нажмем кнопку Опубликовать.

ВНИМАНИЕ

Если в качестве веб-сервера вы используете Internet Information Server (IIS), то, чтобы в файловом варианте работы пример реализации механизма Web-сервиса корректно работал, надо дать пользователю, от имени которого работает IIS (IUSR_<XXXX>, IWAM_<XXXX>), права на каталог информационной базы, из которой публикуется Web-сервис.

На этом все необходимые действия в основном приложении мы выполнили. Теперь покажем, как использовать эту функциональность в мобильном приложении.

Использование Web-сервиса в мобильном приложении

Теперь нам нужно доработать мобильное приложение так, чтобы из него можно было вызывать операции Web-сервиса для получения отчета и выполнения обмена данными с основным приложением.

Обмен данными

Откроем мобильное приложение КурьерИнтернетМагазина. Создадим в нем общую команду ОбменСЦентральнойБазой с синонимом Синхронизировать данные. Укажем группу для отображения команды в интерфейсе приложения — Панель действий.Сервис (рис. 4.27).

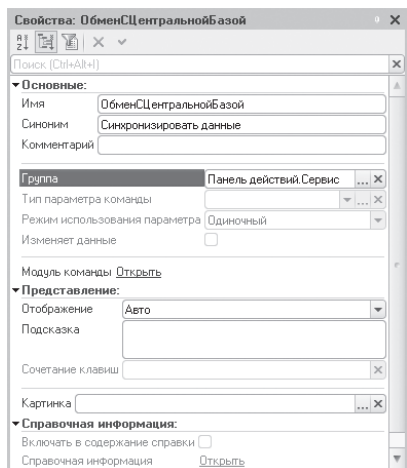


Рис. 4.27. Общая команда «ОбменСЦентральнойБазой»

Таким образом, команда для выполнения синхронизации данных будет находиться в группе Сервис главного меню мобильного приложения.

Откроем модуль команды и заполним обработчик команды следующим образом (листинг 4.39).

Листинг 4.39. Обработчик команды «ОбменСЦентральнойБазой()»

```
&НаКлиенте
Процедура ОбработкаКоманды(ПараметрКоманды, ПараметрыВыполненияКоманды)

    ТекстОшибки = "";
    Если Обмен.ВыполнитьОбменДанными(ТекстОшибки) Тогда
        ПоказатьПредупреждение(, "Синхронизация данных завершена.",5);
```

```
ОповеститьОБИзменении(Тип("ДокументСсылка.Заказ");  
ОповеститьОБИзменении(Тип("ДокументСсылка.ОбслуживаниеЗаказов"));  
ОповеститьОБИзменении(Тип("СправочникСсылка.Клиенты"));  
ОповеститьОБИзменении(Тип("СправочникСсылка.Товары"));  
ОповеститьОБИзменении(Тип("СправочникСсылка.ПричиныОтказа"));  
ОповеститьОБИзменении(Тип("СправочникСсылка.ХранимыеФайлы"));  
Иначе  
    ПоказатьПредупреждение(, ТекстОшибки);  
КонецЕсли;
```

КонецПроцедуры

В обработчике команды для выполнения синхронизации данных с основным приложением вызывается функция ВыполнитьОбменДанными(), которую мы поместим в общий модуль Обмен мобильного приложения (см. листинг 4.40).

Если эта функция возвращает Истину, то мы выводим сообщение об успешном завершении синхронизации данных и оповещаем об изменении динамические списки объектов, отображаемые в мобильном приложении. Если функция возвращает Ложь, то пользователю выводится текст ошибки, произошедшей при обмене данными.

Для доступа к Web-сервису нам понадобится строковая константа АдресЦентральнойБазы длиной 256 символов. В ней будет храниться имя виртуального каталога на веб-сервере, в который будет выполнена публикация Web-сервиса (рис. 4.28).

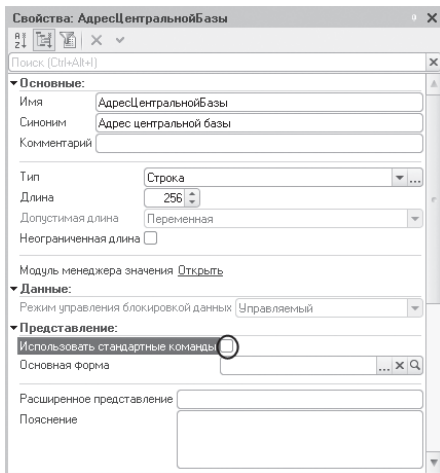


Рис. 4.28. Константа «АдресЦентральнойБазы»

А также нам будет нужна строковая константа КодЦентральнойБазы длиной 9 символов для хранения кода узла обмена основного приложения, с которым будет происходить синхронизация данных.

В палитре свойств у обеих констант снимем флажок Использовать стандартные команды. Для редактирования и записи значений этих констант мы создадим общую форму констант с именем Настройки.

В этой форме при запуске мобильного приложения мы установим значения констант для доступа к Web-сервису, опубликованному основным приложением: КодЦентральнойБазы – ЦО, АдресЦентральнойБазы – http://<IP-адрес компьютера в беспроводной сети>/<Каталог веб-сервера, на котором опубликован Web-сервис> (рис. 4.29).

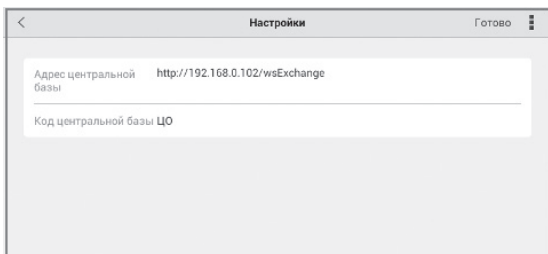


Рис. 4.29. Форма констант «Настройки»

Функцию ВыполнитьОбменДанными() в общем модуле Обмен заполним следующим образом (листинг 4.40).

Листинг 4.40. Функция «ВыполнитьОбменДанными()»

Функция ВыполнитьОбменДанными(ТекстОшибки) Экспорт

```

Прокси = ПолучитьПрокси(ТекстОшибки);
Если Прокси = Неопределено Тогда
    Возврат Ложь;
КонецЕсли;

КодЦБ = Константы.КодЦентральнойБазы.Получить();
ЦентральныйУзелОбмена = ПланыОбмена.Мобильные.НайтиПоКоду(КодЦБ);
Узел = ПланыОбмена.Мобильные.ЭтотУзел();

// Инициализируем обмен с центральной базой.
КодУзла = Прокси.НачатьОбмен(Узел.Код, ЦентральныйУзелОбмена.НомерПринятого,
                                ЦентральныйУзелОбмена.НомерОтправленного);

Если КодУзла = "" Тогда
    ТекстОшибки = "Узел с кодом " + Узел.Код + " не найден!";
    Возврат Ложь;
КонецЕсли;
```

```
// Отправляем данные.  
ДанныеОбмена = Обмен.СформироватьПакетОбмена(ЦентральныйУзелОбмена);  
Прокси.ЗаписатьДанные(Узел.Код, ДанныеОбмена);  
  
// Принимаем данные.  
ДанныеОбмена = Прокси.ПолучитьДанные(Узел.Код);  
Обмен.ПринятьПакетОбмена(ЦентральныйУзелОбмена, ДанныеОбмена);  
  
Возврат Истина;
```

КонецФункции

Сначала мы вызываем функцию `ПолучитьПрокси()`, в которой получаем описание Web-сервиса на основе динамической ссылки и пытаемся подключиться к Web-сервису (листинги 4.41, 4.42).

Листинг 4.41. Функция «ПолучитьПрокси()»

Функция `ПолучитьПрокси(ТекстОшибки)` Экспорт

```
ТекстОшибки = "";  
  
Адрес = Константы.АдресЦентральнойБазы.Получить();  
Адрес = Адрес + "/ws/wsExchange.1cws?wsdl";  
  
Попытка  
    Определения = Новый WSOпределения(Адрес, 60);  
Исключение  
    // Сообщим пользователю о том, что не получилось получить определение сервиса.  
    ТекстОшибки = "Не удалось установить соединение с сервером. Повторите попытку позже.";  
    Возврат Неопределено;  
КонецПопытки;  
  
URI = URIПространстваИменСервиса();  
Прокси = Новый WSПрокси(Определения, URI, "MAExchange", "MAExchangeSoap");  
  
Возврат Прокси;
```

КонецФункции

В данной процедуре создается объект `WSОпределения`, который получает определение Web-сервисов из WSDL-файла. Местоположение WSDL-файла складывается из каталога веб-сервера, на котором опубликован Web-сервис (значение константы `АдресЦентральнойБазы`), и имени файла публикации.

На основе определения Web-сервиса и его свойства `URI` пространства имен создается объект `WSПрокси` и связывает его с точкой подключения Web-сервиса.

Листинг 4.42. Функция для получения URI пространства имен Web-сервиса

```
Функция URIPространстваИменСервиса()
```

```
    Возврат "http://localhost/wsExchange";
```

```
КонецФункции
```

В случае успешного подключения к Web-сервису функция `ПолучитьПрокси()` возвращает объект `WSПрокси`, иначе – функция возвращает значение `Неопределено` и заполняет параметр `ТекстОшибки`.

Вернемся в функцию `ВыполнитьОбменДанными()`.

В случае успешного подключения к Web-сервису мы получаем значение константы `КодЦентральнойБазы`, находим по этому коду узел обмена в плане обмена `Мобильные` и сохраняем ссылку на этот узел в переменной `ЦентральныйУзелОбмена`. В переменной `Узел` сохраняем ссылку на предопределенный узел обмена – базу мобильного приложения (далее мы будем называть его «мобильный узел обмена»).

С помощью объекта `Прокси`, подключенному к Web-сервису, мы получаем доступ к операциям Web-сервиса, определенным в основном приложении.

Сначала мы вызываем операцию `НачатьОбмен` и передаем в нее код мобильного узла обмена и номера сообщений, принятых/отправленных центральным узлом обмена. При выполнении этой операции инициализируется обмен между основным и мобильным приложением (см. листинг 4.22), в случае необходимости синхронизируются номера принятых и отправленных сообщений в узлах обмена и регистрируются изменения центральной базы для мобильного узла обмена.

Затем мы выполняем обмен данными с мобильным приложением в обе стороны.

Сначала вызываем функцию `СформироватьПакетОбмена()` для формирования в мобильном приложении пакета изменений, предназначенных для центрального узла обмена. Эту функцию мы поместим в общий модуль `Обмен` и рассмотрим чуть позже (см. листинг 4.43).

Данные обмена, помещенные в хранилище значения и возвращенные этой функцией, и код мобильного узла обмена мы передаем в операцию Web-сервиса `ЗаписатьДанные`. При выполнении этой операции изменения принятые от мобильного узла обмена записываются в центральную базу данных (см. листинг 4.31).

Таким образом, мы передали данные от мобильного приложения в центральную базу. Теперь выполним обмен в обратном направлении.

Для этого мы вызываем операцию Web-сервиса `ПолучитьДанные` и передаем в нее код мобильного узла обмена. При выполнении этой операции в центральной базе формируется пакет изменений, предназначенных для мобильного узла обмена. В результате в мобильное приложение возвращаются данные обмена, помещенные в хранилище значения (см. листинг 4.24).

Затем мы вызываем процедуру `ПринятьПакетОбмена()` для записи изменений, принятых от центрального узла обмена, в базу мобильного приложения. Эту функцию мы поместим в общий модуль `Обмен` и рассмотрим чуть позже (см. листинг 4.44).

Теперь приведем функции `СформироватьПакетОбмена()`, `ПринятьПакетОбмена()` из общего модуля `Обмен` мобильного приложения (листинги 4.43, 4.44)

Листинг 4.43. Функция «`СформироватьПакетОбмена()`»

Функция `СформироватьПакетОбмена(УзелОбмена)` Экспорт

```
ЗаписьXML = Новый ЗаписьXML;  
  
ЗаписьXML.УстановитьСтроку("UTF-8");  
ЗаписьXML.ЗаписатьОбъявлениеXML();  
  
ЗаписьСообщения = ПланыОбмена.СоздатьЗаписьСообщения();  
ЗаписьСообщения.НачатьЗапись(ЗаписьXML, УзелОбмена);  
  
ЗаписьXML.ЗаписатьСоответствиеПространстваИмен("xsi", "http://www.w3.org/2001/XMLSchema-instance");  
ЗаписьXML.ЗаписатьСоответствиеПространстваИмен("v8", "http://v8.1c.ru/data");  
  
ВыборкаИзменений = ПланыОбмена.ВыбратьИзменения(УзелОбмена,  
ЗаписьСообщения.НомерСообщения);  
  
Пока ВыборкаИзменений.Следующий() Цикл  
    Данные = ВыборкаИзменений.Получить();  
  
    // Записываем данные в сообщение.  
    ЗаписатьДанные(ЗаписьXML, Данные);  
  
КонецЦикла;  
  
ЗаписьСообщения.ЗакончитьЗапись();  
  
Возврат Новый ХранилищеЗначения(ЗаписьXML.Закреть(), Новый СжатиеДанных(9));
```

КонецФункции

Эта функция практически аналогична одноименной функции основного приложения (листинг 4.25), поэтому мы не будем еще раз здесь ее разбирать. Единственное отличие в том, что здесь мы не анализируем данные на предмет переноса. Мы записываем в пакет изменений для центральной базы все изменения, полученные от планшета.

Текст и назначение процедур `ЗаписатьДанные()` (см. листинг 4.28), `ЗаписатьXMLТовары()` (см. листинг 4.29) и `ЗаписатьXMLСклады()` (см. листинг 4.30) уже рассматривались нами в предыдущем разделе. Поэтому мы их не будем здесь еще раз повторять, но в общий модуль `Обмен` их надо внести обязательно.

Листинг 4.44. Процедура «ПринятьПакетОбмена()»

Процедура `ПринятьПакетОбмена(УзелОбмена, ДанныеОбмена)` Экспорт

```
ЧтениеXML = Новый ЧтениеXML;
ЧтениеXML.УстановитьСтроку(ДанныеОбмена.Получить());
ЧтениеСообщения = ПланыОбмена.СоздатьЧтениеСообщения();
ЧтениеСообщения.НачатьЧтение(ЧтениеXML);

ПланыОбмена.УдалитьРегистрациюИзменений(ЧтениеСообщения.Отправитель,
ЧтениеСообщения.НомерПринятого);

НачатьТранзакцию();
Пока ВозможностьЧтенияДанных(ЧтениеXML) Цикл

    Данные = ПрочитатьДанные(ЧтениеXML);

    Если Не Данные = Неопределено Тогда
        // Не переносим изменение, полученное из офиса, если есть регистрация изменения
        // на планшете.
        Если Не ПринятьИзменения(ЧтениеСообщения.Отправитель, Данные) Тогда
            Продолжить;
        КонецЕсли;

        Данные.ОбменДанными.Отправитель = ЧтениеСообщения.Отправитель;
        Данные.ОбменДанными.Загрузка = Истина;

        Данные.Записать();
    КонецЕсли;

КонецЦикла;
ЗафиксироватьТранзакцию();

ЧтениеСообщения.ЗакончитьЧтение();
ЧтениеXML.Закрыть();

КонецПроцедуры
```

Эта функция практически аналогична одноименной функции основного приложения (листинг 4.32), поэтому мы не будем еще раз здесь ее разбирать.

Текст и назначение функций `ВозможностьЧтенияДанных()` (см. листинг 4.33), `ПрочитатьДанные()` (см. листинг 4.34), `ПрочитатьXMLТовары()` (см. листинг 4.36), `ПрочитатьXMLСклады()` (см. листинг 4.35) уже рассматривались нами в предыдущем разделе. Поэтому мы их не будем здесь еще раз повторять, но в общий модуль `Обмен` их надо внести обязательно.

Отличие состоит в процедуре `ПринятьИзменения()` для разрешения коллизий (листинг 4.45).

Листинг 4.45. Функция для разрешения коллизии при обмене данными

Функция `ПринятьИзменения(Отправитель, Данные)` Экспорт

Прием = Истина;

Если `ПланыОбмена.ИзменениеЗарегистрировано(Отправитель, Данные)` Тогда

Если `ТипЗнч(Данные) = Тип("ДокументОбъект.Заказ")` Тогда

Прием = Ложь;

КонецЕсли;

КонецЕсли;

Возврат Прием;

КонецФункции

В этой функции проверяется, зарегистрировано ли изменение объекта в узле обмена (на планшете), для которого отправлены данные. Если да и если тип объекта `ДокументОбъект.Заказ`, то функция возвращает `Ложь`, во всех остальных случаях возвращается `Истина`.

Таким образом, в процедуре `ПринятьПакетОбмена()` мы отклоняем изменения объекта `Документ Заказ`, полученные из офиса, если этот объект уже изменялся на планшете. Во всех остальных случаях мы принимаем изменения полученного объекта.

Процедуры `ПринятьИзменения`, `ПрочитатьXMLТовары`, `ПрочитатьXMLСклады`, `ПрочитатьДанные`, `ВозможностьЧтенияДанных`, `ЗаписатьXMLТовары`, `ЗаписатьXMLСклады`, `ЗаписатьДанные` полностью идентичны одноименным процедурам, описанным нами в модуле плана обмена `Мобильные` при тестировании запасного способа обмена с основным приложением. Поэтому можно их оттуда убрать и вызывать через точку от имени общего модуля, например: `Обмен.ПринятьИзменения(Отправитель, Данные)`. Обратите внимание, что эти процедуры должны быть объявлены, как экспортные.

Проверим, как работает обмен данных с основным приложением на планшете. Команду `ОбменСЦентральнойБазой`, инициирующую обмен, мы поместили в группу `Сервис` главного меню мобильного приложения и установили для нее синоним `Синхронизировать данные`.

Запустим мобильное приложение на планшете. На рабочем столе нашего приложения в нижней части экрана находятся команды интерфейса основного раздела. Нажмем на картинку команды `Настройки`.

Здесь мы должны задать настройки приложения, необходимые для доступа к Web-сервису и выполнения синхронизации данных. Установим значения этих настроек: КодЦентральнойБазы – ЦО, АдресЦентральнойБазы – `http://<IP-адрес компьютера в беспроводной сети>/<Каталог веб-сервера, на котором опубликован Web-сервис>` и нажмем кнопку Готово в правом верхнем углу формы настроек (рис. 4.30).

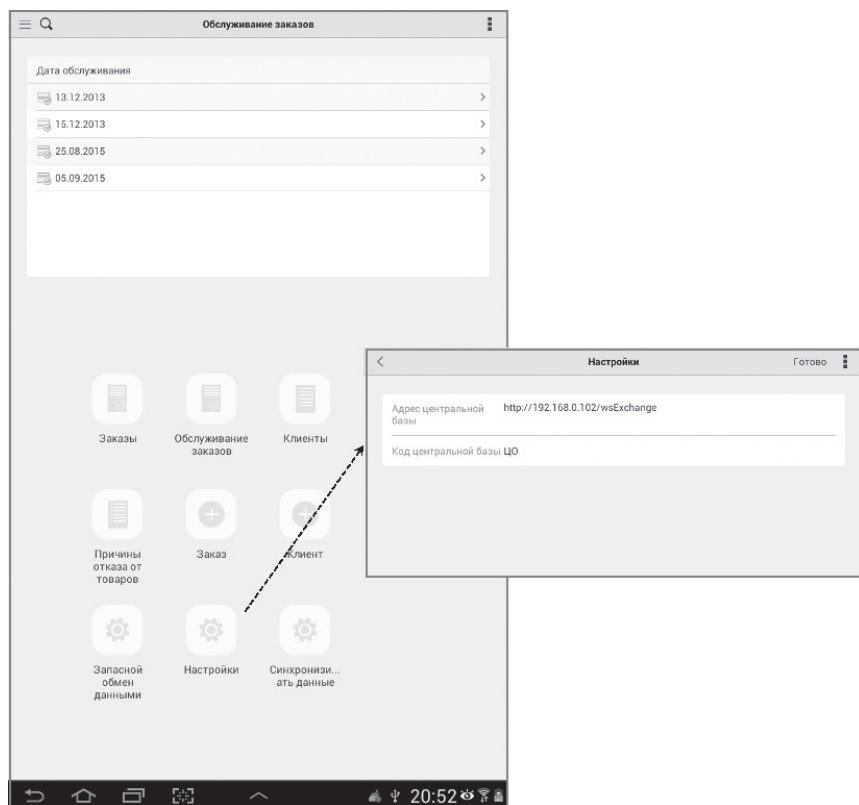


Рис. 4.30. Вызов формы констант «Настройки»

Затем выполним команду Синхронизировать данные (ее можно вызвать из списка команд на рабочем столе или из главного меню приложения), см. рис. 4.31.

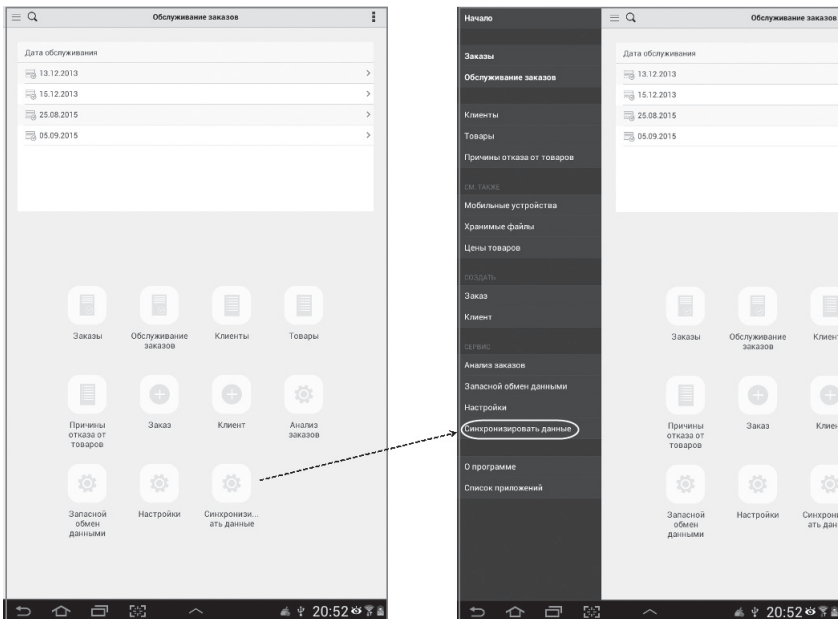


Рис. 4.31. Вызов команды «Синхронизировать данные»

После некоторого ожидания (поскольку происходит обращение к Web-сервису) мы увидим сообщение о завершении синхронизации данных. При этом все динамические списки (списки заказов, товаров, клиентов и т. п.) автоматически обновятся с учетом полученных данных.

В случае ошибки при соединении с веб-сервером будет выведено сообщение с ошибкой.

Получение отчета

Для получения отчета в мобильном приложении, прежде всего, создадим обработку ОстаткиТоваров, в которой будет отображаться наш отчет.

В форме обработки создадим команду Сформировать, по которой отчет будет удаленно формироваться в основном приложении, и перетащим ее в командную панель формы.

А также создадим реквизиты формы ДатаНачала и ДатаОкончания, в которых будут храниться даты отчетного периода. Добавим в форму горизонтальную группу без отображения ОтчетныйПериод и перетащим туда эти реквизиты.

Затем создадим реквизит **Содержимое** типа **ТабличныйДокумент**, который будет содержать данные отчета. Перетащим его в форму и установим у соответствующего элемента формы свойство **Положение заголовка** в значение **Нет**.

Кроме того, создадим реквизит **ИнформацияРасшифровки** типа **Строка**, который будет содержать адрес во временном хранилище, по которому хранятся данные о расшифровке отчета.

В результате форма обработки примет следующий вид (рис. 4.32).

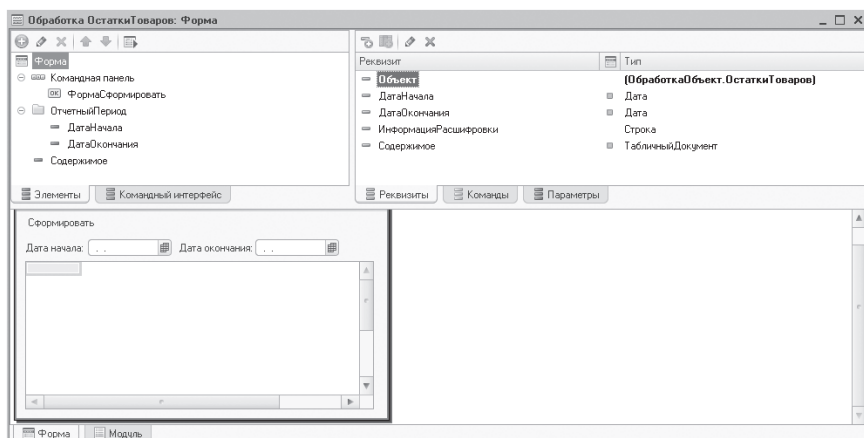


Рис. 4.32. Форма обработки «ОстаткиТоваров» в конфигураторе

Затем создадим клиентский обработчик команды **Сформировать** и заполним его следующим образом (листинг 4.46).

Листинг 4.46. Обработчик команды «Сформировать»

```
&НаКлиенте
Процедура Сформировать(Команда)

    ТекстОшибки = ПолучитьОтчет();
    Если ТекстОшибки <> "" Тогда
        ПоказатьПредупреждение(, ТекстОшибки);
    КонецЕсли;

КонецПроцедуры
```

В этом обработчике вызывается серверная функция **ПолучитьОтчет()**, которая получает отчет и возвращает текст ошибки (листинг 4.47). Если текст ошибки заполнен, то он выводится пользователю.

Листинг 4.47. Функция для получения отчета

```
&НаСервере
Функция ПолучитьОтчет()

    Если ИнформацияРасшифровки <> "" Тогда
        УдалитьИзВременногоХранилища(ИнформацияРасшифровки);
    КонецЕсли;

    ДанныеРасшифровки = Неопределено;
    ТекстОшибки = "";

    Прокси = Обмен.ПолучитьПрокси(ТекстОшибки);
    Если Прокси = Неопределено Тогда
        Возврат ТекстОшибки;
    КонецЕсли;

    Сериализатор = Новый СериализаторXDTO(Прокси.ФабрикаXDTO);
    ИнформацияРасшифровкиXDTO = Неопределено;
    ДокументXDTO = Прокси.ПолучитьОтчет(ДатаНачала, ДатаОкончания, ИнформацияРасшифровкиXDTO);
    ДанныеРасшифровки = Сериализатор.ПрочитатьXDTO(ИнформацияРасшифровкиXDTO);

    Содержимое = Сериализатор.ПрочитатьXDTO(ДокументXDTO);

    Если ДанныеРасшифровки = Неопределено Тогда
        ИнформацияРасшифровки = "";
    Иначе
        ИнформацияРасшифровки = ПоместитьВоВременноеХранилище(ДанныеРасшифровки,
                                                                УникальныйИдентификатор);
    КонецЕсли;

    Возврат ТекстОшибки;

КонецФункции
```

В этой функции мы сначала удаляем из временного хранилища данные расшифровки отчета, если они уже были раньше заполнены.

Затем мы подключаемся к Web-сервису с помощью функции ПолучитьПрокси() общего модуля Обмен, как было подробно объяснено в предыдущем разделе (см. листинг 4.41).

После этого мы создаем объект СериализаторXDTO на основе фабрики XDTO, содержащей определения всех типов XDTO, доступных нашему Web-сервису. Этот объект нужен для чтения данных XDTO, возвращенных при выполнении отчета, и последующей сериализации данных XML в значения платформ «1С:Предприятие».

Затем выполняется операция Web-сервиса ПолучитьОтчет, в которой в основном приложении по переданным параметрам строится отчет ОстаткиТоваровНаСкладах и в виде объекта XDTO возвращается обратно. А также выходной параметр ИнформацияРасшифровкиXDTO операции Web-сервиса ПолучитьОтчет заполняется данными о расшифровке отчета.

После этого данные XDTO читаются с помощью метода ПрочитатьXDTO() объекта Сериализатор. В итоге реквизит формы Содержимое будет содержать результат отчета в виде табличного документа, а переменная ДанныеРасшифровки будет содержать информацию о расшифровке отчета в виде соответствия, которую мы поместим во временное хранилище по адресу, хранящемуся в реквизите формы ИнформацияРасшифровки.

Проверим, как работает наш отчет. Из главного меню мобильного приложения вызовем обработку Остатки товаров и нажмем кнопку Сформировать. В результате отчет примет следующий вид (рис. 4.33).

Остатки товаров							Сформировать
Дата начала		.. Дата окончания					..
Склад	Заказано	Начальный остаток	Приход	Расход	Конечный остаток	В наличии	
Товар							
Размер							
Цвет							
Итого	8		49	7	42	34	
Большой	8		49	7	42	34	
Джинсы	4		15	2	13	9	
44	2		10	2	8	6	
Белый	2		5		5	3	
Черный			5	2	3	3	
46	2		5		5	3	
Черный	2		5		5	3	
Платье	1		8	1	7	6	
44	1		2		2	1	
Красный	1		2		2	1	
46			6	1	5	5	
Белый			3		3	3	
Фиолетовый			3	1	2	2	
Сапоги	1		11	1	10	9	
37	1		5	1	4	3	
Черный	1		5	1	4	3	
38			6		6	6	
Белый			3		3	3	
Черный							
Фиолетовый			3		3	3	
Туфли	2		15	3	12	10	
37			5		5	5	
Черный			5		5	5	
38	2		10	3	7	5	
Черный			5	2	3	3	
Красный	2		5	1	4	2	

Рис. 4.33. Результат отчета «Остатки товаров»

Теперь нам осталось реализовать стандартное поведение результата отчета, при котором, выбрав поле табличного документа быстрым касанием, можно открыть его значение.

Для этого создадим для элемента Содержимое формы обработки ОстаткиТоваров клиентский обработчик события ОбработкаРасшифровки и заполним его следующим образом (листинг 4.48).

Листинг 4.48. Обработчик события «ОбработкаРасшифровки» для поля формы «Содержимое»

```
&НаКлиенте
Процедура СодержимоеОбработкаРасшифровки(Элемент, Расшифровка, СтандартнаяОбработка)

    СтандартнаяОбработка = Ложь;
    Если ИнформацияРасшифровки <> "" Тогда
        ДанныеРасшифровки = ПолучитьИзВременногоХранилища(ИнформацияРасшифровки);
        Если ДанныеРасшифровки <> Неопределено Тогда
            Значение = ДанныеРасшифровки.Получить(Расшифровка);
            Если Значение <> Неопределено И Значение <> Null Тогда
                ПоказатьЗначение(, Значение);
            КонецЕсли;
        КонецЕсли;
    КонецЕсли;
КонецПроцедуры
```

В этом обработчике мы сначала отменяем стандартную обработку расшифровки табличного документа, так как это не отчетная форма, а форма обработки.

Затем мы получаем данные расшифровки в виде соответствия из временного хранилища по адресу, сохраненному в реквизите формы ИнформацияРасшифровки.

После этого значение расшифровки получается из соответствия ДанныеРасшифровки по ключу, который передается в параметре события Расшифровка, и открывается немодальным методом глобального контекста ПоказатьЗначение().

Итак, выполним отчет, быстрым касанием выберем одну из ячеек (например, с названием товара) и нажмем кнопку Расшифровать в командной панели, появившейся вверху формы обработки. В результате будет открыта форма товара, название которого отображено в выбранной ячейке отчета (рис. 4.34).

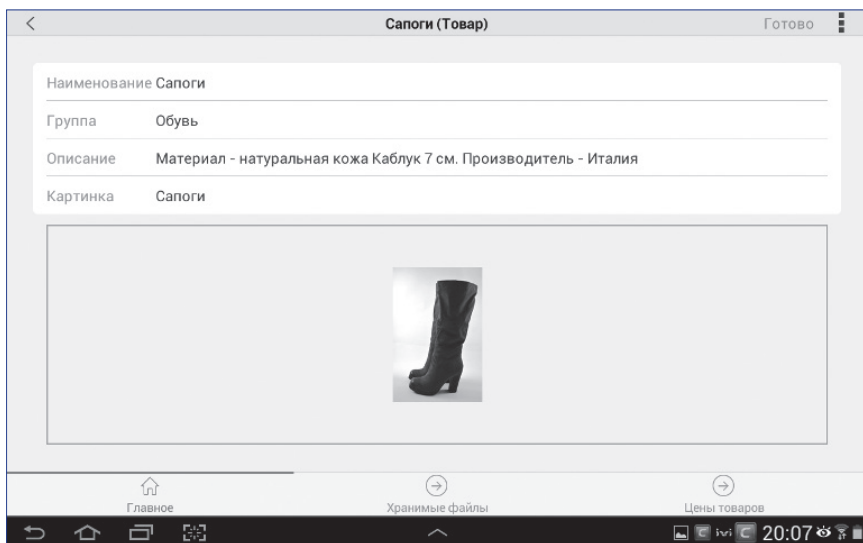


Рис. 4.34. Расшифровка ячейки отчета

Автоматическая синхронизация данных

При разработке формы заказа мы предусмотрели возможность оповещения мобильного приложения об изменении заказа, в случае если заказ записывался с проведением (листинг 4.49).

Листинг 4.49. Обработчик события формы документа «ПослеЗаписи»
формы документа «Заказ»

```
&НаКлиенте
Процедура ПослеЗаписи(ПараметрыЗаписи)

    Если ПараметрыЗаписи.РежимЗаписи = РежимЗаписиДокумента.Проведение Тогда
        Оповестить("ИзменилсяЗаказ");
    КонецЕсли;

КонецПроцедуры
```

В результате при записи заказа с проведением мобильное приложение оповещается о том, что изменился заказ. По условию нашей задачи после этого нужно выполнить автоматическую синхронизацию данных с основным приложением.

Для того чтобы мобильное приложение могло обработать оповещение, посланное формой заказа, нужно подключить обработчик оповещения

в модуле управляемого приложения в обработчике события ПриНачалеРаботыСистемы. Мы уже создавали этот обработчик для обработки локальных напоминаний при разработке формы клиента (см. раздел «Напоминание о звонке (локальное уведомление)» на стр. 141). Внесем в него следующие изменения (листинг 4.50).

Листинг 4.50. Обработчик события «ПриНачалеРаботыСистемы»
в модуле управляемого приложения

Процедура ПриНачалеРаботыСистемы()

```
ОписаниеОповещения = Новый ОписаниеОповещения("ОбработкаУведомлений", УведомленияКлиент);  
ДоставляемыеУведомления.ПодключитьОбработчикУведомлений(ОписаниеОповещения);
```

```
ПодключитьОбработчикОповещения("ОбработчикОповещения");
```

КонецПроцедуры

Затем в модуле управляемого приложения поместим саму процедуру обработчика оповещения, в которой выполняется код общей команды для обмена данными ОбменСЦентральнойБазой (листинг 4.51).

Листинг 4.51. Обработчик оповещения

Процедура ОбработчикОповещения(ИмяСобытия, Параметр, Источник) Экспорт

```
Если ИмяСобытия = "ИзменилсяЗаказ" Тогда  
    ТекстОшибки = "";  
    Если Обмен.ВыполнитьОбменДанными(ТекстОшибки) Тогда  
        ПоказатьПредупреждение(, "Синхронизация данных завершена.", 5);  
        ОповеститьОБИзменении(Тип("ДокументСсылка.Заказ"));  
        ОповеститьОБИзменении(Тип("ДокументСсылка.ОбслуживаниеЗаказов"));  
        ОповеститьОБИзменении(Тип("СправочникСсылка.Клиенты"));  
        ОповеститьОБИзменении(Тип("СправочникСсылка.Товары"));  
        ОповеститьОБИзменении(Тип("СправочникСсылка.ПричиныОтказа"));  
        ОповеститьОБИзменении(Тип("СправочникСсылка.ХранимыеФайлы"));  
    Иначе  
        ПоказатьПредупреждение(, ТекстОшибки);  
    КонецЕсли;  
КонецЕсли;
```

КонецПроцедуры

Таким образом, как только курьер выполнит заказ и нажмет кнопку Готово, будет выполнена автоматическая синхронизация данных с основным приложением. Тем самым, с одной стороны, добавляется еще одна удобная возможность для курьера, а с другой стороны, обеспечивается оперативность информации как на планшете, так и в интернет-магазине.

Работа с push-уведомлениями

В процессе работы интернет-магазина может возникнуть необходимость срочно проинформировать курьера, работающего удаленно на мобильном устройстве, о наступлении каких-то событий в офисе. Например, о том, что изменилась важная для курьера информация и что надо выполнить обмен данными между основным и мобильным приложением.

Такие уведомления, которые отправляются из центральной базы удаленным мобильным устройствам, называются push-уведомлениями.

Общая информация

Прежде всего, надо понимать, что в получении и рассылке push-уведомлений (далее просто «уведомлений») участвуют три стороны:

- Собственно мобильное приложение, которое получает уведомления и реализует реакцию на них со стороны мобильного устройства.
- Прикладное решение (созданное на платформе «1С:Предприятия»), которое занимается рассылкой уведомлений. Оно выступает в роли отправителя уведомлений.
- Сервис доставки уведомлений. Мы для простоты и удобства будем использовать специальный вспомогательный сервис для отправки уведомлений, разработанный фирмой «1С». Сервис расположен по адресу <https://pushnotifications.1c.com>.

В общем случае отправка уведомления выглядит следующим образом:

- отправитель формирует уведомление;
- отправитель определяет список получателей уведомления;
- отправитель подключается к сервису уведомлений;
- отправитель передает сервису уведомление и список получателей;
- сервис обеспечивает доставку уведомлений на мобильные устройства;
- мобильное приложение на устройстве обеспечивает обработку уведомлений.

Чтобы получение и рассылка push-уведомлений стали возможны, мобильное приложение и отправитель должны «уметь» взаимодействовать друг с другом через некоторый публичный программный интерфейс (веб-сервис):

- отправитель уведомления должен опубликовать веб-сервис и получить через него идентификатор получателя уведомлений от мобильного приложения;

- получатель уведомления (мобильное приложение) должен обратиться к веб-сервису и передать через него идентификатор получателя уведомлений основному приложению.

ПОДРОБНЕЕ

Документация 1С:Предприятие 8.3. Руководство разработчика. Глава 26. Разработка решений для мобильной платформы > Особенности использования > Специальные возможности > Работа с уведомлениями > Push-уведомления.

Пример работы с push-уведомлениями

Особенности получения и рассылки push-уведомлений мы рассмотрим на конкретном примере.

Предположим, в заказах, уже находящихся в работе у курьера на планшете, изменилась какая-то важная информация. Например, клиент позвонил в офис и изменил адрес или время доставки. После записи изменений заказа в офисе основное приложение должно послать курьеру, обслуживающему данный заказ, сообщение с информацией об этих изменениях.

Или, например, в офисе был добавлен новый товар. В этом случае основное приложение должно послать всем мобильным приложениям, получающим от него уведомления, сообщение об этом с предложением выполнить обмен данными с основным приложением.

Основное приложение

Для реализации первой задачи в основном приложении откроем форму объекта документа Заказ и добавим в нее два новых реквизита: ВажнаяИнформация типа Строка (в нем будет храниться информация, передаваемая курьеру) и ОтправитьУведомление типа Булево (в нем будет храниться признак необходимости отправки уведомления).

При создании формы заказа в обработчике события формы ПриСозданииНаСервере признак отправки уведомлений ОтправитьУведомление установим в значение Ложь (листинг 4.52).

Листинг 4.52. Обработчик события формы «ПриСозданииНаСервере»

```
&НаСервере  
Процедура ПриСозданииНаСервере(Отказ, СтандартнаяОбработка)
```

```
    ОтправитьУведомление = Ложь;  
    ПересчитатьИтоговуюСумму();
```

```
КонецПроцедуры
```

Затем создадим обработчики событий ПриИзменении полей формы АдресДоставки и ДополнительнаяИнформация и заполним их следующим образом (листинги 4.53, 4.54).

Листинг 4.53. Обработчик события при изменении поля формы «АдресДоставки»

```
&НаКлиенте
Процедура АдресДоставкиПриИзменении(Элемент)

    Если Объект.СтатусЗаказа =
        ПредопределенноеЗначение("Перечисление.СтатусыЗаказов.ВРаботе") Тогда
        ОтправитьУведомление = Истина;
        Если ВажнаяИнформация <> "" Тогда
            ВажнаяИнформация = ВажнаяИнформация + Символы.ПС +
                "Адрес доставки: " + СокрЛП(Объект.АдресДоставки);
        Иначе
            ВажнаяИнформация = "Адрес доставки: " + СокрЛП(Объект.АдресДоставки);
        КонецЕсли;
    КонецЕсли;

КонецПроцедуры
```

Листинг 4.54. Обработчик события при изменении поля формы
«ДополнительнаяИнформация»

```
&НаКлиенте
Процедура ДополнительнаяИнформацияПриИзменении(Элемент)

    Если Лев(Объект.ДополнительнаяИнформация, 1) = "!" И Объект.СтатусЗаказа =
        ПредопределенноеЗначение("Перечисление.СтатусыЗаказов.ВРаботе") Тогда
        ОтправитьУведомление = Истина;
        Если ВажнаяИнформация <> "" Тогда
            ВажнаяИнформация = ВажнаяИнформация + Символы.ПС +
                "Дополнительно: " + СокрЛП(Объект.ДополнительнаяИнформация);
        Иначе
            ВажнаяИнформация = "Дополнительно: " + СокрЛП(Объект.ДополнительнаяИнформация);
        КонецЕсли;
    КонецЕсли;

КонецПроцедуры
```

В этих обработчиках, если заказы находятся в работе у курьера (статус заказа – В работе) и если заказ содержит важную информацию (реквизит ДополнительнаяИнформация начинается с символа «!»), признак отправки уведомления ОтправитьУведомление устанавливается в значение Истина и заполняется строковый реквизит ВажнаяИнформация, который после записи заказа будет передан курьеру в тексте уведомления.

Собственно формирование и отправку уведомления мы реализуем в обработчике события формы заказа ПослеЗаписиНаСервере. Создадим его и заполним следующим образом (листинг 4.55).

Листинг 4.55. Обработчик события формы «ПослеЗаписиНаСервере»

```
&НаСервере
Процедура ПослеЗаписиНаСервере(ТекущийОбъект, ПараметрыЗаписи)

    Если ОтправитьУведомление = Истина Тогда
        Уведомление = Новый ДоставляемоеУведомление();
        Уведомление.Текст = "Заказ " + ТекущийОбъект.Номер +
            " - Изменилась важная информация." + Символы.ПС + ВажнаяИнформация;
        РаботаСДоставляемымиУведомлениям.ОтправитьУведомление(Уведомление,
                                                                    ТекущийОбъект.Курьер);
    КонецЕсли;
КонецПроцедуры
```

В этом обработчике, если признак отправки уведомления установлен, мы создаем объект ДоставляемоеУведомление, заполняем его свойство Текст так, чтобы проинформировать курьера, что в заказе изменилась важная информации (что именно изменилось, содержится в реквизите ВажнаяИнформация).

Затем мы вызываем процедуру ОтправитьУведомление общего модуля РаботаСДоставляемымиУведомлениями, в которую передаем само уведомление и ссылку на курьера, которому его надо доставить. В этой процедуре (мы рассмотрим ее позже, см. листинг 4.60) и происходит определение списка получателей уведомления и отправка уведомления через вспомогательный сервис «1С».

Теперь реализуем второй пример – рассылку уведомлений при добавлении в офис нового товара.

Для этого откроем форму элемента справочника Товары, создадим у формы обработчик события ПередЗаписьюНаСервере и заполним его следующим образом (листинг 4.56).

Листинг 4.56. Обработчик события формы «ПередЗаписьюНаСервере»

```
&НаСервере
Процедура ПередЗаписьюНаСервере(Отказ, ТекущийОбъект, ПараметрыЗаписи)

    Если ТекущийОбъект.ЭтоНовый() Тогда
        ПараметрыЗаписи.Вставить("Уведомление", Истина);
    КонецЕсли;
КонецПроцедуры
```

В этом обработчике, в случае если записывается новый элемент справочника, мы добавляем в структуру ПараметрыЗаписи элемент Уведомление со значением Истина.

Затем создадим у формы обработчик события ПослеЗаписиНаСервере и заполним его следующим образом (листинг 4.57).

Листинг 4.57. Обработчик события формы «ПослеЗаписиНаСервере»

```
&НаСервере
Процедура ПослеЗаписиНаСервере(ТекущийОбъект, ПараметрыЗаписи)

    Если ПараметрыЗаписи.Свойство("Уведомление", Истина) Тогда
        Уведомление = Новый ДоставляемоеУведомление();
        Уведомление.Текст = "Добавлен новый товар";
        Уведомление.Данные = "1";
        РаботаСДоставляемымиУведомлениям.ОтправитьУведомление(Уведомление, Неопределено);
    КонецЕсли;

КонецПроцедуры
```

В этом обработчике, в случае если в параметрах записи содержится признак записи нового товара, мы создаем объект ДоставляемоеУведомление, заполняем его свойство Текст, а свойству Данные присваиваем значение «1». При получении уведомления с такими данными мобильное приложение «поймет», что ему надо выполнить обмен данными с основным приложением.

Затем мы вызываем процедуру ОтправитьУведомление общего модуля РаботаСДоставляемымиУведомлениями, в которую передаем само уведомление и значение Неопределено, поскольку информация о новом товаре актуальна для любого курьера, работающего с мобильным приложением. В этой процедуре (мы рассмотрим ее позже, см. листинг 4.60) и происходит определение списка получателей уведомления и его отправка.

Чтобы отправить уведомление мобильному приложению, необходимо настроить параметры подключения основного приложения к сервисам рассылки уведомлений. Как уже говорилось, для отправки уведомлений мы будем использовать специальный вспомогательный сервис, разработанный фирмой «1С».

Чтобы подключиться к нему, нужно будет перед рассылкой уведомлений зарегистрировать отправителя уведомлений на этом сервисе по адресу <https://pushnotifications.1c.com> и получить для него ключ доступа, который будет храниться в константе КлючДоступаОтправителя.

Добавим эту константу в основное приложение. Установим ее свойства: Синоним — Ключ доступа, Тип — Строка, Длина — 54, Использовать стандартные команды — Нет (рис. 4.35).

Создадим общую форму констант с именем Настройки. Чтобы сгенерировать ключ отправителя на вспомогательном сервисе «1С», добавим

в форму команду `ПолучитьКлюч` и перетащим ее в командную панель формы настроек. Создадим обработчик этой команды и заполним ее следующим образом (листинг 4.58).

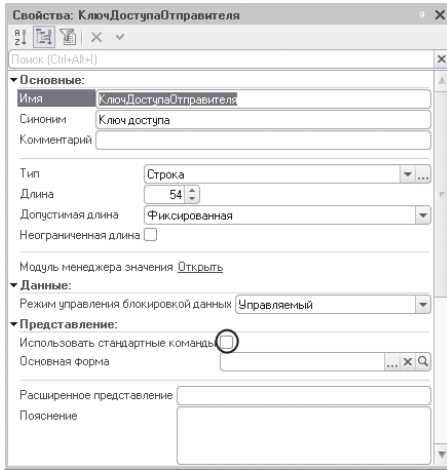


Рис. 4.35. Константа «КлючДоступаОтправителя»

Листинг 4.58. Обработчик команды «ПолучитьКлюч»

```
&НаКлиенте
Процедура ПолучитьКлюч(Команда)

    ПерейтиПоНавигационнойСсылке("https://pushnotifications.1c.com/push/publishers/new");

КонечПроцедуры
```

Таким образом, нажав на кнопку `Получить` ключ в форме `Настройки`, специалист по внедрению сможет перейти на сайт сервиса «1С», зарегистрироваться там (если это еще не было сделано) и зарегистрировать приложение – отправителя уведомлений. Полученный ключ нужно будет скопировать и вставить в форму настроек в поле `Ключ доступа`.

Чтобы сделать интерфейс более дружелюбным к тем, кто будет настраивать для приложения возможность рассылки уведомлений, можно добавить расширенную подсказку для поля `КлючДоступаОтправителя`.

У этого поля вызовем из контекстного меню пункт `Показать расширенную подсказку` и заполним свойство `Заголовок` этой подсказки поясняющим текстом (рис. 4.36).

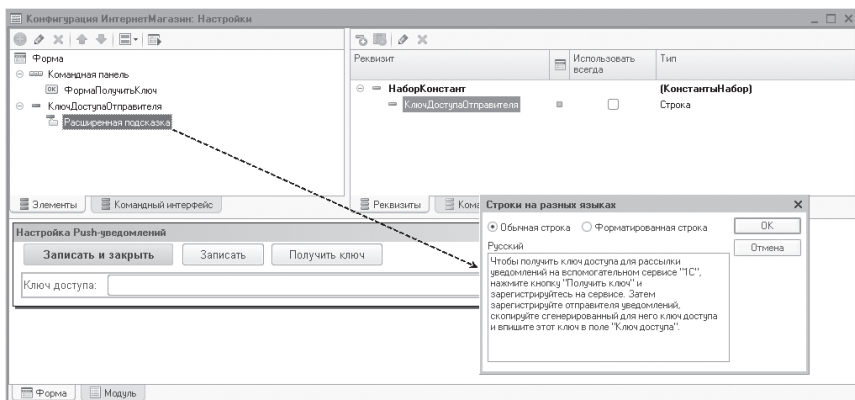


Рис. 4.36. Форма «Настройки» в конфигураторе

Затем у поля `КлючДоступаОтправителя` установим свойство `ОтображениеПодсказки` в значение `Кнопка`. А также установим свойство `Заголовок формы` как `Настройка Push-уведомлений` и отключим флажок `АвтоЗаголовков`.

В результате в режиме `1С:Предприятие` справа от поля `Ключ доступа` в форме настроек появится кнопка в виде вопросительного знака, нажав на которую, можно увидеть расширенную подсказку (рис. 4.37).

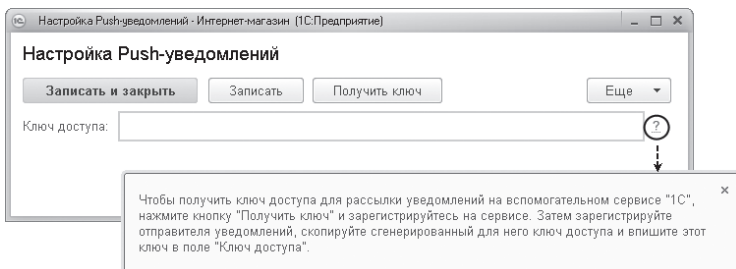


Рис. 4.37. Форма «Настройки» в режиме «1С:Предприятие»

Затем откроем окно редактирования плана обмена `Мобильные` и на закладке `Данные` добавим реквизит `ИдентификаторПодписчикаДоставляемыхУведомлений` типа `ХранилищеЗначения`. В нем будет храниться идентификатор получателя уведомлений для узла плана обмена, полученный от мобильного приложения через операцию веб-сервиса, публикуемого основным приложением.

Действия мобильного приложения по передаче идентификатора получателя уведомлений для конкретного мобильного устройства в основное приложение мы рассмотрим немного позже. А сейчас добавим операцию Web-сервиса для записи этого идентификатора в реквизит плана обмена ИдентификаторПодписчикаДоставляемыхУведомлений.

Для этого откроем окно редактирования Web-сервиса MAExchange на закладке Прочее и добавим для него новый пакет XDTO `http://v8.1c.ru/8.2/data/ext`, который содержит описание типов параметров и возвращаемых значений новой операции Web-сервиса (рис. 4.38).

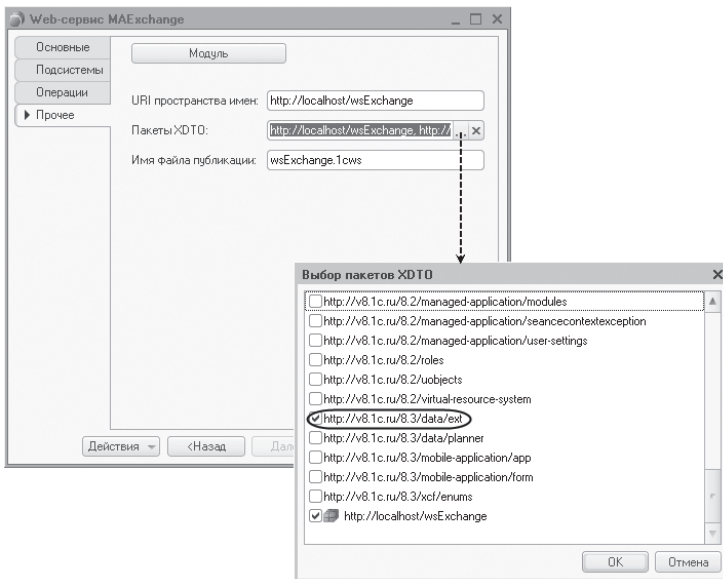


Рис. 4.38. Список пакетов XDTO Web-сервиса «MAExchange»

Затем перейдем на закладку Операции и добавим операцию Web-сервиса НовыйИдентификаторПодписчикаУведомлений (для записи идентификатора получателя уведомления в реквизит плана обмена) со свойствами:

- тип возвращаемого значения – string из пространства имен `http://www.w3.org/2001/XMLSchema`;
- Возможно пустое значение – установлен;
- Имя процедуры – НовыйИдентификаторПодписчикаУведомлений.

Для этой операции определим параметры, которые будут передаваться в процедуру, описанную в модуле Web-сервиса и указанную в свойстве Имя процедуры (рис. 4.39):

■ КодУзла со свойствами:

- Тип значения – string из пространства имен <http://www.w3.org/2001/XMLSchema>;
- Возможно пустое значение – не установлен;
- Направление передачи – Входной.

■ ИдентификаторXDTO со свойствами:

- Тип значения – DeliverableNotificationSubscriberID из пространства имен <http://v8.1c.ru/8.2/data/ext>;
- Возможно пустое значение – установлен;
- Направление передачи – Входной.

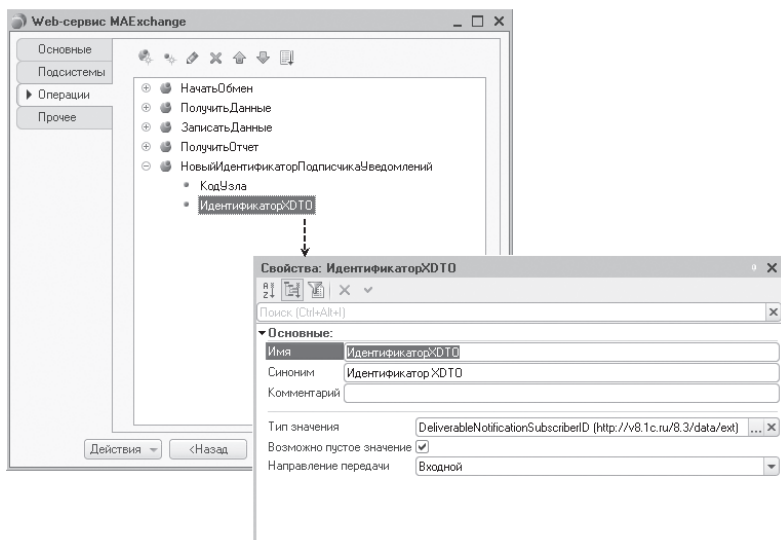


Рис. 4.39. Свойства параметров операции Web-сервиса
«НовыйИдентификаторПодписчикаУведомлений»

Теперь откроем модуль Web-сервиса и поместим в нем функцию, которая будет вызываться при выполнении новой операции (листинг 4.59).

Листинг 4.59. Функция «НовыйИдентификаторПодписчикаУведомлений»

Функция НовыйИдентификаторПодписчикаУведомлений(КодУзла, ИдентификаторXDTO)

```
Идентификатор = СериализаторXDTO.ПрочитатьXDTO(ИдентификаторXDTO);  
УзелОбмена = ПланыОбмена.Мобильные.НайтиПоКоду(КодУзла);
```

```
Если УзелОбмена.Пустая() Тогда  
    ВызватьИсключение("Неизвестное устройство - " + КодУзла);  
КонецЕсли;
```

```
Узел = УзелОбмена.ПолучитьОбъект();  
Узел.ИдентификаторПодписчикаДоставляемыхУведомлений = Новый  
    ХранилищеЗначения(Идентификатор);  
Узел.Записать();
```

КонецФункции

В эту функцию передается код узла обмена (базы на планшете), которая является получателем уведомления, и его идентификатор, полученный при запуске мобильного приложения на конкретном мобильном устройстве, сериализованный в XDTO.

ИдентификаторXDTO читается с помощью метода ПрочитатьXDTO объекта глобального контекста СериализаторXDTO и сохраняется в переменной Идентификатор.

Узел обмена находится по коду в плане обмена Мобильные. От найденной ссылки получается объект, и его реквизит ИдентификаторПодписчикаДоставляемыхУведомлений заполняется значением переменной Идентификатор, помещенной в хранилище значения. Затем объект записывается.

Таким образом, с помощью операции Web-сервиса НовыйИдентификаторПодписчикаУведомлений для каждого узла обмена будет получен и сохранен идентификатор получателя уведомлений, который однозначно определяет базу-получателя на планшете. Как мы покажем позже, этот идентификатор будет регулярно обновляться.

Теперь, наконец, займемся процедурой отправки уведомлений. Создадим в основном приложении общий неглобальный серверный модуль РаботаСДоставляемымиУведомлениям и напомним в нем процедуру ОтправитьУведомление(). Первым параметром в нее передается само уведомление, во втором параметре содержится либо ссылка на пользователя мобильного приложения (курьера, которому надо передать уведомление), либо Неопределено (листинг 4.60).

Листинг 4.60. Процедура «ОтправитьУведомление»

Процедура ОтправитьУведомление(Уведомление, Пользователь) Экспорт

```

Выборка = ПланыОбмена.Мобильные.Выбрать();
Пока Выборка.Следующий() Цикл
    Если Выборка.ИдентификаторПодписчикаДоставляемыхУведомлений <> Неопределено Тогда
        Если Пользователь = Неопределено ИЛИ Пользователь = Выборка.Курьер Тогда
            Идентификатор =
                Выборка.ИдентификаторПодписчикаДоставляемыхУведомлений.Получить();
            Если Идентификатор <> Неопределено Тогда
                Уведомление.Получатели.Добавить(Идентификатор);
            КонецЕсли;
        КонецЕсли;
    КонецЕсли;
КонецЦикла;

Если Уведомление.Получатели.Количество() > 0 Тогда
    ДанныеАутентификации = "";
    Сертификат = Неопределено;
    ДанныеАутентификации = СокрЛП(Константы. КлючДоступаОтправителя.Получить());

    УдаленныеТокены = Новый Массив;
    ОтправкаДоставляемыхУведомлений.Отправить(
        Уведомление, ДанныеАутентификации, УдаленныеТокены, Истина);
    НеИспользоватьИдентификаторы(УдаленныеТокены);
КонецЕсли;
КонецПроцедуры

```

В этой процедуре сначала определяется массив получателей уведомления (объектов типа ИдентификаторПодписчикаДоставляемыхУведомлений) – список конкретных информационных баз в определенном приложении на конкретном мобильном устройстве.

Для этого обходится выборка узлов плана обмена Мобильные и для каждого узла анализируется значение реквизита ИдентификаторПодписчикаДоставляемыхУведомлений. Если он определен, проверяется: если параметр Пользователь, переданный в процедуру, не определен или если в нем содержится ссылка на курьера, которая находится в реквизите Курьер текущего узла плана обмена, то этому узлу нужно отправить уведомление. В этом случае идентификатор подписчика доставляемых уведомлений получается из соответствующего реквизита узла плана обмена и добавляется в массив получателей уведомления (Уведомление.Получатели.Добавить(Идентификатор)).

Если массив получателей уведомления содержит хотя бы один элемент, то в переменной ДанныеАутентификации мы сохраняем значение константы КлючДоступаОтправителя, в которой мы запомним ключ доступа, полученный для отправителя уведомлений на сервисе «1С» при настройке push-уведомлений. При вводе (копировании) ключа доступа с сервиса

можно допустить ошибку, поэтому на всякий случай мы программно удаляем незначащие символы в начале и в конце ключа.

Затем с помощью свойства глобального контекста `ОтправкаДоставляемыхУведомлений` мы обращаемся к менеджеру отправки доставляемых уведомлений и методом `Отправить` отправляем уведомление всем получателям уведомления, определенным выше. В первом параметре мы передаем само `Уведомление` (размер которого сильно ограничен), во втором – `ДанныеАутентификации`, которые необходимы для подключения к сервису отправки уведомлений (в нашем случае это ключ доступа к вспомогательному сервису «1С»).

Третьим параметром мы передаем в метод `Отправить` пустой массив `УдаленныеТокены`, в который возвращается массив строк с идентификаторами устройств, которые не подходят для отправки и, соответственно, должны быть исключены из будущих рассылок. Это справедливо только для устройств под управлением Android, на устройствах под iOS этот параметр игнорируется. И четвертым параметром мы передаем `Истину`, так как для отправки уведомлений используем вспомогательный сервис.

В случае если массив `УдаленныеТокены` заполнен, надо исключить устройства, на которые не удалось доставить уведомления (например, пользователь удалил мобильное приложение), из будущих рассылок. Для этого вызывается процедура `НеИспользоватьИдентификаторы` (расположенная в том же общем модуле `РаботаСДоставляемымиУведомлениями`), в которую передается массив строк с идентификаторами таких устройств (листинг 4.61).

Листинг 4.61. Процедура «НеИспользоватьИдентификаторы»

Процедура `НеИспользоватьИдентификаторы(Токены)`

```
Если Токены.Количество() > 0 Тогда
    Выборка = ПланыОбмена.Мобильные.Выбрать();
    Пока Выборка.Следующий() Цикл
        Если Выборка.ИдентификаторПодписчикаДоставляемыхУведомлений <>
            Неопределено Тогда
                Идентификатор =
                    Выборка.ИдентификаторПодписчикаДоставляемыхУведомлений.Получить();
                Если Идентификатор <> Неопределено И
                    Токены.Найти(Идентификатор.ИдентификаторУстройства) Тогда
                    Узел = Выборка.ПолучитьОбъект();
                    Узел.ИдентификаторПодписчикаДоставляемыхУведомлений = Неопределено;
                    Узел.Записать();
                КонецЕсли;
            КонецЕсли;
        КонецЦикла;
    КонецЕсли;
КонецПроцедуры
```

Если массив `УдаленныеТокены` заполнен, выборка узлов плана обмена Мобильные обходится в цикле, и для каждого узла анализируется значение реквизита `ИдентификаторПодписчикаДоставляемыхУведомлений`. Если он определен, значение идентификатора извлекается из хранилища значения методом `Получить` и преобразуется в строку. Если в полученном идентификаторе устройства содержится такая же строка, как и в массиве, переданном в процедуру в качестве параметра, то для текущего элемента выборки получается объект, его реквизит `ИдентификаторПодписчикаДоставляемыхУведомлений` устанавливается в значение `Неопределено` и объект записывается.

Таким образом, пока мобильное приложение, работающее на конкретном мобильном устройстве, снова не передаст свой идентификатор подписчика доставляемых уведомлений в основное приложение, это устройство не попадет в список рассылки.

На этом мы закончили подготовку основного приложения к отправке push-уведомлений. Осталось добавить в мобильное приложение функциональность по регулярному получению и передаче в основное приложение идентификатора получателя уведомлений для конкретного мобильного устройства. А также реализовать реакцию мобильного приложения на получение push-уведомлений.

Мобильное приложение

Итак, добавим в мобильное приложение константу `ИдентификаторПодписчикаУведомлений` типа `ХранилищеЗначения`, в которой будет храниться идентификатор получателя уведомлений, обновляемый при каждом запуске системы. Дело в том, что идентификатор подписчика уведомлений может измениться, поэтому его рекомендуется получать и обновлять регулярно.

Процедуру обновления идентификатора подписчика уведомления мы будем вызывать в модуле управляемого приложения в обработчике события `ПриНачалеРаботыСистемы`. Мы уже создавали этот обработчик для обработки локальных напоминаний при разработке формы клиента (см. раздел «Напоминание о звонке (локальное уведомление)» на стр. 141). Внесем в него следующие изменения (листинг 4.62).

Листинг 4.62. Обработчик события «ПриНачалеРаботыСистемы» в модуле управляемого приложения

```
Процедура ПриНачалеРаботыСистемы()
```

```
// Push- и локальные уведомления
```

```
// идентификатор подписчика надо получать регулярно, он может измениться.
```

```
УведомленияКлиент.ОбновитьИдентификаторПодписчикаУведомлений();
```



```
// Подключение обработчика локальных и push-уведомлений.  
ОписаниеОповещения = Новый ОписаниеОповещения("ОбработкаУведомлений", УведомленияКлиент);  
ДоставляемыеУведомления.ПодключитьОбработчикУведомлений(ОписаниеОповещения);  
  
ПодключитьОбработчикОповещения("ОбработчикОповещения");
```

КонецПроцедуры

Саму процедуру ОбновитьИдентификаторПодписчикаУведомлений() поместим в общий модуль УведомленияКлиент. Мы уже создавали этот модуль для обработки локальных напоминаний при разработке формы клиента (см. раздел «Напоминание о звонке (локальное уведомление)» на стр. 141). Откроем его и поместим в нем процедуру ОбновитьИдентификаторПодписчикаУведомлений (листинг 4.63).

Листинг 4.63. Процедура «ОбновитьИдентификаторПодписчикаУведомлений»

Процедура ОбновитьИдентификаторПодписчикаУведомлений() Экспорт

```
НомераПриложений = Новый Массив();  
// Идентификатор приложения на промежуточном сервисе "1С".  
НомераПриложений.Добавить("797785657074");  
  
Попытка  
    ИдентификаторПодписчикаУведомлений = ДоставляемыеУведомления.  
        ПолучитьИдентификаторПодписчикаУведомлений(НомераПриложений);  
    ТекстОшибки = "";  
    УведомленияСервер.ПередатьИдентификаторПодписчикаУведомлений(  
        ИдентификаторПодписчикаУведомлений, ТекстОшибки);  
    Если ТекстОшибки <> "" Тогда  
        Сообщить(ТекстОшибки);  
    КонецЕсли  
Исключение  
    Инфо = ИнформацияОбОшибке();  
    ПоказатьИнформациюОбОшибке(Инфо);  
КонецПопытки
```

КонецПроцедуры

В процедуре сначала создается массив НомераПриложений. В него могут быть добавлены номера уже собранных мобильных приложений, зарегистрированных на сервисе доставки уведомлений. Мобильная платформа разработчика уже зарегистрирована на специальном вспомогательном сервисе «1С», который мы будем использовать для доставки уведомлений под номером 797785657074.

Поэтому мы добавляем в массив НомераПриложений строку «797785657074».

Затем с помощью свойства глобального контекста `ДоставляемыеУведомления` мы обращаемся к менеджеру доставляемых уведомлений и методом `ПолучитьИдентификаторПодписчикаУведомлений` получаем уникальный идентификатор подписчика уведомлений для конкретного мобильного устройства, на котором работает мобильное приложение.

Затем с помощью процедуры `ПередатьИдентификаторПодписчикаУведомлений` общего серверного модуля `УведомленияСервер` мы передаем полученный идентификатор подписчика в основное приложение. Создадим этот общий модуль и поместим в нем указанную процедуру. В качестве параметров в нее передаются полученный идентификатор подписчика уведомлений и пустая строка для заполнения текста ошибки, которая может возникнуть при подключении к веб-сервису (листинг 4.64).

Листинг 4.64. Процедура «`ПередатьИдентификаторПодписчикаУведомлений`»

```
Процедура ПередатьИдентификаторПодписчикаУведомлений(ИдентификаторПодписчикаУведомлений,
ТекстОшибки) Экспорт

    СтарыйИдентификатор =
        Константы.ИдентификаторПодписчикаУведомлений.Получить().Получить();

    Если СтарыйИдентификатор <> ИдентификаторПодписчикаУведомлений Тогда
        Прокси = Обмен.ПолучитьПрокси(ТекстОшибки);
        Если Прокси = Неопределено Тогда
            ВызватьИсключение ТекстОшибки;
        КонецЕсли;

        Узел = ПланыОбмена.Мобильные.ЭтотУзел();
        Сериализатор = Новый СериализаторXDTO(Прокси.ФабрикаXDTO);
        ИдентификаторXDTO = Сериализатор.ЗаписатьXDTO(ИдентификаторПодписчикаУведомлений);
        Прокси.НовыйИдентификаторПодписчикаУведомлений(Узел.Код, ИдентификаторXDTO);

        Константы.ИдентификаторПодписчикаУведомлений.Установить(
            Новый ХранилищеЗначения(ИдентификаторПодписчикаУведомлений));
    КонецЕсли;

КонецПроцедуры
```

В процедуре сначала получается старый идентификатор из значения константы `ИдентификаторПодписчикаУведомлений`. Значение константы извлекается из хранилища значения методом `Получить` и преобразуется в строку.

Если старый идентификатор не совпадает с новым, переданным в процедуру в первом параметре, мы подключаемся к Web-сервису, опубликованному основным приложением, при помощи функции `ПолучитьПрокси()` общего модуля `Обмен`, как это делалось при получении отчета

или при выполнении операций обмена. Если при подключении произошла ошибка, текст ошибки выводится пользователю.

Затем получается предопределенный узел плана обмена Мобильные для данной информационной базы, запущенной в мобильном приложении.

После этого мы создаем объект СериализаторXDTO на основе фабрики XDTO, содержащей определения всех типов XDTO, доступных нашему Web-сервису. С помощью этого объекта сериализуем новый ИдентификаторПодписчикаУведомлений в XDTO.

И затем мы вызываем операцию Web-сервиса НовыйИдентификаторПодписчикаУведомлений, в которую передаем код узла нашей информационной базы и сериализованный идентификатор подписчика уведомлений. В результате выполнения этой операции в основном приложении переданный идентификатор будет сохранен в реквизите ИдентификаторПодписчикаДоставляемыхУведомлений для переданного узла обмена.

После этого значение нового идентификатора подписчика уведомлений помещается в хранилище значения и устанавливается в качестве значения константы ИдентификаторПодписчикаУведомлений.

Теперь нам осталось только доработать обработчик уведомлений, чтобы мобильное приложение реагировало на получение push-уведомлений. Мы уже создавали этот обработчик в общем модуле УведомленияКлиент для обработки локальных напоминаний при разработке формы клиента (см. раздел «Напоминание о звонке (локальное уведомление)» на стр. 141). Внесем в него следующие изменения (листинг 4.65).

Листинг 4.65. Процедура «ОбработкаУведомлений»

Процедура ОбработкаУведомлений(Уведомление, Локальное, Показано, Параметры) Экспорт

```
Если Уведомление.Данные = "1" Тогда
    ТекстВопроса = Уведомление.Текст + Символы.ПС +
        "Синхронизировать данные сейчас?";
    Оповещение =
        Новый ОписаниеОповещения("ОбработкаУведомленийЗавершение", ЭтотОбъект);
    ПоказатьВопрос(Оповещение, ТекстВопроса, РежимДиалогаВопрос.ДаНет);
ИначеЕсли Локальное = Истина Тогда
    ПараметрыФормы = Новый Структура("Текст, Данные", Уведомление.Текст, Уведомление.Данные);
    ОткрытьФорму("ОбщаяФорма.Звонок", ПараметрыФормы);
Иначе
    Если Не Показано Тогда
        ПоказатьПредупреждение(, Уведомление.Текст);
    КонецЕсли;
КонецЕсли;
```

КонецПроцедуры

Обработка получения push-уведомлений мало чем отличается от обработки локальных уведомлений.

Когда в основном приложении в заказе меняется какая-то важная для курьера информация, то ему приходит push-уведомление просто с текстом сообщения. В случае если приложение не запущено или работает в фоновом режиме, оно показывается средствами операционной системы (в списке уведомлений планшета).

Если приложение запущено и активно, то уведомление сразу передается в мобильное приложение и не показывается пользователю (параметр Показано = Ложь). В этом случае (в последней секции оператора условия) мы для привлечения внимания курьера показываем его немодальным методом ПоказатьПредупреждение.

Когда в основном приложении добавляется новый товар, то от него приходит push-уведомление с соответствующим текстом сообщения и свойством Данные, равным «1». В этом случае мы задаем курьеру вопрос о необходимости синхронизации данных.

Для этого мы используем немодальный метод глобального контекста ПоказатьВопрос, который выводит пользователю вопрос, но не ожидает его ответа. Первым параметром в этот метод передается описание процедуры ОбработкаУведомленийЗавершение, которая будет выполнена, когда пользователь ответит на вопрос (листинг 4.66).

Листинг 4.66. Процедура «ОбработкаУведомленийЗавершение»

Процедура ОбработкаУведомленийЗавершение(Результат, Параметры) Экспорт

```
Если Результат = КодВозвратаДиалога.Да Тогда
    ТекстОшибки = "";
    Если Обмен.ВыполнитьОбменДанными(ТекстОшибки) Тогда
        ПоказатьПредупреждение(, "Синхронизация данных завершена.", 5);
        ОповеститьОБИзменении(Тип("ДокументСсылка.Заказ"));
        ОповеститьОБИзменении(Тип("ДокументСсылка.ОбслуживаниеЗаказов"));
        ОповеститьОБИзменении(Тип("СправочникСсылка.Клиенты"));
        ОповеститьОБИзменении(Тип("СправочникСсылка.Товары"));
        ОповеститьОБИзменении(Тип("СправочникСсылка.ПричиныОтказа"));
        ОповеститьОБИзменении(Тип("СправочникСсылка.ХранимыеФайлы"));
    Иначе
        ПоказатьПредупреждение(, ТекстОшибки);
    КонецЕсли;
КонецЕсли;
КонецПроцедуры
```

В этой процедуре, если пользователь ответил утвердительно, мы дублируем код общей команды для обмена данными ОбменСЦентральнойБазой. В результате выполняется синхронизация данных с основным приложением.

Тестирование отправки и получения push-уведомлений

Итак, сначала настроим в основном приложении отправку push-уведомлений через вспомогательный сервис «1С». Для этого в группе Сервис выполним команду Настройки и откроем форму Настройка Push-уведомлений.

Нажмем на кнопку Получить ключ и зарегистрируемся на открывшемся сайте сервиса «1С». Заполним поле Наименование отправителя, перейдем в раздел Доставляемые уведомления > Отправители уведомлений и зарегистрируем там приложение — отправителя уведомлений кнопкой Зарегистрировать отправителя. Сгенерированный для него ключ из поля Ключ доступа отправителя скопируем и вставим в форму настроек в поле Ключ доступа. Нажмем Записать и закрыть (рис. 4.40).

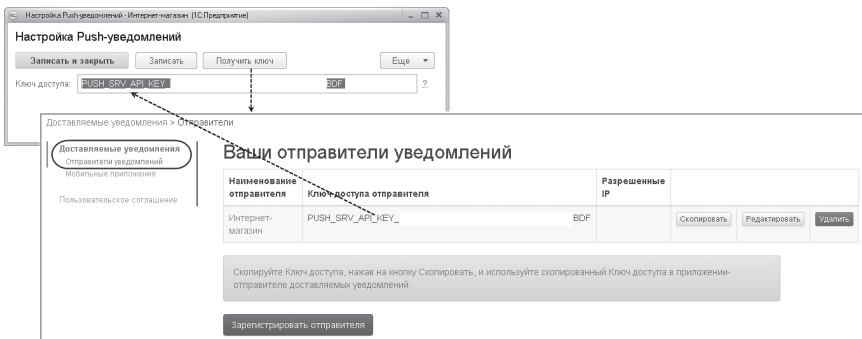


Рис. 4.40. Настройка push-уведомлений в основном приложении

ПРИМЕЧАНИЕ

Ключ доступа отправителя на рис. 4.40 частично стерт, поскольку это конфиденциальная информация.

Теперь запустим на планшете мобильную платформу разработчика и откроем информационную базу Курьер Интернет-магазина (Морозов), в которой работает курьер Морозов. При запуске информационная база в нашем мобильном приложении получает идентификатор подписчика уведомлений и передает его в основное приложение через веб-сервис. «Поработаем» с мобильным приложением, чтобы оно было активно.

Затем откроем в основном приложении заказ ЦО-000003 для курьера Морозов, введем информацию в поле Дополнительно и нажмем Провести и Заккрыть. В мобильном приложении тут же (поскольку оно активно), без

значка в списке уведомлений планшета и звукового сигнала, появится следующее уведомление (рис. 4.41).

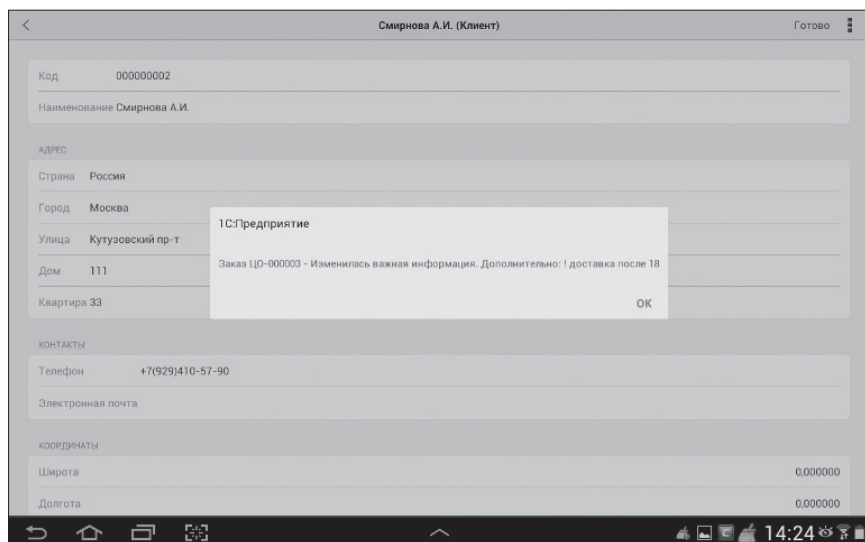


Рис. 4.41. Push-уведомление, полученное от основного приложения для курьера «Морозов»

Затем запустим на планшете какое-то другое приложение (т.е. наше приложение будет работать в фоновом режиме). Откроем в основном приложении тот же заказ ЦО-000003, изменим информацию в поле Адрес доставки и в поле Дополнительно и проведем его.

На планшете прозвучит стандартный сигнал, установленный в системе для уведомлений, и в списке уведомлений в правом углу планшета появится значок уведомления  (это справедливо для мобильных устройств, работающих под управлением ОС Android). При нажатии на значок уведомления наше мобильное приложение будет активизировано и появится следующее сообщение (рис. 4.42).

Выйдем из приложения и окажемся в списке информационных баз мобильной платформы разработчика. Откроем в основном приложении тот же заказ ЦО-000003, изменим информацию в поле Адрес доставки и проведем его. На планшете появится текст сообщения, и будет предложено перейти к соответствующему приложению (рис. 4.43).

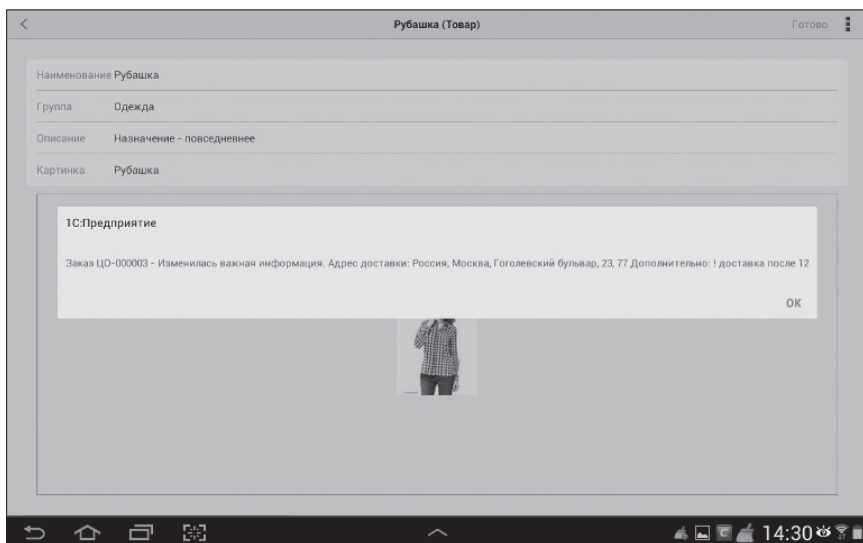


Рис. 4.42. Push-уведомление, полученное от основного приложения для курьера «Морозов»

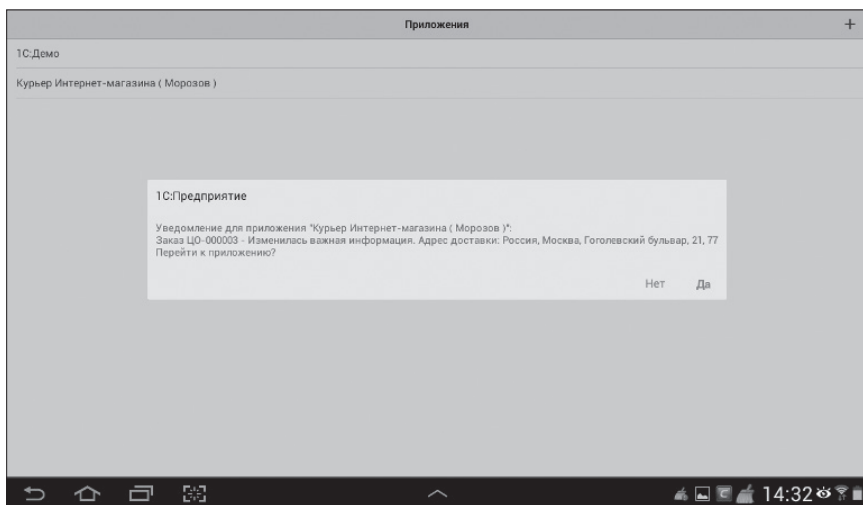


Рис. 4.43. Push-уведомление, полученное от основного приложения для курьера «Морозов»

Если мобильная платформа разработчика не запущена, то текст уведомления будет показан только в списке уведомлений. При нажатии на него будет открыто мобильное приложение, но текст уведомления больше не появится.

Итак, мы убедились, что при изменении важной информации в заказе из офиса на планшет тому курьеру, который обслуживает заказ, приходят уведомления об этом. Все работает так, как мы и хотели!

Теперь протестируем ситуацию, когда в офисе добавляется новый товар.

Запустим на планшете мобильную платформу разработчика и окажемся в списке информационных баз. Затем в основном приложении добавим товар и запишем его. В мобильном приложении появятся текст уведомления для нашего мобильного приложения и предложение перейти к нему (рис. 4.44).

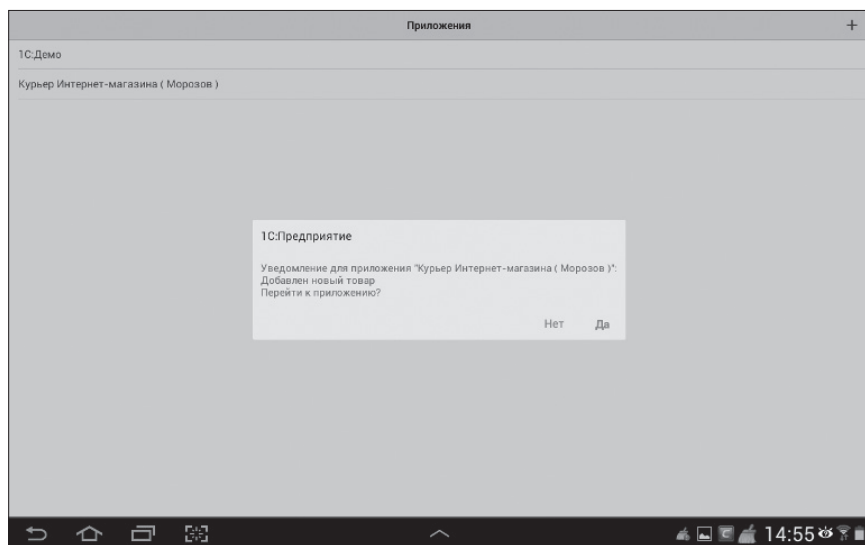


Рис. 4.44. Push-уведомление, полученное от основного приложения
при добавлении нового товара

После перехода к приложению курьеру будет задан вопрос о необходимости синхронизировать данные с основным приложением (рис. 4.45).

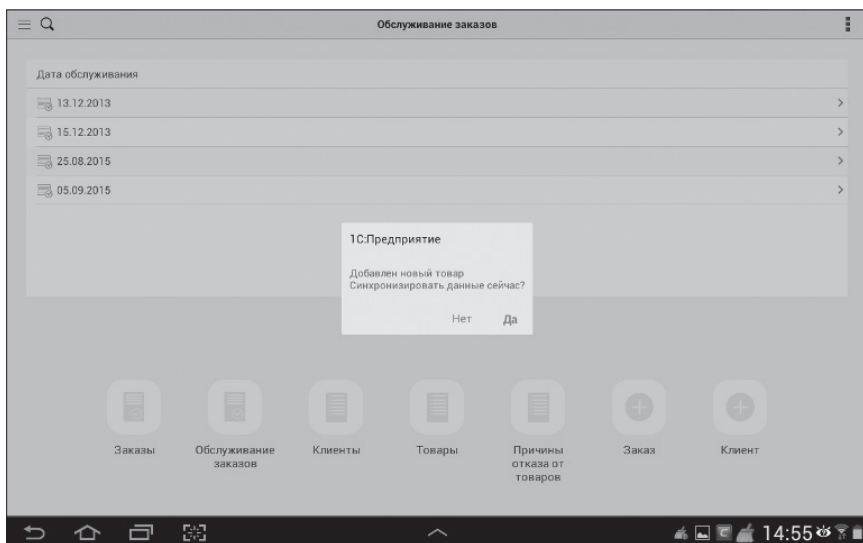


Рис. 4.45. Push-уведомление, полученное от основного приложения при добавлении нового товара

Таким образом, в случае добавления нового товара в офисе в наше мобильное приложение приходят уведомления из основного приложения с признаком необходимости синхронизации данных, что нам и требовалось.

Глава 5.

Сборка мобильного приложения

Итак, мы разработали мобильное приложение для работы курьера интернет-магазина при доставке товаров клиентам. Конечно, оно достаточно схематично и не может охватить всех задач, реально возникающих в процессе работы курьера. Но оно реализует всю ту функциональность, которую мы хотели показать в этой книге.

Теперь, после того как разработка закончена, нам осталось только собрать наше мобильное приложение в один файл и загрузить его на планшет.

Хотя для сборки мы будем использовать специальную конфигурацию Сборщик мобильных приложений, облегчающую процесс сборки, все равно в первый раз сделать это не просто и не быстро. Поэтому нужно запастись терпением и внимательно и аккуратно выполнить описанную ниже последовательность действий.

Сборщик мобильных приложений

Конфигурация Сборщик мобильных приложений поставляется в составе мобильной платформы. В первой главе книги в разделе «Мобильная платформа "1С:Предприятия"» на стр. 22 мы распаковывали архив с мобильной платформой на компьютер. В этом каталоге есть папка MobileAppMaker с файлом Setup.exe для установки шаблона конфигурации. Запустим этот файл и установим шаблон конфигурации в каталог шаблонов «1С:Предприятия» (рис. 5.1).

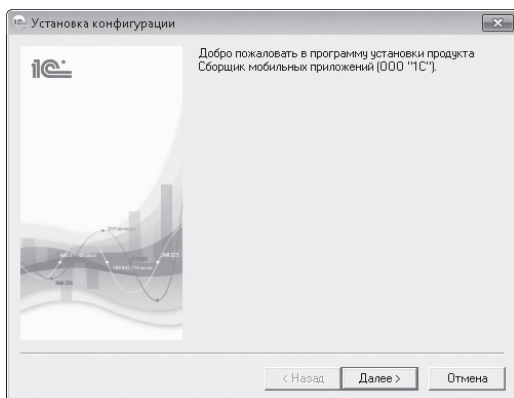


Рис. 5.1. Установка шаблона конфигурации «Сборщик мобильных приложений»

Затем добавим новую информационную базу в список информационных баз «1С:Предприятия» и создадим информационную базу из созданного ранее шаблона (рис. 5.2).

Затем откроем эту базу в конфигураторе и добавим пользователя Администратор со свойствами Аутентификация 1С:Предприятия, ролями Администратор и Пользователь и языком Русский (рис. 5.3).

Сохраним конфигурацию, закроем ее и откроем в режиме 1С:Предприятие от имени пользователя Администратор. Сейчас эта база пустая. Мы должны заполнить в ней все необходимые параметры для сборки, которые будут сохраняться и использоваться для дальнейших сборок.

Сначала (если база пустая) на начальной странице приложения открывается общая справочная информация о сборщике мобильных приложений. Перейти к ней также можно из главного меню – Главное меню > Справка > Содержание справки > Сборщик мобильных приложений. Кроме того, из отдельных форм конфигурации отрываются дополнительные страницы справки о сборке мобильных приложений (рис. 5.4).

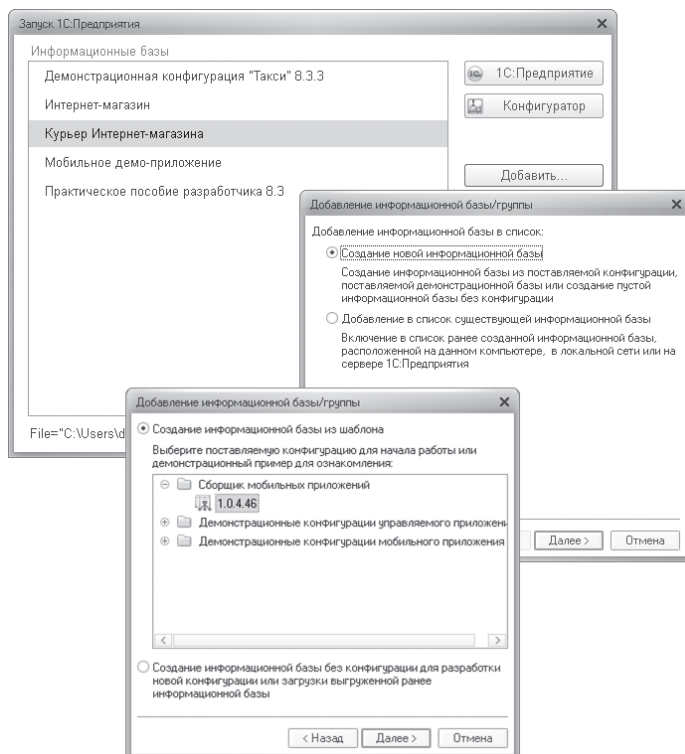


Рис. 5.2. Создание информационной базы «Сборщик мобильных приложений» из шаблона

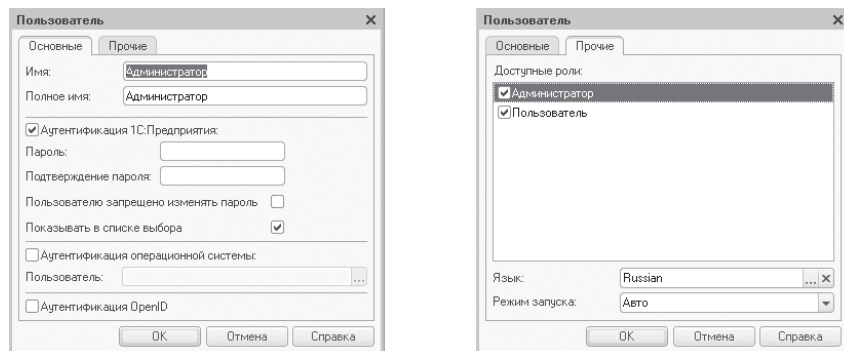


Рис. 5.3. Создание пользователя «Администратор»

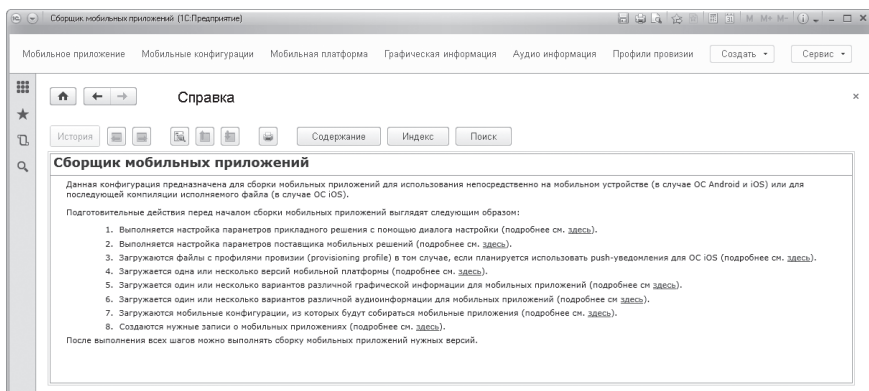


Рис. 5.4. Справка по конфигурации «Сборщик мобильных приложений»

Настройка параметров прикладного решения

Сначала мы должны настроить параметры сборщика. Для этого из меню Сервис вызовем пункт Настройки параметров приложения. Собирать мобильное приложение для Apple мы сейчас не будем, поэтому соответствующий флажок оставляем пустым.

В форме настроек нам нужно заполнить таблицу Каталоги компонентов на компьютерах, участвующих в процессе сборки, которая будет содержать пути к программным компонентам, необходимым для сборки мобильного приложения. Для этого нажмем кнопку Создать над этой таблицей (рис. 5.5).

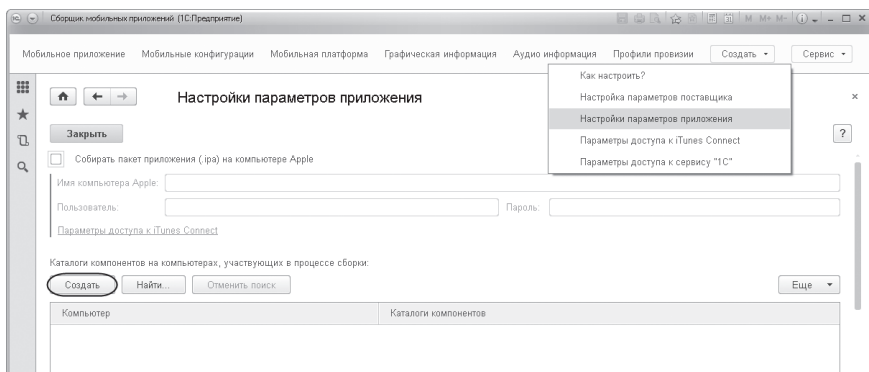


Рис. 5.5. Создание записи таблицы «Каталоги компонентов...»

Откроется форма Пути к компонентам. Вызвав справку из этой формы, можно увидеть ссылки для получения программных компонентов и их описание (рис. 5.6).

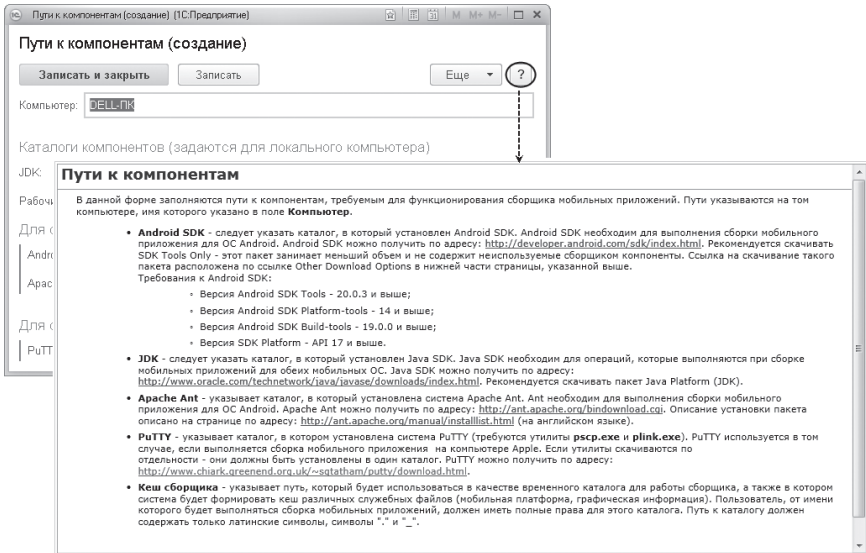


Рис. 5.6. Справка, описывающая пути к компонентам

Сначала нужно установить Java SDK и в поле JDK указать каталог, в который этот компонент установлен. Java SDK можно получить по адресу: <http://www.oracle.com/technetwork/java/javase/downloads/index.html>. Рекомендуется скачивать пакет Java Platform (JDK).

На открывшейся странице вверху нужно нажать на кнопку Download (рис. 5.7).

На следующей странице нужно принять лицензионное соглашение (установить отметку Accept License Agreement) и нажать на ссылку с нужным дистрибутивом в колонке Download (для 64-битной Windows – это пакет `jdk-8u60-windows-x64.exe`), рис. 5.8.



Рис. 5.7. Получение Java SDK

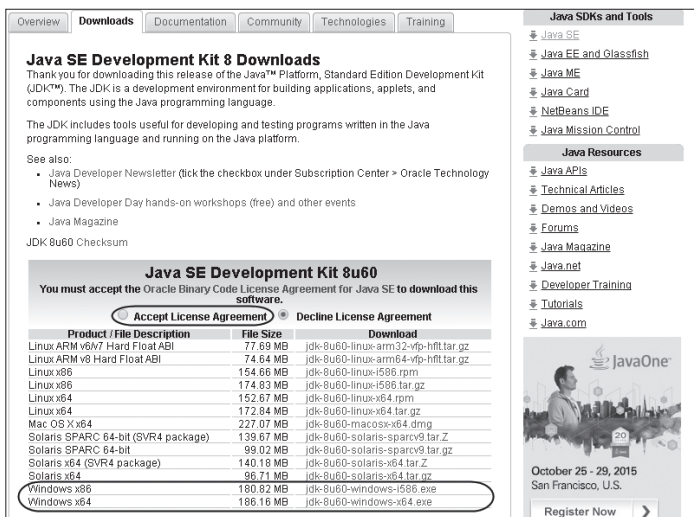


Рис. 5.8. Получение Java SDK

Полученный инсталлятор нужно запустить и установить Java SDK, например, в каталог: C:\Program Files\Java\jdk1.8.0_60 (рис. 5.9).

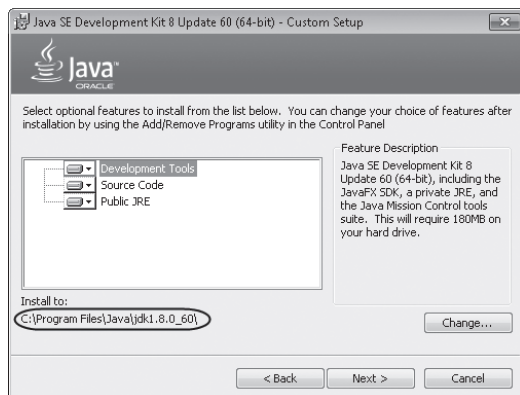


Рис. 5.9. Установка Java SDK

Затем этот путь нужно указать в поле JDK формы настройки путей к компонентам приложения Сборщик мобильных приложений (рис. 5.10).

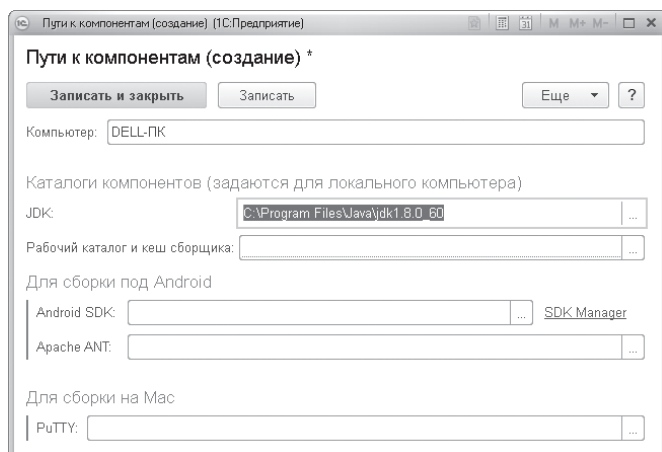


Рис. 5.10. Настройка путей к компонентам приложения
«Сборщик мобильных приложений»

В следующем поле формы настройки Рабочий каталог и кеш сборки нужно указать любой каталог на латинице, в котором программа сборки

будет размещать служебные файлы. Пользователь, от имени которого будет выполняться сборка мобильных приложений, должен иметь полные права для этого каталога (рис. 5.11).

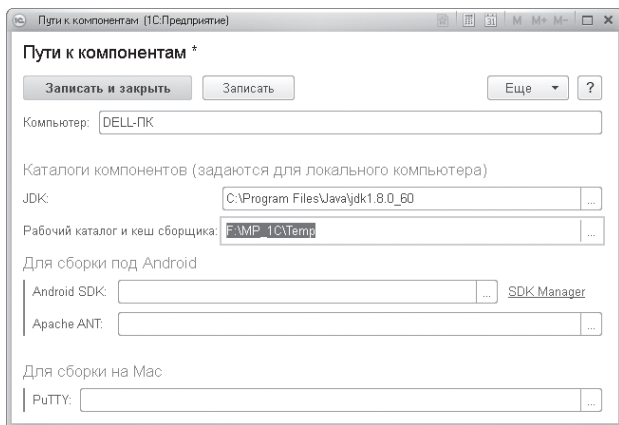


Рис. 5.11. Настройка путей к компонентам приложения
«Сборщик мобильных приложений»

В поле Android SDK укажем путь к каталогу, в котором находится SDK Manager. Android SDK мы устанавливали в главе 1, в разделе «Android SDK» на стр. 18 (рис. 5.12).

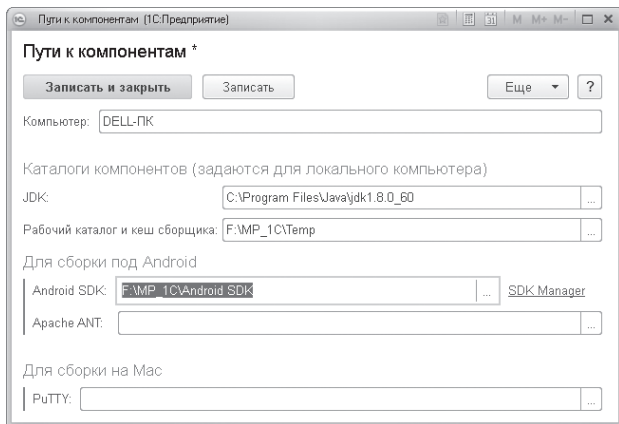


Рис. 5.12. Настройка путей к компонентам приложения
«Сборщик мобильных приложений»

Затем нужно установить Apache ANT и в поле Apache ANT указать каталог, в который этот компонент установлен. Apache ANT необходим для выполнения сборки мобильного приложения для ОС Android. Apache Ant можно получить по адресу: <http://ant.apache.org/bindownload.cgi>.

С этой страницы нам нужно скачать архив `apache-ant-1.9.6-bin.zip` (рис. 5.13).

Downloading Apache Ant

Use the links below to download a binary distribution of Ant from one of our mirrors. It is good practice to [verify the integrity](#) of the distribution files, especially if you are using one of our mirror sites. In order to do this you must use the signatures from our [main distribution directory](#).

Ant is distributed as `zip`, `tar.gz` and `tar.bz2` archives – the contents are the same. Please note that the `tar.*` archives contain file names longer than 100 characters and have been created using GNU tar extensions. Thus they must be untarred with a GNU compatible version of `tar`.

In addition the [JPackage project](#) provides RPMs at their own distribution site.

If you do not see the file you need in the links below, please see the [master distribution directory](#) or, preferably, its [mirror](#).

Mirror

You are currently using <http://apache-mirror.rbc.ru/pub/apache/>. If you encounter a problem with this mirror, please select another mirror. If all mirrors are failing, there are *backup* mirrors (at the end of the mirrors list) that should be available.

Other mirrors: <http://apache-mirror.rbc.ru/pub/apache/>

Current Release of Ant

Currently, Apache Ant 1.9.6 is the best available version, see the [release notes](#).

Note

Ant 1.9.6 was released on 02-Jul-2015 and may not be available on all mirrors for a few days.

Tar files may require `gnu tar` to extract

Tar files in the distribution contain long file names, and may require `gnu tar` to do the extraction.

- `zip` archive: [apache-ant-1.9.6-bin.zip](#) [PGP] [SHA1] [SHA512] [MD5]
- `tar.gz` archive: [apache-ant-1.9.6-bin.tar.gz](#) [PGP] [SHA1] [SHA512] [MD5]
- `tar.bz2` archive: [apache-ant-1.9.6-bin.tar.bz2](#) [PGP] [SHA1] [SHA512] [MD5]

Рис. 5.13. Получение Apache ANT

Разархивируем этот файл на компьютер и укажем путь к нему в форме настроек путей к компонентам (рис. 5.14).

Затем нужно установить систему PuTTY и в поле PuTTY указать каталог, в который этот компонент установлен. PuTTY можно получить по адресу: <http://www.chiark.greenend.org.uk/~sgtatham/putty/download.html>.

PuTTY используется в том случае, если выполняется сборка мобильного приложения для Apple. Для сборки мобильных приложений требуются утилиты `pscp.exe` и `plink.exe`. На всякий случай скачаем весь установочный пакет `putty-0.65-installer.exe` (рис. 5.15).

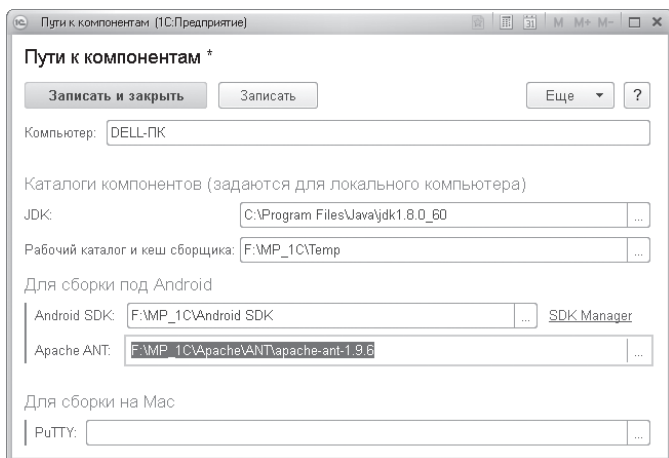


Рис. 5.14. Настройка путей к компонентам приложения «Сборщик мобильных приложений»

Binaries

The latest release version (beta 0.65)

This will generally be a version I think is reasonably likely to work well. If you have a problem with the release version, it's out the latest development snapshot (below) to see if I've already fixed the bug, before reporting it to me.

The last release was signed using the old release keys (before the 2015 rollover), so it has separate RSA and DSA signatures.

For Windows on Intel x86

PuTTY:	putty.exe	(or by FTP)	(RSA sig)	(DSA sig)
PuTTYtel:	puttytel.exe	(or by FTP)	(RSA sig)	(DSA sig)
PSCP:	pscp.exe	(or by FTP)	(RSA sig)	(DSA sig)
PSFTP:	psftp.exe	(or by FTP)	(RSA sig)	(DSA sig)
Plink:	plink.exe	(or by FTP)	(RSA sig)	(DSA sig)
Pageant:	pageant.exe	(or by FTP)	(RSA sig)	(DSA sig)
PuTTYgen:	puttygen.exe	(or by FTP)	(RSA sig)	(DSA sig)

A .ZIP file containing all the binaries (except PuTTYtel), and also the help files

Zip file:	putty.zip	(or by FTP)	(RSA sig)	(DSA sig)
-----------	---------------------------	-------------	-----------	-----------

A Windows installer for everything except PuTTYtel

Installer:	putty-0.65-installer.exe	(or by FTP)	(RSA sig)	(DSA sig)
------------	--	-------------	-----------	-----------

Checksums for all the above files

MD5:	md5sums	(or by FTP)	(RSA sig)	(DSA sig)
SHA-1:	sha1sums	(or by FTP)	(RSA sig)	(DSA sig)
SHA-256:	sha256sums	(or by FTP)	(RSA sig)	(DSA sig)
SHA-512:	sha512sums	(or by FTP)	(RSA sig)	(DSA sig)

Рис. 5.15. Получение PuTTY

Полученный инсталлятор нужно запустить и установить PuTTY, например, в каталог: C:\Program Files (x86)\PuTTY (рис. 5.16).

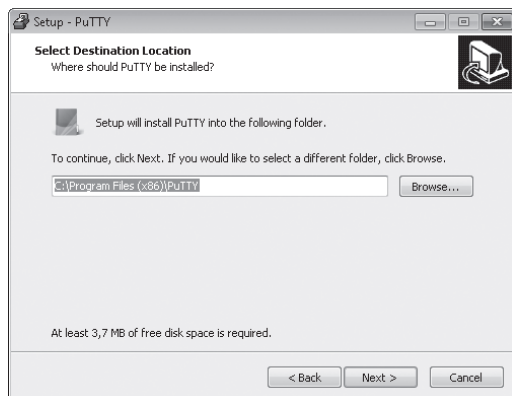


Рис. 5.16. Установка PuTTY

Затем укажем путь, полученный при установке PuTTY, в форме настройки путей к компонентам (рис. 5.17).

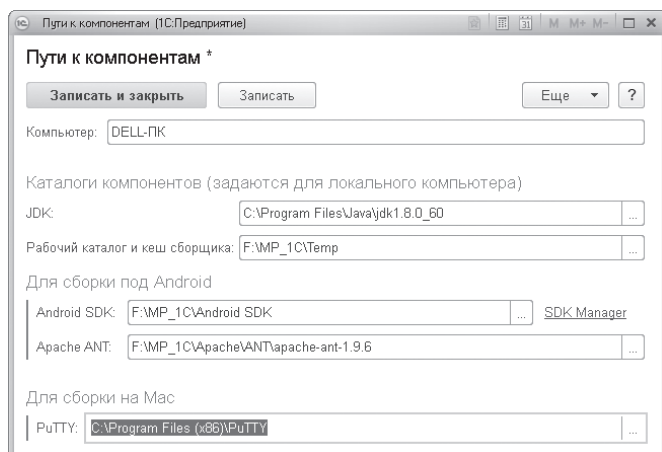


Рис. 5.17. Настройка путей к компонентам приложения
«Сборщик мобильных приложений»

На этом настройка путей к компонентам завершена. Нажмем **Записать и закрыть**.

Настройка параметров поставщика

Теперь нам нужно настроить параметры поставщика. Для этого вызовем из меню Сервис пункт Редактировать параметры поставщика.

Откроется форма Поставщика, в которой нужно на закладке Общие параметры указать произвольное наименование поставщика, а также задать Префикс идентификатора приложения. Это поле должно заполняться на латинице и начинаться со строки «com». Правила заполнения этого поля можно посмотреть в контекстной справке, открывающейся по кнопке со знаком «?».

Затем нужно отметить, для каких операционных систем выполняется сборка мобильного приложения. В нашем случае установим флажок Для ОС Android.

Для работы с push-уведомлениями через вспомогательный сервис «1С» укажем параметры доступа к сервису «1С». Для этого нажмем кнопку Добавить над таблицей внизу формы поставщика. В открывшемся окне Параметры доступа к вспомогательному сервису «1С» отметим опцию Регистрировать для – выбранного пользователя, выберем пользователя сборщика – Администратор и укажем электронный адрес и пароль, под которым мы ранее регистрировались на сервисе при тестировании работы с push-уведомлениями. Нажмем кнопку Сохранить и закрыть. Кроме того, можно зарегистрироваться на сервисе «1С» непосредственно из этой формы с помощью кнопки Зарегистрироваться в сервисе «1С», если этого еще не было сделано (рис. 5.18).

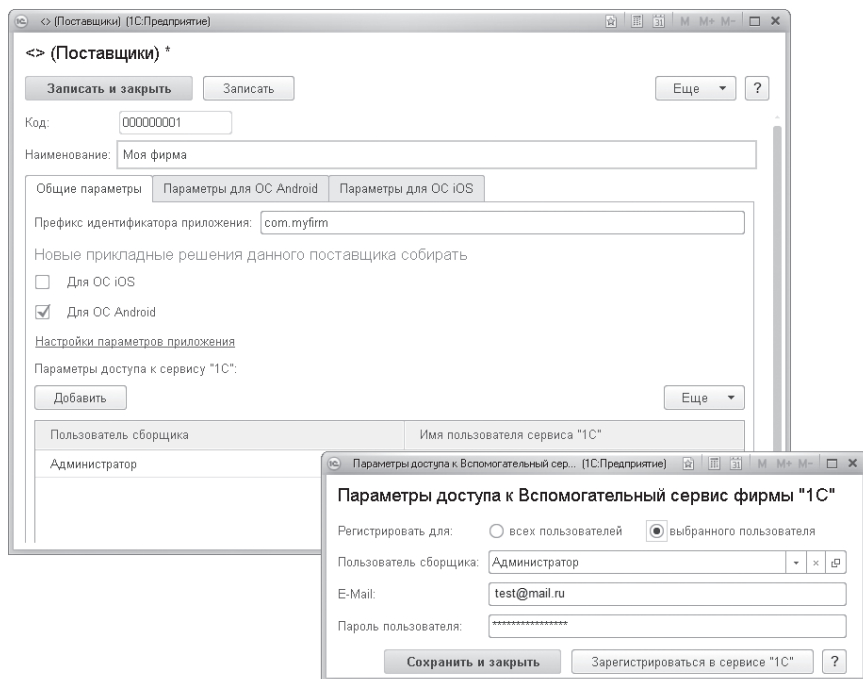


Рис. 5.18. Настройка параметров поставщика приложения
«Сборщик мобильных приложений»

Кроме того, вызвать окно настройки параметров доступа к сервису «1С» можно из меню Сервис, пункт Параметры доступа к сервису «1С».

После этого необходимо на закладке Параметры для ОС Android заполнить группу полей Ключ разработчика. Для этого сначала создадим ключ разработчика, нажав на ссылку Создать ключ разработчика. В открывшейся форме Создание ключа разработчика произвольно заполним поля (для поля Страна нужно указать код России в стандарте ISO – ru) и нажмем кнопку Сформировать ключ (рис. 5.19).

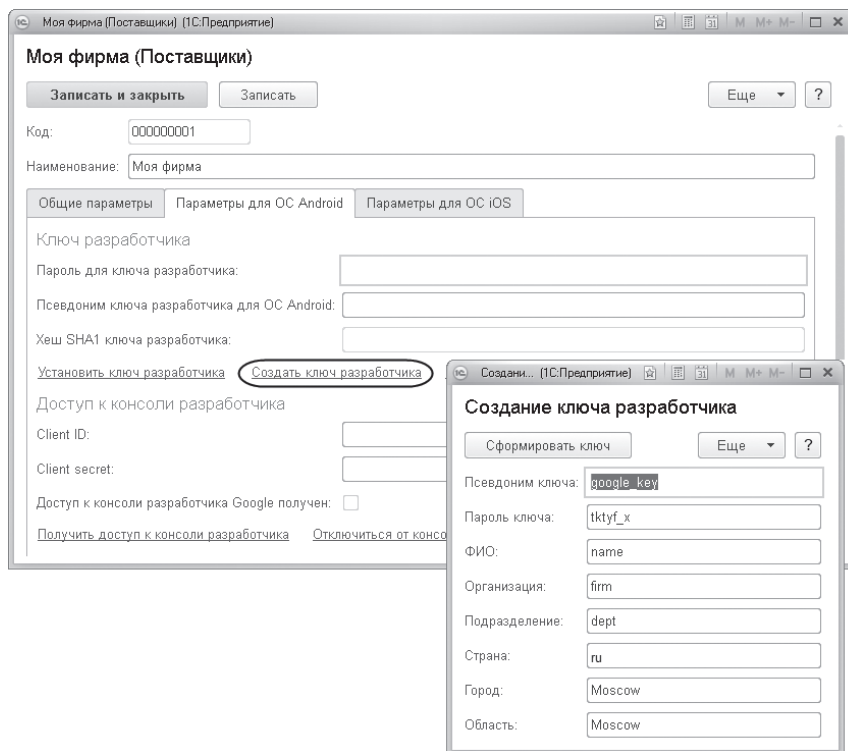


Рис. 5.19. Настройка параметров поставщика приложения
«Сборщик мобильных приложений»

После этого поля-параметры ключа разработчика заполнятся автоматически (рис. 5.20).

Значение в поле Хеш SHA1 ключа разработчика будет использоваться в дальнейшем для получения ключа работы с картами Google. Это значение необходимо в том случае, если мобильное приложение будет использовать средства геопозиционирования на платформе Android.

На этом настройка параметров поставщика завершена. Нажмем Записать и закрыть.

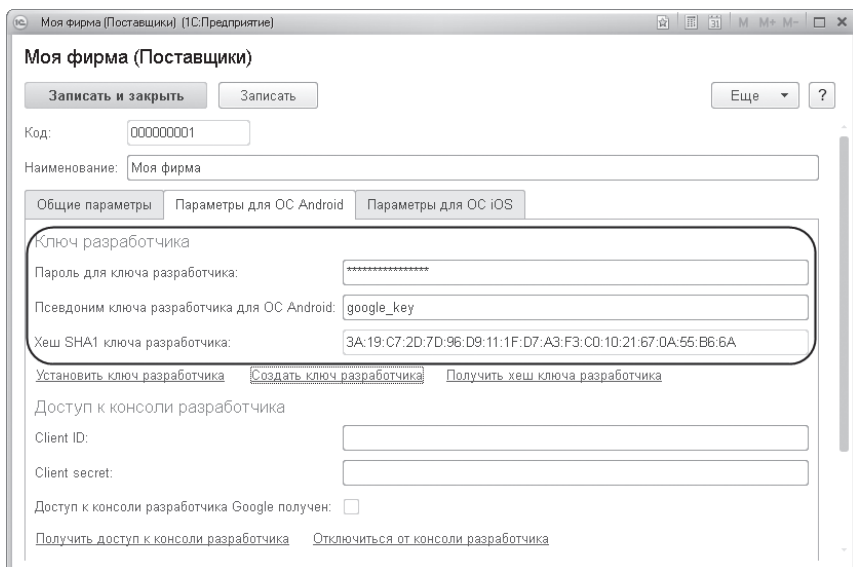


Рис. 5.20. Настройка параметров поставщика приложения
«Сборщик мобильных приложений»

Загрузка мобильной платформы

Теперь нам нужно загрузить мобильную платформу «1С:Предприятия», под управлением которой будет работать собранное мобильное приложение. Версий мобильной платформы может быть несколько, но они должны быть не ниже версии 8.3.4.

Для загрузки и хранения различных версий мобильной платформы предназначен справочник Мобильные платформы. Для каждой версии платформы в этом справочнике должна быть создана отдельная запись.

Из командной панели приложения откроем справочник Мобильные платформы и нажмем кнопку Создать. После этого появится диалог выбора файла, в котором нужно выбрать файл с архивом мобильной платформы mobile.zip, который мы сохраняли на компьютере при получении мобильной платформы в первой главе книги, в разделе «Мобильная платформа "1С:Предприятия"» на стр. 22. Выберем его и нажмем кнопку Открыть.

В случае успешной загрузки платформы откроется форма создания элемента справочника Мобильные платформы, в которой поля Версия мобильной

платформы и Наименование заполняются автоматически, и появится флажок
Файлы мобильной платформы загружены (рис. 5.21).

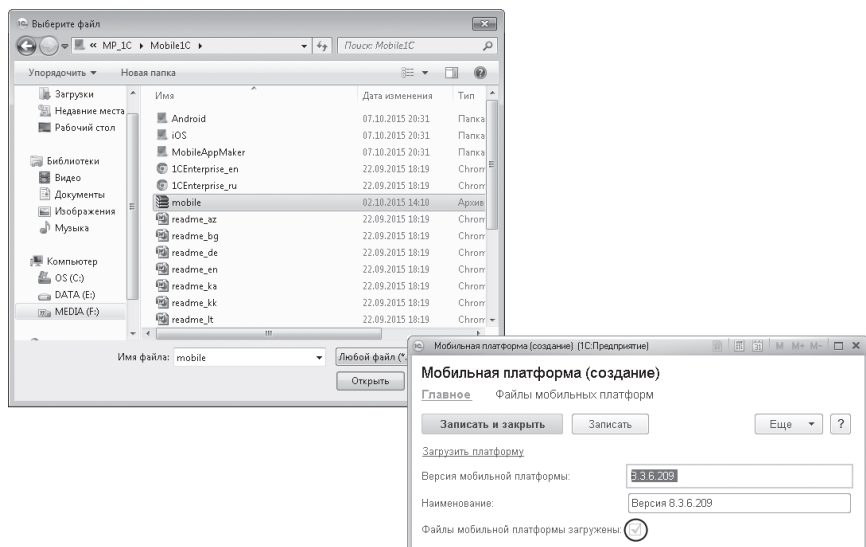


Рис. 5.21. Загрузка мобильной платформы

Нажмем Записать и закрыть.

Загрузка мобильной конфигурации

Теперь нам нужно загрузить разработанную нами мобильную конфигурацию КурьерИнтернетМагазина. Откроем эту конфигурацию в конфигураторе. В палитре свойств конфигурации зададим свойства Поставщик – myfirm и Версия – 1.0.0 (рис. 5.22).

Затем выгрузим эту конфигурацию в файл, выполнив команду конфигуратора Конфигурация > Мобильное приложение > Записать в файл...

Для загрузки и хранения различных версий конфигураций мобильных приложений предназначен справочник Мобильные конфигурации. Справочник предполагает двухуровневую структуру: группы описывают прикладные решения, а элементы в группах – различные версии конфигураций этих прикладных решений. Для загрузки новой версии конфигурации необходимо зайти в группу, соответствующую прикладному решению, и в этой группе создать новый элемент.

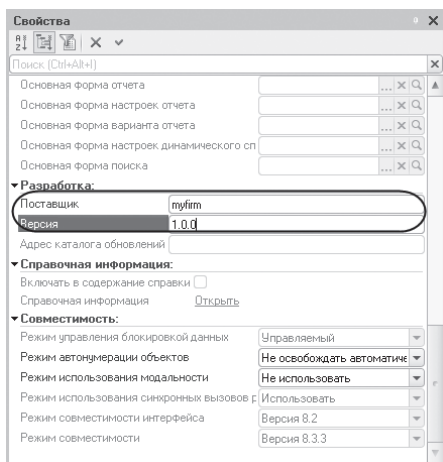


Рис. 5.22. Свойства мобильной конфигурации «КурьерИнтернетМагазина»

Из командной панели приложения откроем справочник Мобильные конфигурации и нажмем кнопку Создать группу с наименованием нашей конфигурации Курьер Интернет-магазина (рис. 5.23).

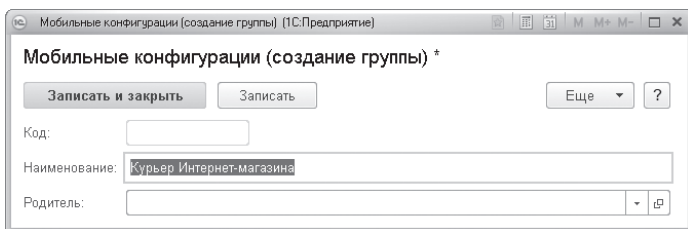


Рис. 5.23. Создание группы справочника «Мобильные конфигурации»

Затем в этой группе создадим новый элемент справочника.

После этого появится диалог выбора файла, в котором нужно выбрать файл 1сета.xml, в который мы только что сохраняли нашу мобильную конфигурацию. Выберем его и нажмем кнопку Открыть.

В случае успешной загрузки конфигурации все поля формы будут заполнены автоматически и не должны изменяться вручную. В таблице разрешений будут указаны все разрешения по работе мобильного приложения с мультимедиа, геопозиционированием, уведомлениями и т.п., которые мы установили при его разработке. Кроме того, будут показаны

сообщения о невозможности работы с календарями и контактами, разрешения на которые мы не устанавливали (рис. 5.24).

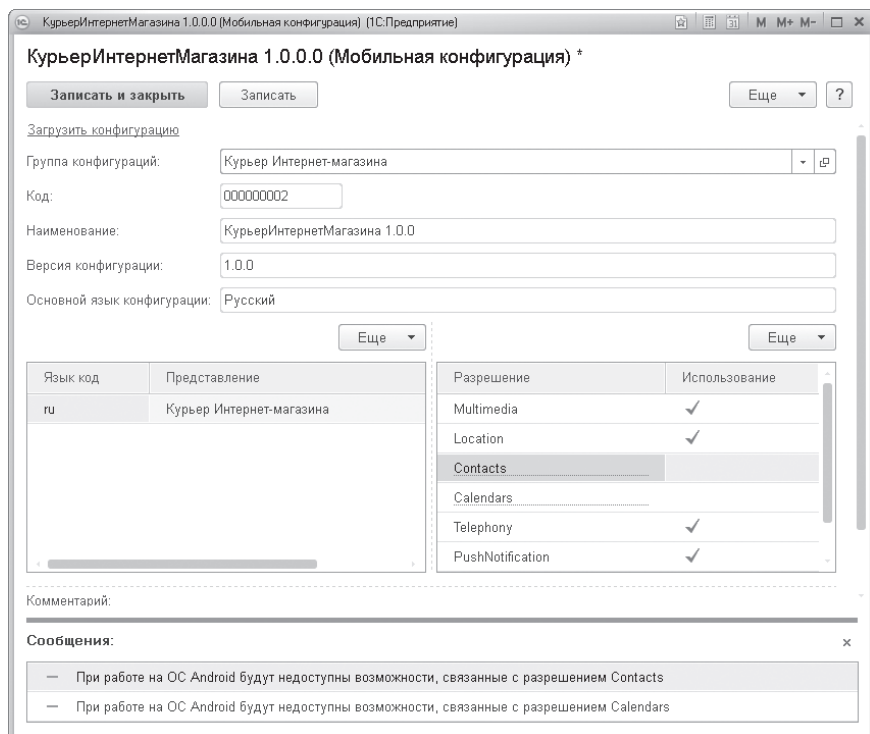


Рис. 5.24. Загрузка мобильной конфигурации

Нажмем Записать и закрыть.

Описание параметров мобильного приложения

Теперь нам нужно описать параметры мобильного приложения, которое мы будем собирать, в справочнике Мобильные приложения.

Справочник должен обладать двухуровневой структурой, где группа описывает основные параметры сборки, а элемент группы уточняет параметры сборки для конкретной версии мобильного приложения. Для каждого мобильного приложения должна быть создана отдельная группа, и для

каждой версии мобильного приложения в данной группе нужно создать свой элемент.

Из командной панели приложения откроем справочник Мобильные приложения и нажмем кнопку Создать группу. В открывшейся форме зададим наименование мобильного приложения Курьер Интернет-магазина.

Поставщик у нас один – Моя фирма. Он заполнится автоматически. А также установится флажок Для ОС Android. Поле Мобильная платформа оставим пустым – при сборке автоматически будет использована самая «свежая» версия платформы.

В поле Идентификатор решения укажем произвольную строку на латинице. Следующее за ним поле заполнится автоматически (рис. 5.25).

Мобильное приложение (создание группы) *

Записать и закрыть Записать Работа с сервисом "1С"...

Еще ?

Код:

Наименование: Курьер Интернет-магазина

Поставщик: Моя фирма

Мобильное приложение собирать

☐ Для ОС iOS

Сертификат для сборки:

Профиль провизии:

Идентификатор в AppStore: 0

☒ Для ОС Android

Ключ для карт Google

Параметр получения ключа для работы с картами Google: 3A19:C7:2D:7D:96:D9:11:1F:D7:A3:F3:C0:10:21:67:0A:55:B6:6A:com.myfirm.myapp

Ключ для работы с картами Google:

Мобильная платформа:

Графические ресурсы:

Аудиоресурсы:

Идентификатор решения: myapp com.myfirm.myapp Проверить уникальность идентификатора

Версия приложения: Номер сборки: 1

Добавить

Еще

N	Источник
---	----------

Рис. 5.25. Создание группы справочника «Мобильные приложения»

После этого автоматически заполнится поле Параметр получения ключа для работы с картами Google (в это поле подставляется значение параметра Хеш SHA1 ключа разработчика поставщика Моя фирма из формы настройки параметров поставщика, см. рис. 5.20 + строка идентификатора решения) – это нам понадобится для получения ключа для работы с картами Google. Для этого мы должны обратиться к сервису Google и после получения ключа записать его в поле Ключ для работы с картами Google.

Если у вас нет аккаунта в Google, то надо его создать и зарегистрироваться по ссылке: <https://accounts.google.com/SignUp>.

Затем в Google APIs Console (<https://code.google.com/apis/console/>) нужно создать проект с помощью кнопки Create project, затем ввести произвольное имя проекта и нажать Create (рис. 5.26).

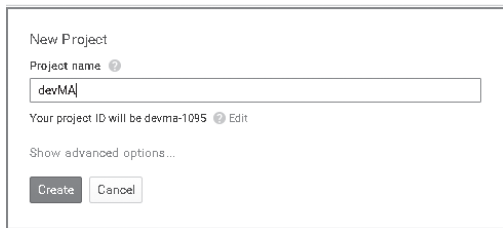


Рис. 5.26. Создание проекта Google APIs Console

Чтобы увидеть список ваших проектов в консоли, может потребоваться еще раз переоткрыть консоль. Затем нужно открыть для редактирования только что созданный проект, нажав на ссылку с его именем.

В списке сервисов Google на закладке APIs нужно выбрать сервис карт Google Maps Android API и включить его нажатием на Enable API (рис. 5.27).

Затем на закладке Credentials нужно создать новый ключ. Для этого выберем из списка Add credentials строку API key и в появившемся окне нажмем Android key (рис. 5.28).

В открывшемся окне нажмем кнопку + Add package name..., в поле Package name скопируем строку полного идентификатора решения из формы настройки параметров приложения (см. рис. 5.25), в поле SHA-1 certificate... – значение параметра Хеш SHA1 ключа разработчика из формы настройки параметров поставщика (см. рис. 5.20) и нажмем кнопку Create (рис. 5.29).

Полученный ключ скопируем в поле Ключ для работы с картами Google формы группы справочника Мобильные приложения, в которой описывается наше мобильное приложение Курьер Интернет-магазина.

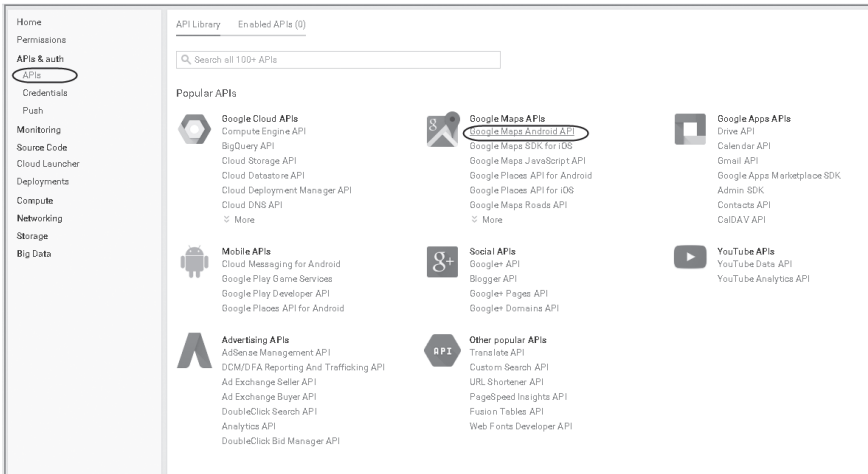


Рис. 5.27. Включение сервиса карт в Google

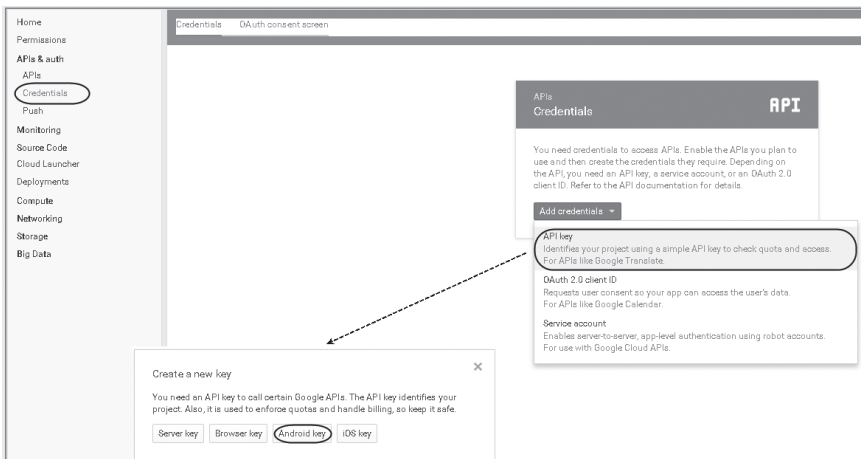


Рис. 5.28. Создание ключа для работы с картами Google

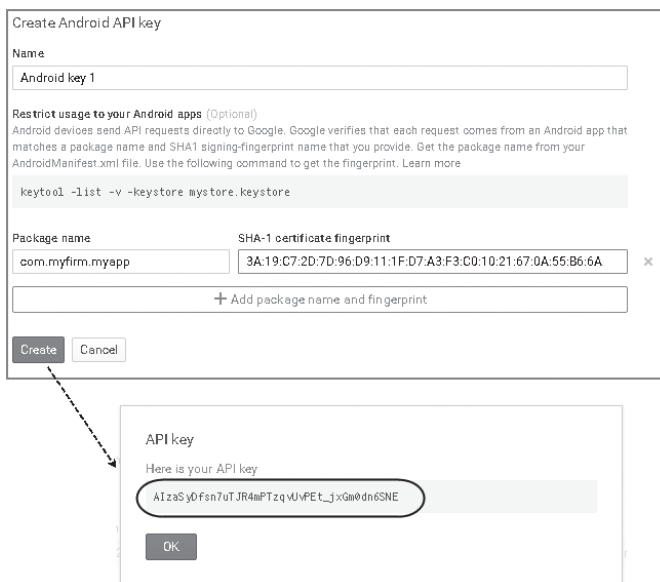


Рис. 5.29. Создание ключа для работы с картами Google

Остальные поля можно не заполнять. Поле Версия приложения будет соответствовать версии используемой мобильной конфигурации, так как при сборке нашего мобильного приложения используется одна мобильная конфигурация.

Поле Номер сборки устанавливается в значение 1 в случае сборки для ОС Android. Данное поле не нужно менять вручную, так как оно будет автоматически увеличивать свое значение при создании новых версий мобильного приложения.

В таблицу в нижней части формы нужно добавить нашу мобильную конфигурацию, которая будет входить в состав мобильного приложения (группу из справочника Мобильные конфигурации), рис. 5.30.

Мобильное приложение (создание группы) * 1С:Предприятие

Записать и закрыть Записать Работа с сервисом "1С"...

Еще ?

Код:

Наименование:

Поставщик:

Мобильное приложение собирать

☐ Для ОС iOS

Сертификат для сборки:

Профиль провизии:

Идентификатор в AppStore:

☒ Для ОС Android

Ключ для карт Google

Параметр получения ключа для работы с картами Google:

Ключ для работы с картами Google:

Мобильная платформа:

Графические ресурсы:

Аудиоресурсы:

Идентификатор решения: Проверить уникальность идентификатора

Версия приложения: Номер сборки:

Добавить

Еще

N	Источник
1	Курьер Интернет-магазина

Рис. 5.30. Создание группы справочника «Мобильные приложения»

Нажмем Записать.

Чтобы мобильное приложение могло получать push-уведомления из основной базы, его надо зарегистрировать на вспомогательном сервисе «1С». Для этого нажмем кнопку Работа с сервисом «1С» (см. рис. 5.30).

При открытии формы Работа с сервисом «1С» по рассылке push-уведомлений произойдет соединение с сервисом на основе заданных нами данных при редактировании параметров поставщика (см. рис. 5.18). Нажмем кнопку Зарегистрировать приложение (см. рис. 5.31).

После регистрации приложения нужно указать, что наше мобильное приложение будет получать push-уведомления с помощью сервиса GCM (Google Cloud Messaging). Для этого нажмем кнопку Подключить к GCM (рис. 5.31).

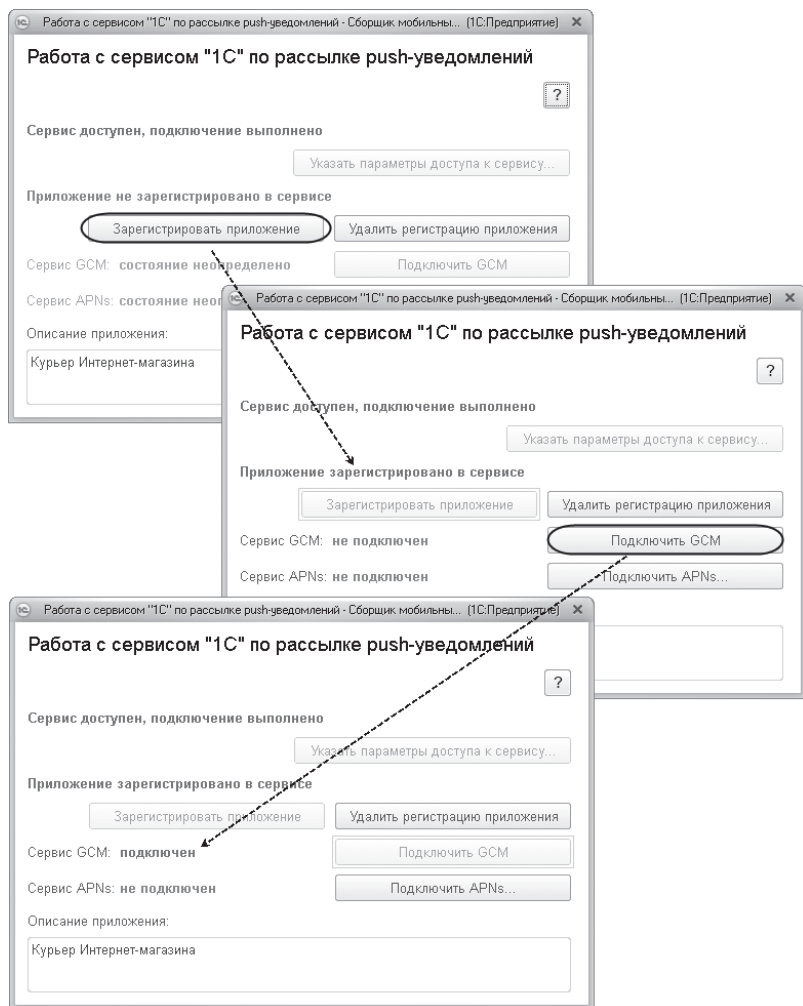


Рис. 5.31. Регистрация мобильного приложения на сервисе «1С»

ПОДРОБНЕЕ

Более подробно о работе с этой формой можно прочитать в контекстной справке, открываемой по кнопке со знаком «?».

Вернемся в форму группы справочника Мобильные приложения и нажмем Записать и закрыть.

Сборка мобильного приложения

Теперь в группе мобильных приложений создадим элемент. Все поля этой формы заполняются автоматически. О назначении этих полей можно прочитать в контекстной справке, открывающейся по кнопке со знаком «?» (рис. 5.32).

Мобильное приложение (создание) [1С:Предприятие]

Наименование:

Поставщик:

Мобильное приложение собирать

☐ Для ОС iOS

Сертификат для сборки:

Профиль провизии:

Идентификатор в AppStore:

☒ Для ОС Android

Ключ для работы с картами Google:

Полный идентификатор приложения:

Мобильная платформа:

Графические ресурсы:

Аудиоресурсы:

Язык представления мобильного приложения

Язык: Представление:

Версия приложения: Номер сборки:

Конфигурации мобильного приложения

N	Конфигурация	Имя	Версия
1	КурьерИнтернетМагазина 1.0.0	Курьер Интернет-магазина	1.0.0

Рис. 5.32. Создание элемента справочника «Мобильные приложения»

Чтобы выполнить сборку мобильного приложения, нажмем кнопку Записать и затем Собрать приложение.

Подождем, пока появятся результаты сборки. Когда сборка закончится, сборщик автоматически перейдет на закладку Результаты сборки – в нашем случае это два файла: один для архитектуры ARM, другой для архитектуры x86 (рис. 5.33).

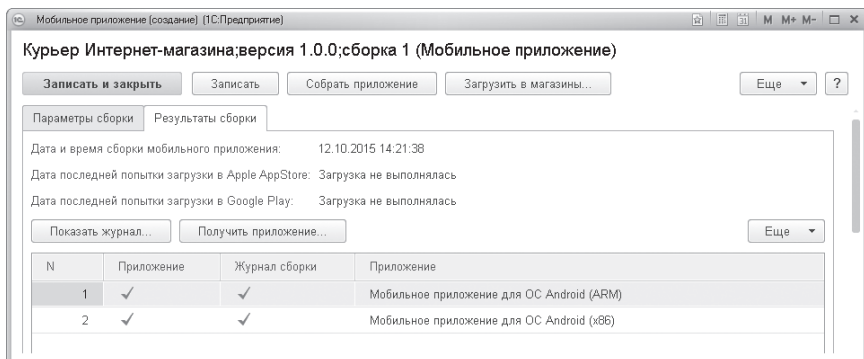


Рис. 5.33. Результаты сборки мобильного приложения

В нашем планшете процессор архитектуры ARM, поэтому выделим строку Мобильное приложение для ОС Android (ARM) и нажмем Получить приложение.

Сохраним полученный файл `com.myfirm.myapp-arm.apk` на компьютер. Затем скопируем этот файл на планшет и запустим его. Приложение с пиктограммой «1С» установится и добавится в список приложений планшета.

Чтобы протестировать его работу, нужно выполнить синхронизацию данных с основным приложением.

Перед этим в форме настроек нужно заполнить адрес и код центральной базы, как показано в разделе «Использование Web-сервиса в мобильном приложении. Обмен данными» (см. стр. 312).

А также нужно заполнить список узлов обмена Мобильные устройства, как показано в разделе «Перенос данных на планшет» (см. стр. 53).

Перед тем как тестировать работу с push-уведомлениями, нужно еще раз переоткрыть мобильное приложение, чтобы оно смогло передать основному приложению свой идентификатор подписчика уведомлений через Web-сервис, параметры доступа к которому вы только что указали.

© ООО «1С-Публишинг», 2015

© Оформление. ООО «1С-Публишинг», 2015

Все права защищены.

Материалы предназначены для личного индивидуального использования приобретателем.

Запрещено тиражирование, распространение материалов, предоставление доступа по сети к материалам без письменного разрешения правообладателей.

Разрешено копирование фрагментов программного кода для использования в разрабатываемых прикладных решениях.

Фирма «1С»

123056, Москва, а/я 64, Селезневская ул., 21

Тел.: (495) 737-92-57

1c@1c.ru, <http://www.1c.ru/>

Издательство ООО «1С-Публишинг»

127434, Москва, Дмитровское ш., 9

Тел.: (495) 681-02-21

publishing@1c.ru, <http://books.1c.ru>

Об опечатках просьба сообщать по адресу books.v8@1c.ru.